

Alexander Barkalov ·
Larysa Titarenko · Kamil Mielcarek ·
Małgorzata Mazurkiewicz ·
Elżbieta Kawecka

Logic Synthesis for VLSI-Based Combined Finite State Machines

Synthesis Targeting ASICs, CPLDs and
FPGAs

Lecture Notes in Electrical Engineering

Volume 922

Series Editors

Leopoldo Angrisani, Department of Electrical and Information Technologies Engineering, University of Napoli Federico II, Naples, Italy

Marco Arteaga, Departament de Control y Robótica, Universidad Nacional Autónoma de México, Coyoacán, Mexico

Bijaya Ketan Panigrahi, Electrical Engineering, Indian Institute of Technology Delhi, New Delhi, Delhi, India

Samarjit Chakraborty, Fakultät für Elektrotechnik und Informationstechnik, TU München, Munich, Germany

Jiming Chen, Zhejiang University, Hangzhou, Zhejiang, China

Shanben Chen, Materials Science and Engineering, Shanghai Jiao Tong University, Shanghai, China

Tan Kay Chen, Department of Electrical and Computer Engineering, National University of Singapore, Singapore, Singapore

Rüdiger Dillmann, Humanoids and Intelligent Systems Laboratory, Karlsruhe Institute for Technology, Karlsruhe, Germany

Haibin Duan, Beijing University of Aeronautics and Astronautics, Beijing, China

Gianluigi Ferrari, Università di Parma, Parma, Italy

Manuel Ferre, Centre for Automation and Robotics CAR (UPM-CSIC), Universidad Politécnica de Madrid, Madrid, Spain

Sandra Hirche, Department of Electrical Engineering and Information Science, Technische Universität München, Munich, Germany

Faryar Jabbari, Department of Mechanical and Aerospace Engineering, University of California, Irvine, CA, USA

Limin Jia, State Key Laboratory of Rail Traffic Control and Safety, Beijing Jiaotong University, Beijing, China

Janusz Kacprzyk, Systems Research Institute, Polish Academy of Sciences, Warsaw, Poland

Alaa Khamis, German University in Egypt El Tagamoa El Khames, New Cairo City, Egypt

Torsten Kroeger, Stanford University, Stanford, CA, USA

Yong Li, Hunan University, Changsha, Hunan, China

Qilian Liang, Department of Electrical Engineering, University of Texas at Arlington, Arlington, TX, USA

Ferran Martín, Departament d'Enginyeria Electrònica, Universitat Autònoma de Barcelona, Bellaterra, Barcelona, Spain

Tan Cher Ming, College of Engineering, Nanyang Technological University, Singapore, Singapore

Wolfgang Minker, Institute of Information Technology, University of Ulm, Ulm, Germany

Pradeep Misra, Department of Electrical Engineering, Wright State University, Dayton, OH, USA

Sebastian Möller, Quality and Usability Laboratory, TU Berlin, Berlin, Germany

Subhas Mukhopadhyay, School of Engineering & Advanced Technology, Massey University,

Palmerston North, Manawatu-Wanganui, New Zealand

Cun-Zheng Ning, Electrical Engineering, Arizona State University, Tempe, AZ, USA

Toyooki Nishida, Graduate School of Informatics, Kyoto University, Kyoto, Japan

Luca Oneto, Department of Informatics, BioEngineering, Robotics, University of Genova, Genova, Genova, Italy

Federica Pascucci, Dipartimento di Ingegneria, Università degli Studi "Roma Tre", Rome, Italy

Yong Qin, State Key Laboratory of Rail Traffic Control and Safety, Beijing Jiaotong University, Beijing, China

Gan Woon Seng, School of Electrical & Electronic Engineering, Nanyang Technological University, Singapore, Singapore

Joachim Speidel, Institute of Telecommunications, Universität Stuttgart, Stuttgart, Germany

Germano Veiga, Campus da FEUP, INESC Porto, Porto, Portugal

Haitao Wu, Academy of Opto-electronics, Chinese Academy of Sciences, Beijing, China

Walter Zamboni, DIEM - Università degli studi di Salerno, Fisciano, Salerno, Italy

Junjie James Zhang, Charlotte, NC, USA

The book series *Lecture Notes in Electrical Engineering* (LNEE) publishes the latest developments in Electrical Engineering—quickly, informally and in high quality. While original research reported in proceedings and monographs has traditionally formed the core of LNEE, we also encourage authors to submit books devoted to supporting student education and professional training in the various fields and applications areas of electrical engineering. The series cover classical and emerging topics concerning:

- Communication Engineering, Information Theory and Networks
- Electronics Engineering and Microelectronics
- Signal, Image and Speech Processing
- Wireless and Mobile Communication
- Circuits and Systems
- Energy Systems, Power Electronics and Electrical Machines
- Electro-optical Engineering
- Instrumentation Engineering
- Avionics Engineering
- Control Systems
- Internet-of-Things and Cybersecurity
- Biomedical Devices, MEMS and NEMS

For general information about this book series, comments or suggestions, please contact leontina.dicecco@springer.com.

To submit a proposal or request further information, please contact the Publishing Editor in your country:

China

Jasmine Dou, Editor (jasmine.dou@springer.com)

India, Japan, Rest of Asia

Swati Meherishi, Editorial Director (Swati.Meherishi@springer.com)

Southeast Asia, Australia, New Zealand

Ramesh Nath Premnath, Editor (ramesh.premnath@springernature.com)

USA, Canada

Michael Luby, Senior Editor (michael.luby@springer.com)

All other Countries

Leontina Di Cecco, Senior Editor (leontina.dicecco@springer.com)

**** This series is indexed by EI Compendex and Scopus databases. ****

Alexander Barkalov · Larysa Titarenko ·
Kamil Mielcarek · Małgorzata Mazurkiewicz ·
Elżbieta Kawecka

Logic Synthesis for VLSI-Based Combined Finite State Machines

Synthesis Targeting ASICs, CPLDs
and FPGAs

Alexander Barkalov
Institute of Metrology, Electronics
and Computer Science
University of Zielona Góra
Zielona Góra, Poland

Chair of Computer Science and Information
Technology
Vasyl' Stus Donetsk National University
Vinnytsia, Ukraine

Kamil Mielcarek
Institute of Metrology, Electronics
and Computer Science
University of Zielona Góra
Zielona Góra, Poland

Elżbieta Kawecka
Department of Information Systems
and Computer Networks
Faculty of Technology
The Jacob of Paradies University
Gorzów Wielkopolski, Poland

Larysa Titarenko
Institute of Metrology, Electronics
and Computer Science
University of Zielona Góra
Zielona Góra, Poland

Department of Infocommunications
Kharkov National University of Radio
Electronics
Kharkiv, Ukraine

Małgorzata Mazurkiewicz
Institute of Control and Computation
Engineering
University of Zielona Góra
Zielona Góra, Poland

ISSN 1876-1100

ISSN 1876-1119 (electronic)

Lecture Notes in Electrical Engineering

ISBN 978-3-031-16026-4

ISBN 978-3-031-16027-1 (eBook)

<https://doi.org/10.1007/978-3-031-16027-1>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Preface

Our time is characterized by the fact that computers and embedded systems can be found practically in all fields of human activity. The up-to-day state of the art in modern computer science is characterized by three major factors. The first factor is a development and application of ultra-complex VLSI such as “system-on-programmable chip” with billions of transistors and hundreds of millions of equivalent gates. The second factor is a development of hardware description languages such as VHDL and Verilog permitting the specification of designs with tremendous complexness. The third factor is a wide application of different computer-aided design (CAD) tools to design very complex VLSI-based projects in the satisfactory time. These three factors affected significantly the process of hardware design. Now the hardware design is very similar to the development of computer programs. The mutual application of hardware description languages and CAD tools allows concentrating the designer’s energy on the basic problems of design, whereas a routine work remains the prerogative of computers.

Tremendous achievements in the area of semiconductor electronics result in the development of integrated circuits with very large-scale integration (VLSI), such as application-specific integrated circuits (ASIC), complex programmable logic devices (CPLD) and field-programmable gate arrays (FPGA). Our book targets control units based on the model of combined finite state machine (CFSM). The proposed CFSM models and methods of their synthesis target ASICs, CPLDs and FPGAs. The modern VLSI chips are so huge that enough only a single chip is enough to implement a very complex digital system including a data-path and a control unit. Because of the extreme complexity and different specifics of modern VLSI chips, it is very important to develop effective design methods targeting particular properties of logical elements in use.

As it is known, any digital system can be represented as a composition of a data-path and a control unit (CU). Logic circuits of operational blocks forming a data-path have regular structures. It allows using standard library elements of CAD tools (such as counters, multibit adders, multipliers, multiplexers, decoders and so on) for their design. A control unit coordinates the interplay of other system blocks producing a sequence of control signals. These control signals cause executing some operations

in a data-path. As a rule, control units have irregular structures. It makes process of their design very sophisticated. Many important features of a digital system, such as hardware amount, performance and power consumption, depend to a large extent on characteristics of its CU. Therefore, to design competitive digital systems with VLSI chips, a designer should have fundamental knowledge in the area of synthesis and optimization of logic circuits of CUs. As experience of many scientists shows, design methods used by standard industrial packages produce circuits which are far from optimal. Especially, it is true in the case of designing complex CUs. It means that a designer could be forced to develop his own design methods, next to program them and at last to combine them with standard packages to get a circuit with desired characteristics. To help such a designer, this book is devoted to solution of the problems of logic synthesis and reduction of hardware amount in CUs.

In this book, we discuss a case when a CU is represented by the model of CFSM. The CFSMs combine features of both Mealy and Moore FSMs. They have output signals of both Mealy and Moore types. They have states of Moore FSMs. So, to optimize the circuits of CFSMs, we propose to use optimization methods targeting both Mealy and Moore FSMs. The book contains some original synthesis and optimization methods targeting hardware reduction in VLSI-based CFSM circuits. These methods take into account the peculiarities of both a CFSM model and a VLSI chip in use. Three groups of methods are used for hardware reduction: structural decomposition, proper state assignment and heterogeneous implementation.

The structural decomposition is a method allowing representation of a CFSM logic circuit as a composition of functionally completed blocks. Each block is different from other blocks by a system of implemented Boolean functions. The circuits of these blocks are much simpler than the circuit of CFSM without decomposition. It leads to increasing for the number of logic levels in the final CFSM circuit. But it allows representing some parts of FSM circuit by regular systems of Boolean functions. As a result, the total number of logic elements is less for the decomposed circuit than for its no decomposed counterpart. The proper state assignment minimizes the numbers of arguments (or product terms, or both) in systems of Boolean functions representing a CFSM logic circuit. The heterogeneous implementation assumes that different blocks of decomposed circuit are implemented using different logic elements. For example, the regular parts of these models can be implemented using embedded memory blocks. It results in reducing the irregular part of the control units described by means of Boolean functions. It permits decreasing for the total number of LUTs or PAL-based macrocells in comparison with logic circuits based on homogenous implementation. In our book, control algorithms are represented by graph schemes of algorithms (GSA). This choice is based on the obvious fact that this specification provides the simple explanation of the methods proposed by the authors.

We propose to use two new approaches for structural decomposition of CFSM circuits. The first of them is called a twofold state assignment. The twofold state assignment targets FPGA-based CFSMs. It is based on a partition of the set of states by classes of states such that any Boolean function inside a class could be implemented using a single LUT. Each internal state is encoded in two ways: as an

element of the set of FSM states and as an element of some class of states. The final circuit has three levels of logic and regular interconnections. The second approach is a mixed encoding of microoperations. It can be used for both CPLD- and FPGA-based CFSMs. This method assumes that some microoperations are represented by one-hot codes, while the others are considered as elements of collections of microoperations. It allows reducing hardware in the part of CFSM circuit generating Mealy-type microoperations. We combine the proposed methods with such known approaches of structural decomposition as the replacement of logical conditions, encoding of collections of microoperations, expansion of code space, nonstandard representation of state codes and transformation of object codes.

The process of CFSM logic synthesis can be viewed as a transformation of a control algorithm into some tables describing the behavior of CFSM blocks. These tables are used to find the systems of Boolean functions, which can be used to implement logic circuits of particular CFSM blocks. In order to implement corresponding circuits, this information should be transformed using data formats of particular industrial CAD systems. We do not discuss this step in our book. Our book contains a lot of examples illustrating design of CFSM circuits based on using the proposed methods. Some examples are illustrated by logic circuits. The main part of the book contains eight chapters.

Chapter 1 provides some basic information. It is shown that control algorithms could be implemented using either microprogram control units or finite state machines. The language of GSA is introduced. Next, the connections are shown with GSAs and state transition graphs of Mealy, Moore and combined FSMs. Classical principles are discussed for logic synthesis of Mealy and Moore FSMs. The FSM logic circuits are implemented with gates and flip-flops. Next, the methods of synthesis for microprogram control units are discussed. The last part of the chapter deals with peculiarities of combined FSMs and methods of their synthesis.

Chapter 2 deals with analysis of methods of logic synthesis targeting VLSI. It starts with analysis of evolution of logic elements. The peculiarities of synthesis are shown for PROMs, PLAs and PALs. Next, three classes of VLSI chips are considered, namely ASIC, CPLD and FPGA. The methods of structural decomposition are discussed. Main idea of these methods is reduced to diminishing the numbers of literals in systems of Boolean functions due to increasing the number of logic levels in FSM circuits. Methods of replacement of logical conditions and encoding of collections of microoperations are analyzed. Next, these methods are applied for reducing hardware in ASIC-, CPLD- and FPGA-based FSM circuits. It is shown that memory blocks allow implementing systems of regular Boolean functions. The methods basing of classes of pseudoequivalent states are discussed for Moore FSM. VLSI-based structural diagrams of FSM circuits based on different methods of structural decomposition are shown.

Chapter 3 begins from the trivial matrix implementation of CFSM circuits. To diminish the chip area occupied by the trivial matrix circuit, the methods of structural decomposition are used. In the beginning, the CFSM with the replacement of logical conditions is synthesized. Next, it is shown that the area reduction is possible due to encoding of collections of microoperations of both Mealy and Moore types. Next, the

method of encoding of the fields of compatible microoperations is used to optimize the area required for CFSM circuit. The methods of optimization based on the classes of pseudoequivalent states (PES) of Moore FSMs are discussed. Three groups of these methods are considered: the optimal state assignment, the complete transformation of states and the partial transformation of states into classes of PES. Next, the methods of expansion of code space, nonstandard representation of state codes and refined state assignment are discussed. The last part of the chapter is devoted to the analysis of different possible models of ASIC-based CFSMs. These models combine different methods of structural decomposition used for both Mealy and Moore FSMs. The examples of synthesis are given for majority of proposed CFSM models.

Chapter 4 is devoted to reduction of ASIC chip areas occupied by CFSM logic circuits. All methods are based on using the classes of PES. It starts from optimization of CFSM with optimal state assignment. The optimization is achieved due to replacement of logical conditions, encoding of collections of microoperations of both Mealy and Moore types. Next, these methods are applied for CFSMs with complete transformation of states. Also, the method of encoding of the fields of compatible microoperations is used for the hardware reduction. Next, these methods are used for optimization CFSMs with partial transformation of states. The last parts of the chapter are devoted to optimization based on expansion of code space and nonstandard representation of state codes. The examples of synthesis are given for majority of proposed CFSM models.

Chapter 5 deals with synthesis of CFSMs using the homogenous CPLDs, which include either PAL- or PLA-based macrocells. It starts from synthesis of trivial single-level circuits. The PAL-based circuits are mostly analyzed. Next, different methods of optimization combined with the optimal state assignment are discussed. Further, the methods are discussed based on the transformation of state codes into codes of classes of PES. This approach is combined with replacement of logical conditions and encoding of collections of microoperations. Next, the optimization methods are discussed based on the primary encoding of the classes of PES. Both methods of expansion of code space and nonstandard representation of state codes are discussed. They are combined with other methods of hardware reduction. All proposed methods target reducing the numbers of product terms in systems of Boolean functions representing CFSM circuits. The examples of synthesis are given for majority of proposed CFSM models.

Chapter 6 deals with synthesis of CFSMs using the heterogeneous CPLDs, which include PAL-based macrocells and embedded memory blocks. In the beginning, circuits based on embedded memory blocks are analyzed. Next, different models of CFSMs with trivial state assignment are discussed. The EMBs are used to implement systems of regular functions, and PAL-based macrocells implement irregular functions. Next, it is shown how to use EMBS for replacement of logical conditions. Different optimization methods are discussed for CFSMs with optimal state assignment. Next, the CFSM models based on the primary encoding of classes are proposed. The conditions are shown allowing execution of proposed methods. The last part of the chapter is devoted to synthesis methods based on transformation of

object codes. They are based on the representation of states by collections of microoperations and identifiers, as well as representation of microoperations by states and identifiers. All proposed methods target reducing the numbers of product terms in systems of Boolean functions representing CFSM circuits. The examples of synthesis are given for majority of proposed CFSM models.

Chapter 7 is devoted to the hardware reduction based on transformation of state codes into the codes of classes of PES. The chapter starts from the complete state transformation. The method is applied for CFSMs implemented with either homogeneous or heterogeneous CPLDs. These methods are applied if outcome of the optimal state assignment is far from optimal. It is shown how to combine this approach with other methods of structural decomposition. Next, the CFSMs with the partial state transformation are analyzed. It is proposed to use two codes for classes of PES. It can be done due to a wide fan-in of PAL macrocells. Next, this approach is combined with other methods of structural decomposition. The same is done for CFSMs with PLA-based macrocells. Next, it is shown how to optimize hardware sharing EMBs and macrocells to implement the same systems of Boolean functions. At last, different methods of object transformation are discussed for heterogeneous CPLDs. All proposed methods target reducing the numbers of product terms in systems of Boolean functions representing CFSM circuits. The examples of synthesis are given for majority of proposed CFSM models.

Chapter 8 is devoted to the design of FPGA-based CFSMs. Only heterogeneous models are discussed. CFSM circuits are implemented using lookup table (LUT) elements and EMBs. All proposed methods target reducing the numbers of arguments in systems of Boolean functions representing CFSM circuits. In the beginning, the trivial models of CFSMs are discussed. Next, it is shown how to apply different methods of structural decomposition to improve the characteristics of CFSMs with trivial state assignment. Next, the optimization methods based on the optimal state assignment are discussed. Further, the methods of structural decomposition are used for hardware reduction in CFSMs with expansion of code space and nonstandard representation of state codes. Next, the methods of hardware reduction based on mixed encoding of collections of microoperations is discussed. In this case, some collections are encoded by one-hot codes and some by binary codes with minimum amount of bits. The last part of the chapter is devoted to twofold state assignment in CFSMs. In this case, each state is encoded as an element of the set of states and as an element of some class of partition of the set of states. It leads to CFSM circuit having exactly three levels of logic and regular interconnections. The examples of synthesis are given for majority of proposed CFSM models.

Let us point out that our book is the first book entirely devoted to the problems associated with synthesis and optimization of VLSI-based combined finite state machines. We hope that the book will be interesting and useful for students and Ph.D. students in the area of computer science, as well as for designers of modern

digital systems. We think that proposed CFSM models enlarge the class of models applied for the implementation of control units with modern VLSI chips.

Zielona Góra, Poland
April 2020

Alexander Barkalov
Larysa Titarenko

Contents

1	Control Algorithms and Finite State Machines	1
1.1	Methods of Implementation of Control Algorithms	1
1.2	Basic Models of Finite State Machines	5
1.3	Synthesis of Mealy FSM	9
1.4	Synthesis of Moore FSM	12
1.5	Synthesis of Microprogram Control Units	17
1.6	Background of Combined FSMs	25
	References	30
2	VLSI-based Logic Synthesis	33
2.1	Evaluation of Logic Elements	33
2.2	Logic Synthesis with ASICs	40
2.3	Logic Synthesis with CPLDs	50
2.4	Logic Synthesis with FPGAs	60
	References	67
3	ASIC-based Synthesis of CFSMs	77
3.1	Trivial Matrix Implementation	77
3.2	Structural Decomposition for Matrix CFSMs	80
3.3	PES-based Matrix Circuits of Moore FSMs	93
3.4	Analysis of CFSM-based Matrix Circuits	106
	References	110
4	Optimization of ASIC-based CFSMs	111
4.1	CFSMs with Optimal State Assignment	111
4.2	CFSMs with Transformation of State Codes	119
4.3	CFSMs with Partial Code Transformation	128
4.4	CFSMs with Primary Encoding of Classes of PES	137
	References	143

5 Homogenous CPLD-Based Synthesis of CFSMs 145

5.1 Preliminary Information 145

5.2 Synthesis of P CFSM 147

5.3 Synthesis of CFSMs with Optimal State Assignment 154

5.4 Synthesis of CFSMs with Transformation of State Codes 159

5.5 Synthesis of CFSMs with Primary Encoding of Classes 169

References 178

6 Heterogeneous CPLD-based Synthesis of CFSMs 179

6.1 Preliminary Information 179

6.2 Synthesis of CFSMs with Trivial State Assignment 182

6.3 Synthesis of CFSMs with Optimal State Assignment 190

6.4 Synthesis of CFSMs with Primary Encoding of Classes 195

6.5 Synthesis of CFSMs with Transformation of Object Codes 203

References 213

7 CPLD-Based Synthesis with Transformation of State Codes 215

7.1 Synthesis with Complete State Transformation 215

7.2 Partial State Transformation: Homogenous CPLDs 222

7.3 Partial State Transformation: Heterogenous CPLDs 230

7.4 Combining Different Methods of Object Transformation 241

References 248

8 FPGA-Based Synthesis of CFSMs 249

8.1 Preliminary Information 249

8.2 Primary Encoding of Classes of PES for FPGA-Based GFSMs 256

8.3 FPGA-Based Synthesis with Transformation of Class Codes 265

8.4 Twofold State Assignment in CFSMs 277

References 285

Conclusion 287

Index 289

Abbreviations

ADC	Analog-to-digital converter
ASIC	Application specified integrated circuit
BA	Block of addressing
BF	Block of functions
BMO	Block of microoperations
CC	Combinational circuit
CD	Coder
CDS	Cyber-digital system
CFSM	Combined finite state machine
CLB	Configurable logic block
CM	Control memory
CMCU	Compositional microprogram control unit
CMO	Collection of microoperations
CMOS	Complement metal-oxygen semiconductor
CPLD	Complex programmable logic devices
CPS	Cyber-physical system
CPU	Central processing unit
CS	Chip select
CSA	Combined state assignment
CST	Complete state transformation
CT	Counter
CU	Control unit
DAC	Digital-to-analog converter
DC	Decoder
DP	Data path
DRG	Distributed register
DST	Direct structure table
ECE	Extended class encoding
ECMO	Encoding of collections of microoperations
EFCMO	Encoding of fields of compatible microoperations
EMB	Embedded memory block

EMBer	Logic block consisting from EMBs
ES	Embedded system
FB	Function block
FCMO	Fields of compatible microoperations
FF	Flip-flop
FPGA	Field-programmable gate arrays
FPLD	Field-programmable logic device
FSM	Finite state machine
GAL	Generic array logic
GFT	Generalized formula of transition
GSA	Graph scheme of algorithm
IC	Integrated circuit
IMF	Input memory function
LAB	Logic array block
LB	Logic block
LBC	Logic block cluster
LC	Logical condition
LCS	Linear chain of states
LUT	Lookup table
LUTer	Logic block consisting from LUTs
MC	Macrocell
MCU	Microprogram control unit
MI	Microinstruction
MO	Microoperation
MPI	Matrix of programmable interconnections
MX	Multiplexer
OC	Object of control
OSA	Optimal state assignment
PAL	Programmable array logic
PALer	Block consisting from PALs
PEP	Primary encoding of classes of PES
PES	Pseudoequivalent states
PI	Programmable interconnect
PIM	Programmable interconnect matrix
PLA	Programmable logic arrays
PLAer	Block consisting from PLAs
PLB	Programmable logic block
PROM	Programmable read-only memory
PST	Partial state transformation
RAM	Random access memory
RG	Register
RLC	Replacement of logical conditions
ROM	Read-only memory
RSA	Refined state assignment
SBF	System of Boolean functions

SEP	Secondary encoding of the classes of PES
SFT	System of formulae of transitions
SOP	Sum of products
SPLD	Simple programmable logic device
SRAM	Static random access memory
ST	Structure table
STG	State transition graph
STR	State transformer
TOS	Transformation of object codes
TSA	Twofold state assignment
VLSI	Very large-scale integration circuit

Chapter 1

Control Algorithms and Finite State Machines



Abstract The chapter provides some basic information. It is shown that control algorithms could be implemented using either microprogram control units or finite state machines. The language of GSA is introduced. Next, the connections are shown with GSAs and state transition graphs of Mealy, Moore and combined FSMs. Classical principles are discussed for logic synthesis of Mealy and Moore FSMs. The FSM logic circuits are implemented with gates and flip-flops. Next, there are discussed the methods of synthesis for microprogram control units. The last part of the chapter deals with peculiarities of combined FSMs and methods of their synthesis.

1.1 Methods of Implementation of Control Algorithms

As a rule, a control unit (CU) is a part of some closed loop system [3, 19, 36]. Such systems include a control unit and an object of control (Fig. 1.1).

A control unit manages the behaviour of an object of control (OC) using a control loop. A CU executes some control algorithm. Using a control algorithm, a CU generates a sequence of control signals. The purpose of the control is to change the state of the OC in accordance with the control algorithm. After receiving control signals, the OC performs some prespecified actions. For example, the object opens the garage door or performs the addition of floating-point numbers. A CU checks status signals reflecting the state of the algorithm execution process. Some external data are used to control the process of control. For example, it could be start or stop commands or a control algorithm code.

In the case of cyber-physical systems (CPS) [3], the control unit is represented as a controller (or microcontroller), while the object of control is some physical object. As a rule, the physical object is represented by some electromechanical system [36]. In the case of CPS, the controller is an embedded system (ES) [19] incorporated in the physical object. Embedded systems are becoming more and more popular, and literally people are surrounded by them from all sides [32, 35]. In turn, this means that we are surrounded from all sides by control units.

In the case of CPS, the control loop includes sensors, analog-to-digital converters, controllers, digital-to-analog converters, and actuators (Fig. 1.2).

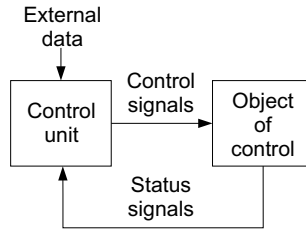


Fig. 1.1 A closed loop system

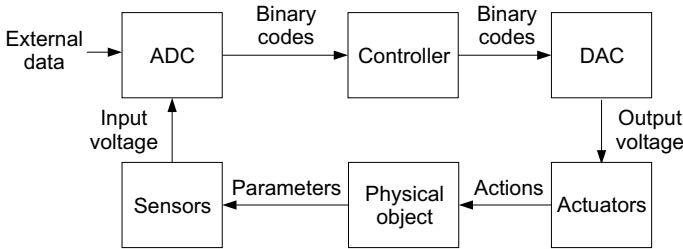
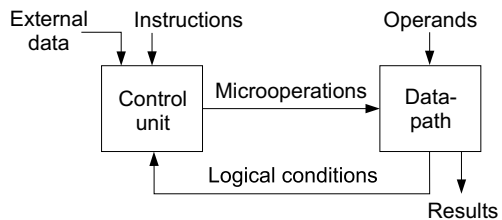


Fig. 1.2 Structural diagram of CPS

In such a system, sensors measure the values of the monitored parameters of the physical object and form the appropriate levels of electrical voltages. Because a controller “understands” only binary codes, analog-to-digital converters (ADC) transform outputs of sensors into binary codes. The same transformation is done for external data (for example, for infrared signal from remote control). The controller transforms input codes into output binary codes using the control algorithm. Digital-to-analog converters (DAC) transform outputs of the controller into electrical voltages used for controlling actuators. For example, the DAC outputs close the motor start relay and turn on the flickering lamp mode. Next, the movement of the rotor through the worm gear opens the gate for the waiting car. In this case, the gate is a physical object.

In the case of a central processing unit (CPU), both objects are digital. The object of control is a data-path (DP) [1, 28, 29]. In this case, the status signals are logical conditions and control signals are microoperations (Fig. 1.3).

Fig. 1.3 Structural diagram of CPU



The model (Fig. 1.3) is based on the principle of microprogram control proposed by M. Wilkes in 1951 [50]. This principle is the following [50, 51]:

1. Each computer instruction is considered as a complex action, which is represented as a sequence of elementary operations (microoperations) on the words of information (operands).
2. The logical conditions are used to control the order of microoperations in the sequence. Both microoperations and logical conditions are Boolean variables [37, 38].
3. Execution of operations is specified by a control algorithm. It is represented in the terms of logical conditions and microoperations. It is called a “microprogram”.
4. The microprogram determines the order of testing the values of logical conditions and sequences of microoperations.

In most modern digital systems, there are two main parts: a control unit and an executive unit (it is a data-path in the case of CPU) [36]. Since both parts are digital, we can call such systems cyber-digital systems (CDS). In CDS, a control algorithm could be represented either as a microprogram kept in the memory of CU (firmware) or as a hardwired automaton (hardware) [14].

Let us point out that in the case of CPSs the controllers could be represented by different computers [2]. In the case of a nuclear power plant, this may be a mainframe computer. In the case of a smart house, this may be a microcontroller. But all these systems are united by the fact that the control algorithm is represented as a program stored in the controller’s memory (software) [25, 28].

So, there are three ways to implement control algorithms (Fig. 1.4). The software way is used in cyber-physical systems. The firmware and hardware ways are used in cyber-digital systems. The firmware realization leads to microprogram control units (MCU) [41]. The hardware realization leads to hardwired sequential circuit [1, 23].

In our book, we will focus our attention on the firmware and hardware ways for implementing control units. To implement the hardwired control, we will use finite state machines (FSM) [4, 5]. We start from Mealy and Moore FSMs. Next, we show that an MCU is a special case of Moore FSM. At last, we discuss combined finite state machines (CFSM). Note that CFSMs are the main theme of our book.

Fig. 1.4 Methods of control algorithm’s implementation

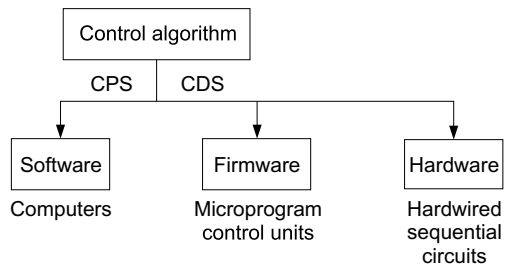
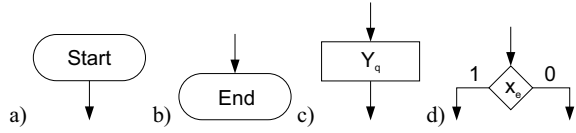


Fig. 1.5 Types of GSA vertices



As it is for our previous books [13, 52], we use the language of graph-schemes of algorithms (GSA) [4, 5] in this book. A GSA is used for the representation of a control algorithm. It is a connected directed graph, characterized by a finite set of vertices. There are four types of GSA vertices (Fig. 1.5).

The start vertex corresponds to the initial point of a control algorithm. It has no input arc. The end vertex corresponds to the ending point of a control algorithm. It has no output arc. The operator vertex includes a collection of microoperations (CMO) $Y_q \subseteq Y$, where $Y = \{y_1, \dots, y_N\}$ is a finite set of microoperations. Each microoperation (MO) evokes some action executed by the executive unit of a CDS. The conditional vertex includes some logical condition (LC) $x_e \in X$, where $X = \{x_1, \dots, x_L\}$ is a finite set of LCs. It has two outputs corresponding to the different values of a Boolean variable $x_e \in X$. The start vertex is shown in Fig. 1.5a, the end vertex in Fig. 1.5b, the operator vertex in Fig. 1.5c, and the conditional vertex in Fig. 1.5d.

Let us consider GSA Γ_1 (Fig. 1.6).

There are four operator and three conditional vertices in GSA Γ_1 . The following sets could be derived from GSA Γ_1 : $X = \{x_1, x_2, x_3\}$ and $Y = \{y_1, \dots, y_5\}$. The MOs $y_n \in Y$ form $Q = 4$ different CMOs: $Y_1 = \{y_1, y_2\}$, $Y_2 = \{y_3\}$, $Y_3 = \{y_2, y_4\}$, and $Y_4 = \{y_1, y_5\}$. In general case, the same CMOs can be written in different operator vertices. The same is true for LCs and conditional vertices [4].

Fig. 1.6 Graph-scheme of algorithm Γ_1

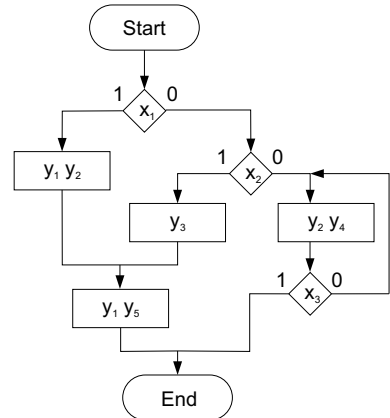
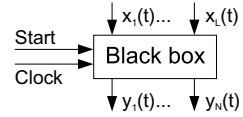


Fig. 1.7 An FSM as a black box



1.2 Basic Models of Finite State Machines

An FSM is a sequential digital device [17] transforming input binary vectors $X(t)$ into output binary vectors $Y(t)$, where $t = 0, 1, 2, \dots$ —is a discrete time clock. The transformation is determined by a specification. In this book, we discuss only finite synchronous machines starting operation only in the presence of an external signal Start. The course of the discrete time is given by pulses Clock generated by an external oscillator of periodic signals. An FSM could be represented by a black box shown in Fig. 1.7.

As follows from Fig. 1.7, the vector $X(t)$ is represented as $\langle x_1(t), \dots, x_L(t) \rangle$ and the vector $Y(t)$ as $\langle y_1(t), \dots, y_N(t) \rangle$. The variables $x_e \in X$ and $y_n \in Y$ are Boolean.

For sequential devices, the values of the components of the output vector at the instant t depend on the values of the components of all input vectors received by the device input at the instants $t = 0, 1, \dots, t - 1, t$ [4, 17]. So, the FSM output depends on the prehistory of FSM's operation [4]. It means that an FSM should have a memory.

The FSM memory is represented by states forming the set $A = \{a_1, \dots, a_M\}$. To represent these states, special binary codes $K(a_m)$ are used. The codes include R binary state variables $T_r \in T$:

$$M \geq R \leq \lceil \log_2 M \rceil. \quad (1.1)$$

State codes are kept into special register (RG) consisting of synchronous “master-slave” flip-flops [47]. As a rule, D flip-flops are the elements of an RG. To change the content of RG, special input memory functions (IMF) are used. They form the set $\Phi = \{D_1, \dots, D_R\}$.

In this book, we will discuss FSMs with registers having exactly

$$R = \lceil \log_2 M \rceil \quad (1.2)$$

flip-flops. It means that $|\Phi| = |T| = \lceil \log_2 M \rceil$.

Now, it is possible to represent an FSM as a composition of combinational circuit (CC) and state register (Fig. 1.8). The device operates as following.

In the instance $t = 0$, the FSM receives the pulse Start. It causes loading the code $K(a_1)$ of the initial state $a_1 \in A$ into RG. During each cycle of operation, some values of MOs are generated by the CC. At the same time, the CC generates some values of input memory functions $D_r \in \Phi$. The pulse Clock allows replacement of

Fig. 1.8 An FSM as composition of combinational circuit and RG

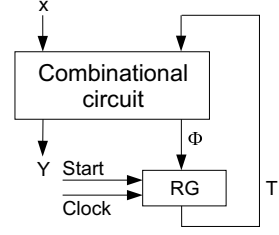
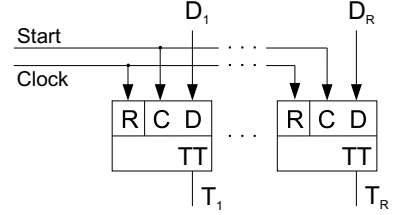


Fig. 1.9 Organization of RG



the code $K(a_m)$ of the current state $a_m \in A$ by the code $K(a_s)$ of next state (state of transition) $a_s \in A$. This code is loaded into RG. The process is repeated till the final state of FSM is reached.

Flip-flops are standard elements of digital circuits [47]. To organize the RG, their informational inputs should be connected with corresponding IMFs. All flip-flops have shared pulses Start and Clock (Fig. 1.9).

So, to implement the logic circuit of FSM, it is necessary: 1) to implement the circuit CC and 2) to connect CC and RG into a loop (Fig. 1.8). This postulate is fully consistent with the structural completeness theorem by Victor Glushkov [31].

There are two basic models of FSMs [31, 38]. For both of them, next states are determined by the same function of transition

$$A \times X \rightarrow A. \quad (1.3)$$

The system (1.3) is determined as

$$\Phi = \Phi(T, X). \quad (1.4)$$

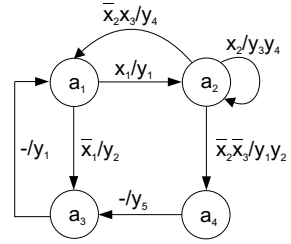
The outputs of FSM, are determined by its function of outputs [4, 38]. In the case of Mealy FSM [4], it is the function

$$A \times X \rightarrow Y. \quad (1.5)$$

In the case of Moore FSM it is the function

$$A \rightarrow Y. \quad (1.6)$$

Fig. 1.10 State transition graph of Mealy FSM S_1



The function (1.5) corresponds to the system of Boolean functions

$$Y = Y(T, X). \quad (1.7)$$

The output function (1.6) corresponds to the SBF

$$Y = Y(T). \quad (1.8)$$

Let us discuss the basic methods of specification of Mealy and Moore FSMs. Both FSMs are represented by a six-component vector

$$S = \langle A, X, Y, \delta, \lambda, a_1 \rangle. \quad (1.9)$$

In (1.9), A is a finite set of internal states, X is a set of LCs, Y is a set of MOs, δ is the function of transitions represented as (1.3), λ is function of outputs represented as either (1.7) or (1.8), and a_1 is an initial state of FSM.

There are two main approaches to represent the vector (1.9). An FSM could be represented either by a state transition graph (STG) or a state transition table (STT) [4, 38].

Let us discuss the STG of Mealy FSM S_1 (Fig. 1.10). Its nodes correspond to FSM states and arcs to interstate transitions.

The following sets could be derived from the STG shown in Fig. 1.10: $A = \{a_1, \dots, a_4\}$, $X = \{x_1, x_2, x_3\}$, and $Y = \{y_1, \dots, y_5\}$. It gives three first components of the vector (1.9). The arcs of STG are marked by the pairs $\langle X_h, Y_h \rangle$ where X_h is an input signal determining the transition number h ($h \in \{1, \dots, H\}$) and Y_h is a CMO generated during this transition. So, arcs and their marking determine functions δ and λ of (1.9). There is the state $a_1 \in A$ explicitly represented on the STG. Let us point out that the input signal X_h is a conjunction of logical conditions (or their compliments) determining the transition $\langle a_m, a_s \rangle$.

For example, the following formulae could be obtained from Fig. 1.10: $a_1 \times \bar{x}_1 = a_3$ and $a_1 \times \bar{x}_1 = y_2$. So, this arc uniquely determines the elements of formulae (1.3) and (1.5). We hope that the reader has no difficulties in finding the other components of these formulae.

Table 1.1 STT of Mealy FSM S_1

a_m	a_s	X_h	Y_h	h
a_1	a_2	x_1	y_1	1
	a_3	$\bar{x}_1$	y_2	2
a_2	a_2	x_2	$y_3 y_4$	3
	a_1	$\bar{x}_2 x_3$	y_4	4
	a_4	$\bar{x}_2 \bar{x}_3$	$y_1 y_2$	5
a_3	a_1	1	y_1	6
a_4	a_3	1	y_5	7

An STT is just a representation of an STG as the list of transitions $\langle a_m, a_s, X_h, Y_h, h \rangle$, where h is the number of transition. Table 1.1 shows the STT corresponding to STG (Fig. 1.10).

As follows from Table 1.1, there are $H = 7$ transitions among the states of S_1 . Unconditional transitions are marked with 1 in the column X_h . This 1 can be considered as the probability of transition $\langle a_m, a_s \rangle$. Let us point out, the disjunction of variables X_h for transitions from each state $a_m \in A$ is also equal to 1. For example, $X_3 \vee X_4 \vee X_5 = x_2 \vee \bar{x}_2 x_3 \vee \bar{x}_2 \bar{x}_3 = 1$. For the STG, the unit probability is indicated by the symbol “—”.

The STG (Fig. 1.11) represents some Moore FSM S_2 . It has $M = 3$, $L = 2$, and $N = 4$.

As follows from (1.8), microoperations $y_n \in Y$ of Moore FSM depend only on states. Because of it, the CMOs $Y(a_m)$ are written near the corresponding states $a_m \in A$ of STG (Fig. 1.11).

Each line of STT of Moore FSM corresponds to the record $\langle a_m, Y(a_m), a_s, X_h, h \rangle$. For example, there is $H = 5$ for STT of Moore FSM S_2 (Table 1.2).

Both STG and STT represent only the behaviour of a corresponding FSM. This is a behaviour specification [29, 38] that does not explain how to get a logic circuit with such a behaviour. To get a circuit, the procedure of synthesis should be applied [29].

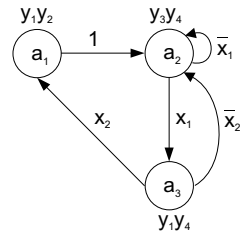
Fig. 1.11 State transition graph of Moore FSM S_2 

Table 1.2 STT of Moore FSM S_2

a_m	$Y(a_m)$	a_s	X_h	h
a_1	$y_1 y_2$	a_2	1	1
a_2	$y_3 y_4$	a_3	x_1	2
		a_2	$\bar{x}_1$	3
a_3	$y_1 y_4$	a_1	x_2	4
		a_2	$\bar{x}_2$	5

1.3 Synthesis of Mealy FSM

The synthesis is reduced to the sequential conversion of the initial specification to the specification defining the logic circuit of FSM [29]. In this book, a GSA Γ is used as an initial specification of the FSM. The following method could be used for FSM synthesis [4]:

1. Constructing the marked GSA Γ .
2. Executing transition from GSA to STG.
3. Constructing the STT on the base of STG.
4. Executing the state assignment.
5. Constructing the direct structure table (DST) of Mealy FSM.
6. Constructing the systems (1.4) and (1.7) on the base of DST.
7. Implementing the FSM logic circuit.

The following procedure [4] allows obtaining the marked GSA Γ :

1. The output of Start vertex is marked by the initial state a_1 .
2. The input of End vertex is marked by the final state a_1 .
3. Inputs of vertices connected with outputs of operator vertices are marked by unique states starting from a_2 .
4. Any input could be marked only once.

Applying this procedure to GSA Γ_1 , we can get the marked GSA Γ_1 shown in Fig. 1.12a. The corresponding STG is shown in Fig. 1.12b.

The FSM performs its functions cyclically. So, its final state corresponds to the beginning of a new cycle of operation. Because of it, both initial and final states are denoted by a_1 . In order not to complicate the Fig. 1.12b, we show the state a_1 twice. We hope that the transition from the marked GSA Γ_1 to STG is clear for the reader.

The transition from the STG to STT is a trivial happening. Let GSA Γ_1 represent Mealy FSM S_3 . Its STT has $H = 6$ rows (Table 1.3).

States $a_m \in A$ are abstract issues. To implement the FSM circuit, states must be represented by some physical objects. Such objects are vectors $K(a_m)$ consisting of state variables $T_r \in T$ and called state codes. The transition from states $a_m \in A$ to state codes $K(a_m)$ is called a state assignment.

Table 1.4 Direct structure table of Mealy FSM S_3

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
a_1	00	a_2	01	x_1	$y_1 y_2$	D_2	1
		a_2	01	$\bar{x}_1 x_2$	y_3	D_2	2
		a_3	10	$\bar{x}_1 \bar{x}_2$	$y_2 y_4$	D_1	3
a_2	01	a_1	00	1	$y_1 y_5$	–	4
a_3	10	a_1	00	x_3	–	–	5
		a_3	10	$\bar{x}_3$	$y_2 y_4$	D_1	6

$$F_h = \bigwedge_{r=1}^R T_r^{l_{mr}} \cdot X_h \quad (h \in \{1, \dots, H\}). \quad (1.10)$$

The first member of (1.10) is a conjunction A_m corresponding to the code $K(a_m)$ of the current state $a_m \in A$ from the h -th row of DST. There is $l_{mr} \in \{0, 1\}$, $T_r^\circ = \bar{T}_r$, $T_r^1 = T_r$ ($r \in \{1, \dots, R\}$).

The following SBFs could be derived from Table 1.4:

$$D_1 = F_3 \vee F_6; \quad D_2 = F_1 \vee F_2. \quad (1.11)$$

$$\begin{aligned} y_1 &= F_1 \vee F_4; & y_4 &= F_3 \vee F_6; \\ y_2 &= F_1 \vee F_3 \vee F_6; & y_5 &= F_3 \vee F_6. \\ y_3 &= F_2; \end{aligned} \quad (1.12)$$

The system (1.11) corresponds to (1.4), the system (1.12) to (1.7). The terms F_h are determined as:

$$\begin{aligned} F_1 &= \bar{T}_1 \bar{T}_2 x_1; & F_2 &= \bar{T}_1 \bar{T}_2 \bar{x}_1 x_2; & F_3 &= \bar{T}_1 \bar{T}_2 \bar{x}_1 \bar{x}_2; \\ F_4 &= \bar{T}_1 \bar{T}_2; & F_5 &= T_1 \bar{T}_2 x_3; & F_6 &= T_1 \bar{T}_2 \bar{x}_3. \end{aligned} \quad (1.13)$$

Each term (1.13) corresponds to an AND gate, each function (1.11)–(1.12) corresponds to an OR gate. Using this fact, it is possible to implement the logic circuit of Mealy FSM S_3 (Fig. 1.13).

In fact, we used the canonical method of structure synthesis [31] to get the circuit of S_3 . The circuit can be simplified using different optimization approaches [43, 44, 46, 49, 53]. We discuss those methods in next Chapters of this book.

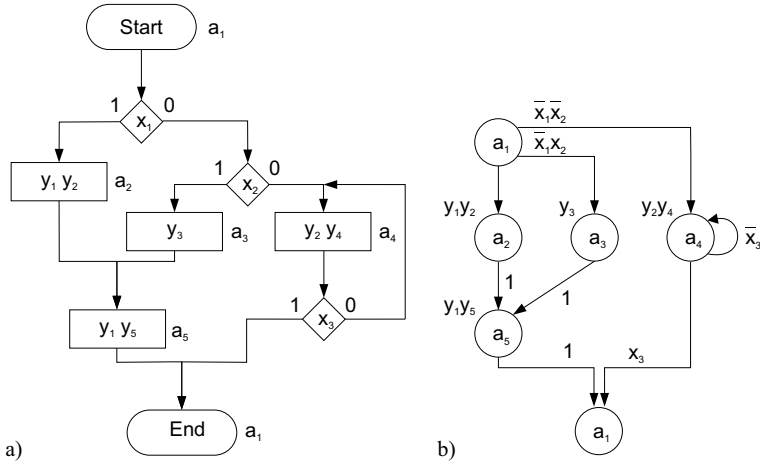


Fig. 1.14 Marked GSA Γ_1 **a** and STG of Moore FSM S_3 **b**

Table 1.6 Direct structure table of Moore FSM S_3

a_m	$Y(a_m)$	$K(a_m)$	a_s	$K(a_s)$	X_h	Φ_h	h
a_1	—	000	a_2	001	x_1	D_3	1
			a_3	010	$\bar{x}_1 \bar{x}_2$	D_2	2
			a_4	011	$\bar{x}_1 \bar{x}_2$	$D_2 D_3$	3
a_2	$y_1 y_2$	001	a_5	100	1	D_1	4
a_3	y_3	010	a_5	100	1	D_1	5
a_4	$y_2 y_4$	011	a_1	000	x_3	—	6
			a_4	011	$\bar{x}_3$	$D_2 D_3$	7
a_5	$y_1 y_5$	100	a_1	000	1	—	8

There is $M = 5$, so, using (1.2) gives $R = 3$. The Moore FSM S_3 is characterized by the sets $A = \{a_1, \dots, a_5\}$, $X = \{x_1, x_2, x_3\}$, $Y = \{y_1, \dots, y_5\}$, $T = \{T_1, T_2, T_3\}$, and $\Phi = \{D_1, D_2, D_3\}$. Let us execute the state assignment in the trivial way: $K(a_1) = 000, \dots, K(a_5) = 100$. Now, we can construct the DST of Moore FSM S_3 (Table 1.6).

Let us construct the system of terms F_h for the discussed case:

$$\begin{aligned}
 F_1 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 x_1; & F_4 &= \bar{T}_1 \bar{T}_2 T_3; & F_7 &= \bar{T}_1 T_2 T_3 \bar{x}_3; \\
 F_2 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{x}_1 x_2; & F_5 &= \bar{T}_1 T_2 \bar{T}_3; & F_8 &= T_1 \bar{T}_2 \bar{T}_3. \\
 F_3 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{x}_1 \bar{x}_2; & F_6 &= \bar{T}_1 T_2 T_3 x_3;
 \end{aligned} \tag{1.14}$$

Using Table 1.6 and system (1.14), the system $D_r \in \Phi$ could be created:

$$D_1 = F_4 \vee F_5; \quad D_2 = F_2 \vee F_3 \vee F_7; \quad D_3 = F_1 \vee F_3 \vee F_7. \tag{1.15}$$

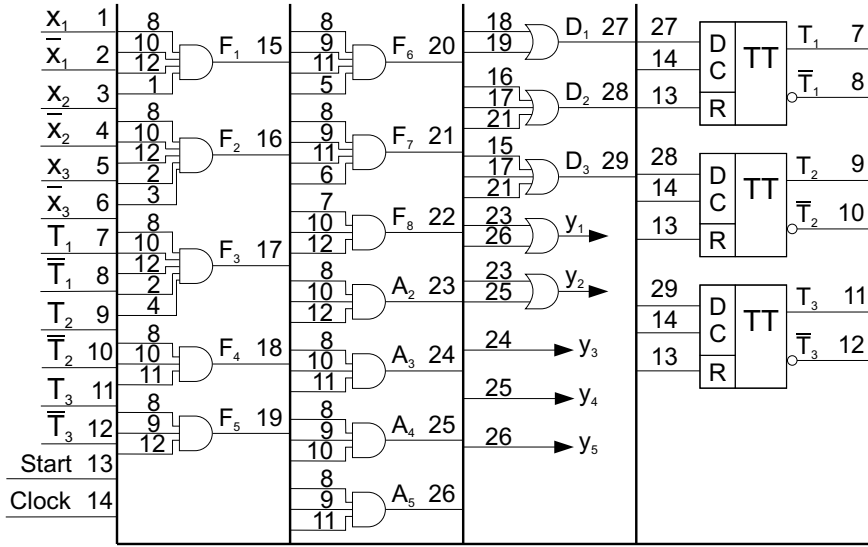


Fig. 1.15 Logic circuit of Moore FSM S_3

The functions $y_n \in Y$ depend only on states. It follows from (1.8). Let us form the system of variables A_m ($m \in \{1, \dots, M\}$):

$$\begin{aligned} A_1 &= \bar{T}_1 \bar{T}_2 \bar{T}_3; & A_3 &= \bar{T}_1 T_2 \bar{T}_3; & A_5 &= T_1 \bar{T}_2 \bar{T}_3. \\ A_2 &= \bar{T}_1 \bar{T}_2 T_3; & A_4 &= \bar{T}_1 T_2 T_3; \end{aligned} \quad (1.16)$$

Using the columns a_m and $K(a_m)$, we can get the following system:

$$\begin{aligned} y_1 &= A_2 \vee A_5; & y_3 &= A_3; & y_5 &= A_5. \\ y_2 &= A_2 \vee A_4; & y_4 &= A_4; \end{aligned} \quad (1.17)$$

Having systems (1.14)–(1.17) allows constructing the logic circuit of Moore FSM S_3 (Fig. 1.15). It requires 12 AND gates, 5 OR gates, and $R = 3$ flip-flops. The equation for A_1 is not used, because A_1 is absent in system (1.17).

Obviously, Mealy and Moore FSM S_3 are equivalent. It follows from the fact that they are synthesised using the same GSA Γ_1 . But there are 6 AND gates and 5 OR gates in the circuit of Mealy FSM S_3 (Fig. 1.13).

Let us point out that the hardware amount (the total number of gates) for Moore FSM could be diminished. To do it, different optimization methods could be used [7–11, 15, 16, 21].

To optimize the logic circuit of Moore FSM, one should take into account [8]:

1. Existence of pseudoequivalent states (PES) of Moore FSM.
2. Dependence of microoperations only on FSM states.

Fig. 1.16 Outcome of optimal state assignment for Moore FSM S_3

		$T_1 T_2$			
		00	01	11	10
T_3	0	a_1	a_3	*	*
	1	a_4	a_2	a_5	*

The states $a_m, a_s \in A$ are pseudoequivalent if the following conditions take places:

$$\delta(a_m, X_h) = \delta(a_s, X_h) \quad (h \in \{1, \dots, H\}). \quad (1.18)$$

Condition (1.18) means that the outputs of operator vertices marked by states a_m and a_s are connected with the input of the same vertex of GSA Γ . For example, the states a_2 and a_3 are PES (Fig. 1.14).

Let Π_A be a partition of the set A by the classes of PES, where $\Pi_A = \{B_1, \dots, B_I\}$. For example, there is the partition $\Pi_A = \{B_1, B_2, B_3, B_4\}$ for FSM S_3 . It includes the classes $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3\}$, $B_3 = \{a_4\}$, and $B_4 = \{a_5\}$. The number of terms in the system (1.4) could be reduced due to the optimal state assignment [8]. It leads to the situation when class is represented by minimal possible amount of generalized intervals of R -dimensional Boolean space. The algorithm JEDI [45], for example, can be used for the optimal state assignment. Also, insignificant state assignments [4] can be used to diminish the number of literals for terms (1.10).

Let us discuss the Karnaugh map (Fig. 1.16) including state codes for Moore FSM S_3 . It contains insignificant state assignments 110, 100, and 101.

Using the basic laws of Boolean algebra [37], we can get the following results:

$$\begin{aligned}
 (a) \quad & 000 \vee 100 = X00; & (b) \quad & 010 \vee 011 = 01X; \\
 (c) \quad & 001 \vee 101 = X01; & (d) \quad & 111 \vee 101 \vee 110 \vee 100 = 1XX; \\
 (e) \quad & 011 \vee 111 = X11; & (f) \quad & 011 \vee 001 = 0X1; \\
 (g) \quad & 010 \vee 110 = X10.
 \end{aligned} \quad (1.19)$$

The Eqs. (a)–(d) from (1.19) give class codes $K(B_1) = X00$, $K(B_2) = X01$, $K(B_3) = X01$, and $K(B_4) = 1XX$. The Eq. (e) gives the function $y_1 = T_2 T_3$, (f) the function $y_2 = \bar{T}_1 \bar{T}_3$, (g) the function $y_3 = T_2 \bar{T}_3$. The functions y_4 and y_5 are determined as $\bar{T}_2 \bar{T}_3$ and T_1 , respectively.

Using classes $B_i \in \Pi_A$ and their codes, the transformed DST of Moore FSM can be constructed [8]. It can be viewed as a DST where states $a_m \in B_i$ are replaced by classes $B_i \in \Pi_A$. It is Table 1.7 in the discussed case.

We use state codes from Fig. 1.16 in Table 1.7. Because there is no column a_m in the transformed DST, we replace the column $Y(a_m)$ of DST by the column $Y(a_s)$.

The IMFs $D_r \in \Phi$ depend on terms (1.10). But now the first members of terms correspond to conjunctions $B_i (i \in \{1, \dots, I\})$. Now, it is the following:

Table 1.7 Transformed DST of Moore FSM S_3

B_i	$K(B_i)$	a_s	$Y(a_s)$	$K(a_s)$	X_h	Φ_h	h
B_1	X00	a_2	y_1y_2	011	x_1	D_2D_3	1
		a_3	y_3	010	$\bar{x}_1, x_2$	D_2	2
		a_4	y_2y_4	001	$\bar{x}_1, \bar{x}_2$	D_3	3
B_2	01X	a_5	y_1y_5	111	1	$D_1D_2D_3$	4
B_3	X01	a_1	–	000	x_3	–	5
		a_4	y_2y_4	001	$\bar{x}_3$	D_3	6
B_4	1XX	a_1	–	000	1	–	7

$$B_i = \bigwedge_{r=1}^R T_r^{l_{ir}}, \quad (1.20)$$

where $l_{ir} \in \{0, 1, X\}$ is a value of the r -th bit of the $K(B_i)$. If $l_{ir} = X$, then $T_r^X = 1$.

The conjunctions B_i are determined as

$$B_1 = \bar{T}_2\bar{T}_3; \quad B_2 = \bar{T}_1T_2; \quad B_3 = \bar{T}_2T_3; \quad B_4 = T_1. \quad (1.21)$$

Using (1.21) and Table 1.7, the following system of terms is derived:

$$\begin{aligned} F_1 &= \bar{T}_2\bar{T}_3x_1; & F_2 &= \bar{T}_2\bar{T}_3\bar{x}_1x_2; & F_3 &= \bar{T}_2\bar{T}_3\bar{x}_1\bar{x}_2; \\ F_4 &= \bar{T}_1T_2; & F_6 &= \bar{T}_2T_3\bar{x}_2. \end{aligned} \quad (1.22)$$

There are only 5 terms in (1.22). Because there are no IMFs in rows 5 and 7 of Table 1.7, there is no need in terms F_5 and F_7 .

The system (1.4) is represented as:

$$D_1 = F_4; \quad D_2 = F_1 \vee F_2 \vee F_4; \quad D_3 = F_1 \vee F_3 \vee F_4 \vee F_6. \quad (1.23)$$

The system (1.8) is represented as:

$$\begin{aligned} y_1 &= T_2T_3; & y_2 &= \bar{T}_1T_3; & y_3 &= T_2\bar{T}_3; \\ y_4 &= \bar{T}_2T_3 = B_3; & y_5 &= T_1 = B_4. \end{aligned} \quad (1.24)$$

Obviously, we can use B_3 and B_4 to form the systems (1.22) and (1.24). The equations (1.21)–(1.24) determine the logic circuit of optimized Moore FSM S_3 (Fig. 1.17). The circuit includes nine AND gates and two OR gates.

To compare FSM circuits implemented with gates, the cost by Quine [37] is used. It is equal to the total number of inputs of gates for a particular circuit. It is equal to 30 for Mealy FSM S_3 , to 55 for Moore FSM S_3 with trivial state codes (Fig. 1.15), and to 30 for PES-based Moore FSM S_3 . So, just a proper state assignment decreases the cost by Quine by 1.83 times.

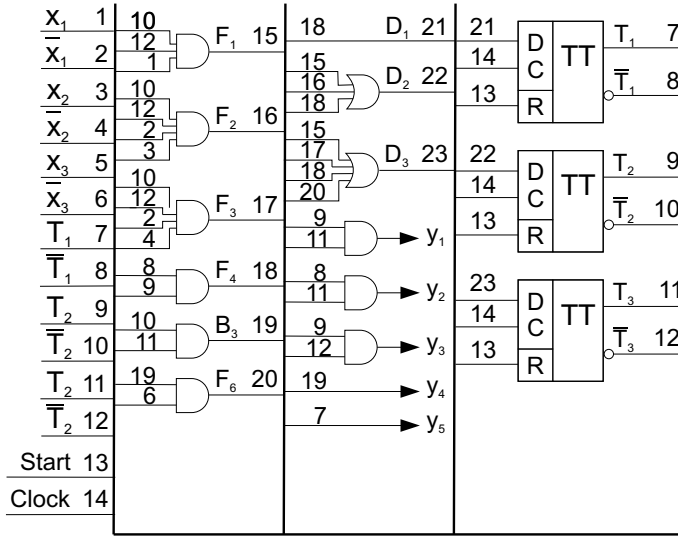


Fig. 1.17 Logic circuit of PES-based Moore FSM S_3

This example shows how the state assignment can change the parameters of an FSM logic circuit. Let us point out that we used so called combined state assignment (CSA) for S_3 [16]. With this approach, each class $B_i \in \Pi_A$ and function $y_n \in Y$ are represented as a single conjunction of state variables $T_r \in T$. Of course, such a result is not always achievable. Also, we use the factorization [4], when the variable B_3 is a part of the term F_6 . Let us point out that factorization increases the FSM cycle time (so, it reduces the FSM performance). But it is rather a general rule: what reduces the hardware also reduces the performance. So, for everything we have to pay.

1.5 Synthesis of Microprogram Control Units

Such an approach was proposed in 1951 by Prof. M. Wilkes [50, 51]. The MCUs were used in computers to organize execution of instructions [24, 27, 33, 42, 48]. Each instruction was represented as a microprogram kept into the special control memory (CU). The microinstructions (MI) are organized using the operational-address principle [34].

In accordance with this principle, an MI includes: (1) an operational part with CMO $Y_q \subseteq Y$ to be executed; (2) an address part with a code of LC $x_e \in X$ to be checked and address fields with addresses of transitions. We discuss the case of an MCU with compulsory addressing of MIs [17].

The MI includes four fields (Fig. 1.18). The field FY includes a code of CMO, the field FX a code of LC, the field FA₀ contains the next address A^{t+1} for an

Fig. 1.18 Format of microinstructions with compulsory addressing

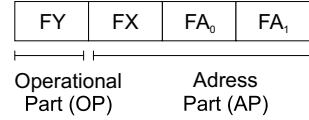
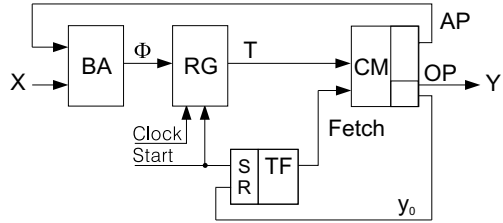


Fig. 1.19 Structural diagram of MCU



unconditional transition or for the case when $x_e^t = 0$, the field FA₁ contains the address of transition A^{t+1} for $x_e^t = 1$. The superscript t means the instant of time.

To fetch an MI from the control memory, its address is loaded into special RG. The content of RG is determined by the block of addressing (BA). To fetch MIs, some enabling signal Fetch should be activated. To do it, a special trigger TF is used. To get Fetch = 1, the pulse Start can be used. To stop the MCU operation, it is necessary to make Fetch=0. Some signal y_0 could be used to do it. This signal corresponds to some bit of the CM. Now, we can show the structural diagram of MCU (Fig. 1.19).

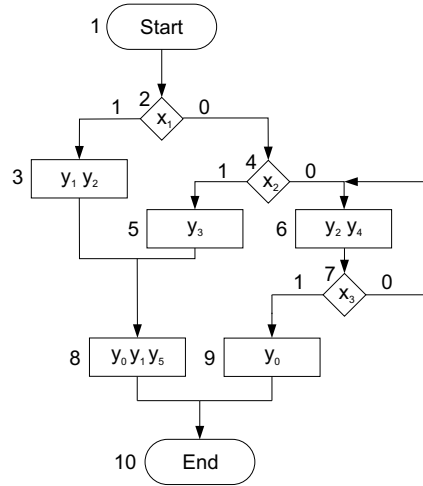
The BA uses the address part (AP) to generate IMFs Φ . The RG is constructed using D flip-flops [17]. So, the functions $D_r \in \Phi$ just repeat addresses from FA₀ or FA₁. It provides the following mode of addressing:

$$A^{t+1} = \begin{cases} [FA_0]^t & \text{if } [FX]^t = \emptyset; \\ [FA_0]^t & \text{if } x_e^t = 0; \\ [FA_1]^t & \text{if } x_e^t = 1. \end{cases} \quad (1.25)$$

The MCU operates as the following. If Start = 1, then Fetch = 1 and zero address is loaded into RG. It starts the operation process. Let some address A^t be in the RG in the instant t ($t = 0, 1, 2, \dots$). The corresponding MI is fetched from CM. Its operational part corresponds to some CMO entering the executive unit. The BA processes both AP and corresponding LC $x_e \in X$. As a result, the address A^{t+1} is generated in accordance with (1.25). The process is terminated if $y_0 = 1$. It corresponds to the last MI of a particular microprogram.

There are the following steps in MCU design method [14, 17]:

1. Transformation of initial GSA Γ .
2. Generation of MIs with a given format.
3. Addressing of microinstructions.
4. Encoding of fields FY and FX.
5. Constructing the content of CM.
6. Implementation of MCU circuit.

Fig. 1.20 Transformed GSA Γ_{1t} 

The transformation of GSA is executed to provide the mode of operating termination [17]. It is executed as the following:

1. If there is an arc connecting some operator vertex with the End vertex, then the MO y_0 is inserted into this operator vertex.
2. If there is arc connecting some conditional vertex with the End vertex, then an additional operator vertex with y_0 is inserted between these vertices.

Applying this approach to GSA Γ_1 (Fig. 1.14a) leads to the transformed GSA Γ_{1t} (Fig. 1.20). We have numbered the vertices of GSA Γ_{1t} . It has $I=10$ vertices. Compared with the initial GSA Γ_1 , the vertex 9 is added.

To generate the MIs with compulsory addressing, it is necessary to analyse pairs of vertices connected by arcs. There are four possible configurations of vertices (Fig. 1.21). To form MIs, we can use the following rules [17]:

1. If i, j are operator vertices (Fig. 1.21a), then the analysis should be continued for the vertex j . The vertex i corresponds to an MI with empty fields FX and FA₁. The field FY containing some code $K(Y_q)$ of CMO $Y_q \subseteq Y$.
2. If i is an operator vertex and j is a conditional one, then they are connected in a single MI. It contains $K(Y_q)$ in the field FY, the code $K(x_e)$ of LC $x_e \in X$ in the field FX. The fields FA₀, FA₁ are not empty. It corresponds to Fig. 1.21b.
3. If both vertices are conditional (Fig. 1.21c), then the vertex j corresponds to an MI with empty field FY. Its field FX contains the code $K(x_e)$. The analysis should be continued for the vertex i .
4. If i is a conditional vertex and j is an operator one (Fig. 1.21d), then the analysis should be continued for both vertices.

The MIs for Γ_{1t} are listed in Table 1.8. It shows pairs of vertices and content of microinstruction fields.

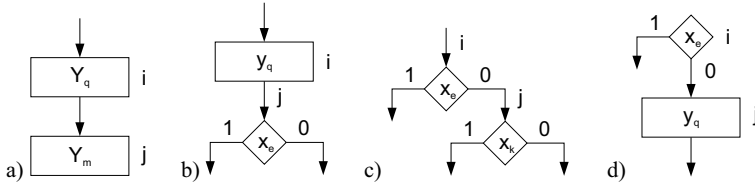


Fig. 1.21 Configurations of pairs

Table 1.8 Microinstructions for GSA Γ_{lr}

MI	i	j	FY	FX	FA ₀	FA ₁	m
MI ₁	2	–	$\emptyset$	x_1	A3	A2	1
MI ₂	3	–	y_1, y_2	$\emptyset$	A6	$\emptyset$	2
MI ₃	4	–	$\emptyset$	x_2	A5	A4	3
MI ₄	5	–	y_3	$\emptyset$	A6	$\emptyset$	4
MI ₅	6	7	y_2, y_4	x_3	A6	A7	5
MI ₆	8	–	y_0, y_1, y_5	$\emptyset$	$\emptyset$	$\emptyset$	6
MI ₇	9	–	y_0	$\emptyset$	$\emptyset$	$\emptyset$	7

There are the following columns in the table of microinstructions: MI (the name of MI); i, j (the pair of vertices corresponding to MI); FY; FX; FA₀ (includes the address of transition A_m of MI _{m} for $x_e^t = 0$ or $[FX]^t = \emptyset$); FA₁ (includes the addresses of transition A_m for $x_e^t = 1$); m (the number of MI). So, there are $M = 7$ microinstructions generated for GSA Γ_{lr} .

To addresses microinstructions, it is necessary R bits. The value of R is determined as (1.2). So, there is $R = 3$ for the discussed case. It gives the sets $T = \{T_1, T_2, T_3\}$ and $\Phi = \{D_1, D_2, D_3\}$. The vertex Start is connected with the vertex 2. So, it should be $A1 = 000$. All other addresses are generated in the trivial way: $A2 = 001, \dots, A7 = 110$.

Now, let us encode the data for fields FY and FX. Let us use the one-hot encoding for the field FY. It means the field has n_0 bits:

$$n_0 = N + 1. \quad (1.26)$$

There is $n_0 = 6$ in the discussed case.

Let us encode logical conditions $x_e \in X$ using the principle of maximal encoding [4]:

$$n_c = \lceil \log_2(L + 1) \rceil. \quad (1.27)$$

So, n_c bits are necessary to encode each LC $x_e \in X$ and the case when $[FX] = \emptyset$. There is $n_c = 2$ in the discussed case.

Let us encode the LCs in the following manner: $K(x_1) = 01$, $K(x_2) = 10$, $K(x_3) = 11$. So, the code 00 indicates that $[FX] = \emptyset$.

Table 1.9 Content of control memory for GSA Γ_{1t}

m	A_m			FY						FX			A_0			A_1		
	T_1	T_2	T_3	y_0	y_1	y_2	y_3	y_4	y_5	d_1	d_2	d_3	d_4	d_5	d_6	d_7	d_8	
1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	1	
2	0	0	1	0	1	1	0	0	0	0	0	1	0	1	0	0	0	
3	0	1	0	0	0	0	0	0	0	1	0	1	0	0	0	0	1	
4	0	1	1	0	0	0	1	0	0	0	0	1	0	1	0	0	0	
5	1	0	0	0	0	1	0	1	0	1	1	1	0	1	1	1	0	
6	1	0	1	1	1	0	0	0	1	0	0	0	0	0	0	0	0	
7	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	

Now, we can transform Table 1.8 into the table of content of CM. It has the following columns: m , A_m , FY, FX, A_0 , A_1 . It is Table 1.9 in the discussed case.

In Table 1.9, we use zero codes for empty fields from Table 1.8. We hope that there is the clear connection between Tables 1.8 and 1.9. There are $n_{CM} = 14$ bits in the words of MI (Table 1.9).

In the general case, we can find n_{CM} as the following:

$$n_{CM} = n_0 + n_C + 2R. \quad (1.28)$$

To implement the MCU circuit, we need a memory block having n_{CM} bits per word and 2^R words (cells).

To implement the MCU circuit, it is necessary to implement the circuits of BA and CM. The BA could be implemented using multiplexers [17].

Let $p = 1$ mean that $RG = [FA_1]$ and $p = 0$ that $RG = [FA_0]$. It means that the equation (1.25) can be represented as

$$A^{t+1} = \bar{p}[FA_0] \vee p[FA_1]. \quad (1.29)$$

As follows from (1.25), $p = 1$ if $x_e^t = 1$ and $\bar{p} = 1$ if $x_e^t = 0$ or $[FX]^t = \emptyset$.

To load the address A^{t+1} , the correct values of functions $D_r \in \Phi$ should be generated. In the discussed case, the equation (1.29) can be represented as

$$D_1 = \bar{p}d_3 \vee pd_6; \quad D_2 = \bar{p}d_4 \vee pd_7; \quad D_3 = \bar{p}d_5 \vee pd_8. \quad (1.30)$$

It is necessary four multiplexers to implement the circuit of BA (Fig. 1.22).

For each combination $\langle d_1 d_2 \rangle$, the value of a particular LC $x_e \in X$ appears as the variable p . If $d_1 = d_2 = 0$, then $p = 0$. Multiplexers MX2–MX4 implement the system (1.30).

We hope that our reader has no problem in connecting together the circuit from Fig. 1.22 with other blocks of MCU (Fig. 1.19). If standard read-only memory (ROM) blocks are used to implement the circuit of CM [52], then the output of TF (Fetch)

Fig. 1.22 Logic circuit of BA

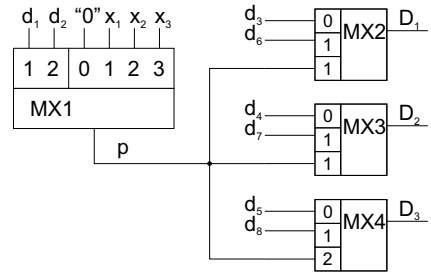
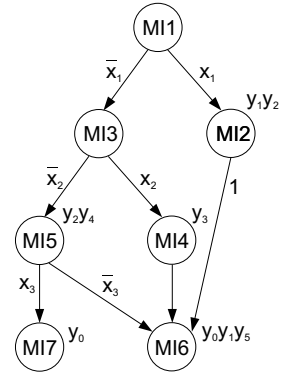


Fig. 1.23 Graph of microinstruction transitions corresponding to GSA Γ_1



should be connected with chip select (CS) input of ROM. All other connections are clear from Fig. 1.19.

Let us transform Table 1.8 into a graph of intermicroinstruction transitions. Its vertices correspond to MIs and arcs to transitions. The arcs are marked either by values of LCs determining conditional transitions or by the symbol 1 for unconditional transitions. It is the graph shown in Fig. 1.23.

As we can see, microoperations $y_n \in Y$ depend on microinstructions. So, microinstructions correspond to FSM states and microinstruction addresses to state codes. Obviously, an MCU is a Moore FSM with the dependence (1.8). Let us point out that many optimization methods used now take start in the methods of optimization of MCU. Let us explain why.

In the sixties of the twentieth century, the computer memory was very expensive [27, 48]. The same applies to the control memory. Because of it, the designers have tried to reduce the required volume of CM. Many methods of hardware reduction used for modern FSMs come from techniques used to minimize the volume (V_{CM}) of control memory.

The following formula allows finding this value:

$$V_{CM} = M \cdot n_{CM}. \quad (1.31)$$

Fig. 1.24 Format of MIs with two logical conditions to be checked

FY	K(x ₁)	K(x ₂)	FA00	FA01	FA10	FA11
----	--------------------	--------------------	------	------	------	------

As follows from (1.31), reducing the value of V_{CM} is possible either by the reducing the number of microinstructions (M) or by reducing their width (n_{CM}). Let us point out that these parameters are interrelated [17].

Only a single LC could be checked during execution of any MI of MCU (Fig. 1.19). It results in an increase in the number of MIs compared to the numbers of operator vertices of GSA Γ . For example, there are 5 states in Moore FSM S_3 , but $M = 7$ for the equivalent MCU. We treat two control units equivalent if they are designed using the same GSA Γ .

Because of this drawback, it is necessary more clock cycles for a control algorithm' execution using MCU. We can increase the number of LCs to be checked. For example, if it is possible to check 2 conditions, then the MIs have the following format (Fig. 1.24).

As you can see, four fields with addresses of transitions are necessary. It means that for unconditional transitions 5 fields will be empty. Therefore, the use of the format (Fig. 1.24) reduces the efficiency of using the control memory.

Let us point out, in fact, the conditions $x_e \in X$ are transformed into a single condition. This mutual condition is represented by the additional variable p . This approach has evolved into a method of replacement of logical conditions [4] that is widely used to optimize the hardware of finite state machines. The method is reduced to replacement of set X by some additional set $P = \{p_1, \dots, p_G\}$ where $G \ll L$ [17].

To reduce the value of n_0 , a binary encoding of CMOs $Y_q \subseteq Y$ could be used [4, 17]. In this case, each CMO Y_q ($q \in \{1, \dots, Q\}$) is encoded by a binary code $K(Y_q)$ having R_Q bits:

$$R_Q = \lceil \log_2 Q \rceil. \quad (1.32)$$

Additional variables $z_r \in Z = \{z_1, \dots, z_{R_Q}\}$ are used to represent the codes $K(Y_q)$.

To generate the MOs $y_n \in Y$, it is necessary: 1) to transform codes $K(Y_q)$ into one-hot codes $Y1, \dots, YQ$ and 2) to transform variables $Y1, \dots, YQ$ into MOs [17]. The first stage is executed by a decoder (DC), the second by a coder (CD). It leads to the following structural diagram (Fig. 1.25).

Let us point out that the system (DC–CD) can be implemented as a ROM having V_Y bits:

$$V_Y = Q \cdot (N + 1). \quad (1.33)$$

Fig. 1.25 Structural diagram of MCU with binary encoding of CMOs

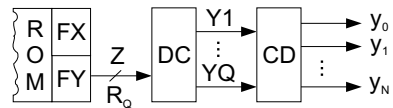
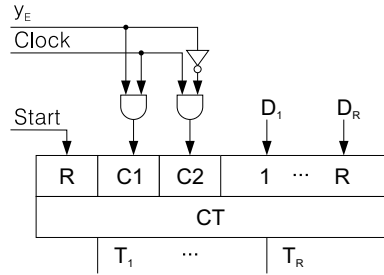


Fig. 1.26 Organization of address counter



Now, there are V_{CM}^1 bits in the CM:

$$V_{CM}^1 = M \cdot (R_Q + n_c + 2R). \quad (1.34)$$

We can find the gain G_{CM} in bits of CM due to the using this approach:

$$G_{CM} = V_{CM} - (V_{CM}^1 + V_Y). \quad (1.35)$$

Obviously, the more is a value of G_{CM} , the more effective the application of this approach is. But it has a drawback. Namely, the transformation $K(Y_q) \rightarrow Y_q \rightarrow y_n \in Y_q$ increases the cycle time of MCU in comparison with MCU (Fig. 1.19).

The value of V_{CM} can be reduced if some addresses of transitions are made implicit. Let us consider the following style of calculating the address A^{t+1} :

$$A^{t+1} = \begin{cases} A^t + 1 & \text{if } [FX]^t = \emptyset; \\ A^t + 1 & \text{if } x_e^t = 1; \\ [FA]^t & \text{if } x_e^t = 0. \end{cases} \quad (1.36)$$

As follows from (1.36), the increment of the current address can produce the next address. In this case the address register RG should be replaced by an address counter (CT) [17]. It is necessary to have only one field of next address (for $x_e = 0$). Such a style is named a combined addressing of MIs [17].

There are R address inputs and three control inputs in the CT (Fig. 1.26). If $R = 1$, then $CT := 0$. If $C1 = \text{Clock}$ then $CT := CT + 1$. If $C2 = \text{Clock}$, then $CT := \langle D_1, \dots, D_R \rangle$.

To distribute pulse Clock between control inputs C1 and C2, the special variable y_E is used. If $y_E = 1$, then $C1 = \text{Clock}$, otherwise $C2 = \text{Clock}$. To get the value $\bar{y}_E$, an additional NOT gate is used.

Obviously, there is the same number of MIs for equivalent MCUs with compulsory and combined addressing of MIs. But there are only $n_{CM} = n_0 + n_C + R$ bits in each word of CM. Therefore, the memory volume (V_{CM}) is reduced in the case of combined addressing.

However, using counter increases the cycle time. This is because the incrementing counter takes longer than loading an address into RG. In addition, it is necessary some time to form the data for C1 and C2.

So, the using counters instead of registers reduces the hardware in a control unit. This fact was used by FSM designers [39, 40]. This approach allows developing the theory of both compositional MCU [6, 12] and Moore FSMs with state counters [18]. Some approaches are developed for implementing counter-based Mealy FSMs [19, 20].

So, two main directions of hardware reducing take roots in MCU design methods:

1. **Structural decomposition.** This approach is based on a transformation of FSM circuit [15]. The typical representatives of this direction are the methods of replacement of logical conditions and encoding of the collections of microoperations.
2. **Replacement of state registers by state counters.** It allows simplifying systems of input memory functions [18]. As a result, it is necessary less logic elements to implement an FSM circuit [18, 20].

Now, we return to the main topic of our book. Let us discuss peculiarities of combined finite state machines.

1.6 Background of Combined FSMs

Let us quote the work [4]. “In some applications, it is interesting to use an automaton which combines properties of Mealy and Moore automata. We call it a combined automaton.” Let us call such FSMs combined, too.

An FSM of this type is represented by an 8-tuple vector

$$S = \langle A, X, Y1, Y2, \delta, \lambda_1, \lambda_2, a_1 \rangle. \quad (1.37)$$

We defined the components A , X , δ , and a_1 before. Let us define other components.

The set $Y1$ includes outputs of Mealy FSM, the set $Y2$ of Moore FSM. The following relations take places:

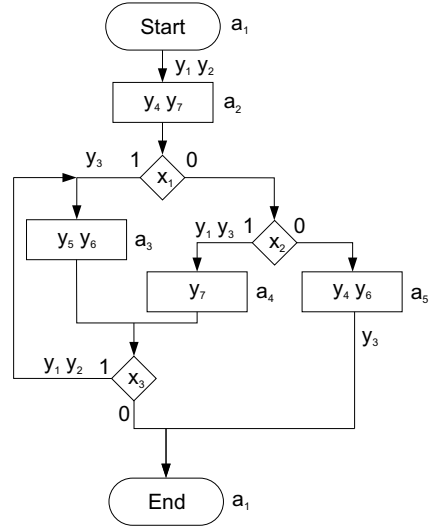
$$Y1 \cup Y2 = Y; \quad (1.38)$$

$$Y1 \cap Y2 = \emptyset. \quad (1.39)$$

The microoperations $y_n \in Y1$ are generated in the current state $a_m \in A$ when the proper logical conditions $x_e \in X$ appear. The microoperations $y_n \in Y2$ are generated till an FSM is in the state $a_m \in A$. The function λ_1 is an output function of Mealy FSM [4, 38]:

$$\lambda_1 : A \times X \rightarrow Y1. \quad (1.40)$$

Fig. 1.27 Graph-scheme of algorithm Γ_2



The function λ_2 is an output function of Moore FSM:

$$\lambda_2 : A \rightarrow Y_2. \quad (1.41)$$

These functions determine the following systems of Boolean functions (SBF):

$$Y1 = Y1(T, X); \quad (1.42)$$

$$Y2 = Y2(T). \quad (1.43)$$

The system $D_r \in \Phi$ is the same as (1.4).

We use the language of GSA to represent CFSMs. Let us consider GSA Γ_2 (Fig. 1.27). It is marked by states of Moore FSM.

The arcs of GSA Γ_2 are marked by CMOs $Y_q \subseteq Y1$. There are CMOs $Y_q \subseteq Y2$ written inside the operator vertices of GSA Γ_2 . The following sets could be found from GSA Γ_2 : $A = \{a_1, \dots, a_5\}$, $X = \{x_1, x_2, x_3\}$, $Y1 = \{y_1, y_2, y_3\}$, and $Y2 = \{y_4, \dots, y_7\}$. So, there is $|Y1| = N_1 = 3$, $|Y2| = N_2 = 4$ and $N = N_1 + N_2 = 7$.

As before, we can construct an STG and an STT using a GSA corresponding to CFSM. In the case of Γ_2 , it is Fig. 1.28 and Table 1.10, respectively. There are CMOs $Y_q \subseteq Y1$ shown in column Y_h and $Y_q \subseteq Y2$ in column $Y(a_m)$. There are $H = 9$ transitions in CFSM for Γ_2 . As follows from the definition of PES [8], states a_3 and a_4 are pseudoequivalent. They can be used for optimization of the CFSM logic circuit.

There are the following steps in the synthesis method of CFSM:

1. Constructing the marked GSA Γ (as for Moore FSM).

Fig. 1.28 State transition graph of CFSM for GSA Γ_2

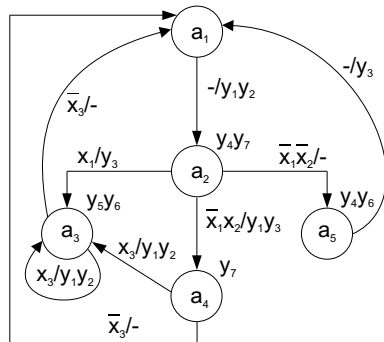


Table 1.10 STT of CFSM for GSA Γ_2

a_m	$Y(a_m)$	a_s	X_h	Y_h	h
a_1	—	a_2	1	$y_1 y_2$	1
a_2	$y_4 y_7$	a_3	x_1	y_3	2
		a_4	$\bar{x}_1 \bar{x}_2$	$y_1 y_3$	3
		a_5	$\bar{x}_1 \bar{x}_2$	—	4
a_3	$y_5 y_6$	a_3	x_3	$y_1 y_2$	5
		a_1	$\bar{x}_3$	—	6
a_4	y_7	a_3	x_3	$y_1 y_2$	7
		a_1	$\bar{x}_3$	—	8
a_5	$y_4 y_6$	a_1	1	y_3	9

2. Execution of transition GSA $\rightarrow$ STT.
3. Execution of state assignment.
4. Constructing the DST of CFSM.
5. Constructing the systems (1.4), (1.42) and (1.43) on the base of DST.
6. Implementing the CFSM logic circuit.

Let us discuss the synthesis of CFSM S_0 corresponding to GSA Γ_2 . Two first steps are already executed.

There is $M = 5$. Using (1.2) gives $R = 3$, $T = \{T_1, T_2, T_3\}$, and $\Phi = \{D_1, D_2, D_3\}$. Let us encode states $a_m \in A$ as shown in Fig. 1.29.

Now, we can construct the DST of S_0 (Table 1.11). Obviously, it includes $H = 9$ rows. We hope that meaning of columns are clear, as well as their contents. We just

Fig. 1.29 State codes of CFSM S_0

		$T_1 T_2$			
T_3		00	01	11	10
		a_1	a_3	a_5	*
	1	*	a_4	a_2	*

Table 1.11 DST of Moore FSM S_0

a_m	$K(a_m)$	$Y(a_m)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
a_1	000	—	a_2	111	1	$y_1 y_2$	$D_1 D_2 D_3$	1
a_2	111	$y_4 y_7$	a_3	010	x_1	y_3	D_2	2
			a_4	011	$\bar{x}_1, x_2$	$y_1 y_3$	$D_2 D_3$	3
			a_5	110	$\bar{x}_1, \bar{x}_2$	—	$D_1 D_2$	4
a_3	010	$y_5 y_6$	a_3	010	x_3	$y_1 y_2$	D_2	5
			a_1	000	$\bar{x}_3$	—	—	6
a_4	011	y_7	a_3	010	x_3	$y_1 y_2$	D_2	7
			a_1	000	$\bar{x}_3$	—	—	8
a_5	110	$y_4 y_6$	a_1	000	1	y_3	—	9

used approaches from previous Sections of this Chapter.

The functions (1.4) and (1.42) depend on terms (1.10). The functions (1.43) depend on the variables $A_m (m \in \{1, \dots, M\})$. Let us find these functions.

There are no functions $D_r \in \Phi$ and $y_n \in Y1$ generated during transitions 6 and 8. So, it is necessary to find the terms F_1 – F_5 , F_7 and F_9 . This system is the following (after minimization):

$$\begin{aligned}
 F_1 &= \bar{T}_2; & F_2 &= T_1 T_3 x_1; & F_3 &= T_1 T_3 \bar{x}_1 x_2; \\
 F_4 &= T_1 T_3 \bar{x}_1 \bar{x}_2; & F_5 &= \bar{T}_1 T_2 \bar{T}_3 x_3; & F_7 &= \bar{T}_1 T_2 T_3 x_3; \\
 F_9 &= T_1 \bar{T}_3.
 \end{aligned} \tag{1.44}$$

To minimize the functions (1.44), we use insignificant state assignments from Karnaugh map (Fig. 1.29). Let us point out that it is possible to optimize some functions $D_r \in \Phi$ and $y_n \in Y1$. They are the following:

$$\begin{aligned}
 D_1 &= F_1 \vee F_4; & D_2 &= F_1 \vee [F_2 \vee F_3 \vee F_4] \vee [F_5 \vee F_7]; \\
 D_3 &= F_1 \vee F_3.
 \end{aligned} \tag{1.45}$$

$$\begin{aligned}
 y_1 &= F_1 \vee F_3 \vee [F_5 \vee F_7]; & y_2 &= F_1 \vee F_3 [F_5 \vee F_7]; \\
 y_3 &= F_3 \vee F_9.
 \end{aligned} \tag{1.46}$$

We can use the expansion law [37] for expressions in the square brackets. It gives the following results:

$$\begin{aligned}
 D_2 &= F_1 \vee T_1 T_3 \vee \bar{T}_1 T_2 x_3 = F_1 \vee F_{10} \vee F_{11}; \\
 y_1 &= F_1 \vee F_3 \vee F_{11}; & y_2 &= F_1 \vee F_3 \vee F_{11}.
 \end{aligned} \tag{1.47}$$

So, the following terms should be implemented: $F_1, F_3, F_4, F_9, F_{10}, F_{11}$. Let us point out that $F_1 = \bar{T}_2$.

The functions (1.43) are represented as the following:

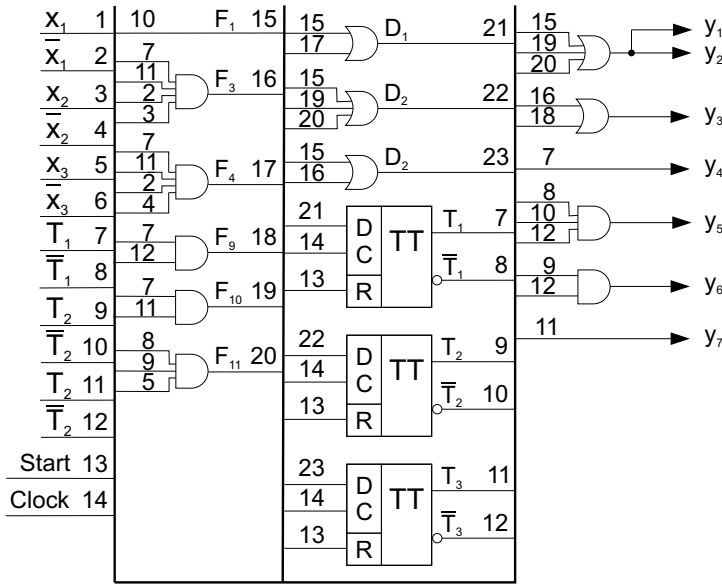


Fig. 1.30 Logic circuit of CFSM S_0

$$\begin{aligned}
 y_4 &= A_2 \vee A_5 = T_1; & y_5 &= A_3 = \bar{T}_1 T_2 \bar{T}_3; \\
 y_6 &= A_3 \vee A_5 = T_2 \bar{T}_3; & y_7 &= A_2 \vee A_4 = T_3.
 \end{aligned}
 \tag{1.48}$$

So, only two terms ($\bar{T}_1 T_2 \bar{T}_3$, $T_2 \bar{T}_3$) should be implemented to get the functions $y_n \in Y_2$.

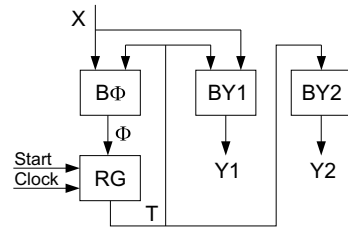
Using these equations, we can construct the logic circuit of CFSM S_0 (Fig. 1.30).

As follows from Fig. 1.30, there are 12 gates in the circuit of S_0 . It has the cost by Quine $C_Q = 32$. In the case of canonical CFSM S_0 , there are 21 gates and $C_Q = 74$. We have got such a gain due to the optimal state assignment (Fig. 1.29). This example shows once more how important the step of state assignment is.

There are three combinational blocks in the structural diagram of CFSM with a basic structure U_1 (Fig. 1.31). The block $B\Phi$ implements the system (1.4), the BY1 the system (1.42) and BY2 the system (1.43).

Of course, these three blocks could be combined in a common combinational part. But we just wanted to emphasize that different blocks have either different input signals or implement different SBFs.

Fig. 1.31 Structural diagram of CFSM U_1



In this book, we discuss CFSMs implemented with very large scale integration (VLSI) chips. As a rule, three problems are connected with the synthesis of VLSI-based FSMs [22, 38, 52]:

1. **The reducing chip area occupied by an FSM circuit** (the problem of hardware reduction). This problem is reduced to diminishing the number of logic elements in an FSM circuit and their interconnections.
2. **The reducing consumed energy.** A solution of this problem leads to either avoiding the use of expensive cooling systems or increasing the battery lifetime. The later is particularly important for mobile and autonomous devices [3, 19, 36]. In addition, the decrease of the degree of heating of the logic elements increases the reliability of FSM operation.
3. **The reducing signal propagation time** (the increase of FSM performance). It is especially important in the case of embedded systems with hard-time constraints [3, 36].

In this book, we discuss and propose methods targeting solution of the first problem. As it is known [29, 30], the reducing hardware leads to decreasing the power consumption and the propagation time.

The methods of hardware reduction depend strongly on logic elements in use [13, 26]. So, we start from analysis of basic VLSI chips. We will discuss three types of chips: application-specific integrated circuits (ASIC), complex programmable logic devices (CPLD) and field-programmable gate arrays (FPGA).

References

1. Adamski M, Barkalov A (2006) Architectural and sequential synthesis of digital devices. University of Zielona Góra Press
2. Alur R (2015) Principles of cyber-physical systems. The MIT Press
3. Ashford LE, Sanjit Arunkumar S (2016) Introduction to embedded systems: A cyber-physical systems approach, 2nd edn. The MIT Press
4. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
5. Baranov S (2008) Logic and system design of digital systems. TUT Press, Tallinn
6. Barkalov A (1983) Microprogram control unit as composition of automate with programmable and hardwired logic. *Automat Comput Sci* 17(4):36–41
7. Barkalov A (1995) Multilevel PLA schemes for microprogram automata. *Cybern Syst Anal* 31(4):489–495

8. Barkalov A (1998) Principles of logic optimization for Moore microprogram automaton. *Cybern Syst Anal* 34(1):54–60
9. Barkalov A, Barkalov A Jr (2005) Design of mealy finite-state machines with the transformation of object codes. *Int J Appl Math Comput Sci* 15(1):151–158
10. Barkalov A, Bukowiec A (2005) Synthesis of mealy finite-states machines for interpretation of verticalized flow-charts. *Theoret Appl Inf* 5(5):39–51
11. Barkalov A, Salomatin V, Starodubov K, Das K (1991) Optimization of Mealy automaton logic using programmable logic arrays. *Cybern Syst Anal* 27(5):789–793
12. Barkalov A, Titarenko L (2008) Logic synthesis for compositional microprogram control units, vol 22. Springer-Verlag, Berlin
13. Barkalov A, Titarenko L (2009) Logic synthesis for FSM-based control units, vol 53. *Lecture Notes in Electrical Engineering*. Springer-Verlag, Berlin
14. Barkalov A, Titarenko L (2009) Synthesis of operational and control automata. UNITECH, Donetsk
15. Barkalov A, Titarenko L, Barkalov A Jr (2012) Structural decomposition as a tool for the optimization of an FPGA-based implementation of a Mealy FSM. *Cybern Syst Anal* 48(2):313–322
16. Barkalov A, Titarenko L, Chmielewski S (2007) Reduction in the number of PAL macrocells into the circuit of a Moore FSM. *Int J Appl Math Comput Sci* 17(4):565–675
17. Barkalov A, Węgrzyn M (2006) Design of control units with programmable logic. University of Zielona Góra Press
18. Barkalov O, Titarenko L, Bieganski J (2018) Logic synthesis for finite state machines based on linear chains of states : Foundations, recent developments and challenges. In: *Studies in Systems, Decision and Control*, Vol. 118. Springer International Publishing Switzerland, Cham
19. Barkalov O, Titarenko L, Mazurkiewicz M (2019). Foundations of embedded systems. In: *Studies in Systems, Decision and Control*, Vol. 195. Springer International Publishing, Cham
20. Barkalov O, Titarenko L, Mielcarek K, Chmielewski S, Design of fpga-based mealy fsms with counters. In: *Mixed Design of Integrated Circuits and Systems—MIXDES*, 25th International Conference, Gdynia, Polska. IEEE Explore, New York, pp 1
21. Bolton M (1990) Digital system design with programmable logic. Addison-Wesley, Boston
22. Brayton R, Hatchel G, McMullen C, Sangiovanni-Vincentelli A (1984) Logic minimization algorithms for VLSI synthesis. Kluwer Academic Publishers, Boston
23. Chattopadhyay S (2005) Area conscious state assignment with flip-flop and output polarity selection for finite state machines synthesis—a genetic algorithm. *Comput J* 48(4):443–450
24. Chu YC (1972) Computer organization and microprogramming. Prentice Hall
25. Clements A (2000) The principles of computer hardware. Oxford University Press Inc, New York
26. Czerwinski R, Kania D (2013) Finite state machine logic synthesis for complex programmable logic devices, vol 231. *Lecture Notes in Electrical Engineering*. Springer-Verlag, Berlin
27. Flynn MJ, Rosin RF (1971) Microprogramming: An introduction and a viewpoint. *IEEE Trans Comput* 20(7):727–731
28. Gajski D (1997) Principles of digital design. Prentice Hall, New York
29. Gajski D, Abdi S, Gerstlauer A, Schirner G (2009) Embedded system design: Modeling, 1st edn. Springer Publishing Company, Incorporated, Synthesis and Verification
30. Garcia-Vargas I, Senhadji-Navarro R (2015) Finite state machines with input multiplexing: A performance study. *IEEE Trans Comput Aided Design Integr Circ Syst* 34(05):867–871
31. Glushkov VM (1965) Synthesis of digital automata. FTD-MT. Translation Division, Foreign Technology Division, 10
32. Guan X, Yang B, Chen C, Dai W, Wang Y (2016) A comprehensive overview of cyber-physical systems: from perspective of feedback system. *IEEE/CAA J Automat Sinica* 3(1):1–14
33. Husson S (1970) Microprogramming: principles and practices. Prentice Hall, Englewood Cliffs, NY
34. Kania D (2002) An efficient algorithm for output coding in PAL-based CPLDs. *Int J Eng* 15(4):325–328

35. Liu Y, Peng Y, Wang B, Yao S, Liu Z (2017) Review on cyber-physical systems. *IEEE/CAA J Automat Sinica* 4(1):27–40
36. Marwedel P (2018) Embedded system design: embedded systems foundations of cyber-physical systems, and the internet of things, 3rd edn. Springer International Publishing
37. McCluskey E (1986) Logic design principles. Prentice Hall, Englewood Cliffs
38. De Micheli G (1994) Synthesis and optimization of digital circuits. McGraw-Hill
39. Papachristou C, Gambhir S (1982) A microsequencer architecture with firmware support for modular microprogramming. *ACM SIGMICRO Newslett* 13(4):105–113
40. Papachristou CA (1981) Hardware microcontrol schemes using PLAs. In: Proceedings of the 14th Annual Workshop on Microprogramming, MICRO 14. IEEE Press, Piscataway, NJ, USA, pp 3–16
41. Patterson D, Hennessy J (1998) Computer organization and design: the hardware/software interface. Morgan Kaufmann, San Moteo, CA
42. Salisbury A (1976) Microprogrammable computer architectures. Elsevier Science, NY
43. Sasao T (1999) Switching theory for logic synthesis. Kluwer Academic Publishers, Boston
44. Sasao T (2011) Memory-based logic synthesis, 1st edn. Springer Publishing Company, Incorporated
45. Sentowich E, Singh K, Lavango L, Moon C, Murgai R, Saldanha A, Savoj H, Stephan P, Bryton R, Sangiovanni-Vincentelli A (1992) SIS: a system for sequential circuit synthesis. Technical report, University of California, Berkely
46. Sklyarova I, Sklyarov V, Sudnitson A (2012) Design of FPGA-based circuits using hierarchical finite state machines. TUT Press, Tallinn
47. Sklyarov V, Sklyarova I, Barkalov A, Titarenko L (2014) Synthesis and optimization of FPGA-based systems, vol 294. Lecture Notes in Electrical Engineering. Springer-Verlag, Berlin
48. Tucker S (1967) Microprogram control for system/360. *IBM Syst J* 6(4):222–241
49. Villa T, Kam T, Brayton R, Sangiovanni-Vincentelli A (1998) A synthesis of finite state machines: logic optimization. Kluwer Academic Publishers, Boston
50. Wilkes M (1951) The best way to design an automatic calculating machine. In: Proceedings of Manchester University Computer Inaugural Conference
51. Wilkes M, Stringer J (1953) Microprogramming and the design of the control circuits in an electronic digital computer. *Proceed Cambridge Philosophical Soc* 49:230–238
52. Wolf W (2004) FPGA-based system design. Prentice Hall PTR, Upper Saddle River, NJ, USA
53. Zeidman B (2002) Designing with FPGAs and CPLDs. CMP Books

Chapter 2

VLSI-based Logic Synthesis



Abstract The chapter deals with analysis of methods of logic synthesis targeting VLSI. It starts with analysis of evolution of logic elements. The peculiarities of synthesis are shown for PROMs, PLAs, and PALs. Next, three classes of VLSI chips are considered, namely ASIC, CPLD and FPGA. The methods of structural decomposition are discussed. Main idea of these methods is reduced to diminishing the numbers of literals in systems of Boolean functions due to increasing the number of logic levels in FSM circuits. Methods of replacement of logical conditions and encoding of collections of microoperations are analysed. Next, these methods are applied for reducing hardware in ASIC, CPLD and FPGA-based FSM circuits. It is shown that memory blocks allow implementing systems of regular Boolean functions. The methods basing of classes of pseudoequivalent states are discussed for Moore FSM. There are shown VLSI-based structural diagrams of FSM circuits based on different methods of structural decomposition.

2.1 Evaluation of Logic Elements

Modern digital systems consist of thousands and millions of CMOS transistors. The abbreviation CMOS stands for complementary metal-oxide-semiconductor [11, 211, 215]. This technology is used in microprocessors, microcontrollers, FPGAs, CPLDs, ASICs, and other digital logic circuits. It was invented by Frank Wanlass in 1963 (Fairchild Semiconductor). A typical design style with CMOS is reduced to using pairs of p-type and n-type metal-semiconductor field-effect transistors [215].

Transistors are combined to create different integrated circuits (IC). The first IC was invented in 1958. The first operational IC was created on September 27, 1960, at Fairchild Semiconductor. It was based on invention of Jack Kilby (Texas Instruments), Kurt Lehovec (Sprague Electric Company) and Robert Noyce (Fairchild Semiconductor).

There is a classification of ICs based on their chip size [211]:

1. SSI (Small Scale Integration) ICs include from 3 to 30 gates per chip.
2. MSI (Medium Scale Integration) ICs include up to 300 gates per chip.
3. LSI (Large Scale Integration) ICs include up to 3000 gates per chip.

Fig. 2.1 Simplified architecture **a** and graphic symbol **b** of PROM

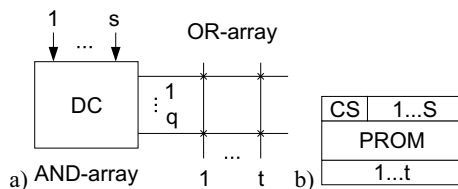
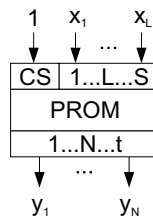


Fig. 2.2 Trivial implementation of SBF by PROM



4. VLSI ICs have more than 3000 gates per chip.

Our book deals mostly with two types of VLSI ICs: ASICs and field-programmable logic devices (FPLD) such as CPLD [99, 205] and FPGA [151, 152]. In the case of ASIC, a digital system is produced by a manufacturer and it cannot be changed by an end user. In the case of FPLD, an end user independently “programs” the functions of logic elements and their interconnections. So, an FPLD is a general purpose chip whose hardware could be configured by the end user to implement a particular project.

The programmable read-only memory (PROM) chips are the first representatives of FPLDs. They were manufactured by Harris Semiconductor in 1970 [151, 152]. They consist of a fixed array of AND gates (AND-array) and a programmable array of OR gates (OR-array). The AND-array implements an address decoder having s inputs and $q = 2^s$ outputs (Fig. 2.1a).

Each output of DC determines an unique memory cell. The content of OR-array is programmable. To show it, we use the sign “X”. So, the sign “X” corresponds to a programmable connection between the arrays of PROM.

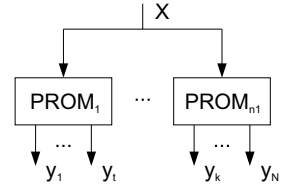
Let us denote the PROM (Fig. 2.1a) as PROM (s, t). Let us use the graphic symbol (Fig. 2.1b) to show such a chip on logic circuits. Let us discuss how to implement the circuit for a Boolean system $Y = F(X)$, where $Y = \{y_1, \dots, y_N\}$ and $X = \{x_1, \dots, x_L\}$, with PROMs [38].

Let the system $Y = F(X)$ be represented by its truth table [153]. There are H rows, L input columns and N output columns in the truth table. The following relation takes place:

$$H = 2^L. \quad (2.1)$$

Four different relations are possible among the values of L, N, s and t [38]. They lead to different amount of chips in the corresponding logic circuit. Let us discuss them:

Fig. 2.3 implementation of SBF with $N > t$



1. $s \geq L, t \geq N$. In this case, it is enough a single chip to implement the circuit (Fig. 2.2). The inputs of PROM are connected with logical variables $x_e \in X$. The functions $y_n \in Y$ are generated by the chip outputs. If there is “0” on the chip select input (CS), then $y_n = 0$ ($n \in 1, \dots, N$). If CS = 1, then outputs are determined by input assignment $\langle x_1, \dots, x_L \rangle$.
2. $s \geq L, t < N$. To implement the circuit, it is necessary to have n_1 chips, where

$$n_1 = \lceil N/t \rceil. \quad (2.2)$$

Address inputs of all PROMs are connected with variables $x_e \in X$. Only t different functions from N available are generated by each PROM (Fig. 2.3).

In fact, such a connection of SBF with $N > t$ results in a single PROM ($s, n_1 \cdot t$). Because of it, this approach is called “the expansion of PROM outputs” [22]. The value of k is determined as $t \cdot (n_1 - 1) + 1$.

3. $s < L, t \geq N$. In this case, it is necessary to have n_2 PROM (s, t) chips [22]:

$$n_2 = \lceil H/q \rceil. \quad (2.3)$$

It leads to three-level circuit (Fig. 2.4).

In this circuit, the $L - s$ leftmost bits of input assignment $\langle x_1, \dots, x_L \rangle$ form a set of variables X^1 , which are connected with inputs of decoder DC, having n_2 outputs. Outputs of the DC are connected with “enable” inputs of corresponding PROMs. Address inputs of all chips are connected with s rightmost bits of input assignment and these variables form a set X^2 . Partial functions Y^i are generated as outputs of microchip i and correspond to subtables of the truth table with rows from $q(i - 1)$ to qi . As it can be seen from Fig. 2.4, OR-gates are used to produce the final values of functions $y_n \in Y$. The approach considered above is rather theoretical, because this level of the circuit could be implemented using the three-stable outputs of PROM chips [204]. This approach is called “the expansion of PROM inputs” [22].

4. $s < L, t < N$. In this case, it is necessary to apply together the expansion of outputs and inputs. As a result, there are

$$n_3 = n_1 \cdot n_2 \quad (2.4)$$

chips in the final circuit.

Fig. 2.4 Implementation of SBF with $L > s$

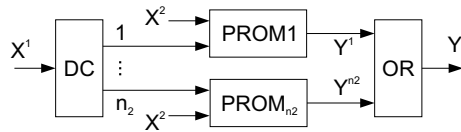
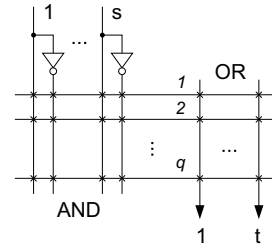


Fig. 2.5 Architecture of PLA



Thanks to regularity of their structure, PROM chips find wide application for implementation of tabular functions. Main drawback of the PROM is doubling of their capacity, if the number of inputs is incremented by 1. Besides, PROMs can not be used for implementation of the SBF satisfying the following conditions:

$$H_1 \ll H, \quad (2.5)$$

where H_1 is the number of input assignments, such that at least one of functions $y_n \in Y$ is equal to 1.

Programmable logic arrays (PLA) are perfect for implementing SBF satisfying 2.5. They were developed in 1970 by Texas Instruments [114, 121]. The peculiarity of PLA is a programmability of both planes AND and OR [13, 15]. It results in better flexibility than the one provided by PROM.

A PLA chip has s inputs, t outputs and q product terms ($q \ll 2^s$). Its structure is shown in Fig. 2.5.

A PROM can implement only perfect sum-of-products (SOP) of SBF. But a PLA can implement the minimal SOP [153]. PLAs are obliged to this property due to programmability of both AND and OR arrays. But this property has a negative effect, too. Namely, the programmability of AND-array leads to reduction of both performance and value of q in comparison with a PROM-based implementation [114].

Let the symbol PLA (s, t, q) stand for a chip having s inputs, t outputs and q terms. Let it be used to implement a system $Y(L, N, H_1)$ having L arguments, N functions and H_1 terms of SOP.

As it is for PROM-targeting design, the number of chips in the final circuit depends on a combination of parameters s, t, q and L, N, H_1 [16, 191]. If $s \geq L$, then four rather simple approaches are possible [16].

1. $t \geq N, q \geq H_1$. It is enough a single PLA chip to implement the circuit for Y . It is similar to the one shown in Fig. 2.2.

Fig. 2.6 Implementation of SBF with expansion of PLA terms

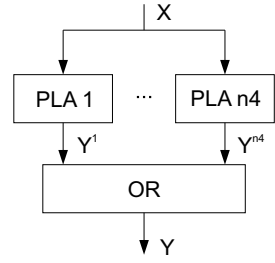
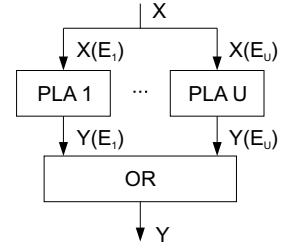


Fig. 2.7 Single-level implementation of SBF with PLA



2. $t < N, q \geq H_1$. It leads to expansion of PLA outputs [22]. The number of chips is determined by 2.2. The circuit is similar to the one shown in Fig. 2.3.
3. $t \geq N, q < H_1$. It leads to necessity of “the expansion of PLA terms” [22]. There are n_4 chips in the circuits:

$$n_4 = \lceil H_1/q \rceil. \quad (2.6)$$

As follows from Fig. 2.6, the eponymous outputs of chips are connected through the OR gates. Unlike the circuit (Fig. 2.4), there is no DC in the case of PLA-based design.

4. $t \geq N, q \geq H_1$. Now, both above mentioned methods should be used. To minimize the hardware, some sophisticated methods [12, 191] should be used.

More complex design methods should be used if

$$s < L. \quad (2.7)$$

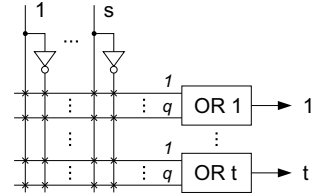
If 2.7 takes place, then the system Y can be represented by a single-level circuit if

$$F_{max} \leq L. \quad (2.8)$$

In 2.8, F_{max} is a maximal amount of literals [12] in terms of SBF Y . The corresponding solution is shown in Fig. 2.7.

During the design of logical circuit, a partition Π_F of the set of terms F should be found, in which $|F| = H_1$, and the number of blocks U attains minimum [12]. Let $X(E_u)$ be the set of logical conditions, which form the terms of set $E_u \in \Pi_F = \{E_1, \dots, E_U\}$, and $Y(E_u)$ be a set of functions depending on terms

Fig. 2.8 Architecture of PAL



$E_u \in \Pi_F$. Partition Π_F should satisfy the following conditions:

$$\begin{aligned} |X(E_u)| &\leq s; \\ |Y(E_u)| &\leq t, \quad (u = 1, \dots, U); \\ |E_u| &\leq q; \\ U &\rightarrow \min. \end{aligned} \quad (2.9)$$

In the 1980s, hundreds and thousands of books and articles were devoted to this problem [13, 15]. The focus was on minimizing the value of U [80, 191]. Even more complex problems arose if the condition 2.8 was violated. It leads to multi-level circuits [22, 191] implementing SBFs.

Let $Q(y_n)$ be a number of product terms in a function $y_n \in Y$. The experience of logical design has shown that the following condition takes place for the most practical digital devices:

$$Q(y_n) \leq 16. \quad (2.10)$$

Due to it, there is a redundancy in PLA where each term could be connected with any output of a chip.

To overcome this drawback, programmable array logic (PAL) chips were introduced in 1978 by Monolithic Memories [81]. The PAL chips target SBFs satisfying 2.10. They include programmable AND-array and t fixed OR arrays Fig. 2.8.

Design methods used in case of PAL are oriented on minimizing $Q(y_n)$ up to some fixed value q , determining the number of AND-arrays connected with a single OR-array [163]. To broaden the application area of these chips, additional elements such as flip-flops, logic gates and multiplexers were added to each PAL output. This new output cell was called Macrocell. It has a feedback path from the output of the cell to the AND-array. All connections inside the macrocell were also programmable and therefore PAL is more flexible than other solutions. These PAL cells were called generic array logic (GAL) and were introduced by Lattice in 1985 [145]. Let us point out, that GAL chips are still manufactured in standalone packages [145] by Lattice and Atmel [9, 145].

The discussed chips belong to the class of Simple Programmable Logic Devices (SPLD) [1]. They have at most 40 inputs/outputs and are equivalent to at most 500 2-input NAND gates [1]. Now, they are practically replaced by CPLD and FPGA chips [115]. These chips were offered to market almost simultaneously.

Fig. 2.9 Simplified architecture of CPLD

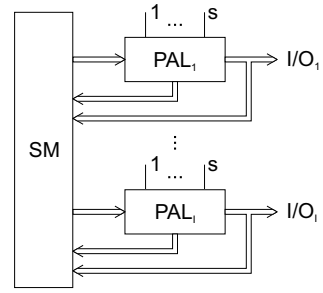
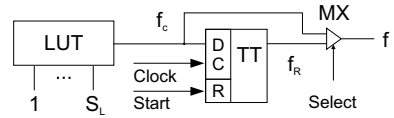


Fig. 2.10 Architecture of PLB



The first CPLD were Mega PAL of MMI [145]. However, the CPLDs proposed by Altera in 1988 are considered classical [1]. It was the CPLD MAX 5000. The typical CPLD includes a switch matrix (SM) and I PAL-based macrocells Fig. 2.9.

In CPLD (Fig. 2.9), each macrocell PAL_i ($i \in \{1, \dots, I\}$) is connected with s fixed chip inputs and a single programmable input/output I/O_i . Their outputs are connected with SM. The typical macrocells include flip-flops and multiplexors.

Now, CPLD are produced by several companies such as Altera (Intel), Xilinx, Cypress, Atmel, Lattice [6, 9, 145, 222]. Modern CPLD mostly use PAL-based macrocells, but PLA-based blocks are known [222]. For example, CPLDs of CXPLA3 family include from 32 to 512 PLA-macrocells. They can replace from 750 to 12000 equivalent system gates [222].

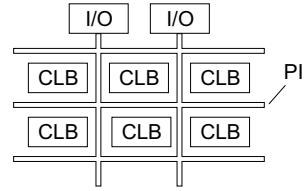
In 1985, first FPGA chip was proposed by Xilinx [222]. These chips target high-performance and large-size digital devices. In FPGA, it is possible to program the content of each programmable logic block (PLB) and interconnections among PLBs [144].

Modern FPGAs include a lot of look-up table (LUT) elements [151, 152, 210]. LUTs are constructed using static random access memory (SRAM) cells and address multiplexors (MX) to choose a particular bit [210]. Each LUT has S_L inputs and $q = 2^{S_L}$ SRAM cells with a single-bit output. This output could be registered using a flip-flop (Fig. 2.10).

A LUT of PLB implements an arbitrary Boolean function f_c depended on S_L arguments. If $\text{Clock}=1$, then $f_R = f_c$, where f_R is a registered value of f_c . The signal "Select" chooses either f_c or f_r as the output function f . So, a LUT can implement any Boolean function. Existence of flip-flop allows creating distributed registers [194].

In this book, we do not discuss detailed information connected with FPGAs. It would be found in different books [74, 151, 152, 194] and sites of manufactures [6, 222]. We discuss only features important for implementing FSM circuits.

Fig. 2.11 Simplified architecture of FPGA



Modern FPGA have a hierarchical architecture. Up to 4 PLBs form a slice. Up to 4 slices form a configurable logic block (CLB) using programmable interconnections (PI). Each CLB can be connected with input/output block. So, the modern FPGA chips have an “island-style” architecture [210] shown in Fig. 2.11.

A CLB can be an embedded memory block (EMB) with ability of changing the number of address inputs, S_A , and cell outputs, t_F [144]. Modern FPGA have a lot of EMBs [210] with the following configurations: $32k \times 1$, $16k \times 2$, $8k \times 4$, $4k \times 8$, $2k \times 16$, $1k \times 32$, 512×64 bits. Those configurations determine the following pairs $\langle S_A, t_F \rangle : \langle 15, 1 \rangle, \langle 14, 2 \rangle, \dots, \langle 9, 64 \rangle$. Using EMB, allows improvement of main characteristics of FSMs such as the hardware amount, consumed power and performance [83, 84, 95].

It is very important to reduce the chip area occupied by FSM circuit [165, 181]. The methods of hardware reduction depend on peculiarities of logic elements used to implement logic circuits [67, 194]. In this book, we discuss this problem for ASIC-, CPLD- and FPGA-based design.

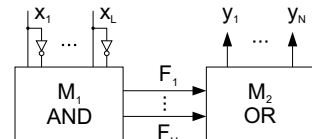
2.2 Logic Synthesis with ASICs

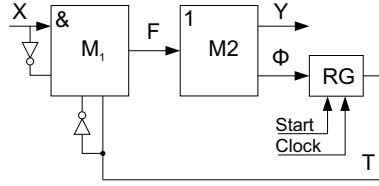
We do not discuss classification of ASICs and different technological aspects. This information can be found, for example, in [195]. We just discuss how to implement FSM circuits using custom matrices [153]. The custom matrices are based on the conception of distributed logic [165].

In this case, it is enough two matrices to implement an SBF Y depended on L arguments and having H product terms (Fig. 2.12).

The matrix M_1 has $2L$ input wires and H output wires. If an argument x_e (or its complement) enters a term F_h , then there is a transistor connecting the wire x_e (or $\bar{x}_e$) with the wire F_h . If a term F_h belongs to SOP of y_n , then there is a transistor connecting wires F_h and y_n . Let us point out that in reality both M_1 and

Fig. 2.12 Matrix implementation of SBF



**Fig. 2.13** Trivial matrix circuit of Mealy FSM**Table 2.1** Structure table of Mealy FSM S_4

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
a_1	00	a_2	01	1	$y_1 y_2$	D_2	1
a_2	01	a_3	10	x_1	y_3	D_1	2
		a_3	10	$\bar{x}_1 x_2$	$y_2 y_4$	D_1	3
		a_4	11	$\bar{x}_1 \bar{x}_2$	$y_1 y_3 y_5$	$D_1 D_2$	4
a_3	10	a_1	00	x_3	$y_1 y_2$	–	5
		a_3	10	$\bar{x}_3$	y_4	D_1	6
a_4	11	a_1	00	x_4	y_5	–	7
		a_4	11	$\bar{x}_4$	$y_1 y_3 y_5$	$D_1 D_2$	8

M_2 are organized as NAND arrays [153]. However, we will call them AND and OR matrices, respectively.

Let a Mealy FSM be represented by systems 1.4 and 1.7. Its trivial matrix circuit [12] includes two matrices and a state register (Fig. 2.13).

There are $2(L + R)$ inputs and H outputs for the matrix M_1 . Each output corresponds to a product term F_h represented as 1.10. There are H inputs and $N + R$ outputs for the matrix M_2 .

Let us find the total area (S_T) of the chip occupied by the matrix circuit shown in Fig. 2.13. It can be found by summation of areas $S(M_1)$ and $S(M_2)$ [12]. Here, $S(M_i)$ stands for the area of the matrix M_i . The area is measured in so called conventional units of area [153]. The following areas can be found for Fig. 2.13:

$$S(M_1) = 2(L + R) \cdot H; \quad (2.11)$$

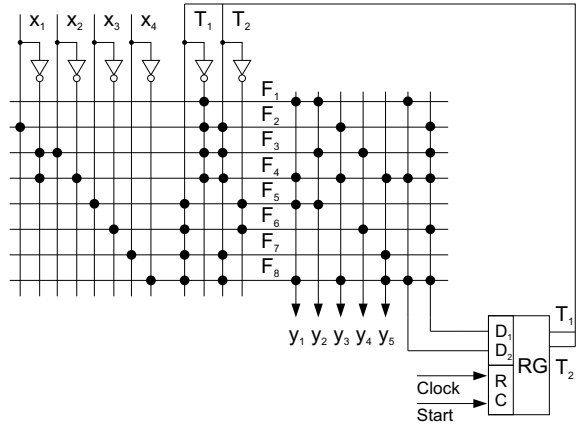
$$S(M_2) = H(N + R); \quad (2.12)$$

$$S_T = (3R + 2L + N) \cdot H. \quad (2.13)$$

The estimations 2.11–2.13 are rather theoretical. They do not include technological coefficients for transistors, wires and spaces among these components. But they allow to compare different implementations of FSM circuits [22, 153].

Let us consider the matrix implementation of Mealy FSM S_4 (Table 2.1).

Fig. 2.14 Trivial matrix implementation of Mealy FSM S_4



There is $H = 8$ for S_4 . So, it has the set $F = \{F_1, \dots, F_8\}$ where $F_1 = \bar{T}_1 \bar{T}_2$, $F_2 = \bar{T}_1 T_2 x_1, \dots, F_8 = T_1 T_2 \bar{x}_y$. The following SBF represents systems 1.4 and 1.6:

$$\begin{aligned} D_1 &= F_2 \vee F_3 \vee F_4 \vee F_6 \vee F_8; & y_1 &= F_1 \vee F_4 \vee F_5 \vee F_8; & y_3 &= F_2 \vee F_4 \vee F_8; \\ D_2 &= F_1 \vee F_4 \vee F_8; & y_2 &= F_1 \vee F_3 \vee F_5; & y_4 &= F_3 \vee F_6; \\ & & & & y_5 &= F_4 \vee F_7 \vee F_8; \end{aligned} \quad (2.14)$$

Using system 2.14 and equations for terms F_h , we can get the trivial matrix implementation of S_4 (Fig. 2.14).

Using 2.11–2.13 gives $S(M_1) = 2(4 + 2) \cdot 8 = 96$, $S(M_2) = 8(5 + 2) = 56$ and $S_T = 152$. The circuit has two levels of matrices among inputs $x_e \in X$ and outputs $y_n \in Y$, $D_r \in \Phi$. So, it is the fastest among all possible solutions for FSM S_4 .

Let us point out that this approach leads to very redundant logic circuits. For example, there are the following characteristics of average practical FSMs [12]: $H \approx 2000$, $R \approx 8$, $N \approx 50$, $L \approx 30$. It gives $S_T \approx 268000$. The value of S_T determines the number of possible interconnections in matrices M_1 and M_2 . Obviously, only some part of possible interconnections is used for a particular FSM. Let a term F_h include L_h letters from input alphabet X , and let N_h microoperations and R_h input memory functions be produced for each FSM transition. Now, we can find real number of interconnections for trivial FSM circuit. To find it, we should replace L and N by L_h and N_h in 2.11–2.13, and R by R_h in 2.12–2.13. It gives us the value

$$S_{TR} = (2L_h + N_h + 2R + R_h) \cdot H. \quad (2.15)$$

The efficiency of using the matrix area, E_T , could be determined as

$$E_T = S_{TR}/S_T. \quad (2.16)$$

Table 2.2 Table of RLC for Mealy FSM S_4

a_m	p_1	p_2	$K(a_m)$
a_1	–	–	00
a_2	x_1	x_2	01
a_3	x_3	–	10
a_4	–	x_4	11

If there is $L_h = N_h = 6$ and $R_h = 4$ for Mealy FSM with average complexity, then $E_T \approx 0, 12$. It means that around 88% of area is wasted in the case of trivial matrix circuit. Because of it, such an approach does not used in practical projects [12].

Obviously, the combining M_1 and M_2 gives a PLA having $s = L + R, t = N + R$, and $q = H$. So, the circuit from Fig. 2.13 is just a P Mealy FSM [16, 191]. To optimize E_T , let us use two methods of structural decomposition [42]: (1) the replacement of logical conditions and (2) the binary encoding of collections of microoperations (CMO).

Let $X(a_m)$ be a set of LCs determining transitions from $a_m \in A$. Let it be $G_m = |X(a_m)|$ and

$$G = \max(G_1, \dots, G_M). \quad (2.17)$$

Let us construct a set $P = \{p_1, \dots, p_G\}$ and replace the LCs $x_e \in X$ by elements of P . It is the case of the replacement of logical conditions (RLC) [12].

As follows from Table 2.1, there are the sets $X(a_1) = 0$, $X(a_2) = \{x_1, x_2\}$, $X(a_3) = \{x_3\}$, and $X(a_4) = \{x_4\}$. It gives $G = 2$ and $P = \{p_1, p_2\}$.

Let us construct table RLC having G columns marked by variables $p_g \in P$, and M rows marked by states $a_m \in A$. If a variable p_g replaces in a state a_m a logical condition x_l , then the symbol x_l is written on the intersection of the row a_m and column p_g of the table. To minimize the hardware amount of logic circuit used for the logical condition replacement, the distribution of logical conditions is executed in such a manner that each variable $x_l \in X$ is always placed in the same column of the table for all states of FSM.

It is Table 2.2 in the discussed case. We add the column $K(a_m)$ in this table to get equations for functions $p_g \in P$.

Let us find the system

$$P = P(T, X). \quad (2.18)$$

Each equation of 2.18 is represented as

$$p_g = \bigvee_{m=1}^M C_{ml} A_m x_l \quad (g \in \{1, \dots, G\}), \quad (2.19)$$

where C_{ml} is a Boolean variable, which is equal to 1, if the variable p_g replaces the logical condition x_l for the state a_m . In the discussed case, it is the following system:

Fig. 2.15 Two-level circuit of RLC for S_4

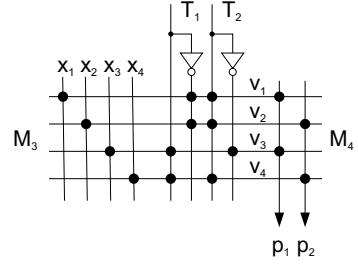
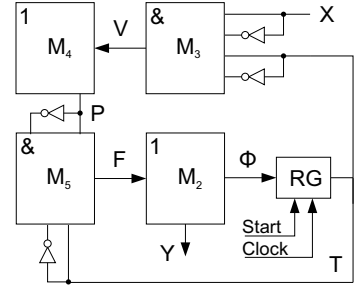


Fig. 2.16 Matrix circuit of MP Mealy FSM



$$\begin{aligned} p_1 &= \bar{T}_1 T_2 x_1 \vee T_1 \bar{T}_2 x_3 = v_1 \vee v_2; \\ p_2 &= \bar{T}_1 T_2 x_2 \vee T_1 T_2 x_4 = v_3 \vee v_4. \end{aligned} \quad (2.20)$$

The system 2.20 can be implemented as a two-level circuit (Fig. 2.15).

In Fig. 2.15, a matrix M_3 implements terms $v_i \in V$ where $V = \{v_1, \dots, v_4\}$. A matrix M_4 implements functions $p_g \in P$ represented by 2.20.

Combining four matrices (Fig. 2.16) gives a matrix circuit of MP Mealy FSM. The symbol “M” stands for the circuit of RLC [20].

The matrix M_5 implements terms of the system

$$F = F(T, P). \quad (2.21)$$

Each term $F_h \in F$ is determined as

$$F_h = A_m P_h \quad (h = \overline{1, h}), \quad (2.22)$$

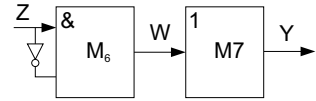
where P_h is a conjunction of variables $p_g \in P$ replacing the conjunction X_h in the term 1.10. So, matrices M_5 and M_2 implement the systems

$$\Phi = \Phi(T, P); \quad (2.23)$$

$$Y = Y(T, P). \quad (2.24)$$

Table 2.3 Structure table of Mealy FSM S_4

a_m	$K(a_m)$	a_s	$K(a_s)$	P_h	Y_h	Φ_h	h
a_1	00	a_2	01	1	$y_1 y_2$	D_2	1
a_2	01	a_3	10	p_1	y_3	D_1	2
		a_3	10	$\bar{p}_1 p_2$	$y_2 y_4$	D_1	3
		a_4	11	$\bar{p}_1 \bar{p}_2$	$y_1 y_3 y_5$	$D_1 D_2$	4
a_3	10	a_1	00	p_1	$y_1 y_2$	–	5
		a_3	10	$\bar{p}_1$	y_4	D_1	6
a_4	11	a_1	00	p_2	y_5	–	7
		a_4	11	$\bar{p}_2$	$y_1 y_3 y_5$	$D_1 D_2$	8

Fig. 2.17 Matrix circuit for system 2.25

To construct systems 2.23–2.24, it is necessary to construct the transformed DST of MP Mealy FSM [20, 39]. It includes the same columns as a DST, but the column X_h is replaced by P_h . It is Table 2.3 in the discussed case.

Using Table 2.3 gives the terms 2.22: $F_1 = \bar{T}_1 \bar{T}_2$, $F_2 = \bar{T}_1 T_2 p_1$, $\dots$, $F_8 = T_1 T_2 \bar{p}_2$. The SBFs 2.23–2.24 are constructed in the trivial way.

The RLC targets optimizing the matrix M_1 (Fig. 2.13) which is replaced by three matrices. To optimize the matrix M_2 , the encoding of CMOs [12] can be used. It leads to PY Mealy FSMs [20].

Let it be Q different CMOs $Y_q \subseteq Y$. Let us encode them by codes $K(Y_q)$ having R_Q bits. The value of R_Q is determined by 1.32. Let us use the elements of a set $Z = \{z_1, \dots, z_{R_Q}\}$ to encode the CMOs.

As a result, the system 1.7 is represented as

$$Y = Y(Z). \quad (2.25)$$

To construct Boolean functions $y_n \in Y$, it is necessary to find the system

$$Z = Z(T, X). \quad (2.26)$$

The variables $z_r \in Z$ forms terms $W_q \in W = \{W_1, \dots, W_Q\}$ of SOPs $y_n \in Y$.

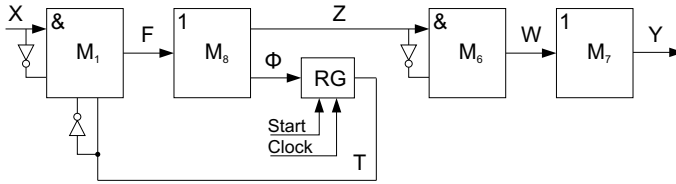
There are two matrices in the circuit implementing 2.25. The matrix M_6 implements terms of 2.25, the matrix M_7 implements functions 2.25. The circuit is shown in Fig. 2.17.

As follows from Table 2.1, there is $Q=6$, where $Y_1 = \{y_1, y_2\}$, $Y_2 = \{y_3\}$, $Y_3 = \{y_2, y_4\}$, $Y_4 = \{y_1, y_3, y_5\}$, $Y_5 = \{y_6\}$, and $Y_6 = \{y_5\}$. It gives $R_Q=3$, $Z=\{z_1, z_2, z_3\}$. Let us encode the CMOs in the trivial way: $K(Y_1) = 000, \dots, K(Y_6) = 101$.

Using Table 2.1 gives the SBF

Table 2.4 Transformed structure table of PY Mealy FSM S_4

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Z_h	Φ_h	h
a_1	00	a_2	01	1	—	D_2	1
a_2	01	a_3	10	x_1	z_3	D_1	2
		a_3	10	$\bar{x}_1 x_2$	z_3	D_1	3
		a_4	11	$\bar{x}_1 \bar{x}_2$	$z_2 z_3$	$D_1 D_2$	4
a_3	10	a_1	00	x_3	—	—	5
		a_3	10	$\bar{x}_3$	z_1	D_1	6
a_4	11	a_1	00	x_4	$z_1 z_3$	—	7
		a_4	11	$\bar{x}_4$	$z_2 z_3$	$D_1 D_2$	8

**Fig. 2.18** Matrix circuit of PY Mealy FSM

$$\begin{aligned} y_1 &= Y_1 \vee Y_4, & y_2 &= Y_1 \vee Y_3, & y_3 &= Y_2 \vee Y_4, \\ y_4 &= Y_3 \vee Y_5, & y_5 &= Y_4 \vee Y_6. \end{aligned} \quad (2.27)$$

To get the system 2.26, it is necessary to replace the column Y_h of initial DST by the column Z_h . If there is 1 in the r -th bit of $K(Y_q)$ from the row h of the DST, then it should be z_r in this row of transformed DST. In our case, it is Table 2.4.

As follows from Table 2.4, there is

$$z_1 = F_6 \vee F_7; \quad z_2 = F_3 \vee F_4 \vee F_8; \quad z_3 = F_2 \vee F_4 \vee F_7 \vee F_8. \quad (2.28)$$

The terms $W_q \in W$ are determined in the trivial way: $W_1 = \bar{z}_1 \bar{z}_2 \bar{z}_3$, $W_2 = \bar{z}_1 \bar{z}_2 \bar{z}_3, \dots$, $W_6 = z_1 \bar{z}_2 z_3$.

There are 4 matrices in the matrix circuit of PY Mealy FSM (Fig. 2.18). The matrix M_2 of P FSM is represented by matrices M_6 – M_8 .

It is possible to optimize the total area of matrices M_6 – M_7 . To do it, it is necessary to encode the CMOs in the optimal way [25, 26]. For example, the algorithm [2] can be used for optimal encoding of CMOs.

Using [2] gives the codes $K(Y_q)$ represented by the Karnaugh map (Fig. 2.19).

Using the codes (Fig. 2.19) allows transforming 2.28 into the system $y_1 = \bar{z}_1 \bar{z}_2$, $y_2 = \bar{z}_1 z_3$, $y_3 = \bar{z}_1 \bar{z}_3$, $y_4 = z_2 z_3$, $y_5 = \bar{z}_2 \bar{z}_3$. This system is implemented by the matrix circuit shown in Fig. 2.20.

As follows from Fig. 2.20, there is no matrix M_7 in the final circuit. There is $S(M_6) + S(M_7) = 66$ for the circuit based on trivial encoding of CMOs. There is

Fig. 2.19 Optimal codes of collections of microoperations for PY Mealy FSM S_4

z_1	$z_2 z_3$			
	00	01	11	10
0	Y_4	Y_1	Y_3	Y_2
1	Y_6	*	Y_5	*

Fig. 2.20 Matrix realization of microoperations for PY Mealy FSM S_4

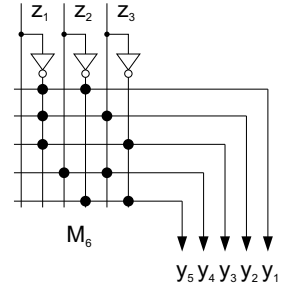
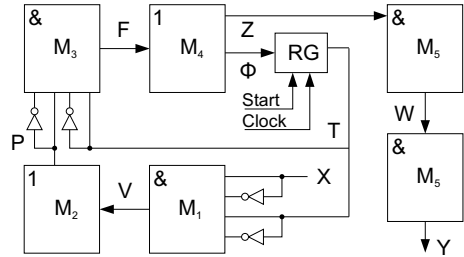


Fig. 2.21 Matrix circuit of MPY Mealy FSM



$S(M_6) = 5 \cdot 5 = 25$ for Fig. 2.20. So, the application of optimal encoding of CMOs leads to the circuit having 2,6 times less hardware amount.

Both discussed methods take their beginning in MCUs proposed by M. Wilices [216, 217]. Now they are used for optimizing ASIC-, CPLD- and FPGA-based circuits of FSMs [38–40].

The joint application of RLC and encoding of CMOs leads to MPY Mealy FSM (Fig. 2.21). Its circuit has 6 matrices. Their functions are clear from the previous text.

There are the following steps in the design method of MPY Mealy FSMs [39, 191]:

1. Construction of marked GSA.
2. Construction of transition table of Mealy FSM.
3. Construction of table of logical condition replacement.
4. State assignment targeted in reduction of hardware for the block RLC.
5. Optimal encoding of collections of microoperations.
6. Construction of transformed structure table of Mealy FSM.
7. Construction of systems for realization of matrices M_1 – M_6 .
8. Design of FSM logic circuit using functions obtained during the step 7.

Table 2.5 Transformed structure table of MPY Mealy FSM S_4

a_m	$K(a_m)$	a_s	$K(a_s)$	P_h	Z_h	Φ_h	h
a_1	00	a_2	01	1	z_3	D_2	1
a_2	01	a_3	10	p_1	z_2	D_1	2
		a_3	10	$\bar{p}_1 p_2$	$z_2 z_3$	D_1	3
		a_4	11	$\bar{p}_1 \bar{p}_2$	–	$D_1 D_2$	4
a_3	10	a_1	00	p_1	z_3	–	5
		a_3	10	$\bar{p}_1$	$z_1 z_2 z_3$	D_1	6
a_4	11	a_1	00	p_2	z_1	–	7
		a_4	11	$\bar{p}_2$	–	$D_1 D_2$	8

In the discussed case, Table 2.5 is the transformed table of MPY Mealy FSM S_4 . It uses codes $K(Y_q)$ taken from Fig. 2.19.

It is really easy to design the logic circuit (matrix realization) of MPY Mealy FSM S_4 , because all its components were already designed. Obviously, the discussed method can be adapted to design either Mealy MP or PY FSM. The results of many researches show that the application of the RLC and optimal encoding of collections of microoperations results in the significant hardware amount reducing, especially for complex FSM having more than 2000 transitions. This approach has one serious drawback, namely, decrease in FSM performance in comparison with the primitive matrix realization of Mealy FSM. It is an effect of the cycle time increase due to increase in the number of levels in the resulting FSM realization (in comparison with the trivial matrix realization of Mealy FSM).

Let us briefly discuss matrix implementation of Moore FSMs. The trivial matrix circuit of P Moore FSM is the same as the one shown in Fig. 2.13. It has the same negative and positive features as P Mealy FSM has. Let us discuss a Moore FSM S_5 represented by its structure table (Table 2.6).

There is a peculiarity of trivial matrix circuit of Moore FSM. It is connected with the dependence 1.8. The outputs $y_n \in Y$ depend on states $a_m \in A$. But there are only terms 1.10 entering the matrix M_2 (Fig. 2.13).

Let us analyse the terms F_4 – F_6 from Table 2.6. Let us find the disjunction $F_4 \vee F_5 \vee F_6$. It is equal to $\bar{T}_1 \bar{T}_2 \bar{T}_3 (x_1 (x_3 \vee \bar{x}_3) \vee \bar{x}_1) = \bar{T}_1 \bar{T}_2 T_3 = A_2$. Let $F(a_m)$ be a set of a terms corresponding to transitions from a state $a_m \in A$. So, to get A_m , the functions $F_h \in F(a_m)$ should be connected with outputs $y_n \in Y$ generated in the state $a_m \in A$. Using this approach gives the circuit shown in Fig. 2.22.

To optimize the value of E_T , we can take into account the dependence 1.8. It allows constructing the circuit of PY Moore FSM (Fig. 2.23)

In PY Moore FSM, the matrix M_1 implements functions $F_h \in F$, the matrix M_2 input memory functions 1.4. The matrix M_3 implements the conjunctions $A_m \in A_0$. The set $A_0 \subseteq A$ includes states $a_m \in A$ where some MOs are generated. The matrix M_4 implements outputs $y_n \in Y$ as disjunctions of corresponding terms $A_m \in A_0$. There are different optimization methods based on existence of pseudoequivalent

Table 2.6 Structure table of Moore FSM S_5

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Φ_h	h
$a_1 (-)$	000	a_2	001	x_1	D_3	1
		a_3	010	$\bar{x}_1x_2$	D_2	2
		a_4	011	$\bar{x}_1\bar{x}_2$	D_2D_3	3
$a_2 (y_1y_3y_4y_5y_6)$	001	a_2	001	x_1x_3	D_3	4
		a_1	000	$x_1\bar{x}_3$	—	5
		a_5	100	$\bar{x}_1$	D_1	6
$a_3 (y_1y_2y_4y_5)$	010	a_2	001	x_1x_3	D_3	7
		a_1	000	$x_1\bar{x}_3$	—	8
		a_5	100	$\bar{x}_1$	D_1	9
$a_4 (y_2y_6)$	*011	a_1	000	x_3x_4	—	10
		a_4	011	$x_3\bar{x}_4$	D_2D_3	11
		a_4	011	$\bar{x}_3$	D_2D_3	12
$a_5 (y_1y_3y_4y_6)$	100	a_1	000	x_3x_4	—	13
		a_4	011	$x_3\bar{x}_4$	D_2D_3	14
		a_4	011	$\bar{x}_3$	D_2D_3	15

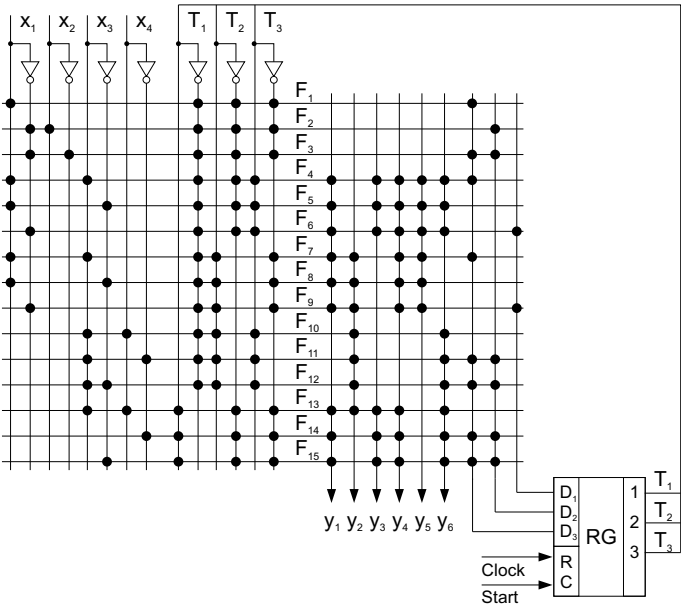


Fig. 2.22 Trivial matrix circuit of P Moore FSM S_5

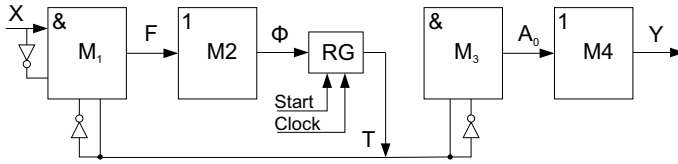
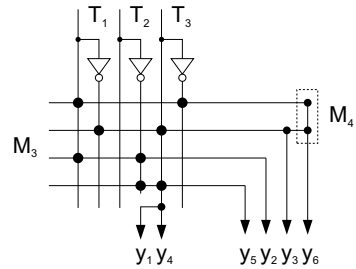


Fig. 2.23 Matrix realization of PY Moore FSM

Fig. 2.24 Refine states codes of Moore FSM S_5

		$T_2 T_3$			
T_1	0	00	01	11	10
		a_1	a_2	a_5	*
1		a_4	a_3	*	*

Fig. 2.25 Matrix implementation of MOs for S_5



states (PES) of Moore FSM [21, 37, 48–50]. We will discuss them in Chap. 3. Now, let us show that it is possible to encode the states $a_m \in A$ in a way optimizing the total area of matrices M_3 and M_4 . This style is called a refined state encoding [192].

Let us encode states of S_5 as it show in the Karnaugh map (Fig. 2.24).

As follows from DST of S_5 (Table 2.6), there is the following system of MOs: $y_1 = y_4 = A_2 \vee A_3 \vee A_5$, $y_2 = A_3 \vee A_4$, $y_3 = A_2 \vee A_5$, $y_5 = A_2 \vee A_3$, $y_6 = A_2 \vee A_4 \vee A_5$. Taking into account the codes from Fig. 2.24, the system of microoperations can be represented as: $y_1 = y_4 = T_3$, $y_2 = T_1 \bar{T}_2$, $y_3 = \bar{T}_1 T_3$, $y_5 = \bar{T}_2 T_3$, $y_6 = \bar{T}_1 T_3 \vee T_1 \bar{T}_3 = y_3 \vee T_1 \bar{T}_3$. This systems is implemented using two matrices (Fig. 2.25).

Let us estimate the economy in chip area due to the refined state, assignment. There is $S(M_3) + S(M_4) = 2R \cdot |A_0| + |A_0| \cdot N = 6 \cdot 4 + 4 \cdot 5 = 44$. After the refined state encoding, there is $S(M_3) + S(M_4) = 5 \cdot 4 + 2 \cdot 1 = 22$. So, the refined state encoding results in 50% of chip areas reducing for S_5 .

2.3 Logic Synthesis with CPLDs

In this book, we do not discuss technical characteristics of different CPLD families. They could be found in technical documentation of manufactures [6, 96, 145, 222]. We discuss CPLDs from logic point of view, as tools for implementing FSM cir-

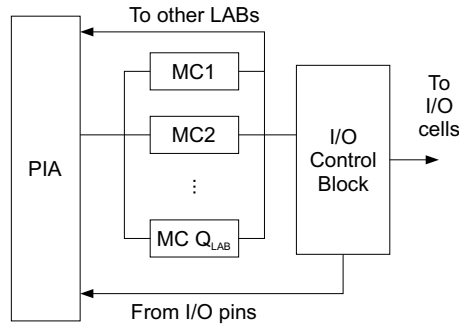


Fig. 2.26 Architecture of LAB

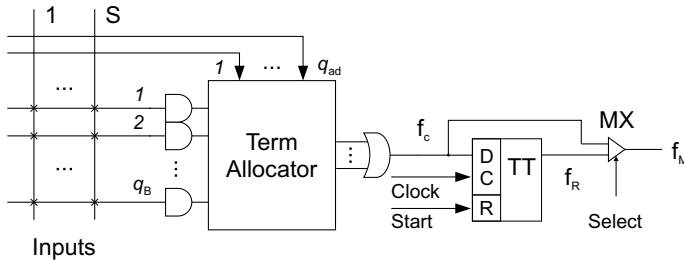


Fig. 2.27 Architecture of MC

cuits. The basic block of CPLD is a macrocell (MC). As a rule, there are PAL-based macrocells and PLA-based macrocells [6, 222]. First of them are characteristic for CPLDs of Intel (Altera). The second are typical for Xilinx. Also, there are heterogeneous CPLD containing PAL-based MCs and configurable RAM. Such CPLDs were produced by Cypress semiconductor corporation [96]. By analogy with embedded RAMs of FPGA chips [222], let us call these memory blocks as embedded memory blocks (EMB). Let us start from PAL-based CPLDs.

Macrocells are connected together to organize a logic array block (LAB). A LAB includes Q_{LAB} MCs having common inputs. Each LAB is connected with other blocks through a programmable interconnect array (PIA) as shown in Fig. 2.26. The PIA can connect any LAB (or output) with other LABs. Using PIA allows connecting I/O with LABs.

Each MC includes q programmable product terms which are a part of AND plane. These terms are connected with an OR-gate and a programmable flip-flop (it could be configured as D, JK, T or RS). Each OR-gate can have q_B basic and q_{ad} additional inputs. They are provided by a term allocator (Fig. 2.27).

As follows from Fig. 2.27, PAL-based CPLDs have all means for implementing circuits of FSMs. Flip-flops can be used to implement a distributed state register (or state counter). Because of it, CPLDs are used for implementing control units such as graphics controllers, LAN controllers, UARTs, cache control circuits and other [88].

Fig. 2.28 Organization of an FB

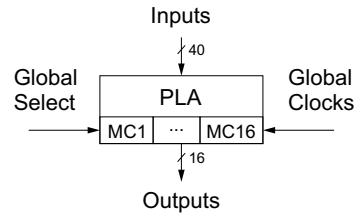
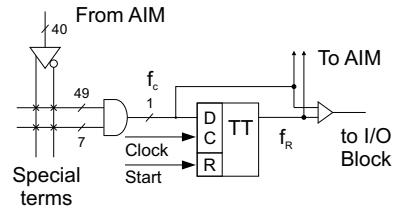


Fig. 2.29 Architecture of CoolRunner CPLD MC



Xilinx Inc. produces CPLD CoolRunner-II Family [222]. They are based on PLA-type macrocells [222]. The simple device (XC2C32A) has 32 macrocells and its operating frequency is equal to 323 MHz. The most complex device (XC2C512) has 512 MCs and its operating frequency is equal to 179 MHz [222]. The macrocells are combined into a function block (FB). Due to PLA-based architecture, all terms of FB could be shared among any of the FB's macrocells. The FBs are connected through advanced interconnect matrix (AIM).

There is an architecture of an FB shown in Fig. 2.28. The simplified architecture of an MC is shown in Fig. 2.29. A PLA of FB has $s = 40$ inputs $t = 16$ outputs and $q = 49$ product terms. It obtains data from AIM and can produce either combinational or registered outputs (or both). These outputs can be used as a feed-back to AIM and as data for I/O Blocks.

There is a programmable flip-flop inside each MC. It can be organized as either D or T flip-flop. In this book, we discuss FSMs having RGs based on D flip-flops. It is the most common case [15, 80, 191]. The state RG (or CT) is distributed among MCs of a chip.

A Delta39K ISR CPLD Family by Cypress Semiconductor Corporation [96] is a very good example of heterogeneous CPLDs. There are 5 devices in this family. Their characteristics are shown in Table 2.7.

1. Upper limit of typical gates is calculated by assuming only 10% of the channel memory is used.
2. Standby $I_C C$ values are with PLL not utilized, no output load and stable inputs.

This architecture is based on Logic Block Clusters (LBC) connected by horizontal and vertical routing channels. Each LBC includes 8 logic blocks, two cluster memory blocks and a single channel memory block. The memory blocks are configured and can be cascaded in width and depth.

Table 2.7 Delta39K™ ISR CPLD Family Members

Device	Typical Gates ¹ (K)	Macrocells	Cluster memory (Kbits)	Channel memory (Kbits)	Maximum I/O pins	f_{MAX2} (MHz)	Speed-tpD pin-to-pin (ns)	Standby I_{CC}^2 $T_A = 25^{\circ}C$	
								3.3/2.5V (mA)	1.8V (mA)
39K30	16–48	512	64	16	176	233	7.2	5	1
39K50	23–72	768	96	24	218	233	7.2	5	1
39K100	46–144	1536	192	48	302	222	7.5	10	2
39K165	77–241	2560	320	80	386	181	8.5	20	4
39K200	92–288	3072	384	96	428	181	8.5	20	4

Each basic block (LB) includes a product term array, an intelligent product-term allocator and 16 macrocells. So, it is a PAL-based circuit. Each PAL has $s = 36$ inputs and up to 16 terms. The allocator can give from 1 to 16 terms to any MC. Moreover, these terms can be shared among different MCs. An LB obtains data from a programmable interconnect matrix (PIM). All 16 outputs of an LB can be connected with other macrocells or memory blocks. So, it is a very flexible architecture.

The architecture of MC is similar to the one shown in Fig. 2.29. But it also includes carry-chains. It allows creating adders and counters. So, these CPLDs can be used to implement FSMs with counters.

Memory blocks are configurable. So, it is possible to change the number of macrocells (V) and their outputs (t_F) for a constant size (V_0). The following relation takes place:

$$V_0 = 2^{S_A} \times t_F. \quad (2.29)$$

In 2.29, the symbol S_A stands for the number of address inputs. Obviously, there is

$$V = 2^{S_A}. \quad (2.30)$$

There is $V_0 = 8192$ bits for a cluster memory block. It has the following configurations: $8K \times 1$, $4K \times 2$, $2K \times 4$ and $1K \times 8$. It corresponds to the pairs $\langle S_A, t_F \rangle$ such as $\langle 13, 1 \rangle$, $\langle 12, 2 \rangle$, $\langle 11, 4 \rangle$ and $\langle 10, 8 \rangle$.

There is $V_0 = 4096$ bits for a channel memory block. It has the following configurations: $4K \times 1$, $2K \times 2$, $1K \times 4$ and 512×8 . Each block can be configured as asynchronous or synchronous dual-port RAM, single-port RAM or ROM.

Now, our reader has all necessary information about CPLD. Let us discuss some design approaches targeting CPLDs. Because CPLD is a product of evolution of SPLDs, the design methods of CPLD-based FSM take roots in SPLD-based design.

There are three types of Boolean functions describing FSM circuits [39, 194]: irregular, regular and multiplexer. Let $Q(y_n)$ be a number of product terms in a SOP of a Boolean function y_n .

The following condition takes place for irregular functions:

$$Q(y_n) \ll 2^L. \quad (2.31)$$

In 2.31, L is a number of arguments in y_n . The following functions are typical for complex FSMs [12]: $L \approx 30$, $Q(y_n) \leq 20$. Such functions are implemented using either PLAs or PALs.

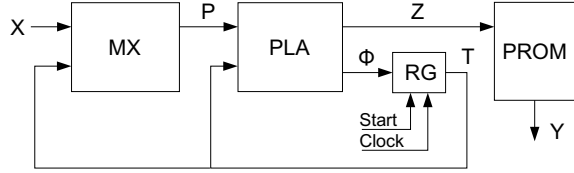
For regular functions, the following condition is true:

$$Q(y_n) > 2^{L-1}. \quad (2.32)$$

Such functions are defined on more than 50% of possible input assignments. Such functions should be implemented using memory blocks.

The multiplexer functions obey to the following rule:

Fig. 2.30 Structural diagram of MPY Mealy FSM



$$Q(y_n) \approx L. \quad (2.33)$$

These functions can be efficiently implemented using multiplexers.

For example, functions 1.4 and 1.7 are irregular, functions 1.8 and 2.25 are regular, functions 2.18 are multiplexer. Such heterogeneity leads to the idea of heterogeneous implementation of FSM logic circuits [16, 191]. It means that some functions are implemented by PLAs (or PALs), some by PROMs (or RAMs). Let us discuss a three-level heterogeneous circuit of MPY Mealy FSM (Fig. 2.30).

In abbreviation MPY, the letter “M” means a multiplexer MX, “P” a PLA, and “Y” a PROM implementing the system 2.25. Let us discuss an example of MPY FSM S_6 represented by Table 2.8.

The following sets can be derived from Table 2.8: $X = \{x_1, \dots, x_8\}$, $Y = \{y_1, \dots, y_9\}$, $A = \{a_1, \dots, a_6\}$, $\Phi = \{D_1, D_2, D_3\}$, $T = \{T_1, T_2, T_3\}$. So, there are $L = 8$, $N = 9$, $M = 6$, $R = 3$ and $H = 15$.

To implement the MX, let us execute the RLC. There are the following sets $X(a_m)$ for S_6 : $X(a_1) = \{x_1, x_2\}$, $X(a_2) = \{x_1, x_3, x_4\}$, $X(a_3) = X(a_6) = \emptyset$, $X(a_4) = \{x_2, x_6\}$, $X(a_5) = \{x_7, x_8\}$. Using 2.17 gives $G = 3$ and $P = \{p_1, p_2, p_3\}$. Using approach from [12] gives the table of RLC represented by Table 2.9.

Using Table 2.9 gives the system 2.18 having the following form:

$$\begin{aligned} p_1 &= x_1(A_1 \vee A_2) \vee x_6A_4 \vee x_7A_5; \\ p_2 &= x_2(A_1 \vee A_4) \vee x_3A_2 \vee x_8A_5; \\ p_3 &= x_4. \end{aligned} \quad (2.34)$$

The system 2.34 determines connections between the control inputs of MX and state variables, as well as information inputs and logical conditions. It is necessary two MXs to implement the system 2.34. The LC $x_4 \in X$ is connected with PLA.

There are $Q = 8$ different CMOs in Table 2.8. They are: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_2\}$, $Y_3 = \{y_3\}$, $Y_4 = \{y_2, y_4\}$, $Y_5 = \{y_5\}$, $Y_6 = \{y_3, y_6\}$, $Y_7 = \{y_7, y_8\}$. Using 1.32 gives $R_Q = 3$ and $Z = \{z_1, z_2, z_3\}$. Because PROMs are used to implement the system 2.25, the codes $K(Y_q)$ do not affect the hardware amount. Let us encode CMOs in the trivial way: $K(Y_1) = 000$, $K(Y_2) = 001, \dots, K(Y_8) = 111$.

Now, it is possible to form the transformed DST of FSM S_6 (Table 2.10). There are columns P_h and Z_h in Table 2.5 replacing the columns X_h and Y_h , respectively.

Next, it is necessary to construct tables with content of MXs, PLA and PROM. Let us show them. Two tables (Tables 2.11 and 2.12) represent MX1 and MX2, respectively. The content of PLA is shown in Table 2.13, the content of PROM in

Table 2.8 Direct structure table of Mealy FSM S_6

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
a_1	000	a_2	001	x_1	$y_1 y_2$	D_3	1
		a_3	010	$\bar{x}_1 x_2$	y_3	D_2	2
		a_3	011	$\bar{x}_1 \bar{x}_2$	$y_2 y_4$	$D_2 D_3$	3
a_2	001	a_4	100	$x_1 x_3$	y_5	D_1	4
		a_5	101	$x_1 \bar{x}_3$	$y_3 y_6$	$D_1 D_3$	5
		a_2	001	$\bar{x}_1 x_4$	$y_1 y_2$	D_3	6
		a_1	000	$\bar{x}_1 \bar{x}_4$	$y_2 y_4$	–	7
a_3	010	a_4	011	1	y_3	$D_2 D_3$	8
a_4	011	a_5	100	$x_2 x_6$	$y_7 y_8$	D_1	9
		a_6	101	$x_2 \bar{x}_6$	$y_8 y_9$	$D_1 D_3$	10
		a_3	010	$\bar{x}_2$	$y_1 y_2$	D_2	11
a_5	100	a_4	011	x_7	y_3	$D_2 D_3$	12
		a_6	101	$\bar{x}_7 x_8$	$y_8 y_9$	$D_1 D_3$	13
		a_5	100	$\bar{x}_7 \bar{x}_8$	–	D_1	14
a_6	101	a_1	000	1	$y_1 y_2$	–	15

Table 2.9 Table of RLC for Mealy FSM S_6

P	a_1	a_2	a_3	a_4	a_5	a_6
p_1	x_1	x_1	–	x_6	x_7	–
p_2	x_2	x_3	–	x_2	x_8	–
p_3	–	x_4	–	–	–	–
$K(a_m)$	000	001	010	011	100	101

Table 2.14. We hope there is the transparent connections between Tables 2.9, 2.11 and 2.12, as well between Table 2.10, 2.13 and 2.14.

We use the symbol “X” to show that this input is insignificant for a given term. We take it from [12].

Using these tables allows constructing a logic circuit of MPY Mealy FSM S_6 (Fig. 2.31). It includes two MXs, a PLA [3, 94] and PROM [74, 216].

MPY Mealy FSMs are based on two basic principles: (1) the structural decomposition [19, 20] and (2) the heterogeneous implementation [19, 191]. The structural decomposition leads to FSM circuits represented as a composition of functional blocks. Each block has unique input variables (at least partially) and output functions different from SBFs implemented by other functional blocks. At the same time, the output functions of some blocks are the arguments of SBFs implemented by other blocks. The heterogeneous implementation is reduced to using different logic elements for creating circuits of different blocks.

Table 2.10 Transformed DST of Mealy FSM S_6

a_m	$K(a_m)$	a_s	$K(a_s)$	P_h	Z_h	ϕ_h	h
a_1	000	a_2	001	p_1	z_3	D_3	1
		a_3	010	$\bar{p}_1 p_2$	z_2	D_2	2
		a_3	010	$\bar{p}_1 \bar{p}_2$	$z_2 z_3$	D_2	3
a_2	001	a_4	011	$p_1 p_2$	z_1	$D_2 D_3$	4
		a_5	100	$p_1 \bar{p}_2$	$z_1 z_3$	D_1	5
		a_2	001	$\bar{p}_1 x_4$	z_3	D_3	6
		a_1	000	$\bar{p}_1 \bar{x}_4$	$z_2 z_3$	–	7
a_3	010	a_4	011	1	z_2	$D_2 D_3$	8
a_4	011	a_5	100	$p_2 p_1$	$z_1 z_2$	D_1	9
		a_6	101	$p_2 \bar{p}_1$	$z_1 z_2 z_3$	$D_1 D_3$	10
		a_3	010	$\bar{p}_2$	z_3	D_2	11
a_5	100	a_4	011	p_1	z_2	$D_2 D_3$	12
		a_6	101	$p_1 \bar{p}_2$	$z_1 z_2 z_3$	$D_1 D_3$	13
		a_5	100	$\bar{p}_1 \bar{p}_2$	–	D_1	14
a_6	101	a_1	000	1	z_3	–	15

Table 2.11 Content of MX1

Inputs	Output
$T_1 T_2 T_3$	p_1
000	x_1
001	x_1
010	–
011	x_6
100	x_7
101	–
110	–
111	–

For example, the SBF 2.18 is implemented by MXs creating the block MX (Fig. 2.30). It has unique arguments $x_e \in X$ and shared arguments $T_r \in T$. The outputs of MX are the inputs of PLA, as well as the shared variables $T_r \in T$. The outputs of PLA enter PROM and RG. So, PROM has only unique inputs $z_r \in Z$, and produces outputs $y_n \in Y$.

Let us denote a functional block consisting of PLAs a PLAer. The block consisting of PAL we denote a PALer. The symbol EMBer stands for a functional block consisting of EMBs. We will use this notation through the further part of our book.

For PAL-based CPLD, an FSM circuit is implemented as a PALer connected with distributed RG (DRG). In the case of PLA-based CPLD, an FSM circuit is implemented as a PLAer and distributed RG. The corresponding structural diagrams

Table 2.12 Content of MX2

Inputs	Output
$T_1 T_2 T_3$	p_2
000	x_2
001	x_3
010	—
011	x_2
100	x_8
101	—
110	—
111	—

Table 2.13 Content of PLA

Inputs		Terms F_h	Outputs	
$T_1 T_2 T_3$	$p_1 p_2 x_4$		$z_1 z_2 z_3$	$D_1 D_2 D_3$
000	1xx	1	001	001
000	01x	2	010	010
000	00x	3	011	010
001	11x	4	100	011
001	10x	5	101	100
001	0x1	6	001	001
001	0x0	7	011	000
010	xxx	8	010	011
011	11x	9	110	100
011	01x	10	111	101
011	x0x	11	001	010
100	1xx	12	010	011
100	01x	13	111	101
100	00x	14	000	100
101	xxx	15	001	000

are shown in Fig. 2.32. In the case of heterogeneous CPLD, a part of the circuit can be implemented as a PALer and a part as an EMBer. For example, there is a structural diagram of P Moore FSM shown in Fig. 2.33. In this case, the SBF 1.4 is represented by the PALer and the SBF 1.8 as the EMBer.

As we noted earlier, there is a huge amount of books and articles related to PLA- and PAL-based synthesis of FSMs. The PLA-based methods can be found in [8, 14, 16, 18, 35, 80, 86, 102–107, 118–120, 134, 146–148, 169, 178, 179, 182, 183, 189, 213, 214]. The works [43–47, 93, 97, 98, 116, 117, 124–131, 135–139, 142, 161, 162, 196–203] are devoted to PAL-based synthesis. The methods of heterogeneous

Table 2.14 Content of PROM

Inputs	Output	<i>q</i>
$z_1z_2z_3$	$y_1y_2y_3y_4y_5y_6y_7y_8y_9$	
000	000000000	1
001	110000000	2
010	001000000	3
011	010100000	4
100	000010000	5
101	001001000	6
110	000000110	7
111	000000011	8

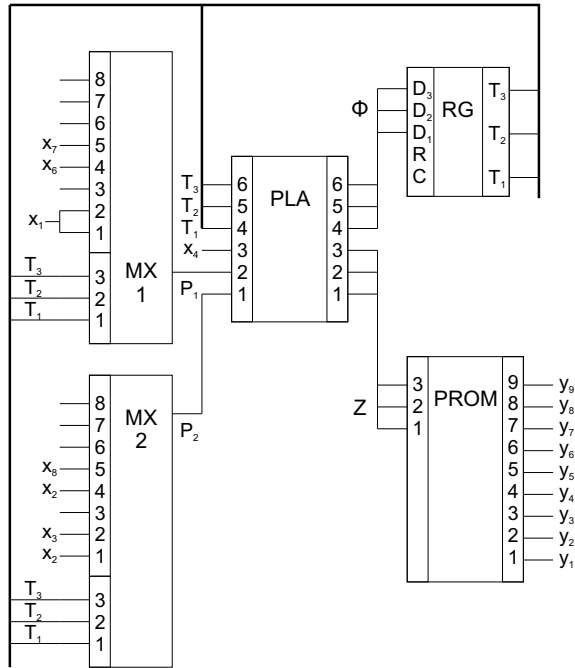


Fig. 2.31 Logic circuit of MPY Mealy FSM S_6

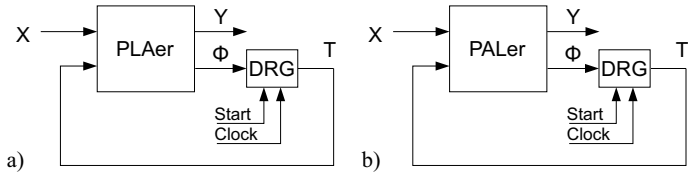


Fig. 2.32 Structural diagrams of PLA-based **a** and PAL-based **b** P Mealy FSM

Fig. 2.33 Structural diagrams of PY Moore FSM

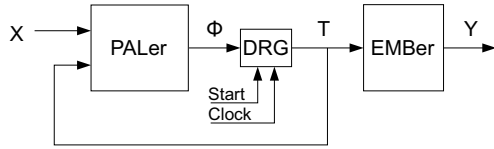
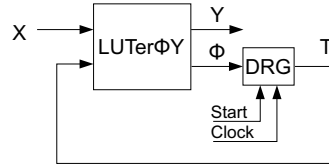


Fig. 2.34 Structural diagram of P FSM



implementation targeting CPLD-based FSMs can be found in [162, 163, 204, 205]. Let us point out that all these methods can be adapted to the features of CFSMs.

2.4 Logic Synthesis with FPGAs

A typical FPGA chip includes hundreds of thousands of LUT elements and around 100Mb of EMBs [6, 222]. To implement an FSM logic circuit, it is enough to use LUTs and/or EMBs and programmable interconnections [194]. As a rule, a LUT has very small amount of inputs ($S_L \leq 6$) [6, 222]. As shown in [144], this value is optimal. EMBs are reconfigurable. There are the following typical configurations of EMB: $36k \times 1$, $18k \times 2$, $8k \times 4$, $4k \times 8$, $2k \times 16$, $1k \times 32$, and 512×64 (bits) [6, 222].

A LUT can implement an arbitrary Boolean function having up to S_L arguments. An EMB can implement an SBF having up to t_F functions and up to S_A arguments. To implement the SBF $Y(L, N)$, it is necessary to find a configuration $\langle S_A, t_F \rangle$ such that the following condition takes place

$$(S_A \geq L) \& (t_F \geq N) \& (V_0 \geq 2^L \cdot N) = 1. \quad (2.35)$$

In 2.35, the value of V_0 is determined by 2.29. We use the symbol $Y(L, N)$ to denote an SBF having L arguments and N outputs.

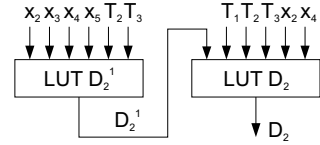
As shown in works [40, 194, 219], there are homogeneous and heterogeneous FPGA-based circuits of FSMs. Let us start from LUT-based FSMs.

Let us denote as LUTer a block including only LUTs. There is the same structural diagram for P Mealy and Moore FSMs (Fig. 2.34).

The LUTer ΦY implements SBFs 1.4, 1.7 or 1.8. The flip-flops of state register are distributed among LUTs. So, we show that it is a DRG.

It is the best LUT-based solution if the following condition takes place:

Fig. 2.35 Circuit for function D_2 if $S_L = 6$



$$NA(f) \leq S_L \quad (f \in \Phi \cup Y). \quad (2.36)$$

The symbol $NA(f)$ in 2.36 stands for the number of arguments in a Boolean function f . It 2.36 is true, then it is necessary exactly a single LUT to implement a circuit for any function. It is really a single-level FSM circuit. It is the fastest, minimal in hardware and requires the least amount of energy among all possible LUT-based solutions [40, 219].

If condition 2.36 is violated for some function $f \in \Phi \cup Y$, then this function should be transformed. To do, different methods of functional decomposition can be used [132, 149, 150, 154, 159, 160, 171, 172, 174–176, 184, 207]. To understand this approach, let us consider the following example.

Let a function D_2 be represented by the following SOP:

$$D_2 = T_1 \bar{T}_2 T_3 x_2 \bar{x}_3 \vee T_1 T_2 T_3 x_4 x_5 \vee T_1 \bar{T}_2 T_3 \bar{x}_2 x_4. \quad (2.37)$$

Let us use LUTs having $S_L = 7$. In this case, it is enough a single LUT to implement a circuit for 2.37. Let us have LUTs with $S_L = 6$. Because $NA(D_2) = 7$, the condition 2.36 is violated.

So, we should represent the function D_2 as a composition of subfunctions having not more than 6 arguments. Let it be $D_2^1 = \bar{T}_2 T_3 x_2 \bar{x}_3 \vee T_2 T_3 x_4 x_5$ and $D_2^2 = T_1 \bar{T}_2 T_3 \bar{x}_2 x_4$. Now, we can represent 2.37 as

$$D_2 = T_1 D_2^1 \vee D_2^2. \quad (2.38)$$

The expression 2.38 corresponds to the two-level circuit shown in Fig. 2.35. Obviously, this circuit has more LUTs, levels of logic, interconnections than a single-level circuit.

The principle of functional decomposition is a basic principle of FPGA-based design [184]. It is based on representation of a Boolean function $F(X)$ in the following form:

$$F(X) = H(X_0, G_1(X_1), \dots, G_I(X_I)). \quad (2.39)$$

Obviously, there are the following relations: $X_0 \subseteq X$, $X_1 \subseteq X$, $\dots$, $X_I \subseteq X$. The equation 2.39 determines the circuit shown in Fig. 2.36.

A positive feature of the functional decomposition is that it allows implementing an FSM circuit in violation of 2.36. A negative influence of it is that it increases the propagation time and the amount of interconnections in comparison with an equivalent single-level circuit. But this drawback follows directly from the limited

Fig. 2.36 Circuit based on functional decomposition

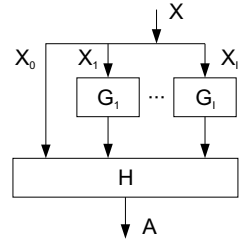
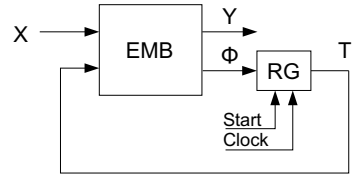


Fig. 2.37 Structural diagram of EMB-based P FSM



number of inputs S_L . An EMB can implement an SBFs 1.4 and 1.7 (Mealy FSM) or 1.4 and 1.8 (Moore FSM). It is possible if the following condition takes place:

$$2^{L+R} \cdot (N + R) \leq V_0. \quad (2.40)$$

In this case, an FSM circuit is implemented as a single EMB (Fig. 2.37).

We show the RG in Fig. 2.37. If an EMB can be synchronized, then RG is absent.

If 2.40 is violated, then it is necessary more than a single EMB to implement an FSM circuit. So, the symbol “EMB” should be replaced by “EMBer” in Fig. 2.37. In many practical cases, a heterogeneous approach should be used [51–56, 82, 85, 100, 111, 112, 133, 140, 141, 170, 173, 177, 186–188, 193, 209].

To optimize the hardware amount in FPGA-based FSM circuits, it is necessary to diminish the numbers of arguments in SBFs. It can be done using the following approaches:

1. A proper state assignment leading to disappearance of some arguments from SBFs 1.4, 1.7, 1.8. The corresponding methods could be found, for example, in [10, 72, 76–79, 87, 89–92, 101, 108–110, 113, 122, 123, 143, 158, 166–168, 180, 206, 212, 220, 221].
2. Structural decomposition of FSM circuit transforming SBFs representing FSM circuits. These methods take roots in MCU [4, 5, 185, 216, 217].

Let us discuss briefly these approaches. Let us start from the state assignment.

If an FSM is specified by GSA, then such sets as X , Y and A are known. But there are no stage codes. To obtain them, the step of state assignment is executed [12]. This step is very important because its outcome has a tremendous influence on the hardware amount (the number of LUTs) in the FSM logic circuit [16, 191]. A strategy of state assignment could target optimization for area, performance, power consumption, or testability.

One of the most popular state assignment algorithms is JEDI which is distributed with the system SIS [190]. JEDI targets a multi-level logic implementation. It is based in the weight assignment for states.

The input dominant algorithm assigns higher weights to pairs of present states which assert similar inputs and produce sets of next states. It allows maximizing the size of common cubes in the implemented logic function. The output dominated algorithm assigns higher weights to pairs of next states which are generated by similar input combinations and similar sets of present states. It maximizes the number of common cubes in the logic function.

In modern industrial packages, different state assignment strategies are used. For example, two optimization criteria are used in the design tool XST of Xilinx: maximum performance and minimum hardware [23]. Seven different approaches are used for state assignment. The automatic state assignment is based on some special algorithm proposed by Xilinx. It has been never published. The method of one-hot encoding is based on the following expression:

$$R = M. \quad (2.41)$$

This method is very popular because: (1) it is very simple and (2) each LUT is connected with flip-flop. So, this conception is implemented very easy in FPGAs. In this case, there is a lot of input memory functions but each of them is relatively simple.

Also, the following methods can be found in XST tool: (1) compact based on 1.2; (2) Grey codes-based; (3) Johnson codes-based and (4) counters-based. As a rule, we will use a compact state assignment and name it a “binary state assignment”. Let us point out that methods of state assignment do not change the structural diagrams of FSM circuits.

The structural decomposition [42] changes the structural diagrams. The most popular are methods of RLC and encoding of CMOs [16, 191]. They can be used in both heterogeneous and homogeneous approaches. For example, four different structural diagrams exist for MP Mealy FSMs. They are shown in Fig. 2.38.

We use a symbol P to show that a block implements the system 2.18. The symbol ΦY stands for blocks implementing systems 2.23–2.24. We propose to use a letter “E” to show that a block is implemented as an EMBer, and “L” as a LUTer. So, the EE MP Mealy FSM is represented by Fig. 2.38a. Other models are: LE MP (Fig. 2.38b), LL MP (Fig. 2.38c) and EL MP (Fig. 2.38d) Mealy FSMs.

The MP FSM can be represented as a pair $\langle M, P \rangle$. So, there are four pairs corresponding to MP Mealy FSMs shown in Fig. 2.38: $\langle E, E \rangle$, $\langle L, E \rangle$, $\langle L, L \rangle$ and $\langle E, L \rangle$.

Also, there are four different structural diagrams of PM Mealy FSM. They corresponds to pairs $\langle E, E \rangle$, $\langle L, E \rangle$, $\langle L, L \rangle$ and $\langle E, L \rangle$, too. For example, the LE PY Mealy FSM is shown in Fig. 2.39.

In LE PY FSM, the LUTer ΦZ implements the functions 1.4 and 2.26. The EMBer Y implements the system 2.25. All other diagrams can be shown in the same manner.

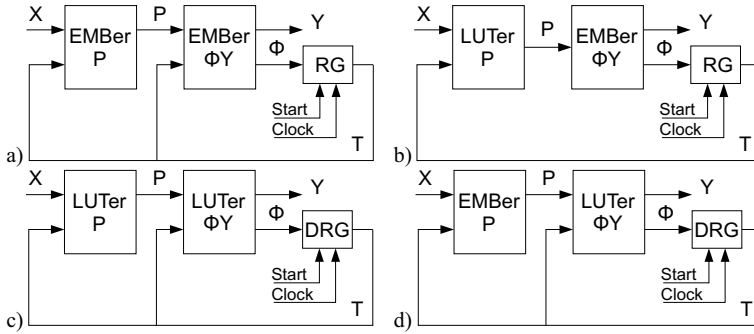


Fig. 2.38 Structural diagrams of MP Mealy FSM

Fig. 2.39 Structural diagram of LE PY Mealy FSM

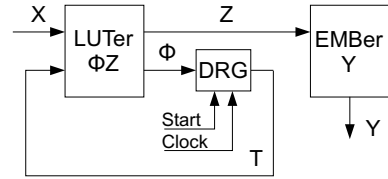
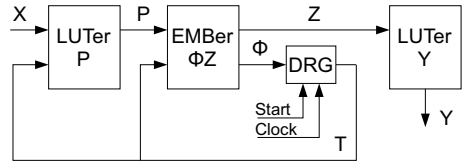


Fig. 2.40 Structural diagram of LEL MPY Mealy FSM



Obviously, there are eight different structures of MPY Mealy FSMs. They corresponds to vectors $\langle M, P, Y \rangle$ from $\langle L, L, L \rangle$ to $\langle E, E, E \rangle$. For example, Fig. 2.40 shows the structural diagram of LEL MPY Mealy FSM.

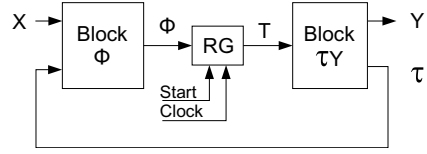
In this circuit, the LUTer P implements the system 2.18 the LUTer Y the system 2.25. The EMBer ΦZ implements the systems 2.23 and

$$Z = Z(T, P). \quad (2.42)$$

To optimize an FSM circuit, it is necessary to take into account the features of both logic elements and the FSM models. There are Moore FSM states in the case of CFSM [12]. Let us discuss possibilities of Moore FSM optimization based on classes of PES [21].

The states $a_m, a_s \in A$ are pseudoequivalent if they mark the vertices of GSA Γ whose outputs are connected with the same input of a GSA's vertex [21]. There is a partition $\Pi_A = \{B_1, \dots, B_I\}$ of the set A where $B_i \subseteq A$ is a class of PES [21]. Two approaches can be used for optimizing Moore FSM circuit [21]:

Fig. 2.41 Structural diagram of P_C Moore FSM



1. **Optimal state assignment** [21]. In this case, each class $B_i \in \Pi_A$ is represented by minimum number of generalized intervals of R -dimensional Boolean space [39]. These intervals are treated as codes $K(B_i)$ of classes $B_i \in \Pi_A$. It leads to P_0 Moore FSM [21, 39]. It has the same structural diagram as P Moore FSM.
2. **Transformation of state codes into class codes** [21]. In this case, each class $B_i \in \Pi_A$ is represented by a code $K(B_i)$ having R_A bits:

$$R_A = \lceil \log_2 I \rceil. \quad (2.43)$$

The class codes are based on class variables $\mathcal{T}_r \in \mathcal{T}$, where $|\mathcal{T}| = R_A$. Let us point out that $I = |\Pi_A|$ in 2.43.

This approach leads to P_C Moore FSM (Fig. 2.41).

In P_C Moore FSM, the Block Φ implements the system

$$\Phi = \Phi(\mathcal{T}, X). \quad (2.44)$$

The Block τY implements the systems 1.8 and

$$\mathcal{T} = \mathcal{T}(T). \quad (2.45)$$

The states $a_m \in A$ can be encoded in different ways: to optimize the system 2.44 or 1.8 or both.

This method belongs to a family of methods of object transformation [23, 41, 66, 75, 155–157]. For example, states can be represented by the pairs (CMO, identifier) or CMOs by pairs (state, identifier). These methods can be applied in both CPLD- and FPGA-based FSMs [39, 40, 194]. In these cases, identifiers replace logical conditions $x_l \in X$ for so named secondary objects [194]. It allows decreasing the number of arguments in corresponding SBFs [39]. The methods target both Mealy and Moore FSMs.

For example, let us discuss a $P_A Y$ Mealy FSM [39]. The subscript “A” means that states $a_m \in A$ are primary objects and CMOs $Y_q \subseteq Y$ are secondary objects. Let it be K identifiers necessary to represent any CMO. In this case, it is enough R_I elements of the set V to encode to identifiers $I_1, \dots, I_K$. The value of R_I is determined as

$$R_I = \lceil \log_2 K \rceil. \quad (2.46)$$

There is a structural diagram of $P_A Y$ Mealy FSM shown in Fig. 2.42.

Fig. 2.42 Structural diagram of $P_A Y$ Mealy FSM

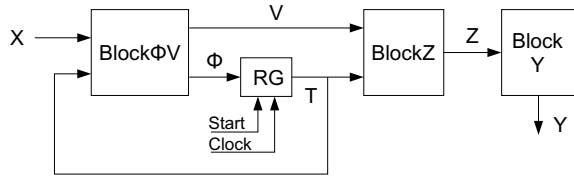
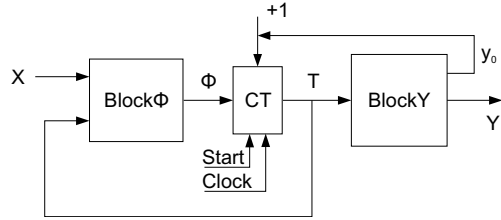


Fig. 2.43 Structural diagram of CMCU



In $P_A Y$ Mealy FSM, the Block ΦV implements systems 1.4 and

$$V = V(T, X). \quad (2.47)$$

The Block Z implements the SBF

$$Z = Z(T, V). \quad (2.48)$$

The Block Y implements the system 2.25.

Also, the hardware amount can be reduced due to taking into account the features of initial GSA Γ . For example, linear GSAs include more than 75% of operator vertices [18]. To use this property, it is possible to replace the state RG by a state counter. In this case, some interstate transitions are executed by incrementing the CT. This approach is used in compositional MCUs (CMCU) [18].

There is a structural diagram of CMCU shown in Fig. 2.43.

In CMCU the Block Φ implements the system 1.4. But these functions are generated only for conditional transitions. The Block Y implements the system 1.8 and additional function

$$y_0 = y_0(T). \quad (2.49)$$

The function $y_0 = 1$ for unconditional transitions. If $y_0 = 1$, then $CT = CT + 1$.

The theory of CMCU is represented in [38]. It is based on the results [17, 24, 36, 59, 60, 68–71, 164, 165, 208, 218]. It is possible to generate $y_0 = 1$ for conditional transitions, too. It is done in FSMs based on linear chains of states [7]. The theory of these FSMs is represented in [73].

Let us point out that all these methods can be adapted to features of CFSMs. In the following chapter we discuss how to synthesize and optimize the circuits of CFSM using ASICs, CPLDs and FPGAs. In the following chapters, we propose synthesis methods based in most of our works [27–34, 57, 58, 61–65].

References

1. <http://www.electronics-tutorial.net>. Accessed: January, 2020
2. Achasova S (1987) Synthesis algorithms for automata with PLAs. M: Soviet radio (in Russian)
3. Adamski M, Barkalov A (2006) Architectural and sequential synthesis of digital devices. University of Zielona Góra Press
4. Agerwala T (1976) Microprogram optimization: a survey. *IEEE Trans Comput* 25(10):962–973
5. Agrawala A, Rauscher T (1976) Foundations of microprogramming. Academic Press, New York
6. Altera (2020) <http://www.altera.com>. Accessed: January
7. Amann R, Baitinger U (1989) Optimal state chains and states codes in finite state machines. *IEEE Trans Comput Aided Design* 8(2):153–170
8. Asahar P, Devidas S, Newton A (1992) Sequential logic synthesis. Kluwer Academic Publishers, Boston
9. Atmel (2020) <http://www.atmel.com>. Accessed: January
10. Bacchetta P, Daldos L, Sciuto D, Silvano C (2000) Low-power state assignment techniques for finite state machines. In: *Proceedings of the 2000 IEEE International Symposium on Circuits and Systems (ISCAS'2000)*, vol 2. IEEE, Geneva, pp 641–644
11. Baker RJ (2010) CMOS circuit design, layout, and simulation, 3rd edn. Wiley-IEEE Press
12. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
13. Baranov S, Barkalov A (1982) Application of programmable logic arrays in digital systems. *Foregin Radioelectron* 6:67–79
14. Baranov S, Levin I, Koren O, Karpovskiy M (2009) Designing fault tolerant FSM by nano-PLA. In: *Proceedings of the 15th IEEE International On-Line Testing Symposium*, pp 229–234
15. Baranov S, Sinjov V (1976) Programmable logic arrays in digital systems. *Foregin Radioelectron* 1:78–84
16. Baranov S, Skliarov V (1986) Digital devices with programmable LSIs with matrix structure. *Radio and Sviaz* (in Russian)
17. Barkalov A (1983) Microprogram control unit as composition of automate with programmable and hardwired logic. *Automat Comput Sci* 17(4):36–41
18. Barkalov A (1983) Synthesis methods for microprogram control units on LSIs with matrix structure. PhD Thesis (in Russian)
19. Barkalov A (1994) Optimization of multilevel circuit of mealy FSM with PLAs. *Control Syst Mach* 93:13–16
20. Barkalov A (1995) Multilevel PLA schemes for microprogram automata. *Cybern Syst Anal* 31(4):489–495
21. Barkalov A (1998) Principles of logic optimization for Moore microprogram automaton. *Cybern Syst Anal* 34(1):54–60
22. Barkalov A (2002) Synthesis of control units with programmable logic devices. Donetsk National Technical University, Donetsk (in Russian)
23. Barkalov A, Barkalov Jr A (2004) Synthesis of finite-state machines with transformation of the object's codes. In: *Proceedings of the International Conference TCSET'2004*. Lviv Polytechnic National University, Publishing House of Lviv Polytechnic, Lviv, Ukraina, pp 61–64
24. Barkalov A, Beleckij O, Nedal A (1999) Applying of optimization methods of Moore automaton for synthesis of compositional microprogram control unit. *Automat Control Comput Sci* 33(1):44–52
25. Barkalov A, Das D (1991) Optimization of logic circuit of microprogrammed mealy automaton on PLA. *Automat Control Comput Sci* 25(3):90–94
26. Barkalov A, Dzhalishvili Z, Salomatin V, Starodubov K (1986) Optimization of a microinstruction address scheme for microprogram control unit with PLA and PROM. *Automat Control Comput Sci* 20(5):83–87

27. Barkalov A, Titarenko L, Visor J, Matvienko A (2015) Realization of combined microprogrammed automaton with fpga. *Probl Inf Control* 3(47):6–12
28. Barkalov A, Titarenko L, Visor J, Matvienko A (2015) Syntthesis of combined microprogrammed automaton with fpga. *Comput Tool Networks Syst* 14:32–39
29. Barkalov A, Titarenko L, Visor J, Matvienko A (2016) Optimal encoding of states in combined microprogrammed automaton. *Control Syst Mach* 6:34–39
30. Barkalov A, Titarenko L, Visor J, Matvienko A (2016) Optimizing block of replacement of logical conditions in combined microprogrammed automaton. 3(55):11–20
31. Barkalov A, Titarenko L, Visor J, Matvienko A (2016) Reducing the number of luts in the circuit of combined automaton. *Control Syst Mach* 3:16–22
32. Barkalov A, Titarenko L, Visor J, Matvienko A (2017) Reducing hardware amount in combined automaton. *Control Syst Mach* 4:44–51
33. Barkalov A, Titarenko L, Visor J, Matvienko A (2017) Structural reduction in combined automata. *Probl Inf Control* 1-2(57-58):12–19
34. Barkalov A, Titarenko L, Vizor J, Matvienko A (2017) Reducing the number of look-up table elements in the circuit of combined microprogrammed automaton. *Comput Tools Networks Syst* 16:20–29
35. Barkalov A, Salomatin V, Starodubov K, Das K (1991) Optimization of Mealy automaton logic using programmable logic arrays. *Cybern Syst Anal* 27(5):789–793
36. Barkalov A, Shwec A (1994) Synthesis of compositional microprogram control unit with modified microinstruction addressing. *Automat Control Comput Sci* 28(5):22–30
37. Barkalov A, Shwec A (1995) Synthesis of compositional microprogram control unit with a code transformer. *Automat Comput Sci* 29(6):16–24
38. Barkalov A, Titarenko L (2008) Logic synthesis for compositional microprogram control units, vol 22. Springer-Verlag, Berlin
39. Barkalov A, Titarenko L (2009) Logic synthesis for FSM-based control units, vol 53. Lecture Notes in Electrical Engineering. Springer-Verlag, Berlin
40. Barkalov A, Titarenko L (2009) Synthesis of operational and control automata. UNITECH, Donetsk
41. Barkalov A, Titarenko L, Barkalov Jr A (2007) Moore FSM synthesis with coding of compatible microoperations fields. In: *Proceedings of IEEE East-West Design & Test Symposium EWDTS'07*. Kharkov National University of Radioelectronics, Yerevan, Armenia, Kharkov, pp 644–646
42. Barkalov A, Titarenko L, Barkalov A Jr (2012) Structural decomposition as a tool for the optimization of an FPGA-based implementation of a Mealy FSM. *Cybern Syst Anal* 48(2):313–322
43. Barkalov A, Titarenko L, Chmielewski S (2007) Optimization of logic circuit of Moore FSM on CPLD. *Pomiary Automat Kontrola* 53(5):18–20
44. Barkalov A, Titarenko L, Chmielewski S (2007) Optimization of moore FSM on CPLD. In: *Proceedings of the Sixth International Conference CAD DD'07*, Vol 2. Minsk, pp 39–45
45. Barkalov A, Titarenko L, Chmielewski S (2007) Optimization of moore FSM on system-on-chip. In: *Proceedings of IEEE East-West Design & Test Symposium EWDTS'07*, Yerevan, Armenia, Kharkov, pp 105–109
46. Barkalov A, Titarenko L, Chmielewski S (2008) Decrease of hardware amount in logic circuit of Moore FSM. *Przegląd Telekomunikacyjny i Wiadomości Telokomunikacyjne* 6:750–752
47. Barkalov A, Titarenko L, Chmielewski S (2008) Optimization of moore control unit with refined state encoding. In: *Proceedings of the 15th International Conference MIXDES*, Poznań. Department of Microeletronics and Computer Science, Technical University of Łódź, Poland, pp 417–420
48. Barkalov A, Titarenko L, Chmielewski S (2008) Optimization of moore FSM on system-on-chip using PAL technology. In: *Proceedings of the International Conference TCSET 2008*. Ministry of Education and Science of Ukraine, Lviv Polytechnic National University, Lviv, Publishing House of Lviv Polytechnic, Lviv-Slavsko, Ukraina, pp 314–317

49. Barkalov A, Titarenko L, Hebda O (2011) Hardware reduction for matrix circuit of control moore automaton. In: Proceedings of IEEE East-West Design & Test Symposium EWDTS 2011, Sevastopol, Ukraina, Kharkov National University of Radio Electronics, pp 94–97
50. Barkalov A, Titarenko L, Hebda O (2011) Synthesis of moore finite state machine with transformation of extended state codes. In: *Pomiary, Automatyka, Kontrola*, Vol. 57(nr 6):652–655
51. Barkalov A, Titarenko L, Kołopeńczyk M (2006) Optimization of control unit with code sharing. In: Proceedings of the 3rd International Workshop of IFAC Discrete–Event System Design (DESDES’06), University of Zielona Góra Press, Rydzyna, pp 195–200
52. Barkalov A, Titarenko L, Kołopeńczyk M (2006) Optimization of control unit with code sharing. In: Proceedings of the IEEE East-West Design & Test Workshop EWDTW’06, Sochi, Kharkov, Kharkov National University of Radioelectronics, pp 171–174
53. Barkalov A, Titarenko L, Kołopeńczyk M (2007) Optimization of control memory size of control unit with codes sharing. In: Proceedings of the IXth International Conference CADSM 2007 “The Experience of Designing and Application of CAD Systems in Microelectronics”. Lviv–Polana, Ukraine, pp 242–245
54. Barkalov A, Titarenko L, Kołopeńczyk M (2013) EMB-based design of mealy FSM. 12th IFAC Conference on Programmable Devices and Embedded Systems. Velké Karlovice, Czechy, pp 215–220
55. Barkalov A, Titarenko L, Mielcarek K, Węgrzyn M (2015) Design of EMB-based mealy FSMs with transformation of output functions. In: *FAC-PapersOnLine*, 48(10):197–201. 2nd IFAC Conference on Embedded Systems, Computer Intelligence and Telematics CESCIT 2015 Maribor, Slovenia, pp 22–24
56. Barkalov A, Titarenko L, Mielcarek K, Zajac W (2015) Designing EMB-based mealy FSM with transformation of collections of output functions. In: *AIP Conference Proceedings*, 1702(1)
57. Barkalov A, Titarenko L, Vizor J, Matvienko A (2018) Encoding of output variables in combined automaton. *Computer Tools Networks Syst* 17:73–80
58. Barkalov A, Titarenko L, Vizor J, Matvienko A, Kadet N (2018) Encoding of collections of microoperations in combined microprogrammed automaton. *Control Syst Mach* 3:42–50
59. Barkalov A, Titarenko L, Wiśniewski R (2006) Optimization of address circuit of compositional microprogram unit. In: Proceedings of the IEEE East-West Design & Test Workshop EWDTW’06. Sochi, Kharkov, Kharkov National University of Radioelectronics, pp 167–170
60. Barkalov A, Titarenko L, Wiśniewski R (2006) Synthesis of compositional microprogram control units with sharing codes and address decoder. In: Proceedings of the International Conference Mixed Design of Integrated Circuits and Systems—MIXDES, Łódź, pp 397–400
61. Barkalov A, Titarenko L, Zeleneva I, Hrushko S (2016) Method of reducing the number of terms under implementing the circuit of combined microprogrammed automaton with cpld. *Bull Nat Univ KPI* 49(1221):25–30
62. Barkalov A, Titarenko L, Zeleneva I, Hrushko S (2017) Encoding of collections of output signals in the circuit of combined microprogrammed automaton with cpld. *Inf Cybern Comput* 2(25):5–11
63. Barkalov A, Titarenko L, Zeleneva I, Hrushko S (2017) Implementing combined FSM with CPLDs. *Int J Soft Eng Comput Syst (USECS)* 3(2):118–124
64. Barkalov A, Titarenko L, Zeleneva I, Hrushko S (2017) Using pseudoequivalent states in combined microprogrammed automaton. *Inf Cybern Comput* 7(27)(1):5–11
65. Barkalov A, Titarenko L, Zeleneva I, Hrushko S (2018) Implementing on fpga of combined finite state machine with counter. In: Proceedings of IEEE 9. International Conference on Dependable Systems, Services and Technologies (DESSERT 2018), Ukraine, Kyev, pp 247–251
66. Barkalov A, Węgrzyn A, Barkalov Jr A (2007) Synthesis of control units with transformation of the codes of objects. In: Proceedings of the IXth International Conference CADSM 2007 (The Experience of Designing and Application of CAD Systems in Microelectronics), Lviv—Polyana, Ukraine, Lviv Polytechnic National University, Lviv, Publishing House of Lviv Polytechnic National University, pp 260–261

67. Barkalov A, Węgrzyn M (2006) Design of control units with programmable logic. University of Zielona Góra Press
68. Barkalov A, Węgrzyn M, Wiśniewski R (2006) Partial reconfiguration of compositional microprogram control units implemented on FPGAs. In: Proceedings of IFAC Workshop on Programmable Devices and Embedded Systems (Brno), pp 116–119
69. Barkalov A, Wiśniewski R (2004) Design of compositional microprogram control units with maximal encoding of inputs. *Radioelectron Inf* 3:79–81
70. Barkalov A, Wiśniewski R (2004) Optimization of compositional microprogram control unit with elementary operational linear chains. *Control Syst Comput* 5:25–29
71. Barkalov A, Wiśniewski R (2005) Optimization of compositional microprogram control units implemented on system-on-chip. *Theoret Appl Inf* 9:7–22
72. Barkalov A, Zelenjova I (2005) Optimization of replacement of logical conditions for an automaton with bidirectional transitions. *Automat Control Comput Sci* 34(5):48–53
73. Barkalov O, Titarenko L, Bieganski J (2018) Logic synthesis for finite state machines based on linear chains of states: foundations, recent developments and challenges. In: *Studies in Systems, Decision and Control*, Vol 118. Springer International Publishing Switzerland, Cham
74. Barkalov O, Titarenko L, Mazurkiewicz M (2019) Foundations of embedded systems. In: *Studies in Systems, Decision and Control*, Vol 195. Springer International Publishing, Cham
75. Barkalov O, Titarenko L, Mazurkiewicz M, Mielcarek K, Encoding of terms in lut-based mealy fsm. In: *Mixed Design of Integrated Circuits and Systems—MIXDES*, 25th International Conference, page 1, Gdynia, Polska. IEEE Explore, New York
76. Barkalov O, Titarenko L, Mielcarek K (2018) Hardware reduction for lut-based mealy fsm. *Int J Appl Math Comput Sci* 28(3):595–607
77. Barkalov O, Titarenko L, Mielcarek K (2018) Twofold state assignment for fpga-based mealy fsm. 7th International Conference on Modern Circuits and Systems Technologies—MOCAST. Thessaloniki, Grecja, New York, IEEE Explore, pp 1–4
78. Barkalov O, Titarenko L, Mielcarek K, Twofold state assignment for lut-based mealy fsm. In: *Mixed Design of Integrated Circuits and Systems—MIXDES*, 25th International Conference, Gdynia, Polska, IEEE Explore, New York, p 1
79. Benini L, De Micheli G (1995) State assignment for low power dissipation. *IEEE J Solid-State Circuits* 30(3):258–268
80. Bibilo P (1992) Synthesis of combinational PLA-structures with VLSIs. Nauka i Technika, Minsk (in Russian)
81. Birkner J (1978) Reduce random-logic complexity. *Electron Design* 26(17):98–105
82. Bomar BW (2002) Implementation of microprogrammed control in FPGAs. *IEEE Trans Ind Electron* 49(2):415–422
83. Borowik G (2007) Synthesis of sequential devices into FPGA with embedded memory blocks. PhD thesis, Warszawa: WUT, PhD Thesis
84. Borowik G (2008) Improved state encoding for FSM implementation in FPGA structures with embedded memory blocks. *Int J Electron Telecommun* 54:9–28
85. Borowik G, Falkowski B, Łuba T (2007) Cost-efficient synthesis for sequential circuits implemented using embedded memory blocks of FPGA's. In: *IEEE Design and Diagnostics of Electronic Circuits and Systems*, pp 99–104
86. Brayton R, Hatchel G, McMullen C, Sangiovanni-Vincentelli A (1984) Logic minimization algorithms for VLSI synthesis. Kluwer Academic Publishers, Boston
87. Brayton R, Rudell R, Sangiovanni-Vincentelli A, Wang A (1987) MIS: a multi-level logic optimization system. *IEEE Trans Comput Aided Design* 6(11):1062–1081
88. Brown S, Rose J (1996) Architecture of FPGAs and CPLDs: a tutorial. *IEEE Design Test Comput* 3:1–41
89. Bukowiec A (2008) Synthesis of finite state machines for programmable devices based on multi-level implementation. PhD thesis, University of Zielona Góra, PhD Thesis
90. Chattopadhyay S (2005) Area conscious state assignment with flip-flop and output polarity selection for finite state machines synthesis—a genetic algorithm. *Comput J* 48(4):443–450

91. Chattopadhyay S, Chaudhuri P (1998) Genetic algorithm based approach for integrated state assignment and flipflop selection in finite state machines synthesis. In: Proceedings of the IEEE International Conference on VLSI Design. Los Alamitos, IEEE Computer Society, pp 522–527
92. Chen C, Zhao J, Ahmadi M (2003) A semi-Gray encoding algorithm for low-power state assignment. In: Proceedings of the 2003 International Symposium on Circuits and Systems, 2003. ISCAS'03, vol 5. IEEE, pp 389–392
93. Chmielewski S (2014) Using structural peculiarities of moore FSM for reduction of number of PALS. PhD Thesis, University of Zielona Góra (in Polish)
94. Clements A (2000) The Principles of Computer Hardware. Oxford University Press Inc, New York
95. Cong J, Yan K (2000) Synthesis for fpgas with embedded memory blocks. In: Proceedings of the 2000 ACM/SIGDA Eighth International Symposium on Field Programmable Gate Arrays, FPGA '00. New York, NY, USA, ACM, pp 75–82
96. Cypress Semiconductor Corporation (2020) Cypress programmable Logic: delta 39K. Data Sheet. <http://cypress.com/pld/delta39k.html>. Accessed: January
97. Czerwiński R, Kania D (2004) State assignment method for high speed FSM. In: Proceedings of Programmable Devices and Systems, pp 216–221
98. Czerwiński R, Kania D (2005) State assignment for PAL-based CPLDs. In: Proceedings of 8th Euromicro Symposium on Digital System Design, pp 127–134
99. Czerwinski R, Kania D (2013) Finite state machine logic synthesis for complex programmable logic devices, vol 231. Lecture Notes in Electrical Engineering. Springer-Verlag, Berlin
100. Das N, Aruna PP (2018) FPGA implementation of reconfigurable finite state machine with input multiplexing architecture using hungarian method. In: International Journal of Reconfigurable Computing
101. De Micheli G, Brayton RK, Sangiovanni-Vincentelli A (2006) Optimal state assignment for finite state machines. IEEE Trans Comput Aided Design Int Circuits Syst 4(3):269–285
102. Debnath D, Sasao T (1999) Multiple-valued minimization to optimize PLA with output EXOR gates. In: Proceedings of IEEE International Symp. on Mupltiple-Valued Logic, pp 99–104
103. Debnath D, Sasao T (2005) Doutput phase optimization for AND-OR-EXOR PLAs with decoders and its application to design of adders. IFICE Trans Inf Syst E88-D(7):1492–1500
104. Devadas S, Ma H (1990) Easily testable PLA-based finite state machines. IEEE Trans Comput Aided Design Int Circuits Syst 9(6):604–611
105. Devadas S, Ma H, Newton A, Sangiovanni-Vincentelli A (1988) MUSTANG: state assignment of finite state machines targeting multilevel logic implementation. IEEE Trans Comput Aided Design 7(12):1290–1300
106. Devadas S, Newton A (1991) Exact algorithms for output encoding, state assignment, and four-level boolean minimization. IEEE Trans Comput Aided Design 10(1):143–154
107. Du X, Hachtel G, Lin B, Newton A (1991) MUSE: a multilevel symbolic encoding algorithm for state assignment. IEEE Trans Computer Aided Design Int Circuits Syst 10(1):28–38
108. El-Maleh A, Sait S, Khan F (2006) Finite state machine state assignment for area and power minimization. In: 2006 IEEE International Symposium on Circuits and Systems. ISCAS, Proceedings. IEEE, pp 5303–5306
109. El-Maleh AH (2017) A probabilistic pairwise swap search state assignment algorithm for sequential circuit optimization. Int VLSI J 56(Supplement C):32–43
110. Escherman B (1993) State assignment for hardwired VLSI control units. ACM Comput Surv 25(4):415–436
111. Garcia-Vargas I, Senhadji-Navarro R, Jiménez-Moreno G, Civit-Balcells A, Guerra-Gutierrez P (2007) ROM-based finite state machine implementation in low cost FPGAs. In: IEEE International Symposium on Industrial Electronics ISIE 2007. IEEE, pp 2342–2347
112. Glaser J, Damm M, Haase J, Grimm C (2011) Tr-fsm: transition-based reconfigurable finite state machine. ACM Trans Reconfig Technol Syst 4(3):23:1–23:14
113. Goren S, Ferguson F (2002) CHESMIN: a heuristic for state reduction of incompletely specified finite state machines. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE'02), pp 248–254

114. Gorman K (1973) Programmable logic array: a new approach to microprogramming. *Electron Design News* 18:68–75
115. Grout I (2011) *Digital systems design with FPGAs and CPLDs*. Elsevier Science, Amsterdam
116. Grzes TN, Solov'ev VV (2014) Sequential algorithm for low-power encoding internal states of finite state machines. *J Comput Syst Sci Int* 53(1):92–99
117. Grzes TN, Solov'ev VV (2015) Minimization of power consumption of finite state machines by splitting their internal states. *J Comput Syst Sci Int* 54(3):367–374
118. Gupta B, Narayanan H, Desai M (1999) A state assignment scheme targeting performance and area. In: *Proceedings of 12th International Conference On VLSI Design*, pp 378–383
119. Habib S (1988) *Microprogramming and firmware engineering methods*. John Wiley and Sons, New York
120. Hassoun S, Sasao T (2002) *Logic synthesis and verification*. Kluwer Academic Publishers
121. Hemel A (1976) The PLA: A “different kind of ROM. *Electron Design* 24:78–84
122. Hu H, Xue H, Bian J (2003) A heuristic state assignment algorithm targeting area. In: *Proceedings of 5th International Conference on ASIC*, vol 1, pp 93–96
123. Huang J, Jou J, Shen W (2000) ALTO: an iterative area/performance algorithms for LUT-based FPGA technology mapping. *IEEE Trans VLSI Syst* 18(4):392–400
124. Kania D (1999) Two-level logic synthesis on PAL-based CPLD and FPGA using decomposition. In: *Proceedings of 25th Euromicro Conference*, pp 278–281
125. Kania D (1999) Two-level logic synthesis on PALs. *Electron Lett* 17:879–880
126. Kania D (2000) Coding capacity of PAL-based logic blocks included in CPLDs and FPGAs. In: *Proceedings of IFAC Workshop on Programmable Devices and Sysytems (PDS'2000)*. Elsevier Science, pp 164–169
127. Kania D (2000) Decomposition-based synthesis and its application in PAL-oriented technology mapping. In: *Proceedings of 26th Euromicro Conference*. Maastricht: IEEE Computer Society Press, pp 138–145
128. Kania D (2002) An efficient algorithm for output coding in PAL-based CPLDs. *Int J Eng* 15(4):325–328
129. Kania D (2002) Logic synthesis of multi-output functions for PAL-based CPLDs. In: *Proceedings of IEEE International Conference on Field-Programmable Technology*, pp 429–432
130. Kania D (2003) An efficient approach to synthesis of multi-output boolean functions on PAL-based devices. *IEEE Proceed Comput Digital Tech* 150:143–149
131. Kania D, Milik A (2010) Logic synthesis based on decomposition for cplds. *Microprocess Microsyst* 34(1):25–38
132. Khamis A, Zydek D, Borowik G, Subbaram Naidu D (2013) *Control system design based on modern embedded systems*, vol 8112. Springer, Berlin Heidelberg, pp 491–498
133. Kilts S (2007) *Advanced FPGA design: architecture, implementation, and optimization*. Wiley-IEEE Press
134. Kim T, Vella T, Brayton R, Sangiovanni-Vincentalli A (1997) *Synthesis of finite state machines: functional optimization*. Kluwer Academic Publishers, Boston
135. Klimovich AS, Solov'ev VV (2010) Transformation of a mealy finite-state machine into a moore finite-state machine by splitting internal states. *J Comput Syst Sci Int* 49(6):900–908
136. Klimovich AS, Solov'ev VV (2011) A method for minimizing moore finite-state machines by merging two states. *J Comput Syst Sci Int* 50(6):907–920
137. Klimovich AS, Solov'ev VV (2012) Minimization of mealy finite-state machines by internal states gluing. *J Comput Syst Sci Int* 51(2):244–255
138. Klimowicz AS, Solov'ev VV (2013) Minimization of incompletely specified mealy finite-state machines by merging two internal states. *J Comput Syst Sci Int* 52(3):400–409
139. Klimowicz AS, Solov'ev VV (2015) Structural models of finite-state machines for their implementation on programmable logic devices and systems on chip. *J Comput Syst Sci Int* 54(2):230–242
140. Kołopieńczyk M, Barkalov A, Titarenko L (2014) Hardware reduction for RAM-based moore FSMs. In: *7th International Conference on Human System Interactions—HSI 2014*, Lisbon, Portugal, pp 255–260

141. Kołopieńczyk M, Titarenko L, Barkalov A (2017) Design of emb-based moore fsm's. *J Circuits Syst Comput* 26(7):1–23
142. Kubica M, Kania D, Kulisz J (2019) A technology mapping of fsm's based on a graph of excitations and outputs. *IEEE Access* 7:16123–16131
143. Kubica M, Opara A, Kania D (2017) Logic synthesis for FPGAs based on cutting of BDD. *Microprocess Microsyst* 52(C):173–187
144. Kuon I, Tessier R, Rose J (2008) FPGA architecture: survey and shallenges–found trends. *Electr Design Automat* 2:135–253
145. Lattice (2020) <http://www.lattice.com>. Accessed: February
146. Levin IS (1986) Decompositional design of automata based on pla with memory. *Automat Control Comput Sci* 20(2):61–68
147. Levin IS (1987) A hierarchical model of the interaction of microprogrammed automata. *Automat Control Comput Sci* 21(3):67–73
148. Lin B, Newton AR (1989) Synthesis of multiple level logic from symbolic high-level description languages. In: *Proceedings of International Conference on VLSI*, pp 187–196
149. Łuba T (1994) Multi-level logic synthesis based on decomposition. *Microprocess Microsyst* 18(8):429–437
150. Łuba T, Rawski M, Jachna Z (2002) Functional Decomposition as a universal method for logic synthesis of digital circuits. In: *Proceedings of IX International Conference MIXDES'02*, pp 285–290
151. Maxfield C (2004) *The design warrior's guide to FPGAs*. Academic Press Inc, Orlando, FL, USA
152. Maxfield C (2008) *FPGAs: instant access*. Newnes
153. McCluskey E (1986) *Logic design principles*. Prentice Hall, Englewood Cliffs
154. Michalski T, Kokosiński Z (2016) Functional decomposition of combinational logic circuits with pkmin. In: *Technical Transactions. Electrical Engineering–Czasopismo Techniczne. Elektrotechnika*, Iss. 2-E:191–202
155. Mielcarek K, Barkalov A, Titarenko L (2016) Designing LUT-based mealy FSM with transformation of collections of output functions. *5th International Conference on Modern Circuits and Systems Technologies–MOCAST*. Thessaloniki, Grecja, IEEE, pp 1–4
156. Mielcarek K, Barkalov O, Titarenko L (2017) Designing HFPGA-based mealy FSMs with transformation of output functions. In: *24th International Conference Mixed Design of Integrated Circuits and Systems—MIXDES*
157. Mielcarek K, Barkalov O, Titarenko L (2017) Designing moore fsm with extended class codes. *6th International Conference on Modern Circuits and Systems Technologies–MOCAST 2017*. Thessaloniki, Grecja, New York, IEEE, pp 1–4
158. Nöth W, Kolla R (1999) Spanning tree based state encoding for low power dissipation. In: *Proceedings of the conference on Design, automation and test in Europe*, Association for Computing Machinery, p 37
159. Nowicka M, Łuba T, Rawski M. FPGA-based decomposition of boolean functions: algorithms and implementation. *Adv Comput Syst* 502–509
160. Opara A, Kubica M, Kania D (2019) Methods of improving time efficiency of decomposition dedicated at fpga structures and using bdd in the process of cyber-physical synthesis. *IEEE Access* 7:20619–20631
161. Palagin A, Barkalov A, Svetc A (1995) Encoding variations of logical conditions to optimize circuits of microprogrammed automata. *Control Syst Mach* 6:9–12
162. Palagin A, Barkalov A, Usifov S, Starodubov K, Svetc A (1991) Realization of microprogrammed automata on CPLD. *Control Syst Mach* 8:18–22
163. Palagin A, Barkalov A, Usifov S, Szwets A (1992) Synthesis of microprogram automata with PLIs. *Kiev: IC of NAN* (92):18–26, 1992
164. Papachristou C, Gambhir S (1982) A microsequencer architecture with firmware support for modular microprogramming. *ACM SIGMICRO Newslett* 13(4):105–113
165. Papachristou CA (1981) Hardware microcontrol schemes using PLAs. In: *Proceedings of the 14th Annual Workshop on Microprogramming, MICRO 14*, Piscataway, NJ, USA, IEEE Press, pp 3–16

166. Park S, Cho S, Yang S, Ciesielski M (2004) A new state assignment technique for testing and low power. In: Proceedings of the 41st annual Design Automation Conference. Association for Computing Machinery, pp 510–513
167. Park S, Yang S, Cho S (2000) Optimal state assignment technique for partial scan designs. *Electron Lett* 36(18):1527–1529
168. Pedram C, Despain A (1998) Low-power state assignment targeting two- and multilevel logic implementations. *IEEE Trans Comput Aided Design Int Circuits Syst* 17(12):1281–1291
169. Pomerancz I, Cheng K (1993) STOIC: state assignment based on output/input functions. *IEEE Trans Comput Aided Design Int Circuits Syst* 12(8):1123–1131
170. Rafla NI, Gauba I (2010) A reconfigurable pattern matching hardware implementation using on-chip RAM-based FSM. In: 2010 53rd IEEE International Midwest Symposium on Circuits and Systems, pp 49–52
171. Rawski M (2007) Decomposition of boolean function sets. *Electron Telecommun Q* 53(3):231–251
172. Rawski M (2007) Efficient variable partitioning method for functional decomposition. *Electron Telecommun Q* 53(1):63–81
173. Rawski M, Borowik G, Łuba T, Tomaszewicz P, Falkowski BJ (2010) Logic synthesis strategy for FPGAs with embedded memory blocks. In: *Electrical Review (Przegląd Elektrotechniczny)*, pp 95–101
174. Rawski M, Łuba T, Jachna Z, Tomaszewicz P (2005) Design of embedded control systems. The Influence of Functional Decomposition on Modern Digital Design Process. Springer-Verlag, Boston, pp 193–203
175. Rawski M, Selvaraj H, Łuba T (2005) An application of functional decomposition in ROM-based FSM implementation in FPGA devices. *J Syst Architect* 51(6–7):423–434
176. Rawski M, Selvaraj H, Łuba T, Szotkowski P (2005) Application of symbolic functional decomposition concept in fsm implementation targeting fpga devices. In: Sixth International Conference on Computational Intelligence and Multimedia Applications (ICCIMA'05), pp 153–158
177. Rawski M, Tomaszewicz P, Borowik G, Łuba T (2011) 5 Logic Synthesis Method of Digital Circuits Designed for Implementation with Embedded Memory Blocks of FPGAs, vol 79. Springer, pp 121–144
178. Rho J, Hatchel F, Somenzi R, Jacoby R (1994) Exact and heuristic algorithms for the minimization of incompletely specified state machines. *IEEE Trans Comput Aided Design* 13(2):167–177
179. Rudell R, Sangiovanni-Vincentelli A (1987) Multiple-valued minimization for PLA optimization. *IEEE Trans Comput Aided Design* 6(5):727–750
180. Sasao T (1984) Input variable assignment and output phase optimization of PLA optimization. *IEEE Trans Comput* 33(10):879–894
181. Sasao T (2011) Memory-based logic synthesis, 1st edn. Springer Publishing Company, Incorporated
182. Saucier G, Depaulet M, Sicard P (1987) ASYL: a rule-based system for controller synthesis. *IEEE Trans Comput Aided Design* 6(11):1088–1098
183. Saucier G, Sicard P, Bouchet L (1990) Multi-level synthesis on programmable devices in the ASYL system. In: Proceedings of Euro ASIC, pp 136–141
184. Scholl C (2001) Functional decomposition with application to FPGA synthesis. Kluwer Academic Publishers, Boston
185. Schwartz S (1968) An algorithm for minimizing read-only memories for machine control. In: IEEE 10th Annual Symposium on Switching and Automata Theory, pp 28–33
186. Senhadji-Navarro R, Garcia-Vargas I (2015) High-speed and area-efficient reconfigurable multiplexer bank for RAM-based finite state machine implementations. *J Circuits Syst Comput* 24(07):1550101
187. Senhadji-Navarro R, García-Vargas I, Guisado JL (2012) Performance evaluation of RAM-based implementation of finite state machines in FPGAs. In: 19th IEEE International Conference on Electronics, Circuits, and Systems (ICECS 2012). Seville, Spain, pp 225–228

188. Senhadji-Navarro R, Garcia-Vargas I, Jiménez-Moreno G, Civit-Balcells A, Guerra-Gutierrez P (2004) ROM-based FSM implementation using input multiplexing in FPGA devices. *Electron Lett* 40(20):1249–1251
189. Sentowich E, Singh K, Lavango L, Moon C, Murgai R, Saldanha A, Savoj H, Stephan P, Bryton R, Sangiovanni-Vincentelli A (1992) SIS: a system for sequential circuit synthesis. In: *Proceedings of the International Conference of Computer Design (ICCD'92)*, pp 328–333
190. Sentowich E, Singh K, Lavango L, Moon C, Murgai R, Saldanha A, Savoj H, Stephan P, Bryton R, Sangiovanni-Vincentelli A (1992) SIS: a system for sequential circuit synthesis. Technical report, University of California, Berkely
191. Skliarov V (1984) Synthesis of automata with matrix LSIs. *Nauka i Technika*, Minsk (in Russian)
192. Skliarova I, Sklyarov V, Sudnitson A (2012) Design of FPGA-based circuits using hierarchical finite state machines. TUT Press, Tallinn
193. Sklyarov V (2000) Synthesis and implementation of RAM-based finite state machines in FPGAs. In: *Proceedings of Field-Programmable Logic and Applications: The Roadmap to Reconfigurable Computing*, Villach, Springer-Verlag, pp 718–728
194. Sklyarov V, Skliarova I, Barkalov A, Titarenko L (2014) Synthesis and optimization of FPGA-based systems, vol 294. *Lecture Notes in Electrical Engineering*. Springer-Verlag, Berlin
195. Smith M (1997) Application-specific integrated circuits. Addison-Wesley, Boston
196. Solov'ev VV (2012) Changes in the length of internal state codes with the aim at minimizing the power consumption of finite-state machines. *J Commun Technol Electron* 57(6):642–648
197. Solov'ev VV (2013) Implementation of finite-state machines based on programmable logic ics with the help of the merged model of mealy and moore machines. *J Commun Technol Electron* 58(2):172–177
198. Solov'ev VV (2014) Complex minimization method for finite state machines implemented on programmable logic devices. *J Comput Syst Sci Int* 53(2):186–194
199. Solov'ev VV (2017) Minimization of mealy finite-state machines by using the values of the output variables for state assignment. *J Comput Syst Sci Int* 56(1):96–104
200. Solov'ev VV, Posrednikova AA (2009) The hierarchical method of synthesis of large-capacity comparators with the use of programmable logic integrated circuits. *J Commun Technol Electron* 54(3):338–346
201. Solovjev V, Czyzy M (1999) Refined CPLD macrocells architecture for effective FSM implementation. In: *Proceedings of the 25th EUROMICRO Conference*, vol 1. Milan, Italy, pp 102–109
202. Solovjev V, Czyzy M (1999) The universal algorithm for fitting targeted unit to complex programmable logic devices. In: *Proceedings of the 25th EUROMICRO Conference*, vol 1. Milan, Italy, pp 286–289
203. Solovjev V, Czyzy M (2001) Synthesis of sequential circuits on programmable logic devices based on new models of finite state machines. In: *Proceedings of the EUROMICRO Conference*, Milan, pp 170–173
204. Solovjov V (1996) Design of functional blocks of digital systems with programmable logic devices. Bestprint (in Russian)
205. Solovjov V (2001) Design of digital systems basing on programmable logic integrated circuits. Hotline-Telecom, Moscow, Moscow (in Russian)
206. Sutter G, Todorovich E, López-Buedo S, Boemo E (2002) Low-power FSMs in FPGA: encoding alternatives. *Integrated Circuit Design. Optimization and Simulation*. Springer-Verlag, Power and Timing Modeling, pp 363–370
207. Sztokowski P, Rawski M (2009) Improvements to symbolic functional decomposition algorithms for FSM implementation in fpga devices. *Electron Telecommun Q* 55(2):335–354
208. Tatalov E (2011) Synthesis of compositional microprogram control units for programmable devices. Master's Thesis, Donetsk National Technical University, Donetsk
209. Tiwari A, Tomko K (2004) Saving power by mapping finite-state machines into embedded memory blocks in FPGAs. In: *Proceedings of the conference on Design, Automation and Test in Europe—Vol 2*. IEEE Computer Society, pp 916–921

- 210. Trimberg S (2015) Three ages of FPGA: a retrospective on the first thirty years of FPGA technology. *IEEE Proceed* 103(3):318–331
- 211. Veendrick H (2017) *Nanometer CMOS ICs: from basics to ASICs*, 2nd edn. Springer International Publishing
- 212. Venkatamaran G, Reddy S, Pomerancz I (2003) GALLOP: genetic algorithm based low power FSM synthesis by simultaneous partitioning and state assignment. In: *Proceedings of 16th International Conference on VLSI Design*, pp 533–538
- 213. Villa T, Saldachna T, Brayton R, Sangiovanni-Vincentelli A (1997) Symbolic two-level minimization. *IEEE Trans Comput Aided Design* 16(7):692–708
- 214. Villa T, Sangiovanni-Vincentelli A (1990) NOVA: State assignment of finite state machines for optimal two-level logic implementation. *IEEE Trans Comput Aided Design* 9(9):905–924
- 215. Weste N, Harris D (2010) *CMOS VLSI design: a circuits and systems perspective*, 4th edn. Addison-Wesley Publishing Company, USA
- 216. Wilkes M (1951) The best way to design an automatic calculating machine. In: *Proceedings of Manchester University Computer Inaugural Conference*
- 217. Wilkes M, Stringer J (1953) Microprogramming and the design of the control circuits in an electronic digital computer. *Proceed Cambridge Philosophical Soc* 49:230–238
- 218. Wiśniewski R (2008) *Synthesis of compositional microprogram control units for programmable devices*. PhD Thesis, University of Zielona Góra
- 219. Wolf W (2004) *FPGA-based system design*. Prentice Hall PTR, Upper Saddle River, NJ, USA
- 220. Wu X, Pedram M, Wang L (2000) Multi-code state assignment for low-power design. *IEEE Proceed Circuits Devices Syst* 147:271–275
- 221. Xia Y, Almani A (2002) Genetic algorithm based state assignment for power and area optimization. *IEEE Proceed Comput Digital Tech* 149:128–133
- 222. Xilinx (2020) <http://www.xilinx.com>. Accessed: January

Chapter 3

ASIC-based Synthesis of CFSMs



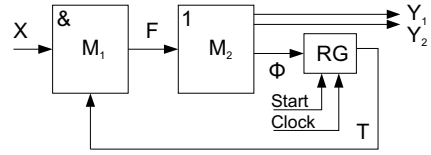
Abstract The Chapter begins from the trivial matrix implementation of CFSM circuits. To diminish the chip area occupied by the trivial matrix circuit, the methods of structural decomposition are used. In the beginning, the CFSM with the replacement of logical conditions are synthesised. Next, it is shown that the area reduction is possible due to encoding of collections of microoperations of both Mealy and Moore types. Next, the method of encoding of the fields of compatible microoperations is used to optimize the area required for CFSM circuit. The methods of optimization based on the classes of pseudoequivalent states (PES) of Moore FSMs are discussed. Three groups of these methods are considered: the optimal state assignment, the complete transformation of states and the partial transformation of states into classes of PES. Next, the methods of expansion of code space, nonstandard representation of state codes and refined state assignment are discussed. The last part of the Chapter is devoted to analysis of different possible models of ASIC-based CFSMs. These models combine different methods of structural decomposition used for both Mealy and Moore FSMs. The examples of synthesis are given for majority of proposed CFSM models.

3.1 Trivial Matrix Implementation

There is the structural diagram of CFSM shown in Fig. 1.31. It includes three blocks ($B\Phi$, $BY1$, $BY2$) and a register. The $B\Phi$ implements input memory functions $D_r \in \Phi$ represented as (1.4). The $BY1$ implements Mealy-type MOs $y_n \in Y1$ (1.42). The $BY2$ implements Moore-type MOs $y_n \in Y2$ (1.43). The RG keeps state codes $K(a_m)$ having R bits (1.2). The functions (1.4) and (1.42) depend on terms (1.10). The functions (1.43) depend on conjunctions A_m corresponding to state codes.

As we show for matrix realization of Moore FSMs, any function A_m can be represented as a conjunction of DST terms corresponding to transitions from the state $a_m \in A$. The same approach can be used in trivial matrix implementation of CFSM (Fig. 3.1).

Fig. 3.1 Trivial matrix circuit of CFSM



Obviously, Fig. 3.1 represents P CFSM. Here, the matrix M_1 implements terms $F_h \in F = \{F_1, \dots, F_H\}$. The terms are represented as (1.10). The matrix M_2 implements functions

$$\Phi = \Phi(F); \quad (3.1)$$

$$Y1 = Y1(F); \quad (3.2)$$

$$Y2 = Y2(F). \quad (3.3)$$

If a control algorithm is represented by a GSA Γ , then we propose the following method of synthesis of trivial matrix CFSM:

1. Marking GSA Γ by states of Moore FSM.
2. Executing the state assignment.
3. Creating the direct structure table of CFSM.
4. Creating the systems (3.1)–(3.3).
5. Representing the systems (3.1)–(3.3) by matrices.

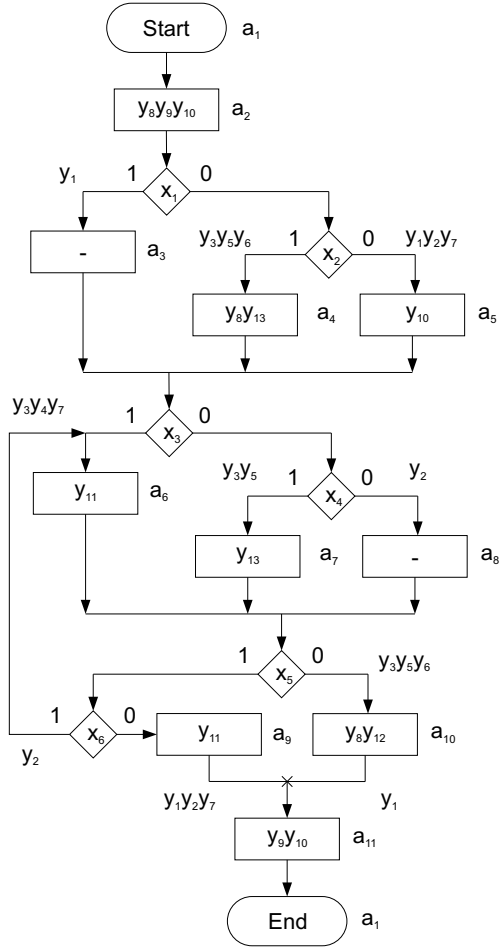
Let us discuss an example of synthesis for CFSM $P(\Gamma_3)$. The symbol $M(\Gamma_i)$ means that an FSM having the structural diagram M is synthesized starting from a GSA Γ_i [5]. The GSA Γ_3 is marked by states of Moore FSM using the rules [4]. It is shown in Fig. 3.2.

Analysis of Fig. 3.2 shows that the CFSM $P(\Gamma_3)$ is characterized by the sets $A = \{a_1, \dots, a_{11}\}$, $X = \{x_1, \dots, x_6\}$, $Y1 = \{y_1, \dots, y_7\}$, $Y2 = \{y_8, \dots, y_{13}\}$, $Y = Y1 \cup Y2 = \{y_1, \dots, y_{13}\}$. So, there is $M = 11$, $L = 6$, $N_1 = 7$, $N_2 = 6$, $N = 13$.

Using (1.2) gives $R = 4$. So, there are the sets $\Phi = \{D_1, \dots, D_4\}$ and $T = \{T_1, \dots, T_4\}$. Let us encode the states $a_m \in A$ in the trivial way: $K(a_1) = 0000$, $K(a_2) = 0001$, ..., $K(a_{11}) = 1010$. Now, it is possible to construct the DST of CFSM $P(\Gamma_3)$. It includes the columns a_m , $K(a_m)$, $Y(a_m)$, a_s , $K(a_s)$, X_h , Y_h , Φ_h , h . The meaning of each column is clear from Chap. 1.

The DST of CFSM $P(\Gamma_3)$ includes $H = 25$ rows. It can be used for creating systems (3.1)–(3.3). These systems are created in the trivial way. For example, there are the following functions: $F_1 = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4$, $F_2 = \bar{T}_1 \bar{T}_2 \bar{T}_3 T_4$, $y_1 = F_2 \vee F_4 \vee F_{23} \vee F_{24}$; $y_8 = F_2 \vee F_3 \vee F_4 \vee F_8 \vee F_9 \vee F_{10} \vee F_{24}$. In the same way, all other functions can be derived from Table 3.1.

The trivial matrix circuit just reflects a DST. In the discussed case, it is the circuit shown in Fig. 3.3. To avoid the doubling of wires, we use the following notation. If the direct value of a variable assigned to a particular wire enters some term, then we use the sign “ $\times$ ”. If it is the compliment value, the sign “ $.$ ” is used. No sign means

Fig. 3.2 Initial GSA I_3 

that there is no connection between a wire and a term. We show only a part of MOs in Fig. 3.3.

To estimate the complexity of this circuit, we can use formulae (2.12)–(2.13). Using them gives $S(M_1) = 2(L + R) \cdot H = 2(6 + 4) \cdot 25 = 500$, $S(M_2) = H \cdot (N + R) = 25 \cdot (13 + 4) = 425$, $S_T = 925$. Let us find the efficiency E_T of this circuit using the expression (2.6).

As we have found, there are 136 interconnections in M_1 and 114 in M_2 . So, there is $S_{TR} = 250$. Using (2.16) gives $E_T = 250/925 = 0.27$. It means that 73% of chip area is not used in the discussed case.

To increase the efficiency of matrix realization, it is possible to use different methods of structural decomposition [8–10]. We start from known methods of RLC and encoding of CMOs [4]. Next, we discuss how to optimize the trivial circuit using PES of Moore FSM [7].

Table 3.1 DST of CFSM $P(\Gamma_3)$

a_m	$K(a_m)$	$Y(a_m)$	a_S	$K(a_S)$	X_h	Y_h	Φ_h	h
a_1	0000	—	a_2	0001	1	—	D_4	1
a_2	0001	$y_8 y_9 y_{10}$	a_3	0010	x_1	y_1	D_3	2
			a_4	0011	$\bar{x}_1 x_2$	$y_3 y_5 y_6$	$D_3 D_4$	3
			a_5	0100	$\bar{x}_1 \bar{x}_2$	$y_1 y_2 y_7$	D_2	4
a_3	0010	—	a_6	0101	x_3	$y_3 y_4 y_7$	$D_2 D_4$	5
			a_7	0110	$\bar{x}_3 x_4$	$y_3 y_5$	$D_2 D_3$	6
			a_8	0111	$\bar{x}_3 \bar{x}_4$	y_2	$D_2 D_3 D_4$	7
a_4	0011	$y_8 y_{13}$	a_6	0101	x_3	$y_3 y_4 y_7$	$D_2 D_4$	8
			a_7	0110	$\bar{x}_3 x_4$	$y_3 y_5$	$D_2 D_3$	9
			a_8	0111	$\bar{x}_3 \bar{x}_4$	y_2	$D_2 D_3 D_4$	10
a_5	0100	y_{10}	a_6	0101	x_3	$y_3 y_4 y_7$	$D_2 D_4$	11
			a_7	0110	$\bar{x}_3 x_4$	$y_3 y_5$	$D_2 D_3$	12
			a_8	0111	$\bar{x}_3 \bar{x}_4$	y_2	$D_2 D_3 D_4$	13
a_6	0101	y_{11}	a_6	0101	$x_5 x_6$	y_2	$D_2 D_4$	14
			a_9	1000	$x_5 \bar{x}_6$	—	D_1	15
			a_{10}	1001	$\bar{x}_5$	$y_3 y_5 y_6$	$D_1 D_4$	16
a_7	0110	y_{13}	a_6	0101	$x_5 x_6$	y_2	$D_2 D_4$	17
			a_9	1000	$x_5 \bar{x}_6$	—	D_1	18
			a_{10}	1001	$\bar{x}_5$	$y_3 y_5 y_6$	$D_1 D_4$	19
a_8	0111	—	a_6	0101	$x_5 x_6$	y_2	$D_2 D_4$	20
			a_9	1000	$x_5 \bar{x}_6$	—	D_1	21
			a_{10}	1001	$\bar{x}_5$	$y_3 y_5 y_6$	$D_1 D_4$	22
a_9	1000	y_{11}	a_{11}	1010	1	$y_1 y_2 y_7$	$D_1 D_3$	23
a_{10}	1001	$y_8 y_{12}$	a_{11}	1010	1	y_1	$D_1 D_3$	24
a_{11}	1010	$y_9 y_{10}$	a_1	0000	1	—	—	25

3.2 Structural Decomposition for Matrix CFSMs

In the case of RLC, the matrix M_1 is replaced by three matrices [4]. The matrix M_V implements terms $V_h \in V = \{V_1, \dots, V_{H1}\}$ of system (2.18). The matrix M_P implements the functions $p_g \in P$ used as arguments of functions (2.23)–(2.24). The matrix V_F implements terms (2.21) used to generate functions $D_r \in \Phi$ and $y_n \in Y1 \cup Y2$. So, there are four matrices in MP CFSM (Fig. 3.4).

In MP CFSM, the matrix M_V implements the functions

$$V = V(T, X). \quad (3.4)$$

The matrix M_P implements the functions

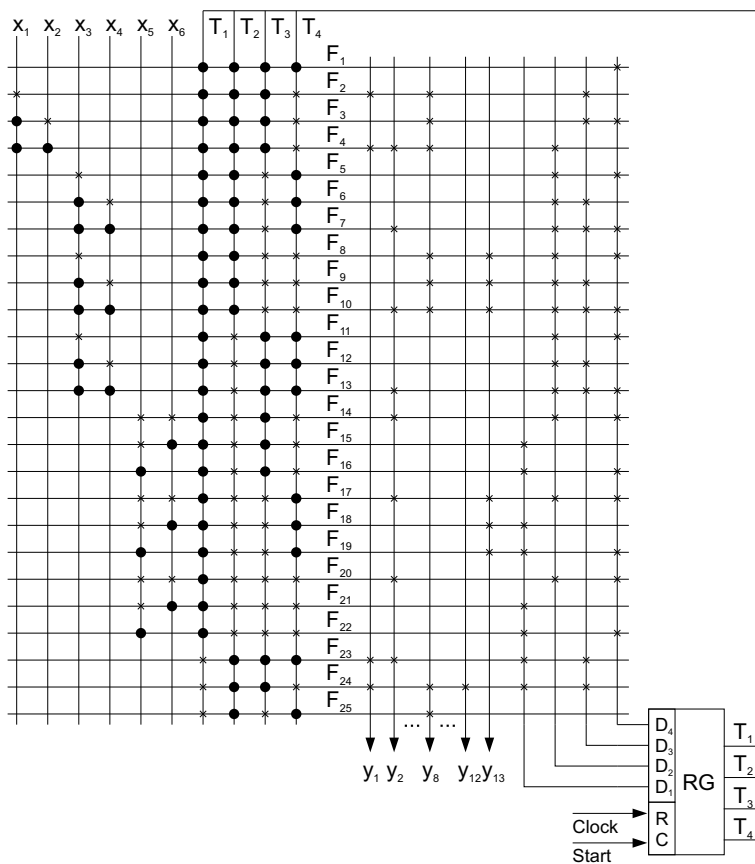


Fig. 3.3 Trivial matrix realization of CFSM $P(F_3)$

Fig. 3.4 Structural diagram of MP CFSM

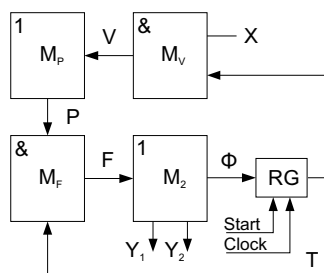


Table 3.2 Table of RLC for CFSM $MP(\Gamma_3)$

P/A	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
p_1	—	x_1	x_3	x_3	x_3	x_5	x_5	x_5	—	—	—
p_2	—	x_2	x_4	x_4	x_4	x_6	x_6	x_6	—	—	—

Fig. 3.5 State codes of CFSM $MP(\Gamma_3)$

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	a_1	a_3	a_2	a_6
	01	a_9	a_4	*	a_7
	11	a_{11}	*	*	*
	10	a_{10}	a_5	*	a_8

$$P = P(V). \quad (3.5)$$

The matrix M_F implements the functions (2.21). The functions (3.1)–(3.3) are generated by the matrix M_2 .

There are the following steps in synthesis method for MP CFSM:

1. Marking the GSA Γ by states of Moore FSM.
2. Execution of RLC.
3. Execution of state assignment optimizing (3.5).
4. Constructing DST of MP CFSM.
5. Creating the systems (3.1)–(3.5).
6. Representing the functions (3.1)–(3.5) by matrices.

Let us apply this method for GSA Γ_3 . The first step is already executed.

Analysis of GSA Γ_3 shows that using (2.17) gives $G = 2$. To optimize the system (2.18), let us use the approach from [4]. It means that the same conditions from different sets $X(a_m)$ must be replaced by the same variables $p_g \in P$. Using this approach gives Table 3.2.

Let us find a partition Π_X of the set A by classes $A(x_e)$. Each class $A(x_e)$ includes states $a_m \in A$ with the following property: $x_e \in X(a_m) \cap X(a_s)$. There are the following classes $A(x_e)$ in discussed case: $A(x_1) = A(x_2) = \{a_2\}$, $A(x_3) = A(x_4) = \{a_3, a_4, a_5\}$, $A(x_5) = A(x_6) = \{a_6, a_7, a_8\}$. Using approach [1] gives us the state codes shown in Fig. 3.5.

There are the following equations $P = P(T, X)$ derived from Table 3.2

$$\begin{aligned} p_1 &= x_1 A_2 \vee x_3 (A_3 \vee A_4 \vee A_5) \vee x_5 (A_6 \vee A_7 \vee A_8); \\ p_2 &= x_2 A_2 \vee x_4 (A_3 \vee A_4 \vee A_5) \vee x_6 (A_6 \vee A_7 \vee A_8). \end{aligned} \quad (3.6)$$

Using state codes (Fig. 3.5) and the law of expansion [17] gives the following equations:

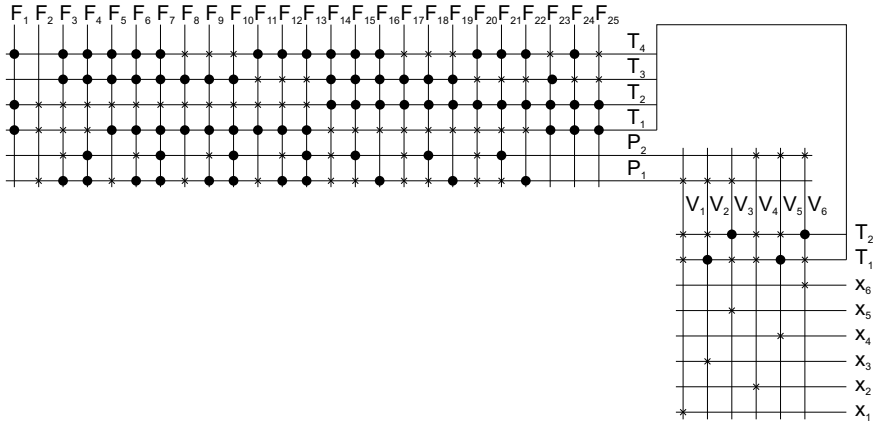


Fig. 3.6 Matrices M_V , M_P , M_F of CFSM $MP(\Gamma_3)$

$$\begin{aligned} p_1 &= T_1 T_2 x_1 \vee \bar{T}_1 T_2 x_3 \vee T_1 \bar{T}_2 x_5 = V_1 + V_2 + V_3; \\ p_2 &= T_1 T_2 x_2 \vee \bar{T}_1 T_2 x_4 \vee T_1 \bar{T}_2 x_6 = V_4 + V_5 + V_6. \end{aligned} \quad (3.7)$$

There are $H_P = 14$ terms in (3.6). But there is $H_P = 6$ in (3.7). So, the proper state assignment and distribution of LCs allow significant optimization for area of matrices M_V and M_P . Besides, there are only 2 state variables in (3.7). It reduces the area of matrix M_V as compared to equations with $R = 4$.

Now, we have systems (3.4)–(3.5). They follow from the system (3.7). To get SBF

$$F = F(T, P), \quad (3.8)$$

it is necessary to construct the DST of MP CFSM.

This table includes the columns a_m , $K(a_m)$, $Y(a_m)$, a_s , $K(a_s)$, P_h , Y_h , h . It is Table 3.3 in the discussed case. It is constructed on the base of Table 3.1 in the way [4]. We use state codes from the Karnaugh map (Fig. 3.5) in Table 3.3.

There are the same circuits of M_2 for $P(\Gamma_3)$ and $MP(\Gamma_3)$. So, let us show only the matrices M_V , M_P and M_F for the discussed case. They form the matrix M_1 from Fig. 3.1. The circuit is shown in Fig. 3.6. Let us point out that only direct values of $x_e \in X$ enter M_V .

Let us find the area $S(M_V) + S(M_P) + S(M_F)$. In the common case, the following formulae can be used for this purpose:

$$S(M_V) = (L + 2R_v) \cdot H_P; \quad (3.9)$$

$$S(M_P) = H_P \cdot G; \quad (3.10)$$

$$S(M_F) = 2(G + R) \cdot H. \quad (3.11)$$

Table 3.3 DST of CFSM $MP(\Gamma_3)$

a_m	$K(a_m)$	$Y(a_m)$	a_s	$K(a_s)$	P_h	Y_h	Φ_h	h
a_1	0000	—	a_2	1100	1	—	$D_1 D_2$	1
a_2	1100	$y_8 y_9 y_{10}$	a_3	0100	p_1	y_1	D_3	2
			a_4	0101	$\bar{p}_1 p_2$	$y_3 y_5 y_6$	$D_2 D_4$	3
			a_5	0110	$\bar{p}_1 \bar{p}_2$	$y_1 y_2 y_7$	$D_2 D_3$	4
a_3	0100	—	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	5
			a_7	1001	$\bar{p}_1 p_2$	$y_3 y_5$	$D_1 D_4$	6
			a_8	1010	$\bar{p}_1 \bar{p}_2$	y_2	$D_1 D_3$	7
a_3	0100	—	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	5
			a_7	1001	$\bar{p}_1 p_2$	$y_3 y_5$	$D_1 D_4$	6
			a_8	1010	$\bar{p}_1 \bar{p}_2$	y_2	$D_1 D_3$	7
a_4	0101	$y_8 y_{13}$	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	8
			a_7	1001	$\bar{p}_1 p_2$	$y_3 y_5$	$D_1 D_4$	9
			a_8	1010	$\bar{p}_1 \bar{p}_2$	y_2	$D_1 D_3$	10
a_5	0110	y_{10}	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	11
			a_7	1001	$\bar{p}_1 p_2$	$y_3 y_5$	$D_1 D_4$	12
			a_8	1010	$\bar{p}_1 \bar{p}_2$	y_2	$D_1 D_3$	13
a_6	1000	y_{11}	a_6	1000	$p_1 p_2$	y_2	D_1	14
			a_9	0001	$p_1 \bar{p}_2$	—	D_4	15
			a_{10}	0010	$\bar{p}_1$	$y_3 y_5 y_6$	D_3	16
a_7	1001	y_{13}	a_6	1000	$p_1 p_2$	y_2	D_1	17
			a_9	0001	$p_1 \bar{p}_2$	—	D_4	18
			a_{10}	0010	$\bar{p}_1$	$y_3 y_5 y_6$	D_3	19
a_8	1010	—	a_6	1000	$p_1 p_2$	y_2	D_1	20
			a_9	0001	$p_1 \bar{p}_2$	—	D_4	21
			a_{10}	0010	$\bar{p}_1$	$y_3 y_5 y_6$	D_3	22
a_9	0001	y_{11}	a_{11}	0011	1	$y_1 y_2 y_7$	$D_3 D_4$	23
a_{10}	0010	$y_8 y_{12}$	a_{11}	0011	1	y_1	$D_3 D_4$	24
a_{11}	0011	$y_5 y_{10}$	a_1	0000	1	—	—	25

In (3.9), the symbol R_V stands for the number of state variables in the system (3.5).

In the discussed case, there is $R = 2$, $H_P = 6$, $G = 2$. So, we have $S(M_V) = (6 + 4) \cdot 6 = 60$, $S(M_P) = 6 \cdot 2 = 12$ and $S(M_F) = 2(2 + 4) \cdot 25 = 300$. So, the matrix M_1 with $S(M_1) = 500$ is replaced by three matrices with total area equal to 372. It gives 25% of area saving.

In the case of CFSMs with average complexity [4], there is $L = 30$, $N = 50$, $H = 2000$, $R = 8$. Let we have $R_v = 6$, $H_P = 60$, $G = 4$. It gives $S_T = 2(30 + 8) \cdot 2000 + 2000(50 + 8) = 152000 + 116000 = 268000$ (convexional area units). At

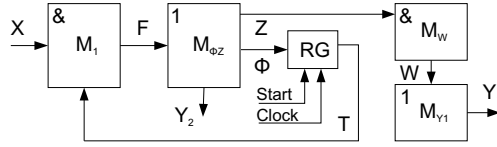


Fig. 3.7 Structural diagram of PY1 CFSM

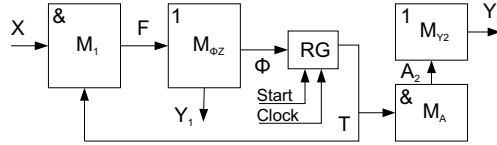


Fig. 3.8 Structural diagram of PY2 CFSM

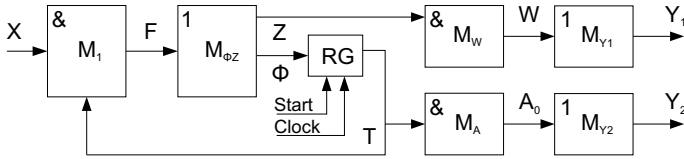


Fig. 3.9 Structural diagram of PY12 CFSM

the same time, there is $S(M_V) + S(M_P) + S(M_F) = 2520 + 240 + 48000 = 50760$. It gives saving of chip area equal to 71%.

In the case of RLC, the area saving is possible due to eliminating the dependence of functions $y_n \in Y$ and $D_r \in \Phi$ on variables $x_e \in X$. Instead of it, these functions depend on additional variables $p_g \in P$, where $|P| \ll |X|$. This example shows the entity of structural decomposition. In this case, there are primary functions ($y_n \in Y$, $D_r \in \Phi$) and primary arguments ($x_e \in X$), as well as secondary arguments ($p_g \in P$). The hardware reduction is achieved by replacing primary arguments in primary functions with secondary arguments. However, this assumes that there is an additional circuit that forms the secondary arguments as secondary functions from the primary arguments. This circuit can be viewed as an overhead necessary for optimization. Naturally, the use of any method of structural decomposition makes sense if the resulting savings are much greater than the overhead.

Now, let us discuss the methods of encoding of collections of microoperations (ECMO). There are two types of CMOs in CFSM: $Y_q \subseteq Y_1$ and $Y(a_m) \subseteq Y_2$. So, there are three different models of CFSM with ECMO.

1. **Encoding of CMOs $Y_q \subseteq Y_1$.** It is reduced to replacement of terms $F_h \in F$ by conjunctions Z_k of additional variables $z_r \in Z$ ($k = \overline{1, K}$). In this case, we have CFSM PY1 (Fig. 3.7).

In PY1 CFSM, the matrix $M_{\phi Z}$ implements systems (3.1), (3.3) and

$$Z = Z(F). \quad (3.12)$$

The matrix M_W implements conjunctions W_h corresponding to terms of the system (2.25). The matrix M_{Y1} implements the system (2.25) represented as

$$Y1 = Y1(W). \quad (3.13)$$

2. **Encoding of CMOs** $Y(a_m) \subseteq Y_2$. The functions $y_n \in Y_2$ depend on state variables $T_r \in T$. So, there is no need in additional variables. It leads to PY2 CFSM (Fig. 3.8).

Two additional matrices appear in this case. The matrix M_A implements H_M conjunctions A_0 for functions

$$Y2 = Y2(A_0). \quad (3.14)$$

The system (3.14) is implemented by the matrix M_{Y2} . The matrix M_Φ implements functions (3.1)–(3.2).

3. **Encoding of both types of CMOs.** It leads to PY12 CFSM (Fig. 3.9). The functions of matrices are clear from the previous text.

To synthesize the circuit of PY1 CFSM starting from a GSA Γ , it is necessary:

1. To mark the GSA Γ by states of Moore FSM.
2. To encode the states $a_m \in A$.
3. To encode the CMOs $Y_q \subseteq Y$.
4. To construct a DST of PY1 CFSM.
5. To construct the functions $F, \Phi, Y2, Z, Y1$.
6. To implement the matrix circuit using the functions from p. 5.

Let us discuss an example of synthesis for CFSM PY1(Γ_3). The steps 1 and 2 are already executed.

There are $Q = 7$ CMOs $Y_q \subseteq Y_1$ in the discusses case: $Y_1 = \emptyset$, $Y_2 = \{y_1\}$, $Y_3 = \{y_3, y_5, y_6\}$, $Y_4 = \{y_1, y_2, y_7\}$, $Y_5 = \{y_3, y_4, y_7\}$, $Y_6 = \{y_3, y_5\}$, and $Y_7 = \{y_2\}$. So, there is the following dependence among $y_n \in Y_1$ and CMOs $Y_q \subseteq Y_1$:

$$\begin{array}{llll} y_1 = Y_2 \vee Y_4 & y_2 = Y_4 \vee Y_7 & y_3 = Y_3 \vee Y_5 \vee Y_6 & y_4 = Y_5 \\ y_5 = Y_3 \vee Y_6 & y_6 = Y_3 & y_7 = Y_4 \vee Y_5. \end{array} \quad (3.15)$$

Let us encode the CMOs in a way minimizing the number of terms (and literals) in (3.15). It can be done using the methods from [1]. The codes $K(Y_q)$ are shown in Fig. 3.10.

Fig. 3.10 Codes of CMOs for PY1(Γ_3)

		$z_1 z_2$			
z_3		00	01	11	10
	0	Y_1	Y_2	*	Y_3
	1	Y_7	Y_4	Y_5	Y_6

Table 3.4 Content of column Z_h of CFSM PY1(Γ_3)

h	Z_h	h	Z_h	h	Z_h	h	Z_h
1	—	7	z_3	13	z_3	19	z_1
2	z_2	8	$z_1 z_2 z_3$	14	z_3	20	z_3
3	z_1	9	$z_1 z_3$	15	—	21	—
4	$z_2 z_3$	10	z_3	16	z_1	22	z_1
5	$z_1 z_2 z_3$	11	$z_1 z_2 z_3$	17	z_3	23	$z_2 z_3$
6	$z_1 z_3$	12	$z_1 z_3$	18	—	24	z_2

In Fig. 3.10 we use the variables $z_r \in Z$ where $|Z| = 3$. We found the value of R_Q from (1.32).

Using codes (Fig. 3.10) allows transforming (3.15) into the following system:

$$\begin{aligned}
 y_1 &= \bar{z}_1 z_2 = W_1; & y_2 &= \bar{z}_1 z_3 = W_2; & y_3 &= z_1; \\
 y_4 &= z_1 z_2 = W_3; & y_5 &= z_1 \bar{z}_2 = W_4; \\
 y_6 &= z_1 \bar{z}_3 = W_5; & y_7 &= z_2 z_3 = W_6.
 \end{aligned} \tag{3.16}$$

As follows from (3.16), there is the set $W = \{W_1, \dots, W_6\}$ with $H_W = 6$.

To construct the DST of CFSM PY1(Γ_3), it is necessary to transform Table 3.1. The column Y_h should be replaced by the column Z_h . In h -th row of DST, there are variables $z_r \in Z$ equal to 1 in the code $K(Y_q)$ of $Y_q \subseteq Y_1$ from the column Y_h of initial DST.

Let us show only the column Z_h of DST (Table 3.4). We show also the numbers of rows in Table 3.4.

Let us explain the filling of Table 3.4. For example, there is $Y_2 = \{y_1\}$ in row 2 of Table 3.1. There is $K(Y_2) = 010$ (Fig. 3.9). So, there is z_2 in row 2 of Table 3.4. Next, there is Y_5 in row 5 of Table 3.1. There is $K(Y_5) = 111$. So, there are symbols z, z_2, z_3 in row 5 of Table 3.4. And so on.

The DST of PY1 CFSM is the base for creating the system (3.12). For example, the following functions can be found:

$$\begin{aligned}
 z_1 &= F_3 \vee F_5 \vee F_6 \vee F_8 \vee F_9 \vee F_{11} \vee F_{12} \vee F_{16} \vee F_{19} \vee F_{22}; \\
 z_2 &= F_4 \vee F_8 \vee F_{11} \vee F_{23} \vee F_{24}; \\
 z_3 &= F_4 \vee F_5 \vee F_6 \vee \dots \vee F_{14} \vee F_{17} \vee F_{20} \vee F_{23}.
 \end{aligned} \tag{3.17}$$

Let us show the part of the circuit of PY1(Γ_3) for matrices $M_{\phi Z}$, M_W , M_{Y_1} (Fig. 3.11). As follows from Fig. 3.11, there is no matrix M_{Y_1} in the discussed case. It follows from the fact that each Eq. (3.16) is represented as a single term. Moreover, there is no term for MO y_3 . It is the same as z_1 (and it is represented by the wire z_1). The disappearance of M_{Y_1} is associated with the optimal encoding of CMOs $Y_q \subseteq Y$. So, this step is very important.

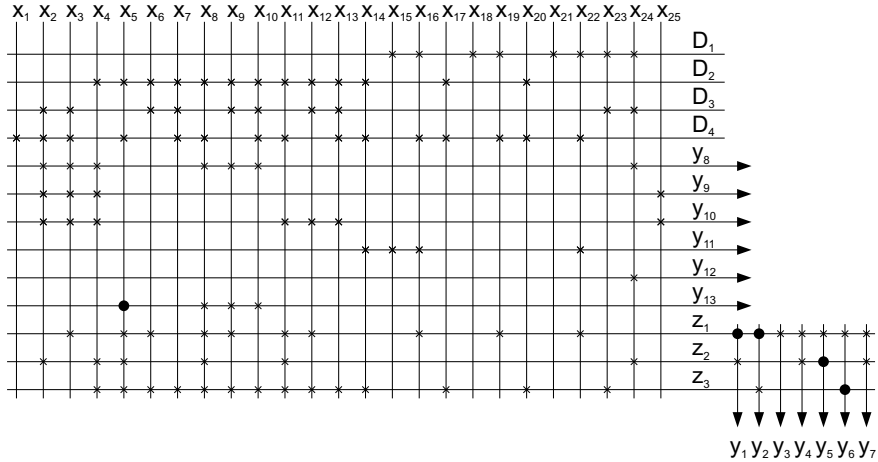


Fig. 3.11 Matrix realization

Let us estimate the complexity of the circuit shown in Fig. 3.11. Generally, it is represented by the formulae:

$$S(M_{\phi Z}) = H \cdot (R + R_Q + N2); \quad (3.18)$$

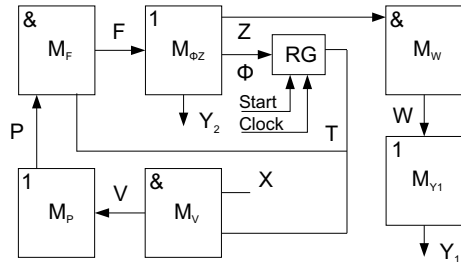
$$S(M_W) = 2R_Q \cdot H_W; \quad (3.19)$$

$$S(M_{Y1}) = H_W \cdot N_1. \quad (3.20)$$

In the discussed case, it is equal to $S(M_{\phi Z}) + S(M_W) = 25(4 + 3 + 6) + 3 \cdot 5 = 325 + 15 = 340$. There is $S(M_2) = 425$. So, using the ECMO gives economy equal to 24%. It is even more significant for complex CFSMs [4].

For a more significant reduction in the area of CFSM circuit, the considered methods can be combined. This generates the following models: MPY1, MPY2, MPY12. For example, the structural diagram of MPY1 CFSM is shown in Fig. 3.12. It can be viewed as a combining the diagrams shown in Figs. 3.4 and 3.7.

Fig. 3.12 Structural diagram of MPY1 CFSM



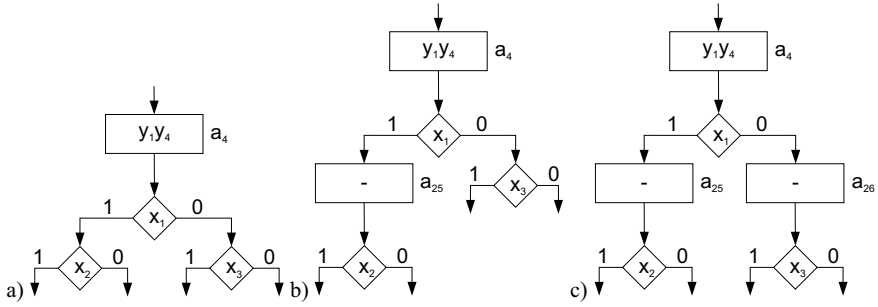


Fig. 3.13 A part of GSA Γ before **a** and after adding one **b** and two **c** operator vertices

The diagrams of MPY2 and MPY12 CFSMs can be obtained in the same manner. As a rule, the following rules have places [6]:

1. The total chip area is reduced with increasing for the numbers of circuit levels.
2. The performance of CFSM is reduced with increasing for the numbers of levels.
3. The less arguments an SBF has, the less the corresponding chip area is. The same is true for terms.

The RLC targets reducing the number of arguments in comparison with a trivial matrix realization. But the value of G is determined by features of the initial GSA Γ . This value can be reduced due to the transformation of initial GSA [5, 19]. Let us explain it.

There is $G_4 = 3$ for the state a_4 (Fig. 3.13a). It means $G \geq G_4$ and G cannot be less than 3.

Let it be $M = 24$. So, there is $R = 5$. Let it be $H = 120$. Let us introduce an operator vertex (Fig. 3.13b) between x_1 and x_2 . Now, $M = 25$ and $H = 121$. We have additional transition $\langle a_4, a_{25} \rangle$. Now, there are $|X(a_4)| = 2$ and $|X(a_{25})| = 1$. So, we have $G \leq G_4 = 2$. If we introduce the state a_{26} (Fig. 3.13c), it gives $M = 26$, $H = 122$ and—the most important— $G_4 = G_{25} = G_{26} = 1$.

So, we can change the value of G by introducing new operator vertices. It leads to increasing for numbers of states and rows of DST, as well as the latency time [4] of an algorithm to be executed. But it allows decreasing G from its maximum determined by an initial GSA to $G - 1, G - 2, \dots, 1$. It leads to $M_{G-1}P, M_{G-2}P, \dots, M_1P$ CFSMs.

Let us discuss an example of synthesis for CFSM $M_1P(\Gamma_3)$. The synthesis method includes a one additional step which is the transformation of initial GSA. In the discussed case, this step is executed in the trivial way.

Three additional operator vertices are introduced into GSA Γ_3 . It leads to the transformed GSA Γ_{3t} (Fig. 3.14). These vertices are marked by state a_{12}, a_{13} , and a_{14} . Now, we have $M = 14$ but there is the same value of $R = 4$.

As follows from GSA Γ_{3t} , it includes only states with $G_m \leq 1 (m = \overline{1, 24})$. So there is $P = \{p_1\}$. Because $G = 1$, the table of RLC is constructed in the trivial way (Table 3.5).

Table 3.5 Table of RLC of CFSM $M_1P(\Gamma_{3t})$

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}	a_{13}	a_{14}
p_1	—	x_1	x_3	x_3	x_3	x_5	x_5	x_5	—	—	—	x_2	x_4	x_6

The following equation can be derived from Table 3.5:

$$p_1 = x_1 A_2 \vee x_3 [A_3 \vee A_4 \vee A_5] \vee x_5 [A_6 \vee A_7 \vee A_8] \vee x_2 A_2 \vee x_4 A_{13} \vee x_6 a_{14}. \quad (3.21)$$

To minimize the number of terms H_V , let us encode states $a_m \in A$ as it is shown in Fig. 3.15. Using these state codes and law of expansion give the following equation for the function $p_1(T, X)$:

$$p_1 = \bar{T}_1 \bar{T}_2 T_4 x_1 \vee \bar{T}_1 T_2 x - 3 \vee t_1 \bar{t}_2 X_5 \vee \bar{T}_1 \bar{T}_2 T_3 x_2 \vee T_1 T_2 \bar{T}_3 x_4 \vee T_1 T_2 T_3 x_6 = V_1 + V_2 + \dots + V_6. \quad (3.22)$$

There is $H_V = L = 6$. It is the minimum possible value of H_V .

Now, there is $H = 24$ (Table 3.6). The difference with Table 3.3 is explained by adding the states a_{12}, a_{13}, a_{14} .

Obviously, there is the same structural diagram of M_1P CFSM as it is shown in Fig. 3.4. The area can be estimated using (3.9)–(3.11) and the formula for $S(M_2)$:

$$S(M_2) = H(R + N_1 + N_2). \quad (3.23)$$

In the discussed case, we have $S(M_V) = (6 + 7) \cdot 6 = 78$, $S(M_P) = 6 \cdot 1 = 6$, $C(M_F) = 2(1 + 4) \cdot 24 = 240$, $S(M_2) = 24(4 + 13) = 408$, $S_T = 732$.

In the case of $MP(\Gamma_3)$, we have $S(M_V) + S(M_P) + S(M_F) = 372$. Adding $S(M_2) = 25(4 + 13) = 425$, we have $S_T = 797$. It gives gain around 8%. Of course, the example is very simple.

Let us consider the system (3.16). The following condition takes place for each function (3.16):

$$Q(y_n) = 1. \quad (3.24)$$

Let us name such functions as decoder functions. There is a method allowing that (3.24) takes place for any function $y_n \in Y$. It is an encoding of fields of compatible microoperations (EFCMO) [2, 3].

This method was proposed in [18]. It was used in IBM 360 family [21]. Let us shortly explain this approach.

The MOs $y_n, y_m \in Y$ are compatible if they do not appear in the same CMOS $Y_q \subseteq Y$ [2]. Let us find the partition $\Pi_Y = \{Y^1, \dots, Y^K\}$ of the set Y by classes of compatible MOs. Let it be $N(Y^k) = |Y^k|$ for $Y^k \in \Pi_Y$. Let us encode MOs $y_n \in Y^k$ by binary codes $K(y_n)$ having R_k bits:

Fig. 3.14 Transformed GSA
 Γ_3

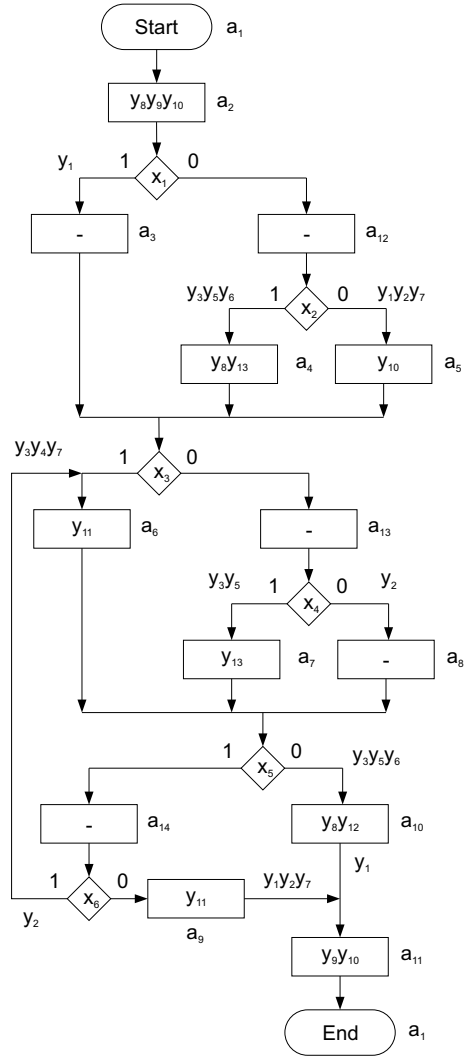


Fig. 3.15 State codes of
CFSM $M_1 P(\Gamma_{3t})$

		$T_1 T_2$			
$T_3 T_4$		00	01	11	10
	00	a_1	a_3	a_{13}	a_6
	01	a_2	a_4	a_9	a_7
	11	*	a_{11}	a_{10}	*
	10	a_{12}	a_5	a_{14}	a_8

Table 3.6 DST of CFSM $M_1 P(\Gamma_{3l})$

a_m	$K(a_m)$	$Y(a_m)$	a_S	$K(a_S)$	P_h	Y_h	Φ_h	h
a_1	0000	–	a_2	0001	1	–	D_4	1
a_2	0001	$y_8 y_9 y_{10}$	a_3	0100	p_1	y_1	D_2	2
			a_{12}	0010	$\bar{p}_1$	–	D_3	3
a_3	0100	–	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	4
			a_{13}	1100	$\bar{p}_1$	–	$D_1 D_3$	5
a_4	0101	$y_8 y_{13}$	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	6
			a_{13}	1100	$\bar{p}_1$	–	$D_1 D_2$	7
a_5	0110	y_{10}	a_6	1000	p_1	$y_3 y_4 y_7$	D_1	8
			a_{13}	1100	$\bar{p}_1$	–	$D_1 D_2$	9
a_6	1000	y_{11}	a_{14}	1110	p_1	–	$D_1 D_2 D_3$	10
			a_{10}	1111	$\bar{p}_1$	$y_3 y_4 y_5$	$D_1 D_2 D_3 D_4$	11
a_7	1001	y_{13}	a_{14}	1110	p_1	–	$D_1 D_2 D_3$	12
			a_{10}	1111	$\bar{p}_1$	$y_3 y_5 y_6$	$D_1 D_2 D_3 D_4$	13
a_8	1010	–	a_{14}	1010	p_1	–	$D_1 D_2 D_3$	14
			a_{10}	1111	$\bar{p}_1$	$y_3 y_5 y_6$	$D_1 D_2 D_3 D_4$	15
a_9	1101	y_{11}	a_{11}	0111	1	$y_1 y_2 y_7$	$D_2 D_3 D_4$	16
a_{10}	1111	$y_8 y_{12}$	a_{11}	0111	1	y_1	$D_2 D - 3D_4$	17
a_{11}	0111	$y_9 y_{10}$	a_1	0000	1	–	–	18
a_{12}	0010	–	a_4	0101	p_1	$y_3 y_5 y_6$	$D_2 D_4$	19
			a_5	0110	$\bar{p}_1$	$y_1 y_2 y_7$	$D_2 D_3$	20
a_{13}	1100	–	a_7	1001	p_1	$y_3 y_5$	$D_1 D_4$	21
			a_8	1010	$\bar{p}_1$	y_2	$D_1 D_3$	22
a_{14}	1110	–	a_6	1000	p_1	y_2	D_1	23
			a_9	1101	$\bar{p}_1$	–	$D_1 D_2 D_4$	24

$$R_k = \left\lceil \log_2(N(Y^k) + 1) \right\rceil \quad (k = \overline{1, K}). \quad (3.25)$$

It is necessary R_Y variables to encode MOs for all classes of Π_Y , where

$$R_Y = \sum_{k=1}^K R_k. \quad (3.26)$$

Let us use variables $z_r \in Z$ to encode the MOs, where $|Z| = R_k$. We use R_1 first variables to encode MOs $y_n \in Y^1$, next R_2 variables $z_r \in Z$ are used to encode MOs $y_n \in Y^2$, and so on. To optimize the final circuit, the following relation should take place:

$$R_y \rightarrow \min. \quad (3.27)$$

We can find a lot of articles from the 1970s devoted to this problem [15, 16, 20, 22]. Let us show that this approach can be used for matrix circuits of CFSMs.

Table 3.7 Codes of compatible MOs of S_7

MO	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	y_{10}	Codes of MOs
z_1	0	*	*	0	0	*	1	1	1	1	$K(y_n) \quad y_n \in Y^1$
z_2	0	*	*	1	1	*	0	0	1	1	
z_3	1	*	*	0	1	*	0	1	0	1	
z_4	*	0	1	*	*	1	*	*	*	*	$K(y_n) \quad y_n \in Y^2$
z_5	*	1	0	*	*	1	*	*	*	*	

Consider the following case. Let some CFSM S_7 have the set $Y_1 = \{y_1, \dots, y_{10}\}$ creating $Q = 8$ CMOs. They are the following: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_2\}$, $Y_3 = \{y_2, y_4\}$, $Y_4 = \{y_3, y_5\}$, $Y_5 = \{y_6, y_7\}$, $Y_6 = \{y_2, y_8\}$, $Y_7 = \{y_3, y_9\}$, $Y_8 = \{y_{10}\}$.

Using approach [11] gives the partition $\Pi_Y = \{Y^1, Y^2\}$, where $Y^1 = \{y_1, y_4, y_5, y_7, y_8, y_9, y_{10}\}$ and $Y^2 = \{y_2, y_3, y_6\}$. It gives $N(Y^1) = 7$, $N(Y^2) = 3$, $R_1 = 3$, $R_2 = 2$, $R_Y = 5$, $Z = \{z_1, \dots, z_5\}$. Let us use $z_1 - z_3$ to encode $y_n \in Y^1$, $z_4 - z_5$ to encode $y_n \in Y^2$. Let us use the codes 000 and 00 to show that there are no MO $y_n \in Y^1$ or $y_n \in Y^2$ in a particular CMO. The codes are shown in Table 3.7.

Now, we can represent each code $K(y_q)$ as a concatenation of codes of MOs. For example, there is $K(y_1) = 00000$, $K(y_2) = 00101, \dots$, $K(y_7) = 11010 = K(y_9) * K(y_3)$, $K(y_8) = 11100 = K(y_{10}) * K(\emptyset)$.

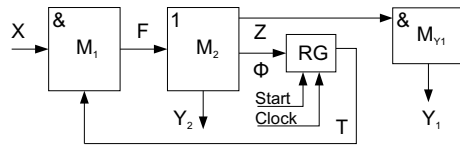
Using Table 3.7 gives the following system (2.25): $y_1 = \bar{z}_1 \bar{z}_2 z_3$, $y_2 = \bar{z}_4 z_5$, $y_3 = z_4 \bar{z}_5, \dots, y_{10} = z_1 z_2 z_3$. So, each equation from (2.25) is represented by a single term. So, there is no need in disjunctive matrix to implement the functions $y_n \in Y^1$.

As a rule, we use the symbol PD to denote the FSMs based on EFCMO. There is the matrix circuit of PD CFSM shown in Fig. 3.16.

This approach can be used together with other methods of structural decomposition. For example, there are MPD , M_2PY2D , $PY2D$ and other. We do not discuss these cases.

3.3 PES-based Matrix Circuits of Moore FSMs

The peculiarity of CFSM is that the initial GSA Γ is marked by states of Moore FSM. So, it is possible to reduce hardware using partition $\Pi_A = \{B_1, \dots, B_I\}$ of the set A by classes of PES [7]. There are three main optimization methods [7–9]:

Fig. 3.16 Matrix circuit of PD CFSM

1. **Optimal state assignment.** The classes $B_i \in \Pi_A$ are represented by intervals of R-dimensional Boolean space. In the best case, each class is represented by a single generalized interval. It is quite possible that only a part T' of the set $T(T' \subseteq T)$ enter terms $F_h \in F$. It means that

$$\Phi = \Phi(T', X). \quad (3.28)$$

In the best case, the number of terms in (3.28) is the same as for equivalent Mealy FSM [8]. In the case of matrix realization, it diminishes the number of inputs and outputs of the matrix M_1 implementing system

$$F = F(T', X). \quad (3.29)$$

The advantage of this approach is absence of additional variables used to encode the classes $B_i \in \Pi_A$. The drawback is reduced to the case when the optimal state assignment is not possible. The method [7] can be used for the optimal state assignment (OSA). It leads to P_0 Moore FSMs.

2. **Transformation of state codes into class codes.** We discuss this approach in Chap. 2. The classes $B_i \in \Pi_A$ are encoded by the class codes $K(B_i)$ having R_A (2.43) bits. To encode the classes, the additional variables $\tau_r \in \mathcal{T}$ are used ($|\mathcal{T}| = R_A$). It leads to PcY Moore FSM. Its structural diagram is shown in Fig. 3.17.

The matrix M_1 implements the system of terms

$$F = F(\mathcal{T}, X). \quad (3.30)$$

The matrix M_2 transforms terms $F_h \in F$ into input memory functions (3.1). The matrix M_3 implements terms A_0 depending on $T_r \in \mathcal{T}$:

$$A_0 = A_0(T). \quad (3.31)$$

The matrix M_4 implements the systems

$$Y = Y(A_0); \quad (3.32)$$

$$\mathcal{T} = \mathcal{T}(A_0). \quad (3.33)$$

Fig. 3.17 Structural diagram of matrix PcY Moore FSM

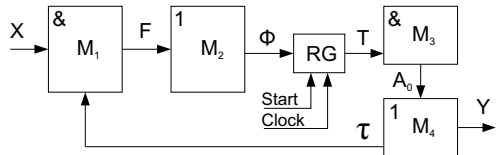
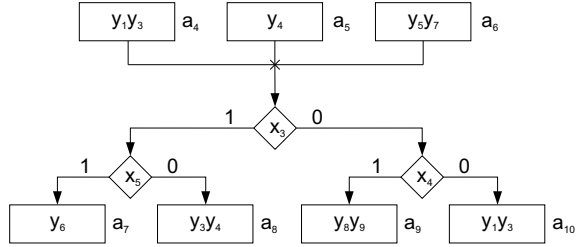


Fig. 3.18 Fragment of GSA Γ_E



Let the following condition take place:

$$R > R_A. \quad (3.34)$$

In this case, it is necessary less inputs in the matrix M_1 than in the case of optimal state assignment. Moreover, the number of outputs of M_1 is the same as for an equivalent Mealy FSM [7, 9]. These two issues are advantages of the transformation.

There are two negative effects of this approach. Firstly, it is impossible to produce the MOs $y_n \in Y$ as outputs of M_2 . For example, there is $K(B_2) = 011$ and $B_2 = \{a_2, a_3, a_4\}$. Let it be three transitions from each of states $a_m \in B_i$ determined by x_1 , $\bar{x}_1x_2$ and $\bar{x}_1\bar{x}_2$. If we find the disjunction $\bar{\tau}\tau_2\tau_3(x_1 \vee \bar{x}_1x_2 \vee \bar{x}_1\bar{x}_2)$, we get $\bar{\tau}\tau_2\tau_3$. So, we get the conjunctions corresponding to class codes. But to form $y_n \in Y$ by M_2 , we need conjunctions corresponding to state codes. Secondly, some area of M_4 is spent on producing variables $\tau_r \in \mathcal{T}$.

3. **Transformation of initial GSA Γ .** Consider the fragment of GSA Γ_E (Fig. 3.19). It corresponds to the fragment of DST (Table 3.8). Let it be $M = 14$. Let us use the trivial approach for state assignment.

Let us introduce the state a_{15} into Γ_E (Fig. 3.19). In this case, Table 3.9 represents the graph from Fig. 3.19.

There are 7 rows in Table 3.9. It gives economy in 58% on the number of DST rows compared to Table 3.8. Consequently, the areas of M_1 and M_2 will decrease to the same extent.

This approach leads to P_T Moore FSM. Its circuit includes two matrices, as it is in the case of P_0 Moore FSM. The matrix M_1 implements terms (1.10), the matrix M_2 functions $Y(F)$ and $\Phi(F)$. Let us find the gain in area due to transition from P to P_T FSMs.

Let I be the number of classes of PES. Each class $B_i \in \Pi_A$ has $|B_i|$ states, each state $a_m \in B_i$ has H_i transitions. The following formulae estimate area of matrices for P Moore FSM:

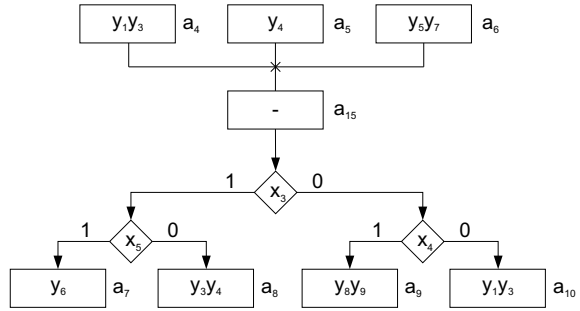
$$S(M_1) = 2(L + R) \cdot H; \quad (3.35)$$

$$S(M_2) = H(R + N); \quad (3.36)$$

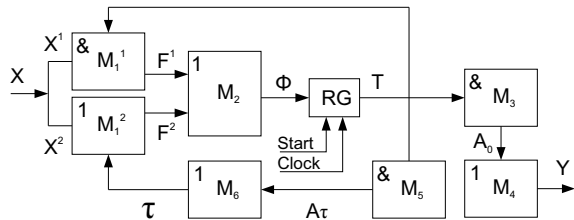
$$S_T(P) = H(N + 2L + 3R). \quad (3.37)$$

Table 3.8 Fragment of DST for Fig. 3.19

a_m	$K(a_m)$	$Y(a_m)$	a_S	$K(a_S)$	X_h	Φ_h	h
a_4	0011	$y_1 y_3$	a_7	0110	$x_3 x_5$	$D_2 D_3$	1
			a_8	0111	$x_3 \bar{x}_5$	$D_2 D_3 D_4$	2
			a_9	1000	$\bar{x}_3 x_4$	D_1	3
			a_{10}	1001	$\bar{x}_3 \bar{x}_4$	$D_1 D_4$	4
a_5	0100	y_4	a_7	0110	$x_3 x_5$	$D_2 D_3$	5
			a_8	0111	$x_3 \bar{x}_5$	$D_2 D_3 D_4$	6
			a_9	1000	$\bar{x}_3 x_4$	D_1	7
			a_{10}	1001	$\bar{x}_3 \bar{x}_4$	$D_1 D_4$	8
a_6	0101	$y_5 y_7$	a_7	0110	$x_3 x_5$	$D_2 D_3$	9
			a_8	0111	$x_3 \bar{x}_5$	$D_2 D_3 D_4$	10
			a_9	1000	$\bar{x}_3 x_4$	D_1	11
			a_{10}	1001	$\bar{x}_3 \bar{x}_4$	$D_1 D_4$	12

Fig. 3.19 Fragment of transformed GSA Γ_E **Table 3.9** Fragment of DST for Fig. 3.19

a_m	$K(a_m)$	$Y(a_m)$	a_S	$K(a_S)$	X_h	Φ_h	h
a_4	0011	$y_1 y_3$	a_{15}	1110	1	$D_1 D_2 D_3$	1
a_5	0100	y_4	a_{15}	1110	1	$D_1 D_2 D_3$	2
a_6	0101	$y_5 y_6$	a_{15}	1110	1	$D_1 D_2 D_3$	3
a_{15}	1110	-	a_7	0110	$x_3 x_5$	$D_2 D_3$	4
			a_8	0111	$x_3 \bar{x}_5$	$D_2 D_3 D_4$	5
			a_9	1000	$\bar{x}_3 x_4$	D_1	6
			a_{10}	1001	$\bar{x}_3 \bar{x}_4$	$D_1 D_4$	7

Fig. 3.20 Structural diagram of $P_{C1}Y$ Moore FSM

Let Δ_i be the gain in the number of rows due to the transformation. It is determined as [7]:

$$\Delta_i = |B_i| \cdot H_i - (|B_i| + H_i). \quad (3.38)$$

For example, there is $|B_i| = 3$, $H_i = 4$ for Γ_E . Using (3.38) gives $\Delta_i = 3 \cdot 4 - (3 + 4) = 5$. Comparison of Tables 3.8 and 3.9 confirms the correctness of the formulae (3.38).

The total area reduction can be found as

$$\Delta_T = \sum_{i=1}^I \Delta_i (N + 2L + 3R). \quad (3.39)$$

If the transformation leads to an increase in the parameter R , the gain Δ_T will decrease. We do not discuss this case.

We assume that the transformation is performed until the bitness of $K(a_m)$ changes. To do it, the algorithm [9] can be used. The meaning of this algorithm is as follows:

1. Find the number M_T of classes $B_i \in \Pi_A$ for which we can transform the fragments of initial GSA Γ :

$$M_T = 2^R - M. \quad (3.40)$$

2. Organize the queue of classes in the descending order of the value Δ_i .
3. Select the first M_T elements of the queue and perform the transformation of the corresponding fragment of GSA.

The main disadvantage of this method is the increase in the execution time of a control algorithm. Some transitions are not related to the execution of microoperations. Therefore, the method can be used if the main optimization criterion is to reduce the chip area.

In articles [12, 13], we propose some additional PES-based methods of optimization targeting matrix circuits. Let us discuss them.

1. **Partial transformation of states.** Let us encode states $a_m \in A$ in the optimal way [7]. Let us represent the partition Π_A as $\Pi_{A1} \cup \Pi_{A2}$, where $\Pi_{A1} \cap \Pi_{A2} = \emptyset$. The set Π_{A1} includes classes $B_i \in \Pi_A$ represented by a single interval of R -dimensional Boolean space. The set Π_{A2} includes classes $B_i \in \Pi_A$ required more than a single generalized interval. Let us encode classes $B_i \in \Pi_{A2}$ by binary codes $K(B_i)$ having R_{A2} bits:

$$R_{A2} = \lceil \log_2(|\Pi_{A2}| + 1) \rceil. \quad (3.41)$$

We add 1 to $|\Pi_{A2}|$ to encode the relation $B_i \notin \Pi_{A2}$.

Let us use variables $\tau_r \in \mathcal{T}$ to encode $B_i \in \Pi_{A2}$, where $|\tau| = R_{A2}$. The classes $B_i \in \Pi_{A1}$ are represented by variables $T_r \in \mathcal{T}$. Now, the set F can be represented

as $F^1 \cup F^2$, where $F^1 \cap F^2 = \emptyset$. The set F^1 includes product terms F_h represented as (1.10). The set F^2 is represented by (3.30). It is quite possible that terms $F_h \in F^1$ depend on variables $x_l \in X^1$ and $F_h \in F^2$ on variables $x_l \in X^2$. In general case, there is $X^1 \cap X^2 \neq \emptyset$.

It is necessary two matrices to implement these terms. It leads to $P_{C1}Y$ Moore FSM (Fig. 3.20).

In $P_{C1}Y$ FSM, the matrix M_1^1 implements terms (1.10) and M_1^2 terms (3.30). The matrices M_3 and M_4 implement microoperations (1.8). The matrix M_6 implements the system

$$\mathcal{T} = \mathcal{T}(A_\tau). \quad (3.42)$$

This approach allows diminishing the total number of terms up to H_0 (H_0 is the number of terms for equivalent Mealy FSM). At the same, time only a part of state codes should be transformed. It leads to a decrease in the area of state transformer compared to an equivalent $P_C Y$ Moore FSM.

In [13], there is proposed a synthesis method for $P_{C1}Y$ Moore FSM. It includes the following steps:

- a. Marking the initial GSA Γ and creating the set A .
- b. Constructing the partition $\Pi_A = \{B_1, \dots, B_I\}$.
- c. Executing the optional state assignment.
- d. Constructing sets Π_{A1} and Π_{A2} .
- e. Executing the class encoding for $B_i \in \Pi_{A2}$.
- f. Constructing DST for classes $B_i \in \Pi_{A1}$.
- g. Constructing DST for classes $B_i \in \Pi_{A2}$.
- h. Constructing a table for state transformer (STR).
- i. Implementing matrix circuit of FSM.

Let us consider an example of synthesis for Moore FSM represented by GSA Γ_4 (Fig. 3.21).

As follows from Fig. 3.21, there is $A = \{a_1, \dots, a_{15}\}$, $X = \{x_1, \dots, x_8\}$, $Y = \{y_1, \dots, y_{12}\}$. Because $M = 15$, there is $R = 4$. So, there are the sets $\Phi = \{D_1, \dots, D_4\}$ and $T = \{T_1, \dots, T_4\}$. Using definition of PES [7], we can find the partition $\Pi_A = \{B_1, \dots, B_6\}$ where $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6\}$, $B_4 = \{a_7, a_8, a_9\}$, $B_5 = \{a_{10}, a_{11}, a_{12}\}$ and $B_6 = \{a_{13}, a_{14}, a_{15}\}$.

Let us encode the states $a_m \in A$ in the optimal way (Fig. 3.22).

Using Fig. 3.22 gives the class codes $K(B_i)$ for classes $B_i \in \Pi_{A1}$. They are the following: $K(B_1) = 0000$, $K(B_2) = XX10$, $K(B_3) = 00X1$, $K(B_4) = 10XX$. There is the set $\Pi_{A2} = \{B_5, B_6\}$. So, it is necessary to transform codes of states $a_{10}, \dots, a_{15}$.

There is $|\Pi_{A2}| = 2$. Using (3.41) gives $R_{A2} = 2$ and $\mathcal{T} = \{\tau_1, \tau_2\}$. Let it be $K(B_5) = X1$ and $K(B_6) = 1X$. We use the code 00 to show that $B_i \notin \Pi_{A2}$.

There are $H^1 = 14$ rows in DST for classes $B_i \in \Pi_{A1}$ (Table 3.10). There are $H^2 = 7$ rows in DST for $B_i \in \Pi_{A2}$ (Table 3.11). Table 3.10 is the base to get terms of a system

$$F = F(T, X) \quad (3.43)$$

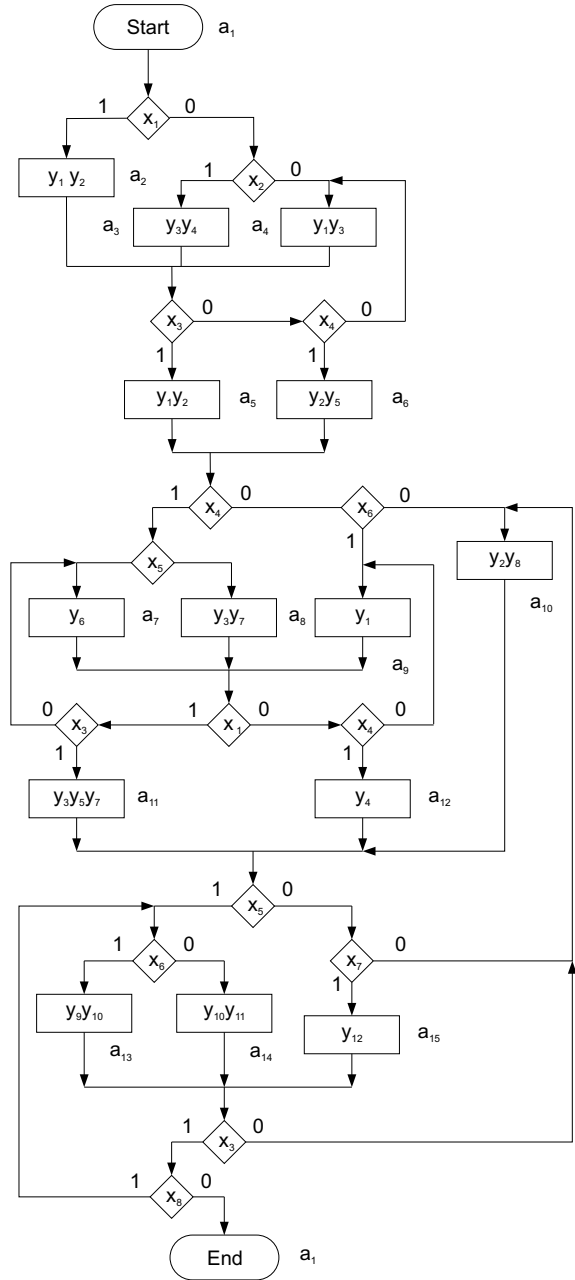
Fig. 3.21 Marked GSA Γ_4 

Fig. 3.22 Optimal state codes for $P(\Gamma_4)$

		$T_3 T_4$			
		00	01	11	10
$T_1 T_2$	00	a_1	a_5	a_6	a_2
	01	a_{10}	a_{11}	a_{12}	a_3
	11	a_{13}	a_{14}	a_{15}	a_4
	10	a_7	a_8	a_9	*

Table 3.10 Direct structure table for Π_{A1}

B_i	$K(B_i)$	a_S	$K(a_S)$	X_h	Φ_h	h
B_1	0000	a_2	0010	x_1	D_2	1
		a_3	0110	$\bar{x}_1 x_2$	$D_2 D_3$	2
		a_4	1110	$\bar{x}_1 \bar{x}_2$	$D_1 D_2 D_3$	3
B_2	XX10	a_5	0001	x_3	D_4	4
		a_6	0011	$\bar{x}_3 x_4$	$D_3 D_4$	5
		a_4	1110	$\bar{x}_3 \bar{x}_4$	$D_1 D_2 D_3$	6
B_3	00X1	a_7	1000	$x_4 x_5$	D_1	7
		a_8	1001	$x_4 \bar{x}_5$	$D_2 D_4$	8
		a_9	1011	$\bar{x}_4 x_6$	$D_1 D_3 D_4$	9
		a_{10}	0100	$\bar{x}_4 \bar{x}_6$	D_2	10
B_4	10XX	a_{11}	0101	$x_1 x_3$	$D_2 D_4$	11
		a_7	1000	$x_1 \bar{x}_3$	D_1	12
		a_{12}	0111	$\bar{x}_1 x_4$	$D_2 D_3 D_4$	13
		a_9	1011	$\bar{x}_1 \bar{x}_4$	$D_1 D_3 D_4$	14

and functions (3.1). Table 3.11 is the base to get terms from the system (3.30) and functions (3.1). Tables 3.10 and 3.11 are based on systems of generalized formulae of transitions (GFT) [8]. Because of it, there is no need in the initial DST of PY Moore FSM.

The table of STR includes the following columns: a_m , $K(a_m)$, B_i , $K(B_i)$, τ_m , m . The column τ_m includes variables $\tau_r \in \mathcal{T}$ equal to 1 in the code $K(B_i)$ of the class $B_i \in \Pi_{A2}$ such that $a_m \in B_i$. There are $M_{A2} = 6$ rows in the table of STR for the discussed case (Table 3.12).

To find the system (1.8), it is necessary to construct table of MOs (Table 3.13). It is formed using the initial GSA Γ in the trivial way.

Using Table 3.10 gives terms $F_h \in F^1$. For example, the following terms can be found: $F_1 = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 x_1$, $F_5 = T_3 \bar{T}_4 \bar{x}_3 \bar{x}_4$, $F_{14} = T_1 \bar{T}_2 \bar{x}_1 \bar{x}_4$. Using these terms, we can find functions $D_r \in \Phi$. For example, there is $D_1 = F_3 \vee F_6 \vee F_7 \vee F_8 \vee F_9 \vee F_{12} \vee F_{14}$.

Using Table 3.11 gives terms $F_h \in F^2$. For example, the following terms can be found: $F_{15} = \tau_2 x_5 x_6$, $F_{18} = \tau_2 \bar{x}_5 \bar{x}_7$, $F_{21} = \tau_1 \bar{x}_3$. These terms form functions (2.44). For example, we can derive the function $D_4 = F_{16} \vee F_{17}$.

Table 3.11 Direct structure table for Π_{A2}

B_i	$K(B_i)$	a_S	$K(a_S)$	X_h	Φ_h	h
B_5	X1	a_{13}	1100	x_5x_6	D_1D_2	15
		a_{14}	1101	$x_5\bar{x}_6$	$D_1D_2D_4$	16
		a_{15}	1111	$\bar{x}_5x_7$	$D_1D_2D_3D_4$	17
		a_{10}	0100	$\bar{x}_5\bar{x}_7$	D_2	18
B_6	1X	a_7	1000	x_3x_8	D_1	19
		a_1	0000	$x_3\bar{x}_8$	–	20
		a_{10}	0100	$\bar{x}_3$	D_2	21

Table 3.12 Table of STR of Moore FSM $P_{C1}Y(\Gamma_4)$

a_m	$K(a_m)$	B_i	$K(B_i)$	τ_m	m
a_{10}	0100	B_5	*1	τ_2	1
a_{11}	0101	B_5	*1	τ_2	2
a_{12}	0111	B_5	*1	τ_2	3
a_{13}	1100	B_6	1*	τ_1	4
a_{14}	1101	B_6	1*	τ_1	5
a_{15}	1111	B_6	1*	τ_1	6

Table 3.13 Table of MOs of Moore FSM $P_{C1}Y(\Gamma_4)$

a_m	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}
$Y(a_m)$	y_1y_2	y_3y_4	y_1y_3	y_1y_2	y_2y_5	y_6	y_3y_7	y_1	y_2y_6	$y_3y_5y_7$	y_4	y_9y_{10}	$y_{10}y_{11}$	y_{12}

The table of STR is used to derive functions (3.42). In the discussed case, it is possible to optimize them:

$$\begin{aligned}\tau_1 &= T_1T_2\bar{T}_3 \vee T_1T_2T_4 = F_{22} \vee F_{23}; \\ \tau_2 &= \bar{T}_1T_2\bar{T}_3 \vee \bar{T}_1T_2T_4 = F_{24} \vee F_{25}.\end{aligned}\tag{3.44}$$

The terms $F_{22} - F_{25}$ are generated by the matrix M_5 having $S(M_5) = 5 \times 4 = 20$. The functions (3.44) are generated by the matrix M_6 having $S(M_6) = 4 \cdot 2 = 8$. So, it is necessary 28 units of area to implement the circuit of STR for Moore FSM $P_{C1}Y(\Gamma_4)$. Here we have $H_\tau = 4$.

In the case of $P_C Y(\Gamma_4)$, there is $R_A = 3$ and $H_\tau = 12$. There is $S(M_5) = 8 \cdot 12 = 96$ and $S(M_6) = 12 \cdot 3 = 36$. So, it is necessary 232 units of area to implement the STR. So, our approach allows saving around 78% of area for implementing the STR circuit.

- Non-standard presentation of state codes.** Let classes $B_i \in \Pi_A$ be encoded using R_A variables $\tau_r \in \mathcal{T}$. Let it be Q different CMOs $Y_q \subseteq Y$ in the initial GSA Γ . Let us encode them using R_Q variables $z_r \in \mathcal{Z}$. The value of R_A is determined by (2.43), the value of R_Q by (1.32).

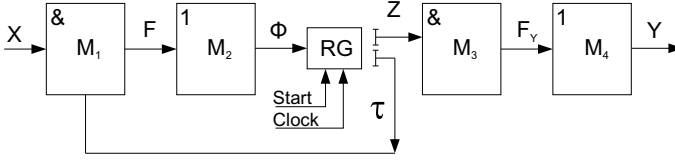


Fig. 3.23 Structural diagram of $P_K Y$ Moore FSM

Let an operator vertex b_i of GSA Γ correspond to a state $a_m \in A$ such that $a_m \in B_i$. Let $Y(a_m) = Y_q$. In this case, the code $K(a_m)$ can be represented by a concatenation of the codes $K(B_i)$ and $K(Y_q)$ [14].

$$K(a_m) = K(B_i) * K(Y_q). \quad (3.45)$$

The asterisk in (3.45) stands for the concatenation of the codes.

This approach leads to $P_K Y$ Moore FSM. Its matrix circuit is shown in Fig. 3.23. In $P_K Y$ FSM, the matrix M_1 implements terms (3.30), the matrix M_2 implements system (3.1). First R_A bits of RG keep variables $\tau_r \in T$; next R_Q bits represent codes $K(Y_q)$. The matrix M_3 implements the terms of the system

$$F_Y = F_Y(Z). \quad (3.46)$$

The matrix M_4 implements the system (2.25).

The main advantage of this approach is the possibility of independent mutual optimization of blocks of functions (BF) and microoperations (BMO). The BF includes matrices M_1 and M_2 , the BMO M_3 and M_4 . The minor drawback of $P_K Y$ FSMs is the increase in the number of outputs of M_2 compared to equivalent PY Moore FSM. But the method makes it possible to guarantee to reduce the number of inputs (from H to H_0) of M_1 . At the same time, the number of inputs of M_2 is decreased from H to H_0 .

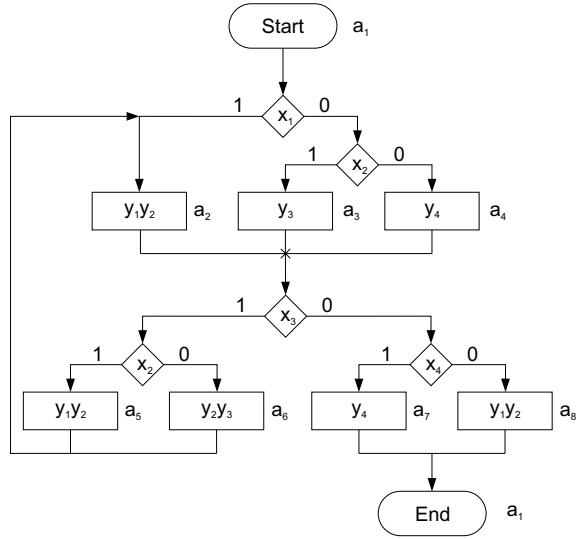
There are the following steps in synthesis method of $P_K Y(\Gamma)$ [12]:

- a. Marking the GSA Γ and creating the set A .
- b. Creating the partition Π_A for the set A .
- c. Encoding of classes of PES $B_i \in \Pi_A$.
- d. Encoding of CMOs $Y_q \subseteq Y$.
- e. Constructing the table of BF.
- f. Constructing the table of BMO.
- g. Creating the matrix circuit of $P_K Y$ FSM.

Let us discuss an example of design of $P_K Y(\Gamma_5)$. The marked GSA Γ_5 is shown in Fig. 3.24.

There is $M = 8$. These states are distributed among the $I = 4$ classes of PES: $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6\}$ and $B_4 = \{a_7, a_8\}$.

Using (2.43) gives $R_A = 2$ and $T = \{\tau_1, \tau_2\}$. Let us encode the classes $B_i \in \Pi_A$ using the frequency approach [4]: the more states a class includes, the more

Fig. 3.24 Marked GSA Γ_5 **Fig. 3.25** Codes of CMOs for $P_K Y(\Gamma_5)$

$z_1 z_2$		00	01	11	10
z_3	0	Y_1	Y_2	*	Y_4
	1	Y_3	Y_5	*	*

zeros its code has. It gives the codes $K(B_1) = 11$, $K(B_2) = 00$, $K(B_3) = 01$, and $K(B_4) = 10$.

There are $Q = 5$ CMOs $Y_q \subseteq Y$ in the discussed case. They are the following: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_2\}$, $Y_3 = \{y_3\}$, $Y_4 = \{y_4\}$, and $Y_5 = \{y_1, y_3\}$. Using (1.32) gives $R_Q = 3$ and $Z = \{z_1, z_2, z_3\}$. Let us encode them in the way [1] (Fig. 3.25). This method allows minimizing the number of terms H_y in functions (2.25). Using contents of CMOs and their codes (Fig. 3.25) gives the following results:

$$\begin{aligned} y_1 = Y_2 \vee Y_5 = z_2; & \quad y_2 = Y_2 = z_2 \bar{z}_3; \\ y_3 = Y_3 \vee Y_5 = z_3; & \quad y_4 = Y_4 = z_1. \end{aligned} \quad (3.47)$$

So, there is $H_Y = 1$ in the discussed case.

There is $R_A + R_Q = 5 > R = 3$. The codes $K(a_m)$ are the following: $K(a_1) = 11000$, $K(a_2) = 00010$, $K(a_3) = 00001$, $K(a_4) = 00100$, $K(a_5) = 01010$, $K(a_6) = 01011$, $K(a_7) = 10100$, $K(a_8) = 10010$. These codes are created using (3.45), codes $K(B_i)$ and codes $K(Y_q)$ from Fig. 3.25.

To construct the table of BF, it is necessary to form the system of GFT. It is the following in the discussed case:

$$\begin{aligned} B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 x_2 a_3 \vee \bar{x}_1 \bar{x}_2 a_4; & B_3 &\rightarrow a_2; \\ B_2 &\rightarrow x_3 x_2 a_5 \vee x_3 \bar{x}_2 a_6 \vee \bar{x}_3 x_4 a_7 \vee \bar{x}_3 \bar{x}_4 a_8; & B_4 &\rightarrow a_1. \end{aligned} \quad (3.48)$$

Using (3.48) allows constructing the table of BF (Table 3.14). Its columns follow from the system (3.48).

There is no need in the table of BMO in the discussed case. The system (3.47) represents both matrices M_3 and M_4 .

Table 3.14 is a base to construct matrices M_1 and M_2 . The system (3.47) gives data for constructing matrices M_3 and M_4 . Now, we can obtain the matrix circuit of Moore FSM $P_K Y(\Gamma_5)$ (Fig. 3.26).

We use RG1 to keep codes $K(B_i)$. Because $K(B_i) = 11$, the S input of RG1 is connected with $Start$. The RG2 keeps codes $K(Y_q)$. Because $Y(a_1) = Y_1$ having $K(Y_1) = 000$, the R input of RG2 is connected with $Start$. Both registers have the shared pulse $Clock$. Concatenation of RG1 and RG2 forms the RG from Fig. 3.23.

As you can see, there is no matrix M_3 in the discussed case, as well as $S(M_4) = 2$. There in $S(M_1) = 12 \cdot 9 = 108$ and $S(M_2) = 9 \cdot 5 = 45$. So, the circuit of $P_K Y(\Gamma_5)$ needs $S_T = 155$.

3. **Expansion of code space.** The main idea of this method [12] is reduced to representing class codes $K(B_i)$ by variables $T_r \in T'$, where $T' \subseteq T$. It does not change the FSM structure compared to PY Moore FSM (Fig. 3.27).

If $|T'| < |T|$, let us denote such an FSM by the symbol $P_E Y$ (“E” stands for “expansion”). The system F is represented as (3.29), the system Φ as (3.28).

Such an approach guarantees the reduction of the number of terms in the system Φ from H to H_0 . It leads to reducing area of matrix M_1 and the number of inputs of matrix M_2 . The disadvantage of this method is connected with its targeting only the block BF’s optimization.

Let us represent the coding space by a Karnaugh map having I_K columns, where

$$I_K = 2^{R_A}. \quad (3.49)$$

Let $|B_i| = m_i$ and $M_0 = \max(m_1, \dots, m_l)$. It is necessary 2^{R_0} cells of Karnaugh map to keep any of classes $B_i \in \Pi_A$. It means that each class $B_i \in \Pi_A$ be represented by a single generalized interval of coding space. The value of R_0 is determined as

$$R_0 = \lceil \log_2 M_0 \rceil. \quad (3.50)$$

If $R_A + R_0 > R$, then the expansion of code space takes place. The value of R_0 can be decreased, if

$$I_K > I. \quad (3.51)$$

In this case, the states $a_m \in B_i$ can be placed in adjacent columns of the map. For example, there is $I = 4$, $R_A = 2$, $M_0 = 3$, $R_0 = 2$ for Moore FSM $P_Y(\Gamma_5)$. So, it is necessary $R_A + R_0 = 4$ state variables to encode the states in the desired manner. Because $R = 3$, there is the expansion of state codes in the discussed case. Let us encode states $a_m \in B_i$ as it is shown in Fig. 3.28.

There is $T' = \{T_1, T_2\}$. These variables form the following class codes: $K(B_1) = 00, \dots, K(B_4) = 11$. It means that there are $H = 9$ rows in the table of BF of

Table 3.14 Table of BF of Moore FSM $P_K Y(\Gamma_5)$

B_i	$K(B_i)$	a_S	$K(a_S)$	X_h	Φ_h	h
B_1	11	a_2	00010	x_1	D_4	1
		a_3	00001	$\bar{x}_1 x_2$	D_5	2
		a_4	00100	$\bar{x}_1 \bar{x}_2$	D_3	3
B_2	00	a_5	01010	$x_3 x_2$	$D_2 D_4$	4
		a_6	01011	$x_3 \bar{x}_2$	$D_2 D_4 D_5$	5
		a_7	10100	$\bar{x}_3 x_4$	$D_1 D_3$	6
		a_8	10010	$\bar{x}_3 \bar{x}_4$	$D_1 D_4$	7
B_3	01	a_2	00010	1	D_4	8
B_4	10	a_1	11000	1	$D_1 D_2$	9

$P_E Y(\Gamma_5)$ (as it is for $P_K Y(\Gamma_5)$). But both matrices M_3 and M_4 are present in the case of $P_E Y(\Gamma_5)$. We can show that there is $H_Y = 7$, $S(M_3) = 2(R_A + R_0) \cdot H_Y = 56$, $S(M_4) = H_Y \cdot N = 28$. So, it is necessary 84 area units to implement the system of MOs.

We can use the refined state assignment (RSA) [12] to reduce this area. Using this approach, we can change the placement of states inside the same column. Obviously, we cannot change the column for a particular state. Using [1] allows creating the Karnaugh map (Fig. 3.29).

Now, we can find that:

$$\begin{aligned}
 y_1 &= A_2 \vee A_5 \vee A_6 \vee A_8 = T_4; \\
 y_2 &= A_2 \vee A_5 \vee A_8 = \bar{T}_3 T_4 \vee T_2 T_4 = F_{10} \vee F_{11}; \\
 y_3 &= A_3 \vee A_6 = T_2 \bar{T}_3 \bar{T}_4 \vee \bar{T}_2 T_3 = F_{12} \vee F_{13}; \\
 y_4 &= A_4 \vee A_7 = T_3 \bar{T}_4 = F_{14}.
 \end{aligned} \tag{3.52}$$

The system (3.52) determines the matrix circuit of BMO (Fig. 3.30). Now, there is $S(M_3) = 6 \cdot 5 = 30$, $S(M_4) = 5 \cdot 3 = 15$, $S(M_3, M_4) = 45$. The MO y_1 is generated as the output of RG. So, using the RSA gives saving in area equal to 46.5% (for M_3 and M_4).

All discussed methods can be used in the case of CFSM. Naturally, they should be adapted to the peculiarities of CFSM. Let us discuss possible models of ASIC-based CFSMs.

3.4 Analysis of CFSM-based Matrix Circuits

Let us construct Table 3.15. It is based on previous material and represents various possible models of ASIC-based CFSMs.

The first column of Table 3.15 corresponds to the RLC. Due to the transformation of the initial GSA, the number of variables $p_g \in P$ can be changed from 1 to (2.7).

Fig. 3.26 Matrix realization of Moore FSM $P_K Y(\Gamma_5)$

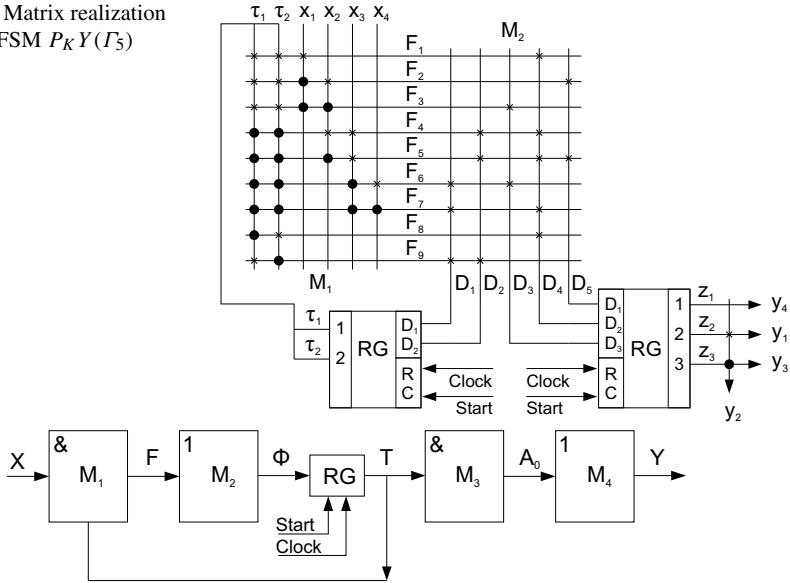


Fig. 3.27 Structural diagram of PY Moore FSM

Fig. 3.28 State codes for Moore FSM $P_E Y(\Gamma_5)$

		$T_1 T_2$			
$T_3 T_4$		00	01	11	10
	00	a_1	a_2	a_7	a_5
	01	*	a_3	a_8	a_6
	11	*	*	*	*
	10	*	a_4	*	*

The second column of Table 3.15 includes different models of P-circuit. Here P is a trivial circuit (Fig. 3.31); P_0 corresponds to optimal state assignment; P_C corresponds to the transformation of state codes into class codes (Fig. 3.17); P_T stands for transformation of initial GSA Γ ; P_{C1} means that only some part of state codes is transformed (Fig. 3.20); P_K corresponds to Moore FSM with nonstandard presentation of state codes (Fig. 3.23); P_E means that there is the expansion of code space (Fig. 3.27). It gives 7 different approaches.

The third column reflects methods of optimization circuits for MOs. In the case of Mealy FSM, it can be either encoding of the CMOs (Y1) or EFCMO (D1). In the case of Moore FSM, it is the circuit corresponding to (3.22).

Now let us analyse how many models of CFSM circuits could be produced from Table 3.15.

1. **Single-level circuits.** There are only 3 single-level models: P , P_0 , P_T . In all other cases, the class codes are used and variables $\tau_r \in \mathcal{T}$ enter the matrix M_1 . So, it is necessary to use the form (3.32) for $y_n \in Y_2$.

Fig. 3.29 Outcome of refined state assignment

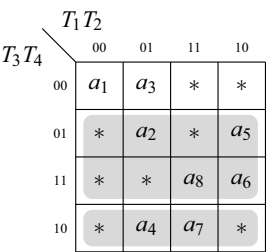


Fig. 3.30 Matrix circuit of BMO

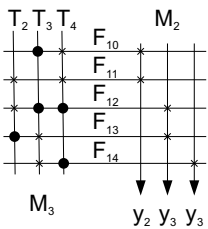


Table 3.15 Models of ASIC-based CFSMs

<i>I</i>	<i>II</i>	<i>III</i>	
		Mealy	Moore
M_1	P	$Y1$	$Y2$
M_2	P_0	$D1$	
$\cdot$	P_C		
$\cdot$	P_T		
M_{G-1}	P_{C1}		
M	P_K		
	P_E		

Fig. 3.31 Structural diagram of M_2P_CD1Y2 CFSM

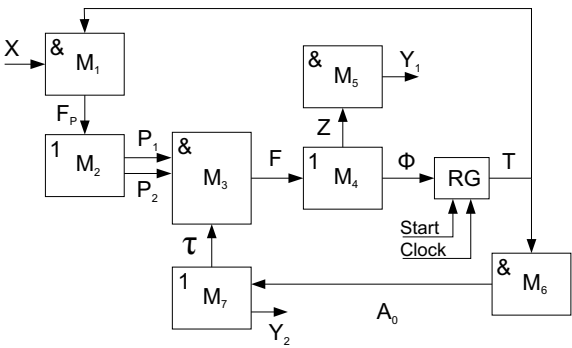
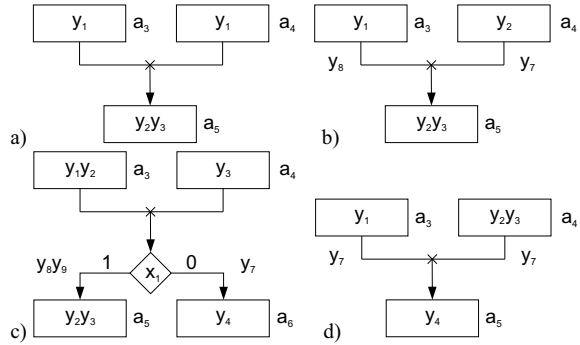


Fig. 3.32 To the definition of PES for CFSMs



2. **Two-level circuits.** There are 4 models connected with (3.32): P_C , P_{C1} , P_K and P_E . It is possible to combine them with either ECMO or EFCMO. It gives 18 models connected with encoding of MOs.

Combining models from columns I and II gives the CFSMs with replacement of logical conditions. To get a two-level model with RLC, it is necessary to use only P , P_0 or P_T models having a single-level. It gives 36 two-level models.

So, there are $18+3G$ different two-level circuits of CFSMs. For example, there are the following two-levels models: $PY1$, P_C , $PD1Y2$, MP , MP_C (and others).

3. **Three-level circuits.** These models combine all approaches listed in Table 3.15.

For example, there is an $M_2P_C D1Y2$ model of CFSM shown in Fig. 3.31.

We hope that this structure naturally follows from the formula $M_2P_C D1Y2$. The part “ M_2 ” shows that there is RLC and a GSA Γ should be transformed to get the set $P = \{p_1, p_2\}$. So, there are terms $F_h = F_k(p_1, p_2, T)$. The existence of T follows from “ P_C ”. Because of “ $D1$ ”, the method of EFCMO is used for MOs $y_n \in Y1$. The symbol “ $Y2$ ” shows that there is the transformation of states $a_m \in A$ into MOs $y_n \in Y2$. Also, there is the transformation of state codes into class codes as follows from “ P_C ”.

To get three-level models, it is necessary to use the RLC in two-level models. There are $(18 + 3G) \cdot G = 18G + 3G^2$ of three-level models.

So, there are $3 + 18 + 3G + 18G + 3G^2 = 21 + 21G + 3G^2$ different models of ASIC-based CFSMs. If there is $G = 4$ for a GSA Γ , then there are 153 different models of CFSMs. Some of them are discussed in Chap. 4.

Before proceeding to these methods, let us clarify the definition of PES. Let us use Fig. 3.32 for the clarification.

Let us start from Fig. 3.32a. The following relations take places for a_3 and a_4 : $\delta(a_3) = \delta(a_4) = a_5$, $\lambda_1(a_3) = \lambda_1(a_4) = \{y_1\}$, $\lambda_2(a_3) = \lambda_2(a_4) = \emptyset$. So, these states are just equivalent [4]. They should be replaced by a single state.

The following relations take places for states shown in Fig. 3.32b. They are the following: $\delta(a_3) = \delta(a_4)$, $\lambda_1(a_3) \neq \lambda_1(a_4)$ ($y_7 \neq y_8$), $\lambda_2(a_3) = \lambda_2(a_4)$. These states are not equivalent despite the presence of the same CMOs in the corresponding operator vertices. They are not pseudoequivalent, too, because $\lambda_1(a_3) \neq \lambda_1(a_4)$. So, in the case of CFSM, states a_3 and a_4 belong to different classes of PES. It

means that the set Π_A has more elements in the case of CFSM than in the case of equivalent Moore FSM.

As follows from Fig. 3.32c, states a_3 and a_4 are in the following relations: $\delta(a_3, X_h) = \delta(a_4, X_h)$, $\lambda_1(a_3, X_h) = \lambda_1(a_4, X_h)$ ($X_h \in \{x_1, \bar{x}_1\}$). These states create a single class of PES for CFSM.

The same is true for states a_3, a_4 from Fig. 3.32d. Only the following condition takes place: $X_h = 1$. So, states $a_3, a_4 \in B_i$.

Now, we can formulate a definition of PES for CFSMs.

Definition 3.1 States $a_m, a_s \in A$ are PES if and only if the following conditions take places:

$$\delta(a_m, X_h) = \delta(a_s, X_h) \quad (h = \overline{1, H}); \quad (3.53)$$

$$\lambda_1(a_m, X_h) = \lambda_1(a_s, X_h) \quad (h = \overline{1, H}). \quad (3.54)$$

The presence of the condition (3.54) is the peculiarity of PES for CFSM. The condition (3.53) is common for both Moore FSM and CFSM.

References

1. Achasova S (1987) Synthesis algorithms for automata with PLAs. M: Soviet radio (in Russian)
2. Agerwala T (1976) Microprogram optimization: a survey. IEEE Trans Comput 25(10):962–973
3. Agrawala A, Rauscher T (1976) Foundations of Microprogramming. Academic Press, New York
4. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
5. Baranov S, Skliarov V (1986) Digital devices with programmable LSIs with matrix structure. Radio and Sviaz (in Russian)
6. Barkalov A (1995) Multilevel PLA schemes for microprogram automata. Cybern Syst Anal 31(4):489–495
7. Barkalov A (1998) Principles of logic optimization for Moore microprogram automaton. Cybern Syst Anal 34(1):54–60
8. Barkalov A, Titarenko L (2009) Logic synthesis for FSM-based control units, vol 53. Lecture Notes in Electrical Engineering. Springer-Verlag, Berlin
9. Barkalov A, Titarenko L (2009) Synthesis of operational and control automata. UNITECH, Donetsk
10. Barkalov A, Titarenko L, Barkalov A Jr (2012) Structural decomposition as a tool for the optimization of an FPGA-based implementation of a Mealy FSM. Cybern Syst Anal 48(2):313–322
11. Barkalov A, Titarenko L, Chmielewski S (2007) Optimization of logic circuit of Moore FSM on CPLD. Pomiary Automat Kontrola 53(5):18–20
12. Barkalov A, Titarenko L, Hebda O (2011) Hardware reduction for matrix circuit of control moore automaton. In: Proceedings of IEEE East-West Design & Test Symposium EWDTS 2011. Kharkov National University of Radio Electronics, Sevastopol, Ukraina, pp 94–97
13. Barkalov A, Titarenko L, Hebda O (2011) Synthesis of moore finite state machine with transformation of extended state codes. Pomiary Automat Kontrola 57(6):652–655

14. Barkalov A, Titarenko L, Mielcarek K, Chmielewski S (2020) Logic synthesis for FPGA-based control units—structural decomposition in logic design, vol 636. Lecture Notes in Electrical Engineering, Springer, Berlin
15. Cong J, Yan K (2000). Synthesis for fpgas with embedded memory blocks. In: *Proceedings of the 2000 ACM/SIGDA Eighth International Symposium on Field Programmable Gate Arrays*, FPGA '00. ACM, New York, NY, USA, pp 75–82
16. Guan X, Yang B, Chen C, Dai W, Wang Y (2016) A comprehensive overview of cyber-physical systems: from perspective of feedback system. *IEEE/CAA J Automat Sinica* 3(1):1–14
17. McCluskey E (1986) Logic design principles. Prentice Hall, Englewood Cliffs
18. Schwartz S (1968) An algorithm for minimizing read-only memories for machine control. In: *IEEE 10th Annual Symposium on Switching and Automata Theory*, pp 28–33
19. Skliarov V (1984) Synthesis of automata with matrix LSIs. Nauka i Technika, Minsk (in Russian)
20. Solovjov V (2001) Design of digital systems basing on programmable logic integrated circuits. Hotline-Telecom, Moscow, Moscow (in Russian)
21. Tucker S (1967) Microprogram control for system/360. *IBM Syst J* 6(4):222–241
22. Wolf W (2004) FPGA-based system design. Prentice Hall PTR, Upper Saddle River, NJ, USA

Chapter 4

Optimization of ASIC-based CFSMs



Abstract The Chapter is devoted to reduction of ASIC chip areas occupied by CFSM logic circuits. All methods are based on using the classes of PES. It starts from optimization of CFSM with optimal state assignment. The optimization is achieved due to replacement of logical conditions, encoding of collections of microoperations of both Mealy and Moore types. Next, these methods are applied for CFSMs with complete transformation of states. Also, the method of encoding of the fields of compatible microoperations is used for the hardware reduction. Next, these methods are used for optimisation CFSMs with partial transformation of states. The last parts of the Chapter are devoted to optimization based on expansion of code space and nonstandard representation of state codes. The examples of synthesis are given for majority of proposed CFSM models.

4.1 CFSMs with Optimal State Assignment

The same rule applies to all PES-based CFSMs: a disjunction of terms corresponding to transition from the state $a_m \in B_i$ produces the code $K(B_i)$. So, it is impossible to produce MOs $y_n \in Y2$ by the matrix M_1 (Fig. 3.1). It is necessary to use matrices M_A and M_{Y2} to generate $y_n \in Y2$. From this rule follows the simplest structural diagram of CFSM (Fig. 4.1).

The matrix M_1 produces terms (3.28), the matrix M_2 functions (3.1)–(3.2). The matrix M_A produces terms from the set F_{Y2} :

$$F_{Y2} = F_{Y2}(T). \quad (4.1)$$

The matrix M_{Y2} implements functions

$$Y2 = Y2(F_{Y2}). \quad (4.2)$$

Let us consider the CFSM $P(\Gamma_6)$. The marked GSA Γ_6 is shown in Fig. 4.2.

Let us construct Table 4.1 reflecting some sets and their parameters extracted from Fig. 4.2.

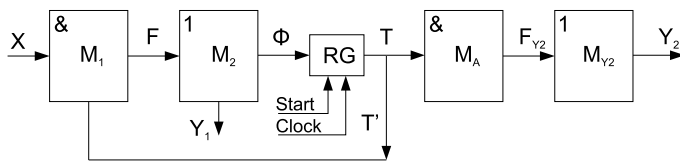


Fig. 4.1 Structural diagram of P_0Y_2 CFSM

Fig. 4.2 Marked GSA Γ_6

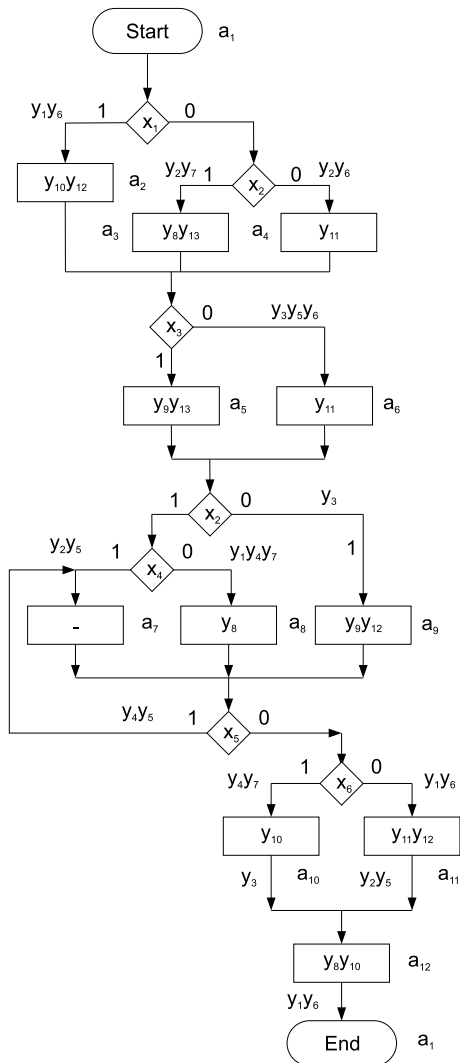


Table 4.1 Characteristics of CFSM $P(\Gamma_6)$

a_m	$X(a_m)$	$Y(a_m)$	H_m	Y_q	Parameters
a_1	$\{x_1, x_2\}$	$\emptyset$	3	$Y_1 = \emptyset$	$M = 12 \quad R = 4$
a_2	$\{x_3\}$	$\{y_{10}, y_{12}\}$	2	$Y_2 = \{y_1, y_6\}$	$L = 6 \quad G = 2$
a_3	$\{x_3\}$	$\{y_8, y_{13}\}$	2	$Y_3 = \{y_2, y_7\}$	$N_1 = 7 \quad Q = 10$
a_4	$\{x_3\}$	$\{y_{11}\}$	2	$Y_4 = \{y_2, y_6\}$	$N_2 = 6 \quad R_Q = 4$
a_5	$\{x_2, x_4\}$	$\{y_9, y_{13}\}$	3	$Y_5 = \{y_3, y_5, y_6\}$	$N = 13$
a_6	$\{x_2, x_4\}$	$\{y_{11}\}$	3	$Y_6 = \{y_2, y_5\}$	$H = 27$
a_7	$\{x_5, x_6\}$	$\emptyset$	3	$Y_7 = \{y_1, y_4, y_7\}$	$H_0 = 13$
a_8	$\{x_5, x_6\}$	$\{y_8\}$	3	$Y_8 = \{y_3\}$	$I = 7 \quad R_A = 3$
a_9	$\{x_5, x_6\}$	$\{y_9, y_{12}\}$	3	$Y_9 = \{y_4, y_5\}$	$S(H_L) = 540$
a_{10}	$\emptyset$	$\{y_{10}\}$	1	$Y_{10} = \{y_4, y_7\}$	$S(H_2) = 459$
a_{11}	$\emptyset$	$\{y_{11}, y_{12}\}$	1	–	$S_T = 999$
a_{12}	$\emptyset$	$\{y_8, y_{10}\}$	1	–	–

The column H_m contains the number of transitions from the state $a_m \in A$. Using it, we find the values of $H = 27$ (for CFSM) and $H_0 = 13$ (for the equivalent Mealy FSM). As follows from Table 4.1, there are the following sets: $A = \{a_1, \dots, a_{12}\}$, $X = \{x_1, \dots, x_6\}$, $Y_1 = \{y_1, \dots, y_7\}$, $Y_2 = \{y_8, \dots, y_{13}\}$, $P = \{p_1, p_2\}$, $Z = \{z_1, \dots, z_4\}$.

There are matrices M_1 and M_2 in the trivial circuit of P CFSM. We can find their areas as following:

$$S(M_1) = 2(L + R) \cdot H; \quad (4.3)$$

$$S(M_2) = H \cdot (N + R). \quad (4.4)$$

Using Table 4.1 and formulae (4.3)–(4.4) gives $S(M_L) = 2(6 + 4) \cdot 27 = 540$, $S(M_2) = 27 \cdot (13 + 4) = 459$. It gives the total area required by the circuit of $P(\Gamma_6)$ equal to $S_T = 999$ (units of area).

In this Chapter, we propose the method of synthesis for P_0Y_2 CFSM. It includes the following steps:

1. Marking the initial GSA Γ .
2. Creating the partition $\Pi_A = \{B_1, \dots, B_I\}$.
3. Executing the optimal state assignment (OSA).
4. Creating the DST of P_0Y_2 CFSM.
5. Creating the system $Y_2 = Y_2(T)$.
6. Implementing the matrix circuit of CFSM.

Let us discuss an example for synthesis of CFSM $P_0Y_2(\Gamma_6)$. The first step is already executed.

To find the set Π_A , we should use the Definition 3.1. As follows from it, states a_{10} and a_{11} should create the separate classes of Π_A .

Fig. 4.3 Outcome of OSA
for CFSM $P_0Y2(\Gamma_6)$

		T_1T_2			
		00	01	11	10
T_3T_4	00	a_1	a_2	a_{10}	a_7
	01	*	a_3	a_{12}	a_8
	11	a_5	*	*	a_9
	10	a_6	a_4	a_{11}	*

Analysis of GSA Γ_6 leads to the partition $\Pi_A = \{B_1, \dots, B_7\}$ with $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6\}$, $B_4 = \{a_7, a_8, a_9\}$, $B_5 = \{a_{10}\}$, $B_6 = \{a_{11}\}$, and $B_7 = \{a_{12}\}$. Let us encode the states $a_m \in A$ in the optimal way (Fig. 4.3).

The following class codes can be obtained from the Karnaugh map (Fig. 4.3): $K(B_1) = 000*$, $K(B_2) = 01**$, $K(B_3) = 001*$, $K(B_4) = 10**$, $K(B_5) = 1100$, $K(B_6) = 111*$, and $K(B_7) = 11*1$. So, each class $B_i \in \Pi_A$ is represented by a single generalized interval of 4-dimensional Boolean space. It is the best possible outcome of OSA [2, 3].

To construct the DST, let us form the system of GFT for the discussed case:

$$\begin{aligned}
 B_1 &\rightarrow x_1a_2 \vee \bar{x}_1x_2a_3 \vee \bar{x}_1\bar{x}_2a_4; \\
 B_2 &\rightarrow x_3a_5 \vee \bar{x}_3a_6; \\
 B_3 &\rightarrow x_2x_4a_7 \vee x_2\bar{x}_4a_8 \vee \bar{x}_2a_5; \\
 B_4 &\rightarrow x_5a_7 \vee \bar{x}_5x_6a_{10} \vee \bar{x}_5\bar{x}_6a_{11}; \\
 B_5, B_6 &\rightarrow a_{12}; \quad B_7 \rightarrow a_1.
 \end{aligned} \tag{4.5}$$

The system (4.5) determines the DST of $P_0Y2(\Gamma_6)$. It includes $H = 14$ rows (Table 4.2). It is equal to $H_0 + 1$, in the discussed case.

Using the column $Y(a_m)$ from Table 4.1 gives the system $Y2(A)$. Using state codes (Fig. 4.3) gives the system $Y2(T)$. It is the following:

$$\begin{aligned}
 y_8 &= A_3 \vee A_8 \vee A_{12} = \bar{T}_3T_4 = F_{15}; \\
 y_9 &= A_5 \vee A_9 = T_3T_4 = F_{16}; \\
 y_{10} &= A_2 \vee A_{10} \vee A_{12} = \bar{T}_2\bar{T}_3\bar{T}_4 \vee T_1T_2\bar{T}_3 = F_{17} \vee F_{18}; \\
 y_{11} &= A_4 \vee A_6 \vee A_{11} = T_3\bar{T}_4 = F_{19}; \\
 y_{12} &= A_9 \vee A_{11} \vee A_2 = T_1T_3 \vee \bar{T}_1T_2\bar{T}_3\bar{T}_4 = F_{20} \vee F_{21}; \\
 y_{13} &= A_3 \vee A_5 = \bar{T}_1\bar{T}_3 = F_{22}.
 \end{aligned} \tag{4.6}$$

The system (4.6) determines the set $F_{Y2} = \{F_{15}, \dots, F_{22}\}$. It gives $H_{Y2} = 8$. There are $H = 14$ terms in systems $\Phi(T, X)$ and $Y1(T, X)$. There are $2(R + L)$ inputs in the matrix M_1 . So, there is no saving in the number of inputs due to OSA (in the discussed case).

Using Table 4.2 and system (4.6) allows obtaining the matrix circuit of CFSM $P_0Y2(\Gamma_6)$ shown in Fig. 4.4. The system $\langle M_1, M_2 \rangle$ is determined by Table 4.2, the system $\langle M_A, M_{Y2} \rangle$ by the system (4.6).

Let us estimate the chip area occupied by this circuit:

Table 4.2 DST of CFSM $P_0Y2(\Gamma_6)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	000*	a_2	0100	x_1	y_1y_6	D_2	1
		a_3	0101	$\bar{x}_1x_2$	y_2y_7	D_2D_4	2
		a_4	0110	$\bar{x}_1\bar{x}_2$	y_2y_6	D_2D_3	3
B_2	01**	a_5	0011	x_3	–	D_3D_4	4
		a_6	0010	$\bar{x}_3$	$y_3y_5y_6$	D_3	5
B_3	001*	a_7	1000	x_2x_4	y_2y_5	D_1	6
		a_8	1001	$x_2\bar{x}_4$	$y_1y_4y_7$	D_1D_4	7
		a_9	1011	$\bar{x}_2$	y_3	$D_1D_3D_4$	8
B_4	10**	a_7	1000	x_5	y_4y_5	D_1	9
		a_{10}	1100	$\bar{x}_5x_6$	y_4y_7	D_1D_2	10
		a_{11}	1110	$\bar{x}_5\bar{x}_6$	y_1y_6	$D_1D_2D_3$	11
B_5	1100	a_{12}	1101	1	y_3	$D_1D_2D_4$	12
B_6	111*	a_{12}	1101	1	y_2y_5	$D_1D_2D_4$	13
B_7	11*1	a_1	0000	1	y_1y_6	–	14

Table 4.3 Table of RLC

$P\Pi_A$	B_1	B_2	B_3	B_4	B_5	B_6	B_7
p_1	x_1	x_3	x_4	x_5	–	–	–
p_2	x_2	–	x_2	x_6	–	–	–
$K(B_i)$	000*	01**	001*	10**	1100	111*	11*1

$$\begin{aligned}
S(M_L) &= 2(6 + 4) \cdot 14 = 280; \\
S(M_2) &= 14(7 + 4) = 154; \\
S(M_A) &= 2 \cdot 4 \cdot 8 = 64; \\
S(M_{Y_2}) &= 4 \cdot 2 = 8; \\
S_T &= S(M_1) + S(M_2) + S(M_A) + S(M_{Y_2}) = 506.
\end{aligned} \tag{4.7}$$

Let us point out that we used so named refined state assignment (RSA) [4, 5]. It targets simultaneous optimization systems of IMFs Φ and outputs of Moore-type $y_n \in Y_2$. Due to it, the matrix M_{Y_2} is so small. Moreover, four MOs $y_n \in Y_2$ are generated as outputs of the matrix M_A . We use the approach [1] to execute RSA.

Let us use the RLC to optimize the circuit of P_0Y2 CFSM. It results in MP_0Y2 CFSM (Fig. 4.5).

We hope that the purpose of each matrix is clear from the previous text. There are additional steps in the synthesis process of MP_0Y2 CFSM. They are clear from the example of synthesis of CFSM $MP_0Y2(\Gamma_6)$.

Let us use the same state codes as shown in Fig. 4.3. As follows from Table 4.1, there is $G = 2$ and $P = \{p_1, p_2\}$. Let us form the table of RLC (Table 4.3).

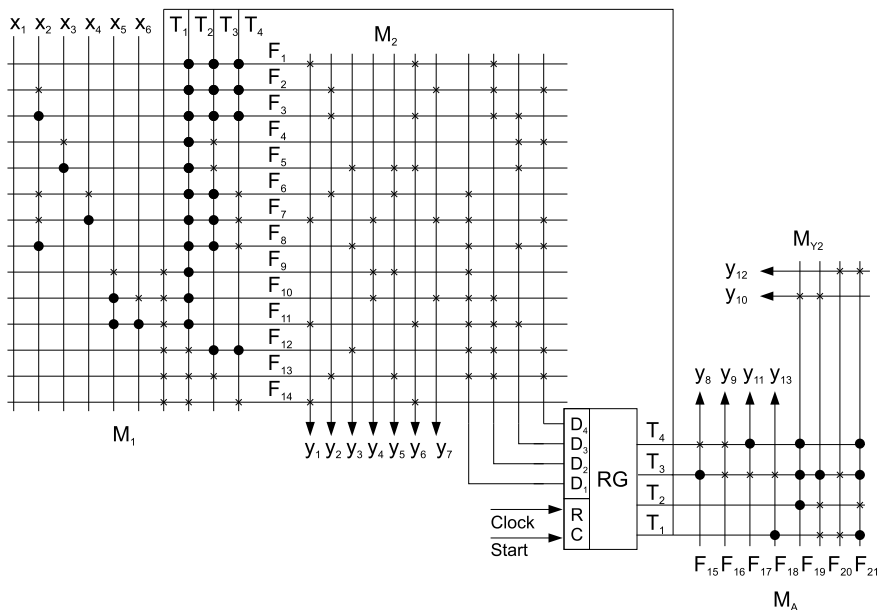
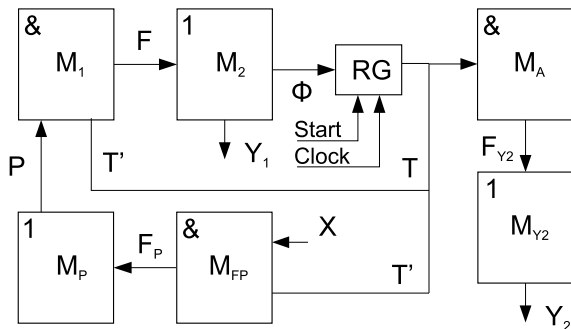


Fig. 4.4 Matrix circuit of CFSM $P_0Y2(I_6)$

Fig. 4.5 Structural diagram of MP_0Y2 CFSM

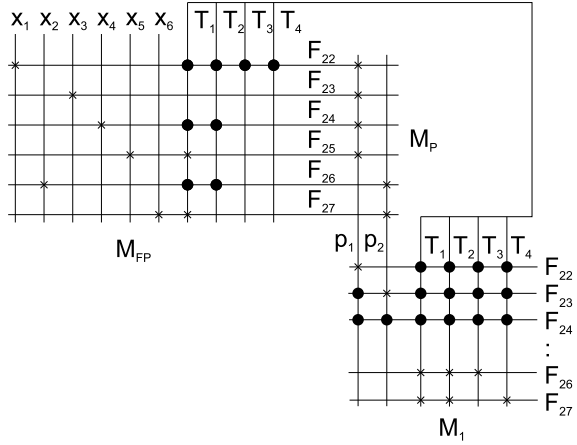


Because transitions from each state $a_m \in B_i$ depend on the same LCs, we execute the distribution of LCs for classes $B_i \in \Pi_A$. If transitions do not depend on LCs for some class $B_i \in \Pi_A$, then the code $K(B_i)$ can be used for minimization of functions $p_g \in P$.

Using this rule, we can get the following system:

$$\begin{aligned} p_1 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 x_1 \vee T_2 x_3 \vee \bar{T}_1 \bar{T}_2 T_3 x_4 \vee T_1 x_5 = F_{22} \vee F_{23} \vee F_{24} \vee F_{25}; \\ p_2 &= \bar{T}_1 \bar{T}_2 x_2 \vee T_1 x_6 = F_{26} \vee F_{27}. \end{aligned} \quad (4.8)$$

Fig. 4.6 A part of matrix circuit of $MP_0Y2(\Gamma_6)$ CFSM



There are exactly $L = 6$ terms in the system (4.8). It is the minimum possible result [6].

To get the DST of MP_0Y2 CFSM, it is necessary to replace the LCs $x_e \in X$ by variables $p_g \in P$. In the discussed case, let us execute the RLC for the system (4.9).

$$\begin{aligned}
 B_1 &\Rightarrow p_1a_2 \vee \bar{p}_1p_2a_3 \vee \bar{p}_1\bar{p}_2a_4; \\
 B_2 &\Rightarrow p_1a_5 \vee \bar{p}_1a_6; \\
 B_3 &\Rightarrow p_2p_1a_7 \vee p_2\bar{p}_1a_8 \vee \bar{p}_2a_9; \\
 B_4 &\Rightarrow p_1a_7 \vee \bar{p}_1p_2a_{10} \vee \bar{p}_1\bar{p}_2a_{12}; \\
 B_5, B_6 &\Rightarrow a_{12}; \\
 B_7 &\Rightarrow a_1.
 \end{aligned} \tag{4.9}$$

Obviously, there is the same number of terms ($H=14$) in both DSTs of $P_0Y2(\Gamma_6)$ and $MP_0Y2(\Gamma_6)$ CFSMs. It is true for any pair of equivalent CFSMs P_0Y2 and MP_0Y2 . The same is true for matrices M_A and M_{Y2} (because the RLC does not changes state codes and, therefore, the functions $y_n \in Y2$). Due to it, let us show only a part of matrix circuit corresponding to matrices M_{FP} , M_P and M_I of the $MP_0Y2(\Gamma_6)$. It is shown in Fig. 4.6.

Let us estimate the chip area for this circuit: $S(M_{FP}) = (6 + 6) \cdot 6 = 72$; $S(M_P) = 6 \cdot 2 = 12$; $S(M_I) = 2(2 + 4) \cdot 14 = 168$. It gives 252 area units for M_{FP} , M_P and M_I . Adding $S(M_2) + S(M_A) + S(M_{Y2})$ gives 478 units of area.

Now, let us apply the methods of ECMOs and RLC to $P_0Y2(\Gamma_6)$. Generally, it gives the model MP_0Y1Y2 shown in Fig. 4.7. The purpose of matrices is clear.

Let us discuss how to synthesize the pair of matrices M_{FY1} and M_{Y1} . As follows from Table 4.1, there are $Q = 10$ different CMOs $Y_q \subseteq Y1$ in the discussed case. Using (1.32) gives $R_Q = 4$ and $Z = \{z_1, \dots, z_4\}$. Let us encode the CMOs $Y_q \subseteq Y1$ in a way optimizing the number of terms in the system

Fig. 4.7 Structural diagram of MP_0Y1Y2 CFSM

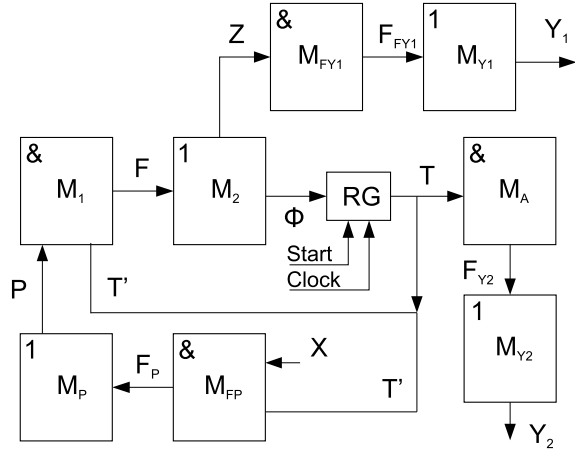


Fig. 4.8 Codes of CMOs for CFSM $MP_0Y1Y2(\Gamma_6)$

	$z_1 z_2$			
$z_3 z_4$	00	01	11	10
00	Y_1	Y_2	*	Y_5
01	*	Y_4	*	Y_8
11	Y_3	*	Y_6	*
10	Y_{10}	Y_7	*	Y_9

$$Y1 = Y1(F_{Y1}). \quad (4.10)$$

Using approach [1] gives codes $K(Y_q)$ shown in Fig. 4.8. Using the codes (Fig. 4.8) and contents of CMOs $Y_q \subseteq Y1$ (Table 4.1) gives the following SBF:

$$\begin{aligned}
 y_1 &= Y_2 \vee Y_7 = z_2 z_3 = F_{28}; \\
 y_2 &= Y_3 \vee Y_4 \vee Y_6 = z_1 z_4 \vee z_2 z_4 = F_{29} \vee F_{30}; \\
 y_3 &= Y_5 \vee Y_8 = z_1 \bar{z}_3 = F_{31}; \\
 y_4 &= Y_7 \vee Y_9 \vee Y_{10} = z_3 \bar{z}_4 = F_{32}; \\
 y_5 &= Y_5 \vee Y_6 \vee Y_9 = z_1 z_2 \vee z_1 \bar{z}_4 = F_{33} \vee F_{34}; \\
 y_6 &= Y_2 \vee Y_4 \vee Y_5 = z_2 \bar{z}_3 \vee z_1 \bar{z}_3 \bar{z}_4 = F_{35} \vee F_{36}; \\
 y_7 &= Y_3 \vee Y_7 \vee Y_{10} = \bar{z}_1 z_3 = F_{37}.
 \end{aligned} \quad (4.11)$$

There are $H_{Y1} = 10$ terms in the system (4.11). Let us point out that MOs $y_1, y_3, y_4, y_7 \in Y1$ are generated directly by M_{FY1} . So, only three MOs are generated by M_{Y1} . There are 6 inputs in this matrix.

Let us estimate the area required for $MP_0Y1Y2(\Gamma_6)$. Obviously, there are the same areas required for M_{FP}, M_P, M_1, M_A and M_{Y2} for equivalent MP_0Y2 and MP_0Y1Y2 automata. There are $R_Q + R$ outputs for M_2 . It gives $S(M_2) = H(R + R_Q) = 14 \cdot (4 + 4) = 112$. We can find $S(M_{FY1}) = 2 \cdot R_Q \cdot H_{Y1} = 8 \cdot 10 = 80$ and $S(M_{Y1}) = 6 \cdot 3 = 18$. It gives $S_T = 534$.

Let us denote as $S_T[\text{model}(\Gamma_j)]$ the area of an CFSM having some model and synthesized using GSA Γ_j . Let us compare CFSMs $P_0Y2(\Gamma_6)$, $MP_0Y2(\Gamma_6)$ and $MP_0Y1Y2(\Gamma_6)$. The follow relation takes place: $S_T[MP_0Y1Y2(\Gamma_6)] > S_T[P_0Y2(\Gamma_6)] > S_T[MP_0Y2(\Gamma_6)]$.

So, there is no gain in area due to encoding of CMOs (in the discussed case). The best result is obtained for $MP_0Y2(\Gamma_6)$.

4.2 CFSMs with Transformation of State Codes

The transformation of state codes leads to P_CY2 CFSM (Fig. 4.9).

In P_CY2 CFSM, the matrix M_1 implements terms F_h from the system (3.30). The matrix M_2 implements functions (3.1) and (3.2). The matrix M_A implements terms $F_h \in F_{Y2\tau}$ determining functions (4.2) and:

$$\tau = \tau(F_{Y2\tau}). \quad (4.12)$$

In this chapter, we propose the synthesis method for $P_CY2(\Gamma_j)$. It includes the following steps:

1. Marking the initial GSA Γ and creating the set A .
2. Creating the partition $\Pi_A = \{B_1, \dots, B_l\}$.
3. Executing the combined state assignment (CSA).
4. Executing the encoding of classes $B_i \in \Pi_A$.
5. Creating the DST of CFSM.
6. Implementing CFSM matrix circuit.

Let us discuss an example of synthesis for CFSM $P_CY2(\Gamma_6)$. The first two steps are already executed.

The aim of CSA is a joint optimization of SBFs (4.2) and (4.12). In the discussed case, we can use the state codes from Fig. 4.3.

Let us encode the classes $B_i \in \Pi_A$ in the following manner:

1. Codes corresponding to numbers from zero onwards are assigned to classes including states with conditional transitions.
2. If $X(a_m) \cap X(a_s) \neq \emptyset$ and $a_m \in B_i$ and $a_s \in B_j$, then classes B_i and B_j should have adjacent codes. Of course, this rule is used if $X(a_m) \neq \emptyset$ and $X(a_s) \neq \emptyset$.

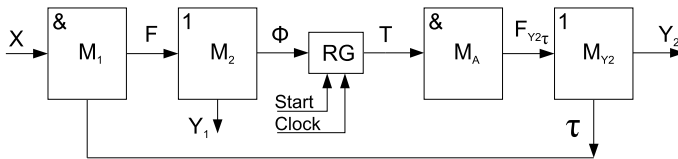


Fig. 4.9 Structural diagram of P_CY2 CFSM

Fig. 4.10 Class codes for $P_C Y2(\Gamma_6)$

		$\tau_1 \tau_2$			
		00	01	11	10
τ_3	0	B_1	B_3	B_2	B_4
	1	B_5	B_6	*	B_7

Table 4.4 DST of CFSM $P_C Y2(\Gamma_6)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	000	a_2	0100	x_1	$y_1 y_6$	D_2	1
		a_3	0101	$\bar{x}_1 x_2$	$y_2 y_7$	$D_2 D_4$	2
		a_4	0110	$\bar{x}_1 \bar{x}_2$	$y_2 y_6$	$D_2 D_3$	3
B_2	110	a_5	0011	x_3	—	$D_3 D_4$	4
		a_6	0010	$\bar{x}_3$	$y_3 y_5 y_6$	D_3	5
B_3	010	a_7	1000	$x_2 x_4$	$y_2 y_5$	D_1	6
		a_8	1001	$x_2 \bar{x}_4$	$y_1 y_4 y_7$	$D_1 D_4$	7
		a_9	1011	$\bar{x}_2$	y_3	$D_1 D_3 D_4$	8
B_4	100	a_7	1000	x_5	$y_4 y_5$	D_1	9
		a_{10}	1100	$\bar{x}_5 x_6$	$y_4 y_7$	$D_1 D_2$	10
		a_{11}	1110	$\bar{x}_5 \bar{x}_6$	$y_1 y_6$	$D_1 D_2 D_3$	11
B_5	001	a_{12}	1101	1	y_3	$D_1 D_2 D_4$	12
B_6	011	a_{12}	1101	1	$y_2 y_5$	$D_1 D_2 D_4$	13
B_7	101	a_1	0000	1	$y_1 y_6$	—	14

In the discussed case, we can obtain the class codes shown in Fig. 4.10. We will explain the second rule a little later.

The DST of $P_C Y2(\Gamma_6)$ is determined by the system (4.5). The only difference is reduced to different class codes (Table 4.4).

The DST determines the system $\langle M_1, M_2 \rangle$. Each row of the DST determines a term F_h from (3.30). There are the following terms, for example, in Table 4.4: $F_1 = \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 x_1$, $F_2 = \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 \bar{x}_1 x_2$, $\dots$, $F_{14} = \tau_1 \bar{\tau}_2 \tau_3$.

To get the content of $\langle M_A, M_{Y2} \rangle$, it is necessary to construct the table of MOs. It is Table 4.5 in the discussed case. We take $Y(a_m)$ from Table 4.1. The column $T(a_m)$ includes variables $\tau_r \in \tau$ equal to 1 in the $K(B_i)$ where $a_m \in B_i$.

The system $Y(T)$ is the same as (4.6). The system $Y(T)$ includes terms $F_{15} - F_{22}$. Let us find the system $T(T)$. It is the following:

$$\begin{aligned}
 \tau_1 &= B_2 \vee B_4 \vee B_7 = \bar{T}_1 T_2 \vee T_1 \bar{T}_2 \vee T_1 T_2 T_4 = F_{23} \vee F_{24} \vee F_{25}; \\
 \tau_2 &= B_2 \vee B_3 \vee B_6 = \bar{T}_1 T_2 \vee \bar{T}_1 \bar{T}_2 T_3 \vee T_1 T_2 T_3 = F_{23} \vee F_{26} \vee F_{27}; \\
 \tau_3 &= B_4 \vee B_6 \vee B_7 \vee B_5 = T_1 \bar{T}_2 \vee T_1 T_2 T_3 \vee T_1 T_2 T_4 \vee T_1 T_2 \bar{T}_3 \bar{T}_4 = \\
 &= F_{24} \vee F_{27} \vee F_{25} \vee F_{28}.
 \end{aligned} \tag{4.13}$$

Table 4.5 Table of MOs

a_m	$K(a_m)$	B_i	$Y(a_m)$	$\mathcal{T}(a_m)$	m
a_1	0000	B_1	–	–	1
a_2	0100	B_2	$y_{10}y_{12}$	$\tau_1 \tau_2$	2
a_3	0101	B_2	y_8y_{13}	$\tau_1 \tau_2$	3
a_4	0110	B_2	y_{11}	$\tau_1 \tau_2$	4
a_5	0011	B_3	y_9y_{13}	τ_2	5
a_6	0010	B_3	y_{11}	τ_2	6
a_7	1000	B_4	–	$\tau_1 \tau_3$	7
a_8	1001	B_4	y_8	$\tau_1 \tau_3$	8
a_9	1011	B_4	y_9y_{12}	$\tau_1 \tau_3$	9
a_{10}	1100	B_5	y_{10}	τ_3	10
a_{11}	1110	B_6	$y_{11}y_{12}$	$\tau_2 \tau_3$	11
a_{12}	1101	B_7	y_8y_{10}	$\tau_1 \tau_3$	12

So, there are $I - 1 = 6$ terms in the system (4.13). It is the number of classes PES whose codes $K(B_i)$ include at least a single unit.

So, we can get matrices M_1 and M_2 from Table 4.4, the matrices M_A and M_{Y_2} from systems (4.6) and (4.13). It gives the circuit of CFSM $P_C Y_2(\Gamma_6)$ shown in Fig. 4.11.

Let us find $S_T[P_C Y_2(\Gamma_6)]$. There are the following areas: $S(M_1) = 2(R_A + L) \cdot H = 252$, $S(M_2) = H(P + N_1) = 154$, $S(M_A) = 2P(H_{Y_2} + H_T) = 112$, and $S(M_{Y_2}) = 10 \cdot 5 = 50$. It gives $S_T = 568$.

Application of RLC turns $P_C Y_2$ into $M P_C Y_2$. Its structural diagram is shown in Fig. 4.12.

In $M P_C Y_2$ CFSM, the matrix M_{FP} implements terms (3.30), the matrix M_P the functions

$$P = P(F_P). \quad (4.14)$$

The matrix M_1 implements terms

$$P = F(\mathcal{T}, P), \quad (4.15)$$

the matrix M_2 the functions (3.1) and (3.2). The purposes of M_A and M_{Y_2} are the same as in CFSM $P_C Y_2$.

The RLC does not affect matrices M_2 , M_A and M_{Y_2} . So let us discuss how to implement matrices M_{FP} , M_P and M_1 for CFSM $M P_C Y_2(\Gamma_6)$.

The distribution of $x_l \in X$ between the classes $B_i \in \Pi_A$ is shown in Table 4.3. Using Table 4.3 and class codes (Fig. 4.10), we can get the following SBF:

Fig. 4.11 Matrix circuit of CFSM $PP_C Y2(\Gamma_6)$

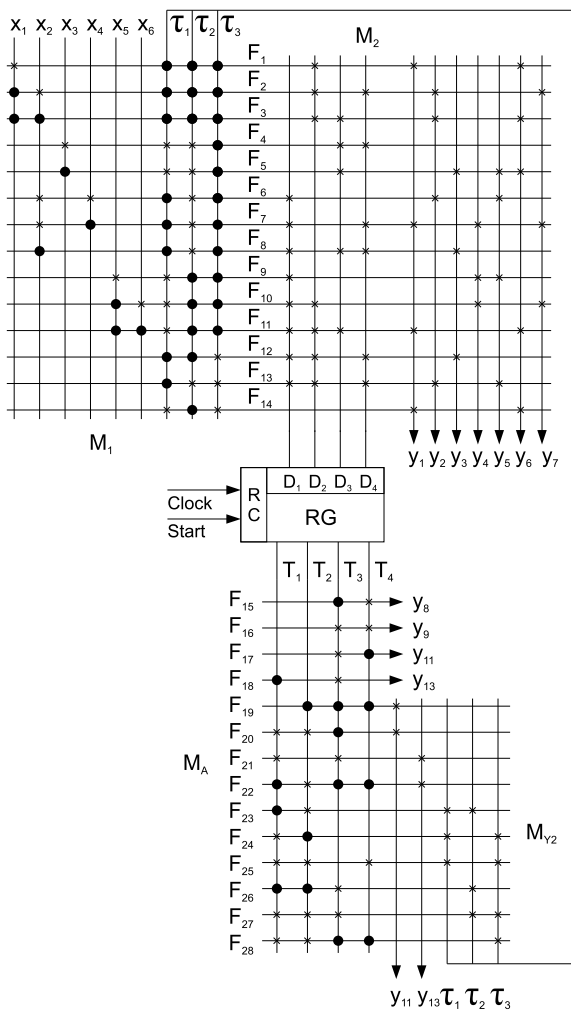
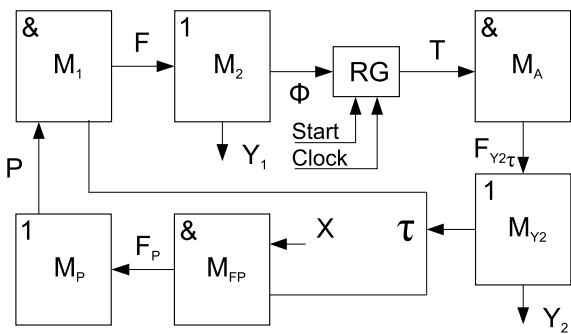


Fig. 4.12 Structural diagram of $MP_C Y2$ CFSM



$$\begin{aligned}
p_1 &= [B_1 \vee B_5]x_1 \vee [B_2 \vee 111]x_3 \vee [B_3 \vee B_6]x_4 \vee [B_4 \vee B_7]x_5 = \\
&= \bar{\tau}_1 \bar{\tau}_2 x_1 \vee \tau_1 \tau_2 x_3 \vee \bar{\tau}_1 \tau_2 x_4 \vee \tau_1 \bar{\tau}_2 x_5 = F_{29} \vee F_{30} \vee F_{31} \vee F_{32}; \\
p_2 &= [B_1 \vee B_3 \vee B_5 \vee B_6]x_2 \vee [B_4 \vee B_7]x_6 = \bar{\tau}_1 x_2 \vee \tau_1 \bar{\tau}_2 x_6 = F_{33} \vee F_{34}.
\end{aligned} \tag{4.16}$$

Let I_C be the number of classes $B_i \in \Pi_A$ including states with conditional transitions. In this case, it is enough R_{AC} bits to encode these classes:

$$R_{AC} = \lceil \log_2 I_C \rceil. \tag{4.17}$$

There is $I_C = 4$ and, therefore, $R_{AC} = 2$ in the discussed case.

Let $H_P = |F_P|$. Obviously, the minimum possible value of H_P is determined as

$$H_P = L. \tag{4.18}$$

There is $L = 6$ in the discussed case.

So, in the best case, the matrix M_{FP} has $L + R_{AC}$ inputs and L outputs. Analysing of system (4.16) shows that it is true for the discussed case. We have got this result due to: (1) proper distribution of LCs (Table 4.3) and (2) proper encoding of classes $B_i \in \Pi_A$ (Fig. 4.10). As follows from the Karnaugh map (Fig. 4.10), there is $\tau_3 = 0$ for all classes with conditional transitions. Moreover, the classes B_1 and B_3 are placed into adjacent cells of the map. It allows obtaining a single term $\bar{\tau}_1 x_2$ instead of two terms.

There is the matrix circuit for M_{FP} , M_P and M_1 shown in Fig. 4.13. It is constructed using the system (4.16) for M_{FP} , M_P and system (4.5) for M_1 . Obviously, the class codes from Fig. 4.10 encode the classes of PES.

Let us find the areas for these matrices. There is $S(M_{FP}) = (L + 2R_{AC}) \cdot H_P = (6 + 4) \cdot 6 = 60$. There is $S(M_P) = H_P \cdot G = 6 \cdot 2 = 12$. There is $S(M_1) = 2(G + R_A) \cdot H = 2(2 + 3) \cdot 14 = 140$. Using areas $S(M_2)$, $S(M_A)$ and $S(M_{Y2})$ from previous example, we can get $S_T[MP_C Y2(\Gamma_6)] = 528$.

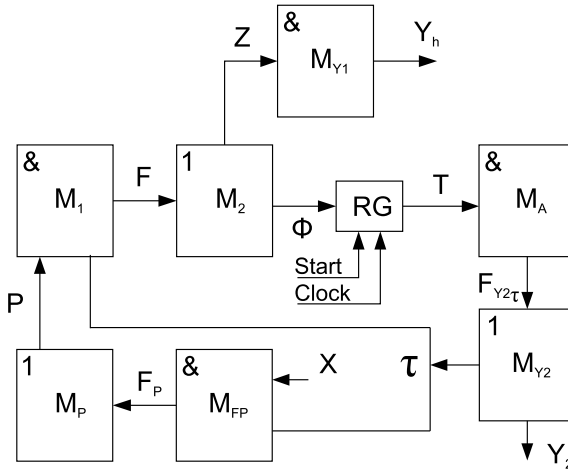
To optimize the area of M_2 , we can use either ECMOs or EFCMOs. It leads to CFSMs $MP_C Y1Y2$ (Fig. 4.14) and $MP_C D1Y2$ (Fig. 4.15), respectively.

Using previous examples, we can find that there is $S_T[MP_C Y1Y2(\Gamma_6)] = 584$. Let us estimate $S(M_2)$ and $S(M_1)$ for CFSM $MP_C D1Y2(\Gamma_6)$.

Let us find the partition $\Pi_{Y1} = \{Y^1, \dots, Y^K\}$ of the set $Y1$ by classes of compatible MOs. There are CMOs $Y_q \subseteq Y_1$ listed in Table 4.1. Using approach [4] gives the partition with classes $Y^1 = \{y_1, y_2, y_3\}$, $Y^2 = \{y_5, y_7\}$ and $Y^3 = \{y_4, y_8\}$. Using (3.25) gives $R_1 = 2$, $R_2 = R_3 = 2$. Because $R_2 = |Y^2|$ and $R_3 = |Y^3|$, there is no sense in encoding of microoperations $y_n \in Y^2 \cup Y^3$. So, there is $Z = \{z_1, z_2\}$.

Let us encode the MOs $y_n \in Y^1$ in the trivial way: $K(y_1) = 01$, $K(y_2) = 10$, and $K(y_3) = 11$. It gives some changes in the DST. Let us show the columns Y_h and h for DST of $MP_C D1Y2(\Gamma_6)$ in Table 4.6.

As follows from Table 4.6, $y_1 = z_2$ and $y_2 = z_1$. So, only a MO $y_3 = z_1 z_2$ should be generated by M_{Y1} . So, there is $S(M_2) = 14 \cdot (4 + 6) = 140$ and $S(M_{Y1}) = 2 \cdot 1 = 2$. It gives $S_T[MP_C D1Y2(\Gamma_6)] = 516$.

Fig. 4.15 Structural diagram of $MP_C D1Y2$ CFSMTable 4.7 Comparison of different CFSMs for Γ_6

Model	M_{FP}	M_P	M_1	M_2	M_A	M_{Y2}	M_{FY1}	M_{Y1}	S_T	NL
P	–	–	540	459	–	–	–	–	999	2
P_0Y2	–	–	280	154	64	8	–	–	506	4
MP_0Y2	72	12	168	154	64	8	–	–	478	6
MP_0Y1Y2	72	12	168	112	64	8	80	18	534	6
P_CY2	–	–	252	154	112	50	–	–	568	4
MP_CY2	60	12	140	154	112	50	–	–	528	6
MP_CY1Y2	60	12	140	112	112	50	80	18	584	6
$MP_C D1Y2$	60	12	140	140	112	50	–	2	516	6

Let us create Table 4.7 with areas of different matrices and models of CFSMs for GSA Γ_6 . There is the number of levels (NL) in the last column of Table 4.7. It may seem strange, but the best features are provided by the model $P_0Y2(\Gamma_6)$. It combines the least area and the four levels of matrices. Of course, this conclusion is valid only for the case of GSA Γ_6 .

Now, let us consider CFSM S_8 . Some of its characteristics are shown in Table 4.8. We explain some parameters a bit later. We can find the value of H as

$$H = \sum_{i=1}^I |B_i| \cdot H_{mi}. \quad (4.19)$$

Using (4.19) gives $H = 82$ (it is placed in Table 4.8).

Let us execute the OSA for CFSM S_8 . Its outcome is shown in Fig. 4.16.

Table 4.8 Characteristics of CFSM S_8

B_i	H_{mi}	Parameters	
$B_1 = \{a_1\}$	4	$L=12$	$H_T = 30$
$B_2 = \{a_1, a_3, a_4\}$	5	$G=3$	$R_A = 3$
$B_3 = \{a_5, a_6, a_7\}$	4	$ Y_1 = 12 = N1$	$R = 3$
$B_4 = \{a_8, a_9, a_{10}\}$	6	$ Y_2 = 10 = N2$	$M = 16$
$B_5 = \{a_{10}, a_{12}, a_{13}\}$	6	$H = 82$	$H_P = 16$
$B_5 = \{a_{14}, a_{15}, a_{16}\}$	5	$H_{opt} = 56$	$H_A = 16$

Fig. 4.16 Outcome of OSA for S_8

		$T_1 T_2$			
$T_3 T_4$		00	01	11	10
	00	a_1	a_5	a_{14}	a_8
	01	a_2	a_6	a_{15}	a_9
	11	a_3	a_7	a_{16}	a_{10}
	10	a_4	a_{11}	a_{12}	a_{13}

As follows from Fig. 4.16, each class $B_i \in \Pi_A (i \neq 1)$ is represented by two intervals of 4-dimensional Boolean space. For example, the class B_2 is represented by codes $00*1$ and $001*$. In fact, the class B_2 is represented by two classes: B_2^1 with the code $K(B_2^1) = 00*1$ and B_2^2 with $K(B_2^2) = 001*$. It means that there are $2H_{m2} = 10$ rows in DST of S_8 for the class $B_2 \in \Pi_A$. This situation is repeated for all classes $B_i \in \Pi_A$. We can find the number of rows in DST of P_0Y2 CFSM corresponding to S_8 . Let us denote it as H_{opt} . We can find it as:

$$H_{opt} = H_{m1} + 2 \sum_{i=2}^I H_{mi}. \quad (4.20)$$

It is equal to 56.

Let us execute transformation of state codes into class codes. There is $R_A = 3$. Obviously, each class $B_i \in \Pi_A$ is represented by a single code. It gives the number of rows of DST H_T determined as

$$H_T = \sum_{i=1}^I H_{mi}. \quad (4.21)$$

It is equal to 30.

Let us find the total areas for P , P_0Y2 and P_CY2 CFSMs corresponding to S_8 :

$$P : S(M_1) = 2(L + R) \cdot H = 2 \cdot 16 \cdot 82 = 2624;$$

$$S(M_2) = H \cdot (N + R) = 82 \cdot 26 = 2132;$$

$$S_T(P) = 4756.$$

$$P_0Y2 : S(M_1) = 2(L + R) \cdot H_{opt} = 16 \cdot 2 \cdot 56 = 1792;$$

$$S(M_2) = H_{opt} \cdot (R + N1) = 56 \cdot 16 = 896;$$

$$S(M_A) = 2R \cdot H_A = 2 \cdot 4 \cdot 16 = 128;$$

$$S(M_{Y2}) = H_A \cdot N2 = 16 \cdot 10 = 160;$$

$$S_T(P_0Y2) = 2976.$$

$$P_CY2 : S(M_1) = 2(L + R_A) \cdot H_T = 2 \cdot 15 \cdot 30 = 900;$$

$$S(M_2) = H_T \cdot (R + N1) = 30 \cdot 16 = 480;$$

$$S(M_A) = 2R \cdot H_A = 2 \cdot 4 \cdot 16 = 128;$$

$$S(M_{Y2}) = H_A \cdot (N2 + R_A) = 16 \cdot 30 = 208;$$

$$S_T(P_CY2) = 1716.$$

So, using the transformation of state codes allows reducing the chip area by 2,77 times compared to P CFSM and by 1,73 times compared to P_0 CFSM (for the case of S_8). Now, let us check the influence of RLC for these models.

We have enough data to estimate the areas of matrices M_{FP} , M_P and M_1 . There is no change for matrices M_2 , M_A and M_{Y2} due to RLC. We can take their areas from previous calculations. So, let us find the total areas for MP , MP_0Y2 and MP_CY2 CFSMs corresponding to S_8 :

$$MP : S(M_{FP}) = (L + 2R) \cdot H_P = 20 \cdot 16 = 320;$$

$$S(M_P) = H_P \cdot G = 16 \cdot 3 = 48;$$

$$S(M_1) = 2(G + R) \cdot H = 14 \cdot 82 = 1148;$$

$$S_T(MP) = 3648.$$

$$MP_0Y2 : S(M_{FP}) = (L + 2R) \cdot H_P = 20 \cdot 16 = 320;$$

$$S(M_P) = H_P \cdot G = 16 \cdot 3 = 48;$$

$$S(M_1) = 2(G + R) \cdot H_{opt} = 14 \cdot 56 = 784;$$

$$S(MP_0Y2) = 2316;$$

$$MP_CY2 : S(M_{FP}) = (L + 2R_A) \cdot H_P = 18 \cdot 16 = 288;$$

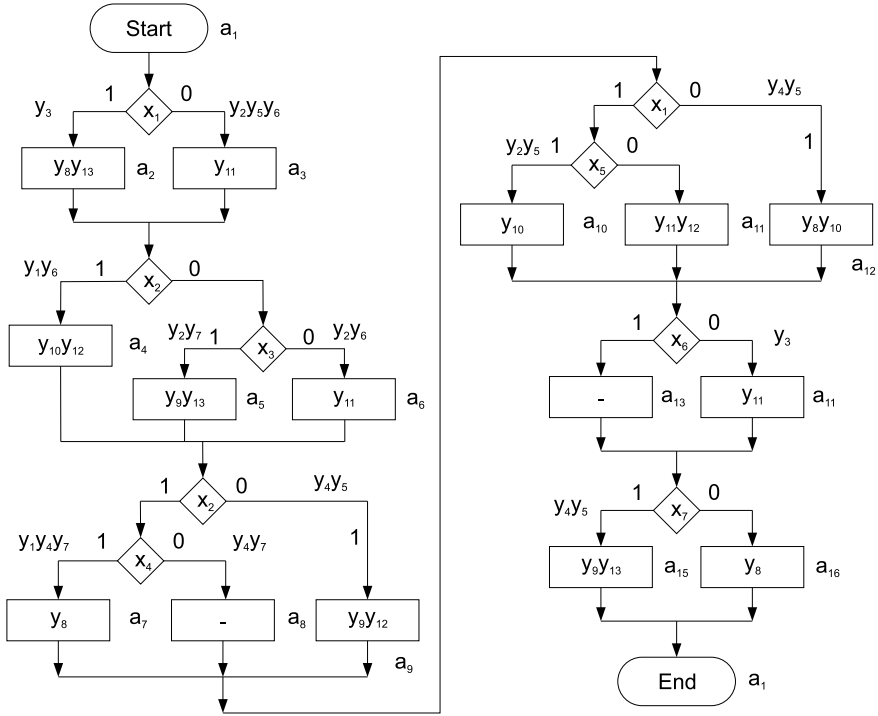
$$S(M_P) = H_P \cdot G = 48;$$

$$S(M_1) = 2(G + R_A) \cdot H_T = 12 \cdot 30 = 360;$$

$$S_T(MP_CY2) = 1032.$$

So, using the RLC allows for further reduction of CFSMs areas. Using it in $P(S_8)$ gives reducing by 30%, in P_0Y2 by 28%, and in P_CY2 by 66%. If we compare different models of CFSM for S_8 , then using the transformation of state codes allows reducing the chip area by 3.53 times compared to $P(S_8)$ and 2.24 times compared to $P_0Y2(S_8)$.

The following conclusion can be made on the base of these examples. If all classes $B_i \in \Pi_A$ are represented by single generalized intervals of R -dimensional Boolean space, then models P_0Y2 provide the best possible solution. If this condition is violated, it is necessary to choose between P_CY2 and P_0Y2 . The more classes represented by more than one interval, the greater the win brings the transition from P_0Y2 to P_CY2 models. Now, let us discuss P_TY2 CFSMs, where only a part of classes $B_i \in \Pi_A$ is encoded.

Fig. 4.17 Marked GSA Γ_7 Fig. 4.18 Outcome of OSA
for CFSM $P_0Y_2(\Gamma_7)$

		T_1T_2			
		00	01	11	10
T_3T_4	00	a_1	a_{13}	a_{10}	a_4
	01	a_2	a_{14}	a_{11}	a_5
	11	a_3	a_{15}	a_{12}	a_6
	10	a_7	a_{16}	a_8	a_9

4.3 CFSMs with Partial Code Transformation

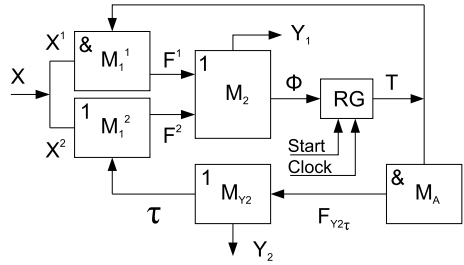
Consider the marked GSA Γ_7 shown in Fig. 4.17. Some characteristics of CFSM for this GSA are shown in Table 4.9. There are the same CMOs $Y_q \subseteq Y_1$ for $P(\Gamma_6)$ and $P(\Gamma_7)$. The same CMOs are written in operator vertices of both Γ_6 and Γ_7 .

We found the values of H and H_T using formulae (4.19) and (4.21), respectively. To find the value of H_{opt} , let us encode the states $a_m \in A$ in the optimal way. To do it, we use state variables $T_r \in T = \{T_1, \dots, T_4\}$ (Fig. 4.18). The classes $B_i \in \Pi_A$ are clear from the column B_i of Table 4.9.

As follows from Fig. 4.18, four classes from Π_A are represented by single generalized 4-dimensional intervals. They are the classes B_1 , B_2 , B_6 and B_7 . They

Table 4.9 Characteristics of CFSMs for Γ_7

a_m	B_i	H_m	a_m	B_i	H_m	Parameters	
a_1	B_1	2	a_9	B_4	3	$M = 16$	$R = 4$
a_2	B_2	3	a_{10}	B_5	2	$L = 7$	$R_A = 3$
a_3	B_2	3	a_{11}	B_5	2	$N1 = 7$	$N = 13$
a_4	B_3	3	a_{12}	B_5	2	$N2 = 6$	$Q = 10$
a_5	B_3	3	a_{13}	B_6	2	$G = 2$	$R_Q = 4$
a_6	B_3	3	a_{14}	B_6	2	$I = 7$	$H_T = 16$
a_7	B_4	3	a_{15}	B_7	1	$H = 38$	$R_{A2} = 2$
a_8	B_4	3	a_{16}	B_7	1	$H_{opt} = 24$	$H_P = 7$

Fig. 4.19 Structural diagram of $P_{C1}Y2$ CFSM

have the codes $K(B_1) = 0000$, $K(B_2) = 00*1$, $K(B_6) = 010*$, and $K(B_1) = 011*$. Other classes are divided by two classes each: $B_3 = B_3^1 \cup B_3^2 = \{a_4, a_5\} \cup \{a_6\}$, $B_4 = B_4^1 \cup B_4^2 = \{a_7, a_9\} \cup \{a_8\}$, $B_5 = B_5^1 \cup B_5^2 = \{a_{10}, a_{11}\} \cup \{a_{12}\}$. They have the following codes: $K(B_3^1) = 100*$, $K(B_3^2) = 10*1$, $K(B_4^1) = *010$, $K(B_4^2) = 1*10$, $K(B_5^1) = 110*$, and $K(B_5^2) = 11*1$. Using (4.20) gives $H_{opt} = 24$. So, there is $H_{opt} > H_T$. It means that we can use the P_T -model of FSM.

Let us consider the $P_M Y2$ CFSM shown in Fig. 4.19. It combines features of $P_0 Y2$ and $P_C Y2$ CFSMs.

The comparison of $P_C Y2$ and $P_{C1} Y2$ CFSMs shown that the only difference is reduced to replacement of M_1 by M_1^1 and M_1^2 . It corresponds to representing the partition Π_A as $\Pi_{A1} \cup \Pi_{A2}$. The set Π_{A1} includes classes of PES represented by single intervals of R -dimensional Boolean space. The set $\Pi_{A2} = \Pi_A / \Pi_{A1}$.

The matrix M_1^1 corresponds to Π_{A1} . It implements system F^1 of terms

$$F_h = F_h(T, X^1). \quad (4.22)$$

The set $X^1 \subseteq X$ includes LCs $x_l \in X$ determining transitions from states $a_m \in B_i$ where $B_i \in \Pi_{A1}$. There are H^1 terms (4.22) generated by M_1^1 .

The matrix M_1^2 corresponds to Π_{A2} . It implements system F^2 of terms

$$F_h = F_h(\mathcal{T}, X^2). \quad (4.23)$$

The set $X^2 \subseteq X$ includes LCs $x_l \in X$ determining transitions from states $a_m \in B_i$ where $B_i \in \Pi_{A2}$. There are H^2 terms (4.23) generated by M_1^2 .

The matrix M_2 generates the systems

$$Y1 = Y1(F^1, F^2); \quad (4.24)$$

$$\Phi = \Phi(F^1, F^2). \quad (4.25)$$

The functions of matrices M_A and M_{Y2} are the same as for $P_C Y2$ CFSM.

Obviously, the following relation takes place:

$$H^1 + H^2 = H_T. \quad (4.26)$$

So, there are H_T inputs in matrix M_2 as it is for $P_C Y2$ CFSM. But only a part of classes PES $B_i \in \Pi_A$ is encoded. It means that the matrix M_{Y2} of $P_T Y2$ is simpler that its counterpart for the equivalent $P_C Y2$ CFSM.

In this book, we propose a method of synthesis for $P_{C1} Y2$ CFSM. It includes the following steps:

1. Marking GSA Γ and creating the set A .
2. Creating the partition Π_A of A by classes of PES.
3. Executing the optimal state assignment.
4. Creating the sets Π_{A1} and Π_{A2} .
5. Encoding of classes of PES from Π_{A2} .
6. Creating the DST for Π_{A1} .
7. Creating the DST for Π_{A2} .
8. Creating the tables of M_2 , M_A and M_{Y2} .
9. Implementing the matrix circuit of CFSM $P_T Y2(\Gamma)$.

Let us discuss an example of synthesis for CFSM $P_{C1} Y2(\Gamma_7)$. The first three steps of the synthesis are already executed. Their outcomes produce the sets $\Pi_{A1} = \{B_1, B_2, B_6, B_7\}$ and $\Pi_{A2} = \{B_3, B_4, B_5\}$.

It is enough R_{A2} bits to encode the classes $B_i \in \Pi_{A2}$:

$$R_{A2} = \lceil \log_2 |\Pi_{A2}| \rceil. \quad (4.27)$$

There is $R_A = 2$ in the discussed case. It gives the set $\mathcal{T} = \{\tau_1, \tau_2\}$. Let us encode the classes $B_i \in \Pi_{A2}$ as the following: $K(B_3) = 00$, $K(B_4) = 01$ and $K(B_5) = 10$. We can use the input assignment $\langle \tau_1 \tau_2 \rangle = 11$ for optimizing functions $F_h \in F^2$.

To form the DST for Π_{A2} , it is necessary to create the system of GFTs for classes $B_i \in \Pi_{A1}$. It is the following system in the discussed case:

$$\begin{aligned} B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 a_3; \\ B_2 &\rightarrow x_2 a_4 \vee \bar{x}_2 x_3 a_5 \vee \bar{x}_2 \bar{x}_3 a_6; \\ B_6 &\rightarrow x_7 a_{15} \vee \bar{x}_7 a_{16}; \\ B_7 &\rightarrow a_1. \end{aligned} \quad (4.28)$$

Table 4.10 DST for Π_{A1} of $P_{C1}Y2(\Gamma_7)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	0000	a_2	0001	x_1	y_3	D_4	1
		a_3	0011	$\bar{x}_1$	$y_3y_5y_6$	D_3D_4	2
B_2	00*1	a_4	1000	x_2	y_1y_6	D_1	3
		a_5	1001	$\bar{x}_2x_3$	y_2y_7	D_1D_4	4
		a_6	1011	$\bar{x}_2\bar{x}_3$	y_2y_6	$D_1D_3D_4$	5
B_6	010*	a_{15}	0111	x_7	y_4y_5	$D_2D_3D_4$	6
		a_{16}	0110	$\bar{x}_7$	—	D_2D_3	7
B_7	011*	a_1	0000	1	—	—	8

Table 4.11 DST for Π_{A2} of $P_{C1}Y2(\Gamma_7)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_3	00	a_7	0010	x_2x_4	$y_1y_4y_7$	D_2	9
		a_8	1110	$x_2\bar{x}_4$	y_4y_7	$D_1D_2D_3$	10
		a_9	1010	$\bar{x}_2$	y_4y_5	D_1D_3	11
B_4	*1	a_{10}	1100	x_1x_5	y_2y_5	D_1D_2	12
		a_{11}	1101	$x_1\bar{x}_5$	—	$D_1D_2D_3$	13
		a_{12}	1111	$\bar{x}_1$	y_4y_5	$D_1D_2D_3D_4$	14
B_5	1*	a_{13}	0100	x_6	—	D_2	15
		a_{14}	0101	$\bar{x}_6$	y_3	D_2D_4	16

The system (4.28) determines the DST (Table 4.10). It has $H^1 = 8$ rows.

As follows from the column $K(B_i)$, there is $T_1 = 0$ in Table 4.10. So, there is no wire with $T_1 = 1$ entering the matrix M_1^2 . Also, there are no functions $D_r \in \Phi$ or $y_n \in Y1$ generated in the row 8. So, there is no output F_8 in the matrix M_1^1 .

To form the DST for Π_{A2} , it is necessary to create the system of GFTs for classes $B_i \in \Pi_{A2}$. It is the following system in the discussed case:

$$\begin{aligned}
 B_3 &\rightarrow x_2x_4a_7 \vee x_2\bar{x}_4a_8 \vee \bar{x}_2a_9; \\
 B_4 &\rightarrow x_1x_5a_{10} \vee x_1\bar{x}_5a_{11} \vee \bar{x}_1a_{12}; \\
 B_5 &\rightarrow x_6a_{13} \vee \bar{x}_6a_{14}.
 \end{aligned} \tag{4.29}$$

The system (4.29) determines DST (Table 4.11). It includes $H^2 = 8$ rows. We number them starting with $H^1 + 1$.

The table of M_2 is creating on the base of tables for partitions Π_{A1} and Π_{A2} . It includes columns F_h , Y_h , Φ_h , h . The data for columns Y_h and Φ_h are taken from corresponding columns of tables for Π_{A1} and Π_{A2} .

In the discussed case, there are $H^1 + H^2 = 16$ rows in the table of M_2 (Table 4.12). The table is created in a trivial way. Because of it, we show only a part of this table.

Table 4.12 Table of M_2 for $P_{C1}Y2(I_7)$

F_h	Y_h	Φ_h	h	F_h	Y_h	Φ_h	h
F_1	y_3	D_4	1	F_9	$y_1y_4y_7$	D_2	9
F_2	$y_3y_5y_6$	D_3D_4	2	F_{10}	y_4y_7	$D_1D_2D_3$	10
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
F_8	—	—	8	F_{16}	y_3	D_2D_4	16

Table 4.13 Table of matrices M_A and M_{Y2} of $P_{C1}Y2(I_7)$

a_m	$K(a_m)$	A_m	B_i	$K(B_i)$	$Y(a_m)$	$\mathcal{T}_m$	m
a_1	0000	$\bar{T}_1\bar{T}_2\bar{T}_3\bar{T}_4$	—	—	—	—	1
a_2	0001	$\bar{T}_1\bar{T}_2\bar{T}_3T_4$	—	—	y_8y_{13}	—	2
a_3	0011	$\bar{T}_1\bar{T}_2T_3T_4$	—	—	y_{11}	—	3
a_4	1000	$T_1\bar{T}_2\bar{T}_3\bar{T}_4$	B_3	00	$y_{10}y_{12}$	—	4
a_5	1001	$T_1\bar{T}_2\bar{T}_3T_4$	B_3	00	y_9y_{13}	—	5
a_6	1011	$T_1\bar{T}_2T_3T_4$	B_3	00	y_{11}	—	6
a_7	0010	$\bar{T}_1\bar{T}_2T_3\bar{T}_4$	B_4	01	y_8	τ_2	7
a_8	1110	$T_1T_2T_3\bar{T}_4$	B_4	01	—	τ_2	8
a_9	1010	$T_1\bar{T}_2T_3\bar{T}_4$	B_4	01	y_9y_{12}	τ_2	9
a_{10}	1100	$T_1T_2\bar{T}_3\bar{T}_4$	B_5	10	y_{10}	τ_1	10
a_{11}	1101	$T_1T_2\bar{T}_3T_4$	B_5	10	$y_{11}y_{12}$	τ_1	11
a_{12}	1111	$T_1T_2T_3T_4$	B_5	10	y_8y_{10}	τ_1	12
a_{13}	0100	$\bar{T}_1T_2\bar{T}_3\bar{T}_4$	—	—	—	—	13
a_{14}	0101	$\bar{T}_1T_2\bar{T}_3T_4$	—	—	y_{11}	—	14
a_{15}	0111	$\bar{T}_1T_2T_3T_4$	—	—	y_9y_{13}	—	15
a_{16}	0110	$\bar{T}_1T_2T_3\bar{T}_4$	—	—	y_8	—	16

Table 4.13 contains information for creating matrices M_A and M_{Y2} in the discussed case. It is creating in the trivial way.

The column $K(a_m)$ determines inputs of M_A . It has 4 inputs. The column A_m determines outputs of M_A and inputs of M_{Y2} . There is no information in rows 1 and 13 of Table 4.13. So, only 14 variables A_m should be implemented by M_A .

The columns $Y(a_m)$ and $\mathcal{T}_m$ determine outputs of M_{Y2} . It has 14 inputs and $N2 + R_{A2} = 8$ outputs.

We show matrices M_1^1 and M_1^2 in Fig. 4.20. There is $X^1 = \{x_1, x_2, x_3, x_7\}$, so there are 8 inputs in M_1^1 . There is $X^2 = \{x_1, x_2, x_4, x_5, x_6\}$. So, there are $R_{A2} + |X^2| = 7$. There is $X^1 \cap X^2 = \{x_1, x_2\}$. So, we minimize the number of inputs for each matrix as compared to $R + L = 11$.

There is the matrix circuit of M_A and M_{Y2} shown in Fig. 4.21. It is based on Table 4.13.

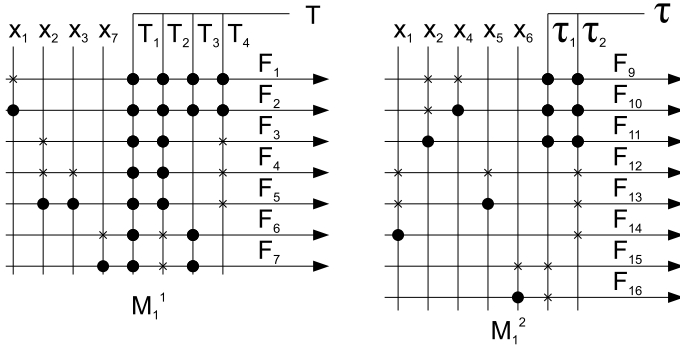


Fig. 4.20 Matrix circuit for M_1^1 and M_1^2 of CFSM $P_{C1}Y2(G_7)$

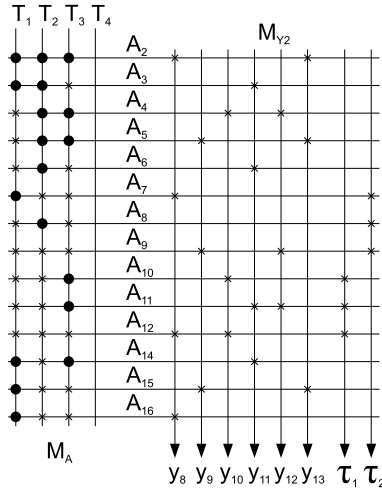


Fig. 4.21 Matrix circuit for M_A and M_{Y2} of CFSM $P_{C1}Y2(G_7)$

Let us find $S_T[P_TY2(G_7)]$. The matrix M_1 consists of matrices M_1^1 and M_1^2 . We can find their areas as

$$\begin{aligned} S(M_1^1) &= 2(|X^1| + R) \cdot H^1 = 105; \\ S(M_1^2) &= 2(|X^2| + R_{A2}) \cdot H^2 = 112. \end{aligned} \quad (4.30)$$

It gives $S(M_1) = 217$. Let us find other areas:

$$\begin{aligned} S(M_2) &= H_T(NL + R) = 176; \\ S(M_A) &= 2R \cdot H_A = 112; \\ S(M_{Y2}) &= H_A(N2 + R_{A2}) = 112. \end{aligned} \quad (4.31)$$

Adding areas from (4.30) and (4.31) gives $S_T = 617$.

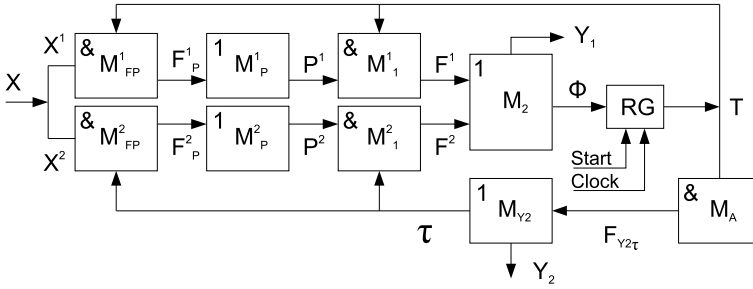


Fig. 4.22 Structural diagram of $MP_{C1}Y2$ CFSM

Table 4.14 Table of RLC for LCs $x_e \in X^1$

$P^1 \setminus B_i$	B_1	B_2	B_6	B_7
p_1	x_1	x_2	—	—
p_2	—	x_3	x_7	—

Now, let us discuss the model $MP_{C1}Y2$. Its structural diagram is shown in Fig. 4.22.

For this model, there is $F_P = F_P^1 \cup F_P^2$, $P = P^1 \cup P^2$, where

$$F_P^1 = F_P^1(T, X^1); \quad (4.32)$$

$$F_P^2 = F_P^2(T, X^2); \quad (4.33)$$

$$P^1 = P^1(F_P^1); \quad (4.34)$$

$$P^2 = P^2(F_P^2); \quad (4.35)$$

$$F^1 = F^1(T, P^1); \quad (4.36)$$

$$F^2 = F^2(T, P^2). \quad (4.37)$$

As follows from Fig. 4.22 and expressions (4.32)–(4.37), the RLC is done independently for sets X^1 and X^2 . There is no change for synthesis of matrices M_2 , M_A and M_{Y2} as compared to equivalent P_TY2 CFSM.

Let us discuss an example of synthesis for CFSM $MP_{C1}Y2(\Gamma_7)$. We have sets X^1 and X^2 from previous example.

Let us execute the RLC for LCs $x_e \in X^1 = \{x_1, x_2, x_3, x_7\}$. The RLC is executed for classes $B_i \in \Pi_{A1} = \{B_1, B_2, B_6, B_7\}$. Let us find the value of $G1 = |P^1|$.

Analysis of Table 4.10 shows that there is $G1 = 2$. It gives the set $P^1 = \{p_1, p_2\}$. The table of RLC for Π_{A1} is represented by Table 4.14.

Table 4.15 Table of RLC for LCs $x_e \in X^2$

$P^2 \setminus B_i$	B_3	B_4	B_5
p_3	x_2	x_1	x_6
p_4	x_4	x_5	–

Table 4.16 DST for Π_{A1} of $MP_{C1}Y2(\Gamma_7)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	0000	a_2	0001	p_1	y_3	D_4	1
		a_3	0011	$\bar{p}_1$	$y_3 y_5 y_6$	$D_3 D_4$	2
B_2	00*1	a_4	1000	p_1	$y_1 y_6$	D_1	3
		a_5	1001	$\bar{p}_1 p_2$	$y_2 y_7$	$D_1 D_4$	4
		a_6	1011	$\bar{p}_1 \bar{p}_2$	$y_2 y_6$	$D_1 D_3 D_4$	5
B_6	010*	a_{15}	0111	p_2	$y_4 y_5$	$D_2 D_3 D_4$	6
		a_{16}	0110	$\bar{p}_2$	–	$D_2 D_3$	7
B_7	011*	a_1	0000	1	–	–	8

Table 4.14 is the base to get terms (4.31) and functions (4.34). They are the following:

$$\begin{aligned} p_1 &= B_1 x_1 \vee B_2 x_2 = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 x_1 \vee \bar{T}_1 \bar{T}_2 T_4 x_2 = F_{17} \vee F_{18}; \\ p_2 &= B_2 x_3 \vee B_6 x_7 = \bar{T}_1 \bar{T}_2 T_4 x_2 \vee \bar{T}_1 T_2 x_7 = F_{19} \vee F_{20}. \end{aligned} \quad (4.38)$$

Let us execute the RLC for LCs $x_e \in X^2 = \{x_1, x_2, x_4, x_5, x_6\}$. The RLC is executed for classes $B_i \in \Pi_{A2} = \{B_3, B_4, B_5\}$.

Analysis of Table 4.11 gives $|P^2| = G2 = 2$. Let us use the variables p_3, p_4 to form P^2 . The table of RLC for Π_{A2} is represented by Table 4.15.

Using Table 4.15 gives terms (4.33) and functions (4.35). They can be found from the following system:

$$\begin{aligned} p_3 &= B_3 x_2 \vee B_4 x_1 \vee B_5 x_6 = \bar{\tau}_1 \bar{\tau}_2 x_2 \vee \tau_2 x_1 \vee \tau_1 x_6 = F_{21} \vee F_{22} \vee F_{23}; \\ p_4 &= B_3 x_4 \vee B_4 x_5 = \bar{\tau}_1 \bar{\tau}_2 x_4 \vee \tau_2 x_5 = F_{24} \vee F_{25}. \end{aligned} \quad (4.39)$$

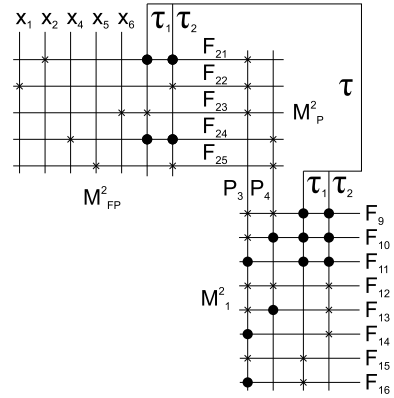
To find the system (4.36), it is necessary to construct the DST for Π_{A1} . It is Table 4.16.

To get Table 4.16, it is enough to replace LCs $x_e \in X^1$ in the system (4.28) by $p_g \in P^1$. If we replace LCs $x_e \in X^2$ in the system (4.29) by $p_g \in P^2$, we can get Table 4.17.

There is no change for matrices M_2 , M_A and M_{Y2} for equivalent $P_{C1}Y2$ and $MP_{C1}Y2$ automata. So, the matrix M_2 is represented by Table 4.12, M_A and M_{Y2} by Table 4.13.

Table 4.17 DST for Π_{A1} of $MP_{C1}Y2(\Gamma_7)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_3	00	a_7	0010	$p_3 p_4$	$y_1 y_4 y_7$	D_2	9
		a_8	1110	$p_3 \bar{p}_4$	$y_4 y_7$	$D_1 D_2 D_3$	10
		a_9	1010	$\bar{p}_3$	$y_4 y_5$	$D_1 D_3$	11
B_4	*1	a_{10}	1100	$p_3 p_4$	$y_2 y_5$	$D_1 D_2$	12
		a_{11}	1101	$p_3 \bar{p}_4$	—	$D_1 D_2 D_4$	13
		a_{12}	1111	$\bar{p}_3$	$y_4 y_5$	$D_1 D_2 D_3 D_4$	14
B_5	1*	a_{13}	0110	p_3	—	$D_2 D_3$	15
		a_{14}	0101	$\bar{p}_3$	y_3	$D_2 D_4$	16

Fig. 4.23 Matrix circuit for MP_{FP}^2 , MP_P^2 and MP_I^2 of CFSM $MP_{C1}Y2(\Gamma_7)$ 

Let us show, for example, the matrix circuit for matrices M_{FP}^2 , M_P^2 and M_I^2 in the discussed case. It is shown in Fig. 4.23. Acting in the same manner, we can get circuits for other matrices and combine them into a single circuit of CFSM $MP_TY2(\Gamma_7)$. We leave this task to our reader.

Now, let us find the area $S_T[MP_{C1}Y2(\Gamma_7)]$. There are the following areas of different matrices for this case:

$$\begin{aligned}
 S(M_{FP}^1) &= (|X^1| + 2R) \cdot |F_P^1| = 40; \\
 S(M_{FP}^2) &= (|X^2| + 2R_{A2}) \cdot |F_P^2| = 45; \\
 S(M_{FP}) &= S(M_{FP}^1) + S(M_{FP}^2) = 85; \\
 S(M_P^1) &= |F_P^1| \cdot |P^1| = 8; \\
 S(M_P^2) &= |F_P^2| \cdot |P^2| = 10; \\
 S(M_P) &= S(M_P^1) + S(M_P^2) = 18; \\
 S(M_I) &= S(M_I^1) + S(M_I^2) = 148; \\
 S_T &= 651.
 \end{aligned} \tag{4.40}$$

We collect information about areas of matrices for seven different models of CFSM based on Γ_7 (Table 4.18).

Table 4.18 Comparison of different CFSMs for (Γ_7)

Model	M_{FP}	M_P	M_1	M_2	M_A	M_{Y2}	S_T	NL
P	–	–	836	646	–	–	1482	2
P_0Y2	–	–	528	264	112	84	988	4
MP_0Y2	105	14	288	264	112	84	867	6
P_CY2	–	–	320	176	112	126	734	4
MP_CY2	91	14	160	176	112	126	679	6
$P_{C1}Y2$	–	–	217	176	112	112	617	4
$MP_{C1}Y2$	85	18	148	176	112	112	651	6

As follows from Table 4.18, the model $P_{C1}Y2$ provides the best results. Let us point out that models with the partial transformation of state codes provide better results than their counterparts. But it is true only for the case of GSA Γ_7 .

4.4 CFSMs with Primary Encoding of Classes of PES

In all previous methods with encoding of $B_i \in \Pi_A$ the variables $\tau_r \in \mathcal{T}$ are generated by M_{Y2} . Let us name this approach the secondary encoding of classes PES (SEP). In the case of SEP, the codes $K(B_i)$ are obtained on the base of the codes $K(a_m)$. There are two methods when class codes are generated by the matrix M_2 . They lead to P_EY2 and P_KY2 automata.

Let us start from P_EY2 CFSM. It has no new blocks as compared to P_0Y2 CFSM. So, the structural diagram of P_EY2 CFSM is the same as the one shown in Fig. 4.1. Let us discuss an example of synthesis for $P_EY2(\Gamma_7)$.

There are the following steps in the corresponding method of synthesis:

1. Marking the GSA Γ and finding the set A .
2. Finding the partition $\Pi_A = \{B_1, \dots, B_I\}$.
3. Executing optimal state encoding using $R + 1$ state variables.
4. Constructing the DST of P_EY2 CFSM.
5. Creating the systems of functions for matrices of the circuit (M_1 , M_2 , M_A , and M_{Y2}).
6. Implementing the CFSM matrix circuit using the obtained SBFs.

In the case of $P_EY2(\Gamma_7)$, two first steps are already executed. There is $M = 16$, $R = 4$ and $\Pi_A = \{B_1, \dots, B_7\}$.

There is $I = 7$ and $R_A = 3$. There is $M_0 = 3$ (the maximum numbers of states in a class $B_i \in \Pi_A$). Using (3.50) gives $R_0 = 2$. So, it is necessary $R_K = R_A + R_0$ bits to encode states $a_m \in A$. There is $R_K = 5$ in the discussed case. So, there is $T = \{T_1, \dots, T_5\}$ and $\Phi = \{D_1, \dots, D_5\}$. Obviously, there is $R_K = R + 1$.

Fig. 4.24 Trivial state codes of $P_E Y2(\Gamma_7)$

$T_4 T_5$ \ $T_1 T_2 T_3$									
		000	001	011	010	110	111	101	100
00		a_1	a_2	a_7	a_4	a_{15}	*	a_{13}	a_{10}
01		*	a_3	a_8	a_5	a_{16}	*	a_{14}	a_{11}
11		*	*	*	*	*	*	*	*
10		*	*	a_9	a_6	*	*	*	a_{12}
		B_1	B_2	B_4	B_3	B_7	*	B_6	B_5

Let us encode the classes $B_i \in \Pi_A$ in the trivial manner: $K(B_1) = 000$, $K(B_2) = 001, \dots, K(B_7) = 110$. It leads to state codes shown in Karnaugh map (Fig. 4.24).

As follows from Fig. 4.24, the classes $B_i \in \Pi_A$ are represented by variables $T_r \in T' = \{T_1, T_2, T_3\}$. The variables $T_4, T_5 \in T$ determine a state inside a class.

Let us estimate the areas of different matrices for the discussed case. We found that $|F_{Y2}| = 14$.

There are the following areas:

$$\begin{aligned}
 S(M_1) &= 2(L + R_A) \cdot H_T = 2(7 + 3) \cdot 16 = 320; \\
 S(M_2) &= H_T(R_K + NL) = 16 \cdot (5 + 7) = 192; \\
 S(M_A) &= 2R_K \cdot |F_{Y2}| = 2 \cdot 5 \cdot 14 = 140; \\
 S(M_{Y2}) &= |F_{Y2}| \cdot N2 = 14 \cdot 6 = 64; \\
 S_T &= 715.
 \end{aligned} \tag{4.41}$$

This result is better than for equivalent $P_0 Y2$ and $P_C Y2$ CFSMs (Table 4.18). Let us try to improve it using the refined state assignment.

In fact, the code $K(B_i)$ is represented as a concatenation of the codes $K(B_i)$, where $a_m \in B_i$, and $C(a_m)$:

$$K(a_m) = K(B_i) * C(a_m). \tag{4.42}$$

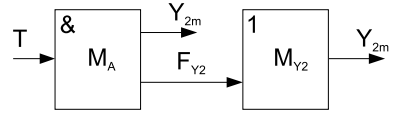
The code $C(a_m)$ is created from the state variables $T_r \in T \setminus T' = T_A$. Obviously, there is $T' \cap T_A = \emptyset$.

So, there is no influence of codes $K(B_i)$ on codes $C(a_m)$ (and otherwise). It means we can assign any codes to classes and states. Let us encode states $a_m \in A$ using the principle of RSA (Fig. 4.25).

To estimate the influence of RSA, let us form the system (1.43). It is the following in the discussed case:

Fig. 4.25 Outcome of RSA for CFSM $P_E Y2(\Gamma_7)$

$T_1 T_2 T_3$		$T_4 T_5$							
		000	001	011	010	110	111	101	100
$T_4 T_5$	00	a_1	a_{10}	a_4	*	a_8	*	*	a_{13}
	01	*	a_{12}	*	a_{16}	a_7	*	a_2	*
	11	*	*	a_5	a_{15}	a_9	*	*	*
	10	*	a_{11}	a_6	*	*	*	a_3	a_{14}
		B_1	B_5	B_3	B_7	B_4	$\emptyset$	B_2	B_6

Fig. 4.26 Matrix circuit for implementing $Y2$ 

$$\begin{aligned}
 y_8 &= A_2 \vee A_7 \vee A_{12} \vee A_{16} = \bar{T}_4 T_5; \\
 y_9 &= A_5 \vee A_9 \vee A_{15} = T_4 T_5; \\
 y_{10} &= A_4 \vee A_{10} \vee A_{12} = \bar{T}_1 T_3 \bar{T}_4; \\
 y_{11} &= A_3 \vee A_6 \vee A_{11} \vee A_{14} = T_4 \bar{T}_5; \\
 y_{12} &= A_4 \vee A_9 \vee A_{11} = \bar{T}_1 T_2 T_3 \bar{T}_4 \vee T_1 T_4 T_5 \vee \bar{T}_1 \bar{T}_2 T_4 = F_{17} \vee F_{18} \vee F_{19}; \\
 y_{13} &= A_2 \vee A_5 \vee A_5 = T_1 \bar{T}_2 T_5 \vee \bar{T}_1 T_4 T_5 = F_{20} \vee F_{21}.
 \end{aligned} \tag{4.43}$$

As follows from (4.40) the set $Y2 = Y2_S \cup Y2_m$, where $Y2_S \cap Y2_m = \emptyset$. The set $Y2_S$ includes MOs represented by a single term. It is the set $Y2_S = \{y_8, \dots, y_{11}\}$. The set $Y2_m$ includes other MOs $y_n \in Y2$. It is the set $Y2_m = \{y_{12}, y_{13}\}$ in the discussed case. The set F_{Y2} includes terms from SOPs of $y_n \in Y2_m$. It is the set $F_{Y2} = \{F_{17}, \dots, F_{21}\}$ in the case of CFSM $P_E Y2(\Gamma_7)$.

In the case of RSA, the system (1.43) is implemented as it is shown in Fig. 4.26.

In the best case, there is $Y2_S = Y2$. It leads to elimination of the matrix M_{Y2} . In the worst case, there is $Y2_S = \emptyset$. It leads to $Y2_m = Y2$. We have such a case in the previous example.

As follows from (4.43), the matrix circuit for M_A and M_{Y2} includes two matrices (Fig. 4.27). There is no variable $\bar{T}_3$ in equations of (4.43). So, there $2R_K - 1 = 9$ inputs in matrix M_A .

Let I_R be the number of inputs of M_A , $NY2_S = |Y2_S|$, $H_{Y2} = |F_{Y2}|$, $NY2_m = |Y2_m|$. In this case, we have:

$$S(M_A) = I_R \cdot (NY2_S + H_{Y2}); \tag{4.44}$$

$$S(M_{Y2}) = H_{Y2} \cdot NY2_m. \tag{4.45}$$

Fig. 4.27 Matrix circuit of $\langle M_A, M_{Y2} \rangle$ for CFSM $P_E Y2(\Gamma_7)$ with RSA

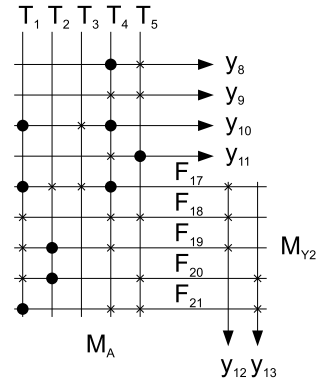
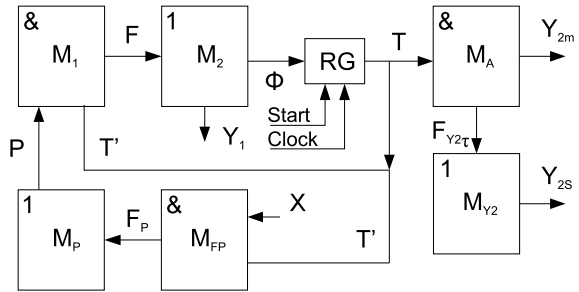


Fig. 4.28 Structural diagram of $M P_{ER} Y2$ CFSM



Using (4.44)–(4.45) and values $I_R = 9$, $NY2_S = 4$, $H_{Y2} = 5$, $NY2_m = 2$ gives $S(M_A) = 81$ and $S(M_{Y2}) = 10$. There is no change for $S(M_L)$ and $S(M_2)$ for the case of RSA. So, using (4.41), (4.44) and (4.45) gives the value $S_T[P_{ER} Y2(\Gamma_7)] = 603$. We use the subscript “R” to show that the RSA is used in $P_E Y2$ CFSM. This result is better than any from Table 4.18.

Combined using RLC and RSA leads to $M P_{ER} Y2$ CFSM. Its matrix circuit is shown in Fig. 4.28.

The functions of all matrices are clear. Let us find the area $S_T[M P_{ER} Y2(\Gamma_7)]$. To do it, we should clarify the parameters $S(M_{FP})$, $S(M_P)$ and $S(M_1)$. They are the following:

$$\begin{aligned} S(M_{FP}) &= (2R_A + L) \cdot H_P = (6 + 7) \cdot 7 = 91; \\ S(M_P) &= H_P \cdot G = 7 \cdot 2 = 14; \\ S(M_1) &= 2(G + R_A) \cdot H_T = 2(2 + 3) \cdot 16 = 160. \end{aligned} \quad (4.46)$$

Using (4.46), $S(M_2) = 192$, $S(M_A) = 81$ and $S(M_{Y2}) = 10$ gives $S_T[M P_{ER} Y2(\Gamma_7)] = 548$. It is the best result among all obtained for synthesis based on the GSA Γ_7 .

Let us point out that the RSA can be used for improving characteristics of any CFSM. But the best result can be obtained if RSA targets only improvement the

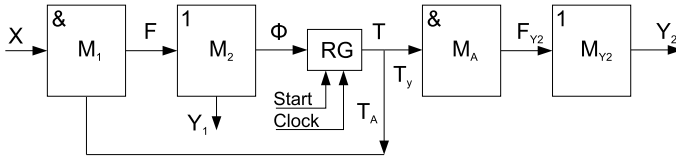


Fig. 4.29 Structural diagram of $P_K Y2$ CFSM

system $Y2$. It is exactly the case for $P_E Y2$ CFSMs. If state assignment is just OSA, then the optimization of $Y2$ is a secondary task.

Now, let us discuss $P_K Y2$ CFSMs. They are based on the state codes' representation (3.45). There is the structural diagram of $P_K Y2$ CFSM shown in Fig. 4.29.

In $P_K Y2$ CFSM, the matrix M_1 implements the terms $F_h \in F$, where

$$F = F(T_A, X). \quad (4.47)$$

The variables $T_r \in T_A$ encode classes $B_i \in \Pi_A$. The matrix M_2 implements systems (3.1)–(3.2). But there is $R = R_A + R_Q$, where R_A is determined as (2.43) and R_Q as (1.32). The variables $T_r \in T_Y$ encode the CMOs $Y(a_m) \subseteq Y2$. As follows from Fig. 4.29, the variables $T_r \in T_Y$ enter M_{FY} and create the terms of the system $Y2$.

Let us discuss an example of synthesis for CFSM $P_K Y2(\Gamma_7)$. We have already the sets A and Π_A . There is $I = 7$, $R_A = 3$, $T_A = \{T_1, T_2, T_3\}$. Let us encode the classes $B_i \in \Pi_A$ in the trivial way: $K(B_1) = 000, \dots, K(B_7) = 110$.

Analysis of GSA Γ_7 shows that there are $Q = 10$ CMOs $Y_q \subseteq Y2$. They are the following: $Y_1 = \{\emptyset\}$, $Y_2 = \{y_8, y_{13}\}$, $Y_3 = \{y_{11}\}$, $Y_4 = \{y_{10}, y_{12}\}$, $Y_5 = \{y_9, y_{13}\}$, $Y_6 = \{y_8\}$, $Y_7 = \{y_9, y_{12}\}$, $Y_8 = \{y_{10}\}$, $Y_9 = \{y_8, y_{10}\}$ and $Y_{10} = \{y_{11}, y_{12}\}$. There is $Q = 10$, $R_Q = 4$, $T_Y = \{T_4, T_5, T_6, T_7\}$. So, there is $R = R_A + R_Q = 7$. It gives the sets $T = \{T_1, \dots, T_7\}$ and $\Phi = \{D_1, \dots, D_7\}$.

The following system represents MOs $y_n \in Y2$ as functions of CMOs $Y_q \subseteq Y2$:

$$\begin{aligned} y_8 &= Y_2 \vee Y_6 \vee Y_9; & y_{11} &= Y_3 \vee Y_{10}; \\ y_9 &= Y_5 \vee Y_7; & y_{12} &= Y_4 \vee Y_7 \vee Y_{10}; \\ y_{10} &= Y_4 \vee Y_8 \vee Y_9; & y_{13} &= Y_2 \vee Y_5. \end{aligned} \quad (4.48)$$

Let us encode the CMOs $Y_q \subseteq Y2$ in a way minimizing the number of terms in (4.45). Using the method [1] gives the codes shown in Fig. 4.30.

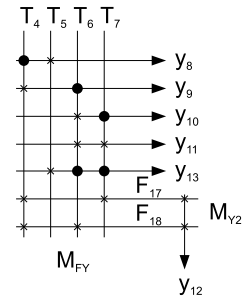
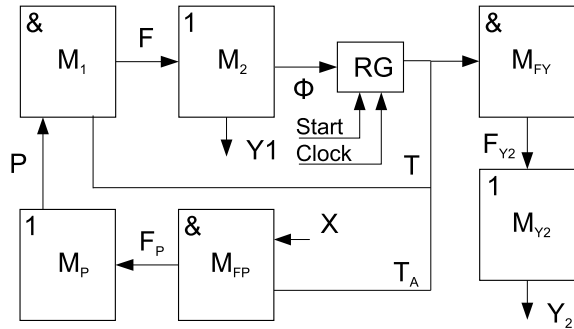
Using codes (Fig. 4.30) turns the system (4.48) in the following system:

$$\begin{aligned} y_8 &= \bar{T}_4 T_5; & y_{10} &= T_6 \bar{T}_7; & y_{12} &= T_4 T_7 \vee T_4 T_6 = F_{17} \vee F_{18}; \\ y_9 &= T_4 \bar{T}_6; & y_{11} &= T_6 T_7; & y_{13} &= T_5 \bar{T}_6 \bar{T}_7. \end{aligned} \quad (4.49)$$

As follows from (4.46), only the MO y_{12} is generated by the matrix M_{Y2} . Other MOs $y_n \in Y2$ are generated by the matrix M_{FY} . It gives the circuit (Fig. 4.31) hav-

Fig. 4.30 Codes of CMOs $Y_q \subseteq Y_7$

		T_4T_5			
		00	01	11	10
T_6T_7	00	Y_1	Y_2	Y_5	*
	01	*	Y_6	Y_7	*
	11	Y_3	*	Y_{10}	*
	10	Y_8	Y_9	Y_4	*

Fig. 4.31 Matrix circuit for M_{FY} and M_{Y2} of CFSM $PP_K Y_2(\Gamma_7)$ **Fig. 4.32** Structure diagram of $MP_K Y_2$ CFSM

ing $S(M_{FY}) + S(M_{Y2}) = 7 \cdot 7 + 2 \cdot 1 = 51$. As follows from Fig. 4.31, the variable $T_5 = 1$. So, we have $(2R_Y - 1)$ inputs of M_{FY} .

Let us estimate areas of M_1 and M_2 for this example:

$$\begin{aligned} S(M_1) &= 2(R_A + L) \cdot H_T = 2(3 + 7) \cdot 16 = 160; \\ S(M_2) &= H_T(R_A + R_Y + N1) = 16(3 + 4 + 7) = 224. \end{aligned} \quad (4.50)$$

So, there is $S_T[PP_K Y_2(\Gamma_7)] = 435$. It is the best result from all discussed before. Let us point out that this value can be diminished due to using the RLC. It leads to $MP_K Y_2$ CFSM (Fig. 4.32).

So, there is a huge potential in methods of primary encoding of classes of PES. Naturally, the degree of optimization depends on the characteristics of initial GSA Γ .

The discussed methods can be used for designing the CPLD- and FPGA-based CFSMs. Naturally, they should be improved taking into account the peculiarities of these chips.

References

1. Achasova S (1987) Synthesis algorithms for automata with PLAs. M: Soviet radio (in Russian)
2. Barkalov A (1995) Multilevel PLA schemes for microprogram automata. *Cybern Syst Anal* 31(4):489–495
3. Barkalov A (1998) Principles of logic optimization for Moore microprogram automaton. *Cybern Syst Anal* 34(1):54–60
4. Barkalov A, Titarenko L (2009) Logic synthesis for FSM-based control units, vol 53. *Lecture Notes in Electrical Engineering*. Springer-Verlag, Berlin
5. Barkalov A, Titarenko L (2009) Synthesis of operational and control automata. UNITECH, Donetsk
6. Barkalov A, Węgrzyn M (2006) Design of control units with programmable logic. University of Zielona Góra Press

Chapter 5

Homogenous CPLD-Based Synthesis of CFSMs



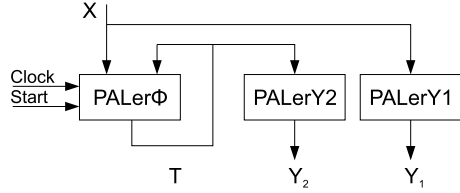
Abstract The chapter deals with synthesis of CFSMs using the homogenous CPLDs, which include either PAL- or PLA-based macrocells. It starts from synthesis of trivial single-level circuits. The PAL-based circuits are mostly analyzed. Next, there are discussed different methods of optimization combined with the optimal state assignment. Further, the methods are discussed based on the transformation of state codes into codes of classes of PES. This approach is combined with replacement of logical conditions and encoding of collections of microoperations. Next, the optimization methods are discussed based on the primary encoding of the classes of PES. Both methods of expansion of code space and nonstandard representation of state codes are discussed. They are combined with other methods of hardware reduction. All proposed methods target reducing the numbers of product terms in systems of Boolean functions representing CFSM circuits. The examples of synthesis are given for majority of proposed CFSM models.

5.1 Preliminary Information

To get optimal characteristics of CFSM circuit, is necessary to take into account the peculiarities of logic elements in use [4, 11]. Let us analyze these features of homogenous CPLDs. This class includes CPLDs having either PAL- or PLA-based macrocells. There are the following peculiarities of PAL-based macrocells:

1. Each MC has only a single output. It means that the optimization of CFSM circuit is reduced to independent optimization of each Boolean function. It means there is no place for collaborative optimization of SBFs. It greatly simplifies the problem of CFSM circuit's optimization.
2. Outputs of MCs are connected with programmable flip-flops. It allows creating the hidden distributed registers for keeping the state codes. As a rule, these flip-flops are tuned as D-type FFs [13].
3. There is a wide fan-in in each macrocell. The number of inputs (S) is enough for implementing any term of standard benchmarks (FSMs collected in the library [8]). It also simplifies the problem of synthesis. In fact, there is no need in the using of RLC for a majority of FSMs.

Fig. 5.1 Structural diagram of PAL-based P CFSM



4. There is a very limited number of terms (q) per a MC. As a rule $q \leq 8$. It brings to the fore a problem of reducing the number of product terms in each Boolean function representing the CFSM circuit. So, it is necessary to use PES-based methods of optimization.

A CFSM circuit is represented by the SFBs: $\Phi = \Phi(T, X)$, $Y1 = Y1(T, X)$ and $Y2 = Y2(T)$. So it can be viewed as a composition of three PALers (Fig. 5.1).

There is a P CSFM shown in Fig. 5.1. There is a distributed RG inside the block $PALer\Phi$. Because of it, pulses *Clock* and *Start* enter this block.

Let the following conditions takes places:

$$Q(f_i) \leq q; \quad (5.1)$$

$$L + R \leq S. \quad (5.2)$$

In this case it is necessary exactly $R + N1 + N2$ macrocells of PAL to implement the circuit of CFSM.

We assume that $f_i \in \Phi \cup Y1 \cup Y2$, so it is $i \in \{1, \dots, R + N1 + N2\}$.

If condition (5.1) is violated, then it is necessary more that a single MC to implement corresponding BF. There are efficient methods allowing reducing the number of MCs in this case [7, 13]. We do not discuss them. We just try to diminish the value of $Q(f_i)$ for maximum possible amount of functions.

Minimizing the number of terms, we assume that this reduces the number of interconnections within the macrocells. In turn, this reduces the energy consumption of CFSM circuit. We also think that the lower is the value $Q(f_i)$, the greater is the probability that only a single MC will be enough for implementing a circuit corresponding to the function f_i . Also, it is necessary to reduce the number of arguments in functions representing CFSM circuit. It allows reducing the number of interconnections between the MCs of this circuit. It significantly diminishes the energy consumed by the circuit.

There are the following peculiarities of PLA-based macrocells:

1. Each MC has S inputs, t outputs and q terms. So, it is equal to $PLA(S, t, q)$ discussed in many references [2, 3, 6, 10]. Because of it, all known optimization methods can be used in CFSM design based on PLA-type CPLDs.
2. Each output of MC is connected with a programmable FF. It allows implementing distributed registers to keep state codes.

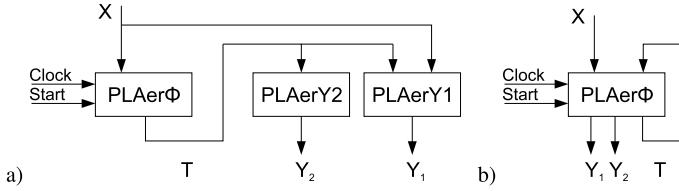


Fig. 5.2 Structural diagram of PAL-based P CFSM as a composition of PLAers (a) and as a single PLAer (b)

3. There is a wide fan-in in MCs. So, to minimize the number of MCs in a circuit, it is necessary to minimize the number of terms in SBFs $\Phi(T, X)$, $Y1(T, X)$ and $Y2(T)$. It is necessary to use the joint minimization of SBF, because each PLA implements an SBF having around t functions.

The trivial structural diagram of P CFSM is shown in Fig. 5.2a. We show separate blocks for each system of functions. Of course, the same PLAers can implement functions from different sets. It leads to the circuit shown in Fig. 5.2b.

In this book, we mostly discuss the design methods targeting PAL-based CPLDs. The PLA-based methods are very similar to ones we discussed for ASIC-based CFSMs. Obviously, the connection of matrices AND and OR gives a PLA. Of course, it is possible to change the values of S , t , q for the ASIC-case. In the case of PLA-based CPLDs, these values are fixed.

Let us name the P CFSM (Fig. 5.1) as an automaton with a base structure. Let us discuss how to design the P CFSM.

5.2 Synthesis of P CFSM

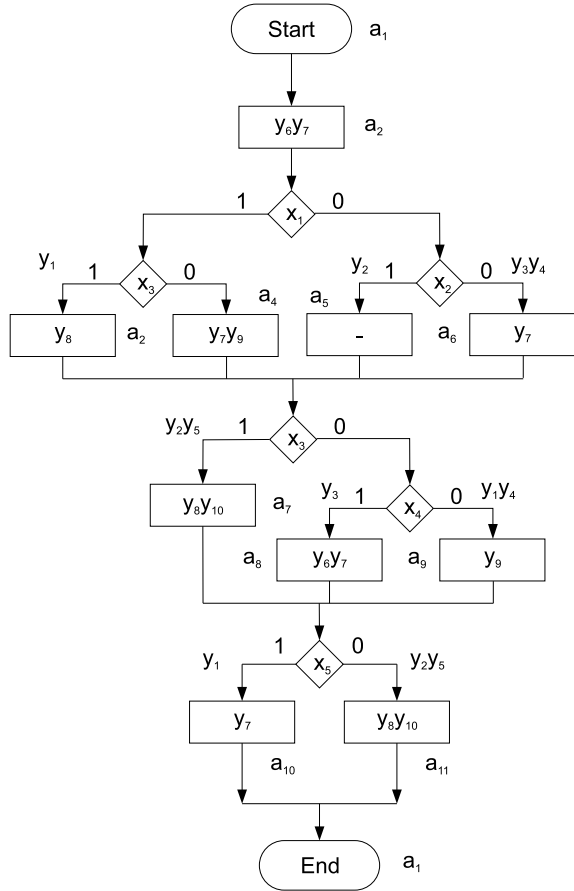
There are the following steps in P CFSM design method:

1. Marking initial GSA Γ by states $a_m \in A$.
2. Executing the state assignment.
3. Constructing the DST of P CFSM.
4. Deriving the systems (1.4), (1.42) and (1.43) from DST.
5. Creating the CFSM logic circuit.

Let us discuss an example of synthesis for CFSM $P(\Gamma_8)$. There is the marked GSA Γ_8 shown in Fig. 5.3.

The following sets can be derived from GSA Γ_8 : $A = \{a_1, \dots, a_{11}\}$, $X = \{x_1, \dots, x_5\}$, $Y1 = \{y_1, \dots, y_5\}$, $Y2 = \{y_6, \dots, y_{10}\}$. It gives $M = 11$, $L = N1 = N2 = 5$, and $N = 10$. Using (1.2) gives $R = 4$, $T = \{T_1, \dots, T_4\}$, $\Phi = \{D_1, \dots, D_4\}$. Let us encode the states in the trivial way: $K(a_1) = 0000, \dots, K(a_{11}) = 1010$.

Using rules [2] gives the DST of CFSM $P(\Gamma_8)$. It has $H = 25$ rows (Table 5.1).

Fig. 5.3 Marked GSA Γ_8 

Functions (1.4) and (1.42) depend on terms (1.10). As follows from Table 5.1, it is necessary to find the terms from F_1 to F_{23} . There are no functions $y_n \in Y_1$ or $D_r \in \Phi$ in the rows 24 and 25. So, the terms F_{24} and F_{25} do not enter SOPs of (1.4) and (1.42).

For example, the following functions can be derived from (Table 5.1):

$$\begin{aligned}
 y_1 &= F_2 \vee F_8 \vee F_{11} \vee F_{14} \vee F_{17} \vee F_{18} \vee F_{20} \vee F_{22}; \\
 y_2 &= F_4 \vee F_6 \vee F_9 \vee F_{12} \vee F_{15} \vee F_{19} \vee F_{21} \vee F_{23}; \\
 D_1 &= F_8 \vee F_{11} \vee F_{14} \vee F_{17} \vee F_{18} \vee F_{19} \vee F_{20} \vee F_{21} \vee F_{22} \vee F_{23}.
 \end{aligned} \tag{5.3}$$

Functions (1.43) depend on terms A_m corresponding to codes $K(a_m)$:

Table 5.1 Direct structure table of CFSM $P(\Gamma_8)$

a_m	$K(a_m)$	$Y(a_m)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
a_1	0000	–	a_2	0001	1	–	D_4	1
a_2	0001	y_6, y_7	a_3	0010	$x_1 x_3$	y_1	D_3	2
			a_4	0011	$x_1 \bar{x}_3$	–	$D_3 D_4$	3
			a_5	0110	$\bar{x}_1 x_2$	y_2	D_2	4
			a_6	0101	$\bar{x}_1 \bar{x}_2$	y_3, y_4	$D_2 D_4$	5
a_3	0010	y_8	a_7	0110	x_3	y_2, y_5	$D_2 D_3$	6
			a_8	0111	$\bar{x}_3 x_4$	y_3	$D_2 D_3 D_4$	7
			a_9	1000	$\bar{x}_3 \bar{x}_4$	y_1, y_4	D_1	8
a_4	0011	y_7, y_9	a_7	0011	x_3	y_2, y_5	$D_2 D_3$	9
			a_8	0111	$\bar{x}_3 x_4$	y_3	$D_2 D_3 D_4$	10
			a_9	1000	$\bar{x}_3 \bar{x}_4$	y_1, y_4	D_1	11
a_5	0100	–	a_7	0011	x_3	y_2, y_5	$D_2 D_3$	12
			a_8	0111	$\bar{x}_3 x_4$	y_3	$D_2 D_3 D_4$	13
			a_9	1000	$\bar{x}_3 \bar{x}_4$	y_1, y_4	D_1	14
a_6	0101	y_7	a_7	0011	x_3	y_2, y_5	$D_2 D_3$	15
			a_8	0111	$\bar{x}_3 x_4$	y_3	$D_2 D_3 D_4$	16
			a_9	1000	$\bar{x}_3 \bar{x}_4$	y_1, y_4	D_1	17
a_7	0110	y_8, y_9	a_{10}	1001	x_5	y_1	$D_1 D_4$	18
			a_{11}	1010	$\bar{x}_5$	y_2, y_5	$D_1 D_3$	19
a_8	0111	y_6, y_7	a_{10}	1001	x_5	y_1	$D_1 D_4$	20
			a_{11}	1010	$\bar{x}_5$	y_2, y_5	$D_1 D_3$	21
a_9	1000	y_9	a_{10}	1001	x_5	y_1	$D_1 D_4$	22
			a_{11}	1010	$\bar{x}_5$	y_2, y_5	$D_1 D_3$	23
a_{10}	1001	y_7	a_1	0000	1	–	–	24
a_{11}	1010	y_8, y_9	a_1	0000	1	–	–	25

$$A_m = \bigwedge_{r=1}^R T_r^{l_{mr}} \quad (m \in \{1, \dots, M\}). \quad (5.4)$$

In (5.4), $l_{mr} \in \{0, 1\}$ is a value of the bit r of the code $K(a_m)$, $T_r^0 = \bar{T}_r$, $T_r^1 = T_r$ ($r \in \{1, \dots, R\}$).

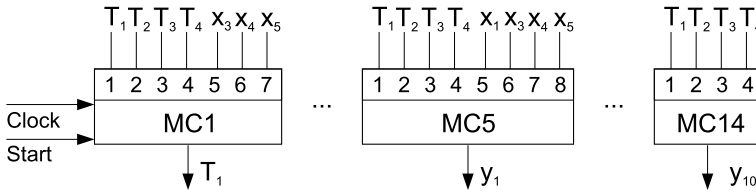
For example, the following functions $y_n \in Y_2$ can be found from the column $Y(a_m)$ of Table 5.1:

$$y_6 = A_2 \vee A_8; \quad y_7 = A_2 \vee A_4 \vee A_6 \vee A_8 \vee A_{10}. \quad (5.5)$$

Let us form Table 5.2 with characteristics of functions $D_r \in \Phi$, $y_n \in Y_1$ and $y_n \in Y_2$. It includes the numbers of terms ($H(f_i)$) and literals ($L(f_i)$) for each function representing CFSM $P(\Gamma_8)$.

Table 5.2 Characteristics of function for $P(\Gamma_8)$

f_i	$H(f_i)$	$L(f_i)$	f_i	$H(f_i)$	$L(f_i)$
D_1	10	7	y_4	5	8
D_2	10	8	y_5	7	6
D_3	13	8	y_6	2	4
D_4	10	9	y_7	5	4
y_1	8	8	y_8	3	4
y_2	8	8	y_9	2	4
y_3	5	8	y_{10}	2	4

**Fig. 5.4** A part of PAL-based circuit of CFSM $P(\Gamma_8)$

As follows from Table 5.2, there is the maximum number $H(f_i) = H(D_3) = 12$ and the maximum number $L(f_i) = L(D_4) = 9$. If we can use CPLD chip with PAL(9,13) (where $S = 9$ and $q = 13$), then it is enough a single PAL to implement any function representing $P(\Gamma_8)$. A part of this circuit is shown in Fig. 5.4.

We denote macrocells as MCi ($i = \overline{1, 14}$). The MC1–MC4 implements IMF D_1 – D_4 . So, the outputs of corresponding PALs are connected with inputs D of flip-flops. The pulses *Start* and *Clock* enter these four MCs to control the operation of FFs.

In the discussed case, the following conditions take places:

$$\max H(f_i) \leq q; \quad (5.6)$$

$$\max L(f_i) \leq S. \quad (5.7)$$

The condition (5.6) is the same as (5.1), the condition (5.7) is rather weaker than (5.2).

If conditions (5.6)–(5.7) take places, then an CFSM is represented by a single-level circuit. There is only a single level of MCs between inputs ($x_l \in X, T_r \in T$) and outputs $D_r \in \Phi, y_n \in Y1 \cup Y2$ of the circuit. It is the fastest solution among all existing. In addition, there is the minimum possible numbers of MCs used by the circuit. So, the conditions (5.6)–(5.7) represent the best combination of characteristics of CFSM and CPLD.

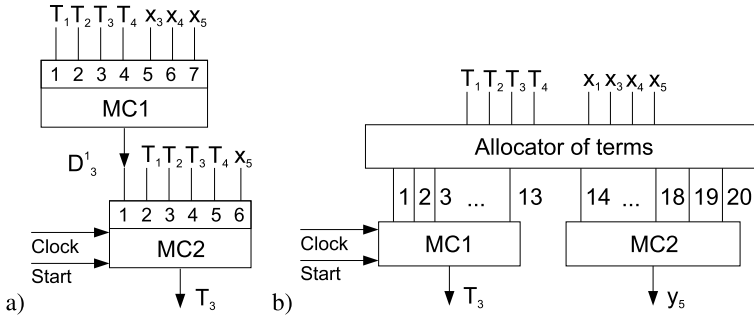


Fig. 5.5 Implementing functions without (a) and with (b) allocator of terms

If the condition (5.6) is violated, then it is necessary more than q terms to implements the circuit for corresponding functions. The solution depends on the peculiarities of a CPLD chip. There are the following situations:

1. There is no possibility to use some terms of other MC. It is the CPLDs without the allocator of terms. In this case, it is necessary more than a single MC to implement the function f_i .

For example, let it be PAL(9,10). In this case, the condition (5.6) is violated, as well as the condition (5.1) for function D_3 with $H(D_3) = 12$. So, it is necessary to divide the SOP of D_3 by 2 parts:

$$\begin{aligned} D_3^1 &= F_2 \vee F_3 \vee F_6 \vee F_7 \vee F_9 \vee F_{10} \vee F_{12} \vee F_{13} \vee F_{15} \vee F_{16}; \\ D_3^2 &= F_{19} \vee F_{21} \vee F_{23}. \end{aligned} \quad (5.8)$$

Analysis of function (5.8) gives $H(D_3^1) = 10$, $L(D_3^1) = 7$, $H(D_3^2) = 3$ and $L(D_3^2) = 5$. Now, conditions (5.6)–(5.7) take places for D_3^1 and D_3^2 . It gives the circuit shown in Fig. 5.5a.

If it is possible to use the allocator, we should find a pair of functions such that

$$H(f_i) + H(f_j) \leq 2q. \quad (5.9)$$

In this case, we can use more than q terms to implement the function violating (5.6).

Let we have PALs with $S = 9$ and $q = 10$. Let us organize the pairs $\langle D_3, y_5 \rangle$ having together 20 terms. Now, we should place these functions in the adjacent MCs. We use 7 mutual terms for y_5 and the rest for D_3 (Fig. 5.5b).

In this case, a MC is treated as an OR gate. For D_3 , the gate output is connected with FF.

There are two levels of logic in the circuit from Fig. 5.5a. Of course, the allocator adds some delay, but the circuit (Fig. 5.5b) has only a single level. Moreover, no macrocells are wasted as elements of the intermediate logic levels.

As follows from this example, the fewer terms are included in the SOPs of functions, the easier the process of implementing the logic circuits. Therefore, all further methods are focused on reducing the number of terms in SBFs representing the logic circuits of CFSMs. In the case of PLA-based CPLDs, it is necessary to execute the joint minimization of SBFs. Let we have $PLA(S, t, q)$ as a MC. Let the condition (5.2) take place. It means that is necessary at least n_0 MCs to implement the circuit:

$$n_0 = \left\lceil \frac{R + N1 + N2}{t} \right\rceil. \quad (5.10)$$

The formula (5.10) determines the minimum possible number of macrocells in a CFSM logic circuit. To implement the circuit, it is necessary to form at least n_0 sets of functions having not more than q mutual terms. To solve this problem, well-known methods [2, 10] can be used.

Let us discuss an example of synthesis the CFSM $P(\Gamma_8)$ using $PLA(9, 4, 16)$. Because of $L + R = 9$, the condition (5.2) takes place. Using (5.10) gives $n_0 = \lceil 14/4 \rceil = 4$. So, it is necessary at least 4 MCs to implement the circuit.

There is $\max H(f_i) = D(D_3) = 13 < q = 16$. So, each function can be implemented using only a single MC. Let us find the partition $\Pi_F = \{F_1, \dots, F_I\}$ of the set $\Phi \cup Y1 \cup Y2$ giving the minimum number of macrocells. In the discussed case, it is possible to implement some functions using resources of more than a single PLA. The number of “redundant” outputs is determined as

$$N0_R = n_0 \cdot t - (R + N1 + N2). \quad (5.11)$$

There is $N0_R = 16 - 14 = 2$ in the discussed case. We can use this redundancy to get a partition Π_F with $I = n_0$.

For example, the following partition can be found $\Pi_F = \{F_1, \dots, F_4\}$. It includes the classes $F_1 = \{D_3, y_2, y_5, y_9\}$, $F_2 = \{D_1, y_1, y_6, y_{10}\}$, $F_3 = \{D_2, y_3, y_8, y_7^1\}$, $F_4 = \{D_4, y_4, y_7^2\}$. We represented the MO y_7 as $y_7^1 \vee y_7^2$ where $y_7^1 = A_2 \vee A_4$, $y_7^2 = A_6 \vee A_8 \vee A_{10}$. There is the logic circuit of CFSM $P(\Gamma_8)$ with $PLA(9, 4, 16)$ shown in Fig. 5.6. We use the symbol X^i to denote the LCs $x_i \in X$ entering $PLAi$ corresponding to $F_i \in \Pi_F$.

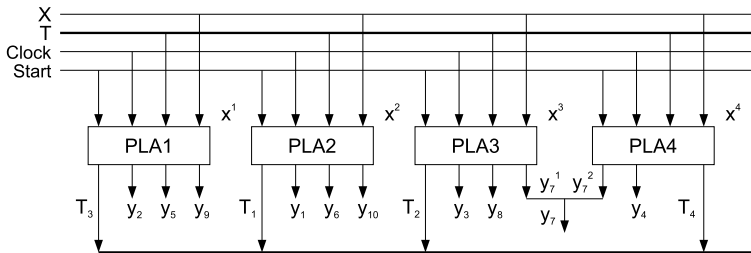


Fig. 5.6 Logic circuit of PLA-based CFSM $P(\Gamma_8)$

Table 5.3 DST for PLA2 of CFSM $P(\Gamma_8)$

a_m	$K(a_m)$	$Y(a_m)$	X_q	Y_q	Φ_q	h	q
a_2	0001	–	x_2x_3	y_1	–	2	1
a_3	0010	–	$\bar{x}_3\bar{x}_4$	y_1	D_1	8	2
a_4	0011	–	$\bar{x}_3\bar{x}_4$	y_1	D_1	11	3
a_5	0100	–	$\bar{x}_3\bar{x}_4$	y_1	D_1	14	4
a_6	0101	–	$\bar{x}_3\bar{x}_4$	y_1	D_1	17	5
a_7	0110	–	x_5	y_1	D_1	18	6
a_7	0110	–	$\bar{x}_5$	–	D_1	19	7
a_8	0111	–	x_5	y_1	D_1	20	8
a_8	0111	–	$\bar{x}_5$	–	D_1	21	9
a_9	1000	–	x_5	y_1	D_1	22	10
a_9	1000	–	$\bar{x}_2$	–	D_1	23	11
a_2	0101	y_6	–	–	–	–	12
a_7	0110	y_{10}	–	–	–	–	13
a_8	0111	y_6	–	–	–	–	14
a_{11}	1010	y_{10}	–	–	–	–	15

To get the function y_7 , we can use, for example, the three-stable buffers [12, 13]. We show such an approach in Fig. 5.6. If we cannot use this approach, then there are two possibilities:

1. The output y_7^1 is connected with an input of PLA4. In this case, the function y_7 is generated by PLA4. It leads to doubling the time of performance, because there are two levels of MCs necessary to generate the function y_7 .
2. The function y_7 is implemented using a separate MC PLA5. It allows to get the maximum operating frequency. But the hardware amount is increased by 25% as compared with the circuit (Fig. 5.6).

To find the information for programming the PLA_i, it is necessary to create DST_i. It is constructed on the base of initial DST and the partition Π_F . It includes the columns a_m , $K(a_m)$, $Y(a_m)$, X_q , Y_q , Φ_q , h , q . The columns X_q , Y_q , Φ_q contain information for columns X_h , Y_h , Φ_h of initial DST, respectively. The column h shows term F_h of initial DST is implemented as the term number q of DST_i.

For example, the following terms should be implemented by PLA2: F_2 , F_8 , F_{11} , F_{14} , F_{17} , F_{18} , F_{19} , F_{20} , F_{21} , F_{22} , F_{23} , A_2 , A_7 , A_8 , A_{10} . The corresponding DST is represented by Table 5.3. To program the connections inside planes AND and OR, it is necessary to form the table of programming. It shows inputs, terms and outputs of a MC_i. Three symbols are used to shown the connections between inputs and terms: (1) “0” means that the complimentary value of input enters the term; (2) “1” that the direct value of input enters the term and (3) “×” means no connection. For example, Table 5.4 shows how to program the PLA2.

Table 5.4 Table of programming for PLA2

Inputs								Terms	Outputs			
T_1	T_2	T_3	T_4	x_2	x_3	x_4	x_5	F_q	y_1	y_6	y_{10}	D_1
0	0	0	1	×	1	×	×	1	1	0	0	0
0	0	1	0	×	0	0	×	2	1	0	0	1
0	0	1	1	×	0	0	×	3	1	0	0	1
0	1	0	0	×	0	0	×	4	1	0	0	1
0	1	0	1	×	0	0	×	5	1	0	0	1
0	1	1	0	×	×	×	1	6	1	0	0	1
0	1	1	0	×	×	×	0	7	0	0	0	1
0	1	1	1	×	×	×	1	8	1	0	0	1
0	1	1	1	×	×	×	0	9	0	0	0	1
1	0	0	0	×	×	×	1	10	1	0	0	1
1	0	0	0	×	×	×	0	11	0	0	0	1
0	1	0	1	×	×	×	×	12	0	1	0	0
0	0	1	0	×	×	×	×	13	0	0	1	0
0	0	1	1	×	×	×	×	14	0	1	0	0
1	0	1	0	×	×	×	×	15	0	0	1	0

We hope there is a transparent connection between Tables 5.1, 5.3 and 5.4. All other tables can be constructed in the similar way.

Now, it is clear the importance of reducing the numbers of terms in SBFs representing CFSM circuits. Let us discuss these methods. All of them are based on partition of the set A by classes of PES.

5.3 Synthesis of CFSMs with Optimal State Assignment

Using the OSA does not change the structural diagram of CFSM compared to Fig. 5.1. Moreover, a part of functions can depend on state variables $T_r \in T'$, where $T' \subseteq T$.

Let us discuss an example of synthesis for CFSM $P_0(I_8)$. Let us use PAL-based macrocells. The method of synthesis is the same as for P CFSM. We just should execute OSA instead of the trivial state assignment. The first step of synthesis is the same as we got in Sect. 5.2. Using Definition 3.1 gives us the partition $\Pi = \{B_1, \dots, B_5\}$ with the following classes: $B_1 = \{a_1\}$, $B_2 = \{a_2\}$, $B_3 = \{a_3, \dots, a_6\}$, $B_4 = \{a_7, a_8, a_9\}$, $B_5 = \{a_{10}, a_{11}\}$. Using [1] gives codes shown in Fig. 5.7.

Using Fig. 5.7 gives the class codes $K(B_1) = 000*$, $K(B_2) = 001*$, $K(B_3) = 01**$, $K(B_4) = 10**$ and $K(B_5) = 11**$.

So, each class $B_i \in \Pi_A$ is represented by a single generalized interval of 4-dimensional Boolean space.

To form DST of P_0 CFSM, let us construct the following system of GFTs:

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	a_1	a_3	a_{10}	a_7
	01	*	a_4	a_{11}	a_8
	11	*	a_6	*	*
	10	a_2	a_5	*	a_9
		B_3	B_5	B_4	

Fig. 5.7 Outcome of OSA for $P_0(\Gamma_8)$ **Table 5.5** DST of CSFM $P_0(\Gamma_8)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	000*	a_2	0010	1	—	D_3	1
B_2	001*	a_3	0100	$x_1 x_3$	y_1	D_2	2
		a_4	0101	$x_1 \bar{x}_3$	—	$D_2 D_4$	3
		a_5	0110	$\bar{x}_1 x_2$	y_2	$D_2 D_3$	4
		a_6	0111	$\bar{x}_1 \bar{x}_2$	$y_3 y_4$	$D_2 D_3 D_4$	5
B_3	01**	a_7	1000	x_3	$y_2 y_5$	D_1	6
		a_8	1001	$\bar{x}_3 x_4$	y_3	$D_1 D_4$	7
		a_9	1010	$\bar{x}_3 \bar{x}_4$	$y_1 y_4$	$D_1 D_3$	8
B_4	10**	a_{10}	1100	x_5	y_1	$D_1 D_2$	9
		a_{11}	1101	$\bar{x}_5$	$y_2 y_5$	$D_1 D_2 D_4$	10

$$\begin{aligned}
B_1 &\rightarrow a_2; \\
B_2 &\rightarrow x_1 x_3 a_3 \vee x_1 \bar{x}_3 a_4 \vee \bar{x}_1 x_2 a_5 \vee \bar{x}_1 \bar{x}_2 a_6; \\
B_3 &\rightarrow x_3 a_7 \vee \bar{x}_3 x_4 a_8 \vee \bar{x}_3 \bar{x}_4 a_9; \\
B_4 &\rightarrow x_5 a_{10} \vee \bar{x}_5 a_{11}; \\
B_5 &\rightarrow a_1.
\end{aligned} \tag{5.12}$$

There is $D_r = 0$ ($r = \overline{1, 4}$) for transitions from $B_5 \in \Pi_A$. So, we do not represent it in DST (Table 5.5).

There are $H_{opt} = 10$ rows in Table 5.5. It is the best possible outcome. It is the same as H_T (4.18). Let us find the systems $\Phi(T', X)$ and $Y1(T', X)$ from Table 5.5. After minimization we have

$$\begin{aligned}
y_1 &= F_2 \vee F_8 \vee F_9 & y_2 &= F_4 \vee F_6 \vee F_{10}; \\
y_3 &= F_5 \vee F_7; & y_4 &= F_5 \vee F_8; \\
y_5 &= F_6 \vee F_{10}; & D_1 &= \bar{T}_1 \bar{T}_2 \vee T_1 \bar{T}_2; \\
D_2 &= \bar{T}_1 \bar{T}_2 T_3 \vee T_1 \bar{T}_2; & D_3 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \vee T_1 \bar{T}_2 T_3 \bar{x}_1; \\
D_4 &= \bar{T}_1 \bar{T}_2 T_3 (x_1 \bar{x}_3 \vee \bar{x}_1 \bar{x}_2) \vee \bar{T}_1 T_2 \bar{x}_3 \bar{x}_4 \vee T_1 \bar{T}_2 \bar{x}_5.
\end{aligned} \tag{5.13}$$

Let us use a CPLD chip with MCs PAL(9,4). Analysis of (5.13) shows that the condition (5.6) takes place. So, there are $R = 4$ PALs in $PALer\Phi$ and $N1 = 5$ PALs in $PALerY1$. It is the best possible result.

Using GSA Γ_8 , we can find the system (1.8). It is the following:

$$\begin{aligned} y_6 &= A_2 \vee A_8; & y_7 &= A_2 \vee A_4 \vee A_6 \vee A_8 \vee A_{10}; \\ y_8 &= A_3 \vee A_7 \vee A_{11}; & y_9 &= A_4 \vee A_9; \\ y_{10} &= A_7 \vee A_{11}. \end{aligned} \quad (5.14)$$

As follows from (5.14), there is $Q(y_7) > q = 4$. So, it is necessary at least two macrocells to implement the circuit for $y_7 \in Y2$.

Let us try to optimize the system $Y2 = Y2(T)$. To do it, let us use the refined state assignment. It leads to P_{OR} CFSM having the same structural diagram as P CFSM.

There are two aims of RSA:

1. To represent as many as possible classes $B_i \in \Pi_A$ by a single generalized intervals of R-dimensional Boolean space. It minimizes the number of terms (and) literals in systems (1.4) and (1.7). It is the primary aim of RSA.
2. To represent as many as possible MOs $y_n \in Y2$ by a minimum number of generalized intervals of R-dimensional Boolean space. It is the secondary aim. It allows minimizing the circuit of $PALerY2$.

Also this approach reduces the number of literals in the implemented SBFs. It allows reducing the number of interconnections (and the consumed energy). Using algorithm [1] gives the codes shown in Fig. 5.8.

Using system (5.14) and state codes (Fig. 5.8) gives the following SBF:

$$\begin{aligned} y_6 &= T_1 \bar{T}_3; & y_7 &= \bar{T}_1 T_2 \bar{T}_3 \vee T_1 \bar{T}_3 \vee T_1 \bar{T}_2 \bar{T}_3; \\ y_8 &= T_3 T_4; & y_9 &= T_2 \bar{T}_3 T_4; \\ y_{10} &= T_1 T_3. \end{aligned} \quad (5.15)$$

As follows from (5.15), it is enough $q = 3$ to implement any function $y_n \in Y2$. There are $N2 = 5$ PALs in the $PALerY2$.

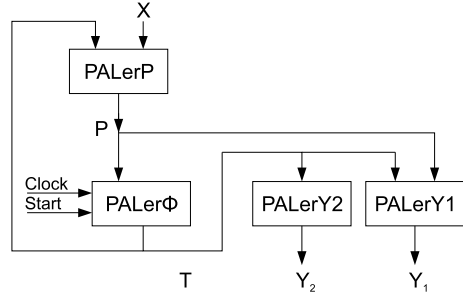
Because state codes are changed, it is necessary to check the systems $\Phi(T, X)$ and $Y1(T, X)$. To do it, let us find the codes $K(B_i)$. There are the following codes: $K(B_1) = 00**$, $K(B_2) = 10*0$, $K(B_3) = 01**$, $K(B_4) = 11**$ and $K(B_5) = 10*1$.

Fig. 5.8 Outcome of OSA
for $P_{OR}(\Gamma_8)$

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	a_1	a_6	a_8	a_2
	01	*	a_4	a_9	a_{10}
	11	*	a_3	a_7	a_{11}
	10	*	a_5	*	*

Table 5.6 Input memory functions for $P_{OR}(\Gamma_8)$

h	1	2	3	4	5	6	7	8	9	10
B_i	B_1	B_2				B_3			B_4	
$K(B_i)$	00**	10*0				01**			11**	
a_s	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
Φ_h	D_1	$D_2 D_3 D_4$	$D_2 D_4$	$D_2 D_3$	D_2	$D_1 D_2 D_3 D_4$	$D_1 D_2$	$D_1 D_2 D_4$	$D_1 D_4$	$D_1 D_3 D_4$

Fig. 5.9 Structural diagram of MP_0 CFSM

Each class $B_i \in \Pi_A$ is represented by a single interval of 4-dimensional space. So, there is no change for the number of terms for functions (1.42). To find the functions $D_r \in \Phi$, let us create Table 5.6. It includes functions $D_r \in \Phi$ generated during each transition.

Analysis of Table 5.6 shows that the functions $D_r \in \Phi$ differ from the ones from (5.13). For example, there is $D_1 = B_1 \vee B_3 \vee B_4 = \bar{T}_1 \bar{T}_2 \vee T_2$. But there is $Q(D_r) \leq q$ for each function $D_r \in \Phi$.

So, the RSA allows implementing each function of $P_{OR}(\Gamma_8)$ by a single PAL. It is the best result. Of course, it cannot be reached in any case. But it can improve the characteristics of P_0 CFSM.

For complex CFSM, it is possible that the condition (5.7) is violated. In this case, it is possible to use the RLC. It leads to MP_0 or MP_{OR} CFSMs. If PALs are used as MCs, then the structural diagram of MP_0 CFSM is shown in Fig. 5.9.

The following functions are executed by the blocks of MP_0 CFSM. The PALerP implements functions (2.18) entering other blocks as logical conditions. The PALerΦ implements system (2.23) and produces state variables $T_r \in T$. These variables are used by all blocks of the circuit. The PALerY1 implements functions

$$Y1 = Y1(T, P), \quad (5.16)$$

the PALerY2 implements the system (1.43).

If the condition (5.1) is violated for a majority of functions $y_n \in Y1$, then it is possible to use the encoding of CMOs created by the MOs. It leads to P_0Y1 CFSM (Fig. 5.10). The same diagram represents the $P_{OR}Y1$ CFSM.

In P_0Y1 CFSM, the PALerZ implements functions (2.26). These functions enter PALerY1 as arguments of functions

Fig. 5.10 Structural diagram of P_0Y1 CFSM

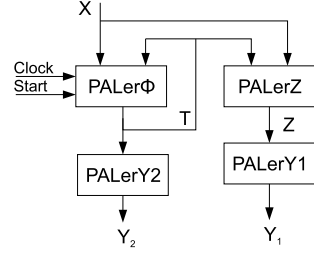
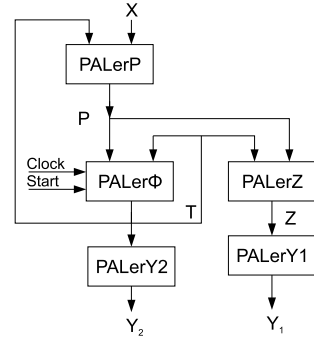


Fig. 5.11 Structural diagram of MP_0Y1 CFSM



$$Y1 = Y1(Z). \quad (5.17)$$

These methods can be combined. It leads to PAL-based MP_0Y1 (or $MP_{OR}Y1$) CFSM (Fig. 5.11).

Each block of MP_0Y1 CFSM implements functions already discussed. Only PALerZ implements functions (2.42) which are different from (2.26). The same structures have PLA-based P_0 , M_{OR} , MP_0 , MP_{OR} , P_0Y1 , $P_{OR}Y1$, MP_0Y1 and $MP_{OR}Y1$ CFSMs.

If some classes $B_i \in \Pi_A$ are represented by few generalized intervals of R -dimensional Boolean space, then $H_{opt} \neq H_T$ ($H_{opt} > H_T$). Let us consider the marked GSA Γ_9 (Fig. 5.12).

There are $M = 16$ states marking the operator vertices of GSA Γ_9 . They form a partition Π_A having $I = 6$ classes. There classes are the following: $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6, a_7\}$, $B_4 = \{a_8, a_9, a_{10}\}$, $B_5 = \{a_{11}, a_{12}, a_{13}\}$, $B_6 = \{a_{14}, a_{15}, a_{16}\}$. There is $R = 4$ and $2^R = 16 = M$. So, there are no insignificant input assignments which can be used for the optimal state assignment.

In this case, it is necessary to transform either all state codes into class codes or only part of state codes. It leads to P_C or P_{C1} CFSMs, respectively. Let us start from P_C CFSMs.

5.4 Synthesis of CFSMs with Transformation of State Codes

In this case, each class $B_i \in \Pi_A$ is encoded by a class code $K(B_i)$ having R_A bits. The value of R_A is determined by (2.43). Let us discuss the case of CFSM corresponding to GSA Γ_9 .

There is $R_A = 3$, which gives set $\tau = \{\tau_1, \tau_2, \tau_3\}$. Let us encode the classes in the trivial manner. It gives the codes $K(B_1) = 000, K(B_2) = 001, \dots, K(B_6) = 101$. The input assignments 110 and 111 can be used for optimizing SBFs representing the CFSM circuit.

Because each MC implements only a single BF for PAL-based CPLDs, we can shown the block PALer as a separate unit of the circuit. It leads to the structural diagram shown in Fig. 5.13.

In P_C CFSM, the PALer Φ implements functions $D_r \in \Phi$, where $\Phi = \Phi(\tau, X)$. The PALer τ implements the system (2.45). The PALerY1 implements the system

$$Y1 = Y1(\tau, X). \quad (5.18)$$

The PALerY2 implements the system (5.17). It is the same as for P or P_0 CFSMs.

In this book, we propose the method of synthesis of P_C CFSM. It includes the following steps:

1. Marking the initial GSA Γ and finding the set A .
2. Creating the partition A by classes of PES.
3. Executing the optimal state assignment.
4. Encoding of classes $B_i \in \Pi_A$.
5. Creating the systems of functions representing blocks of CFSM.
6. Implementing the CFSM logic circuit.

Let us discuss an example of synthesis for CFSM $P_C(\Gamma_9)$. There are already executed two first steps and the step 4. Let us execute the OSA. One of the possible variants of OSA is shown in Fig. 5.14.

As follows from Karnaugh map (Fig. 5.14), only the class $B_1 = \{a_1\}$ is represented by a single interval. All other classes need 2 generalized intervals of 4-dimensional space. So, it is necessary to use the model of P_C CFSM.

Let us estimate the value of H , $H_{opt} = 31$ and H_T for the discussed case. We can use the formulae (4.16)–(4.18). For GSA Γ_9 , there is $H = 45$, $H_{opt} = 31$ and $H_T = 17$.

To find the system (2.44), let us create the system of GFTs and, then, the DST of $P_C(\Gamma_9)$. The following system can be found from GSA Γ_9 :

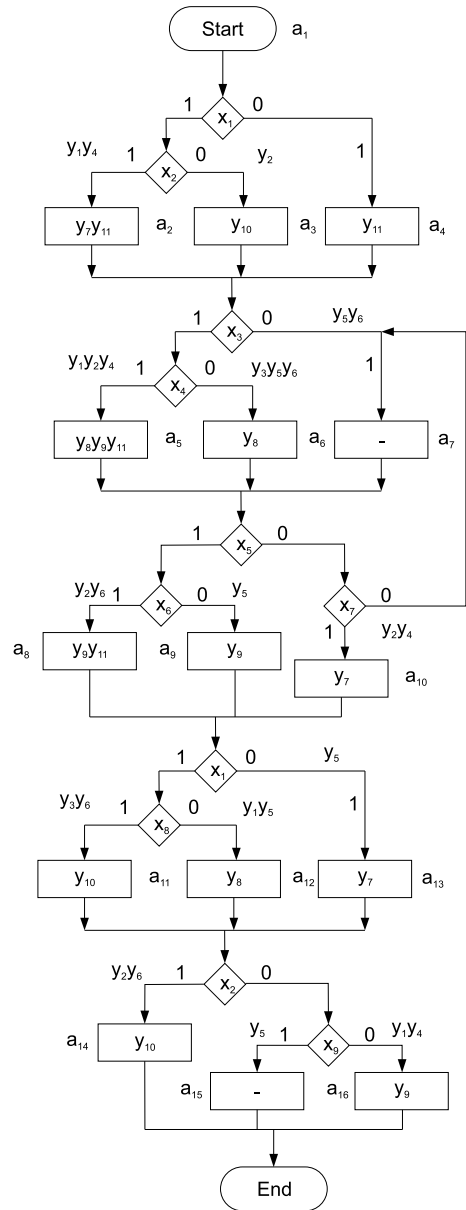
Fig. 5.12 Marked GSA I_9 

Fig. 5.13 Structural diagram of P_C CFSM

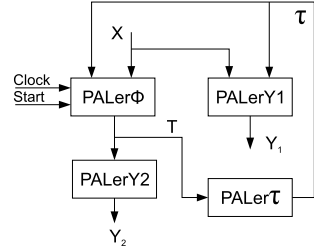


Fig. 5.14 Outcome of OSA for $P(\Gamma_9)$

		$T_1 T_2$			
$T_3 T_4$		00	01	11	10
	00	a_1	a_3	a_{11}	a_7
	01	a_{10}	a_2	a_{13}	a_6
	11	a_8	a_4	a_{12}	a_5
	10	a_9	a_{15}	a_{14}	a_{16}

$$\begin{aligned}
 B_1 &\rightarrow x_1 x_2 a_2 \vee x_1 \bar{x}_2 a_3 \vee \bar{x}_1 a_4; \\
 B_2 &\rightarrow x_3 x_4 a_5 \vee x_3 \bar{x}_4 a_6 \vee \bar{x}_3 a_7; \\
 B_3 &\rightarrow x_5 x_6 a_8 \vee x_5 \bar{x}_6 a_9 \vee \bar{x}_5 x_7 a_{10} \vee \bar{x}_5 \bar{x}_7 a_7; \\
 B_4 &\rightarrow x_1 x_8 a_{11} \vee x_1 \bar{x}_8 a_{12} \vee \bar{x}_1 a_{13}; \\
 B_5 &\rightarrow x_2 a_{14} \vee \bar{a}_2 x_9 a_{15} \vee \bar{x}_2 \bar{x}_9 a_{16}; \\
 B_6 &\rightarrow a_1.
 \end{aligned} \tag{5.19}$$

Using the system (5.19) and GSA Γ_9 allows constructing the DST (Table 5.7). We use codes from Fig. 5.14 in the column $K(a_s)$ of Table 5.7. This table includes only 16 rows which is less than H_T . We can eliminate the transition from B_6 because there are no MOs $y_n \in Y1$ generated during the transitions $\langle a_{14}, a_1 \rangle, \dots, \langle a_{16}, a_1 \rangle$.

Table 5.7 is the base for finding systems $\Phi(\tau, X)$ and $Y1(\tau, X)$. The system $Y2(T)$ can be found from the GSA Γ_9 using state codes from Fig. 5.14. To find the system $\tau(T)$ it is necessary to form a table giving dependence between states $a_m \in A$ and classes $B_i \in \Pi_A$.

For example, the following equations can be obtained from Table 5.7:

$$\begin{aligned}
 y_1 &= \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 x_1 x_2 \vee \bar{\tau}_1 \bar{\tau}_2 \tau_3 x_3 x_4 \vee \bar{\tau}_1 \tau_2 \tau_3 x_1 \bar{x}_8 \vee \tau_1 \bar{\tau}_2 \bar{\tau}_3 \bar{x}_2 \bar{x}_9 = \\
 &= F_1 \vee F_4 \vee F_{12} \vee F_{16}; \\
 D_1 &= \bar{\tau}_1 \tau_3 \vee \bar{\tau}_1 \tau_2 \bar{\tau}_3 \bar{x}_5 \bar{x}_7 \vee \tau_1 \bar{\tau}_2 \bar{\tau}_3 (x_2 \vee \bar{x}_9) = \\
 &= [F_4 \vee F_5 \vee F_6 \vee F_{11} \vee F_{12} \vee F_{13}] \vee F_{10} \vee F_{14} \vee F_{16}.
 \end{aligned} \tag{5.20}$$

As follows from (5.20), the function D_1 is minimized much better than y_1 . This larger depends on: (1) the configuration of GSA Γ_9 and (2) using OSA.

Let us point out of that Fig. 5.14 contains the RSA-based state codes. Due to it, we optimize both functions $\tau(T)$ and $Y2(T)$. The following system $Y2(T)$ can be derived from GSA Γ_9 :

Table 5.7 DST of CSFM $P_C(\Gamma_9)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	000	a_2	0101	x_1x_2	y_1y_2	D_2D_4	1
		a_3	0100	$x_1\bar{x}_2$	y_2	D_2	2
		a_4	0111	$\bar{x}_1$	—	$D_2D_3D_4$	3
B_2	001	a_5	1011	x_3x_4	$y_1y_2y_4$	$D_1D_3D_4$	4
		a_6	1010	$x_3\bar{x}_4$	$y_3y_5y_6$	D_1D_3	5
		a_7	1000	$\bar{x}_3$	y_5y_6	D_1	6
B_3	010	a_8	0011	x_5x_6	y_2y_6	D_3D_4	7
		a_9	0010	$x_5\bar{x}_6$	y_5	D_34	8
		a_{10}	0001	$\bar{x}_5x_7$	y_2y_4	D_4	9
		a_7	1000	$\bar{x}_5\bar{x}_7$	—	D_1	10
B_4	011	a_{11}	1100	x_1x_8	y_3y_6	D_1D_2	11
		a_{12}	1111	$x_1\bar{x}_8$	y_1y_5	$D_1D_2D_3D_4$	12
		a_{13}	1101	$\bar{x}_1$	y_5	$D_1D_2D_4$	13
B_5	100	a_{14}	1110	x_2	y_2y_6	$D_1D_2D_3$	14
		a_{15}	0110	$\bar{x}_2x_9$	y_5	D_2D_3	15
		a_{16}	1010	$\bar{x}_2\bar{x}_9$	y_1y_4	D_1D_3	16

$$\begin{aligned}
y_7 &= A_2 \vee A_6 \vee A_{10} \vee A_{13} = \bar{T}_3T_4; \\
y_8 &= A_5 \vee A_{12} = T_1T_3T_4; \\
y_9 &= A_8 \vee A_9 \vee A_5 \vee A_{16} = \bar{T}_2T_3; \\
y_{10} &= A_3 \vee A_{11} \vee A_{14} = T_2\bar{T}_3\bar{T}_4 \vee T_1T_2\bar{T}_4; \\
y_{11} &= A_2 \vee A_4 \vee A_5 \vee A_8 = \bar{T}_1T_2T_4 \vee \bar{T}_2T_3T_4.
\end{aligned} \tag{5.21}$$

To get the system $\tau(T)$, let us form the table of PALer τ . It has the following columns: a_m , $K(a_m)$, B_i , $K(B_i)$, τ_m , m . It is Table 5.8 in the discussed case.

The following system can be derived from Table 5.8:

$$\begin{aligned}
\tau_1 &= [A_{11} \vee \dots \vee A_{14}] \vee A_{15} \vee A_{16} = T_1T_2 \vee T_2T_3\bar{T}_4 \vee T_1T_3\bar{T}_4; \\
\tau_2 &= [A_5 \vee A_6 \vee A_8 \vee A_{10}] \vee A_7 \vee A_9 = \bar{T}_2T_4 \vee T_1\bar{T}_2\bar{T}_3 \vee \bar{T}_1\bar{T}_2T_3; \\
\tau_3 &= [A_2 \vee A_3 \vee A_4 \vee A_{10}] \vee [A_9 \vee A_{14} \vee A_{15} \vee A_{16}] \vee A_3 = \\
&= \bar{T}_1T_4 \vee T_3\bar{T}_4 \vee \bar{T}_1T_2\bar{T}_3.
\end{aligned} \tag{5.22}$$

The minimization is achieved due to OSA (Fig. 5.14).

Let us point out that, neither OSA nor RSA is intended to minimize the system $Y_1(\tau, X)$. It can be done by encoding of CMOs $Y_q \subseteq Y_1$. The ECMO turns P_C CSFM into P_CY_1 (Fig. 5.15).

The functions of PALer Φ , PALer τ and PALerY2 are the same for P_C and P_CY_1 CFSMs. The PALerZ implements the system (2.26). The functions $z_r \in Z$ (2.26) enter PALerY1 as arguments of functions (2.25).

Fig. 5.16 Codes of CMOs
for CFSM $P_C Y1(\Gamma_9)$

		$z_1 z_2$			
		00	01	11	10
$z_3 z_4$	00	Y_1	Y_2	*	Y_3
	01	Y_9	Y_4	Y_8	Y_7
	11	Y_5	*	*	Y_6
	10	*	Y_{10}	Y_{11}	*

tions $y_n \in Y1$. One of the possible solutions is represented by the Karnaugh map (Fig. 5.16).

Using codes (Fig. 5.16) and content of CMOs $Y_q \subseteq Y1$, we get the following SBF:

$$\begin{aligned}
 y_1 &= Y_2 \vee Y_4 \vee Y_{10} = \bar{z}_1 z_2; & y_2 &= Y_3 \vee Y_4 \vee Y_7 \vee Y_8 = z_2 \bar{z}_3 z_4 \vee z_1 \bar{z}_3; \\
 y_3 &= Y_5 \vee Y_9 = \bar{z}_1 \bar{z}_2 z_4; & y_4 &= Y_2 \vee Y_4 \vee Y_8 = z_2 \bar{z}_3; \\
 y_5 &= Y_5 \vee Y_6 \vee Y_{10} \vee Y_{11} = z_3; & y_6 &= Y_5 \vee Y_6 \vee Y_7 \vee Y_9 = \bar{z}_2 z_4.
 \end{aligned} \tag{5.23}$$

Let us find the maximum possible number of literals in the system $Y1(Z)$. To do it, we can use the following expression:

$$N_{max} = \sum_{n=1}^{N1} Q(y_n) \cdot R_Q. \tag{5.24}$$

In the discussed case, there is $N_{max} = 80$ (because there are 20 terms in the system (5.23)). But there are only $N_R = 15$ literals in terms of (5.23). So, the proper encoding of CMOs can significantly reduce the number of interconnections between the blocks PALerZ and PALerY1. It reduces the consumed energy.

To construct the system $Z(\tau, X)$, it is necessary to construct a DST of $P_C Y1$ CFSM. It is the same as the DST of P_C CFSM, but the column Y_h is replaced by the column Z_h . If $z_r = 1$ in $K(Y_q)$ where $Y_q = Y_h$, then there is the symbol z_r in the column Z_h .

Let us show only a part of DST for $P_C Y1(\Gamma_9)$. It is represented by Table 5.9.

Using Table 5.7 (codes $K(B_i)$), X_h and Table 5.9 (the column Z_h) gives the system $Z(\tau, X)$. For example, the following expression can be derived:

$$\begin{aligned}
 z_1 &= F_2 \vee F_6 \vee [F_7 \vee F_8] \vee F_9 \vee [F_{14} \vee F_{15} \vee F_{16}] = \\
 &= \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 x_1 \bar{x}_2 \vee \bar{\tau}_1 \bar{\tau}_2 \tau_3 \bar{x}_3 \vee \tau_2 \bar{\tau}_3 x_5 \vee \tau_2 \bar{\tau}_3 x_7 \vee \tau_1 \tau_2.
 \end{aligned} \tag{5.25}$$

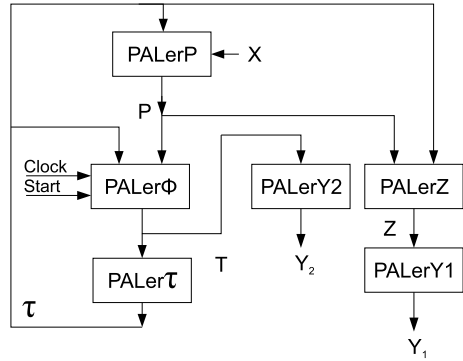
Acting in the same manner, all equations similar to (5.25) can be constructed.

To diminish the number of literals (and interconnections) in functions $\Phi(\tau, X)$, $Z(\tau, X)$, the method of RLC can be used. It leads to $MP_C Y1$ CFSM shown in Fig. 5.17.

In this circuit, the blocks execute the following functions:

Table 5.9 Part of DST of $P_C Y1(\Gamma_9)$

h	q	$K(Y_q)$	Z_h	h	q	$K(Y_q)$	Z_h
1	2	0100	z_2	9	8	1101	$z_2 z_4$
2	3	1000	z_1	10	1	0000	—
3	1	0000	—	11	9	0001	z_4
4	4	0101	$z_2 z_4$	12	10	0110	$z_2 z_3$
5	5	0011	$z_3 z_4$	13	9	0001	z_4
6	6	1011	$z_1 z_3 z_4$	14	7	1001	$z_1 z_4$
7	7	1001	$z_1 z_4$	15	11	1110	$z_1 z_2 z_3$
8	11	1110	$z_1 z_2 z_3$	16	2	1000	z_1

Fig. 5.17 Structural diagram of $MP_C Y1$ CFSM

$$\begin{aligned}
 P &= P(\tau, X) && (PALerP); \\
 \Phi &= \Phi(\tau, P) && (PALer\Phi); \\
 \tau &= \tau(T) && (PALer\tau); \\
 Y_2 &= Y_2(T) && (PALerY_2); \\
 Z &= Z(\tau, P) && (PALerZ); \\
 Y_1 &= Y_1(Z) && (PALerY_1).
 \end{aligned} \tag{5.26}$$

To design the circuit of $MP_C Y1$ CFSM, it is necessary to find all SBFs from (5.26). Obviously, systems τ , Y_2 and Y_1 are the same for equivalent $P_C Y1$ and $MP_C Y1$ CFSMs. Let us find functions depending on variables $p_g \in P$ in the case of $MP_C Y1(\Gamma_9)$.

Let us distribute LCs among $G = 3$ rows of the table of RLC (Table 5.10). We use known methods [9].

To optimize the system P, it is necessary to use the insignificant assignments of variables $\tau_r \in \tau$. The following equations can be derived from Table 5.10:

$$\begin{aligned}
 p_1 &= [B_1 \vee B_4]x_1 \vee B_2x_3 \vee B_3x_5 \vee B_5x_9; \\
 p_2 &= [B_1 \vee B_5]x_2 \vee B_2x_4 \vee B_3x_6 \vee B_4x_8; \\
 p_3 &= x_7.
 \end{aligned} \tag{5.27}$$

Table 5.10 Table of RLC for $MP_C Y1(\Gamma_9)$

P	B_i					
	B_1	B_2	B_3	B_4	B_5	B_6
p_1	x_1	x_3	x_5	x_1	x_9	–
p_2	x_2	x_4	x_6	x_8	x_2	–
p_3	–	–	x_7	–	–	–
$K(B_i)$	000	001	010	011	100	101

Table 5.11 DST of CSFM $MP_C Y1(\Gamma_9)$

B_i	$K(B_i)$	a_s	$K(a_s)$	P_h	Z_h	Φ_h	h
B_1	000	a_2	0101	$p_1 p_2$	z_2	$D_2 D_4$	1
		a_3	0100	$p_1 \bar{p}_2$	z_1	D_2	2
		a_4	0111	$\bar{p}_1$	–	$D_2 D_3 D_4$	3
B_2	001	a_5	1011	$p_1 p_2$	$z_2 z_4$	$D_1 D_3 D_4$	4
		a_6	1010	$p_1 \bar{p}_2$	$z_3 z_4$	$D_1 D_3$	5
		a_7	1000	$\bar{p}_1$	$z_1 z_3 z_4$	D_1	6
B_3	010	a_8	0011	$p_1 p_2$	$z_1 z_4$	$D_3 D_4$	7
		a_9	0010	$p_1 \bar{p}_2$	$z_1 z_2 z_3$	D_3	8
		a_{10}	0001	$\bar{p}_1 x_7$	$x_1 x_2 x_4$	D_4	9
		a_7	1000	$\bar{p}_1 \bar{x}_7$	–	D_1	10
B_4	011	a_{11}	1100	$p_1 p_2$	z_4	$D_1 D_2$	11
		a_{12}	1111	$p_1 \bar{p}_2$	$z_2 z_3$	$D_1 D_2 D_3 D_4$	12
		a_{13}	1101	$\bar{p}_1$	z_4	$D_1 D_2 D_4$	13
B_5	100	a_{14}	1110	p_2	$z_1 z_4$	$D_1 D_2 D_3$	14
		a_{15}	0110	$\bar{p}_2 p_1$	$z_1 z_2 z_3$	$D_2 D_3$	15
		a_{16}	1010	$\bar{p}_2 \bar{p}_1$	z_1	$D_1 D_3$	16

Because $p_3 = x_7$, only two equations from (5.27) should be implemented by PALerP.

To get SBFs for PALer Φ and PALerZ, it is necessary to construct the DST of $MP_C Y1(\Gamma_9)$. We can use the DST (Table 5.7) and replace the column X_h by P_h (using Table 5.10) and Y_h by Z_h (using Table 5.9). It gives Table 5.11. We hope there are the transparent connections between Table 5.11 and other tables.

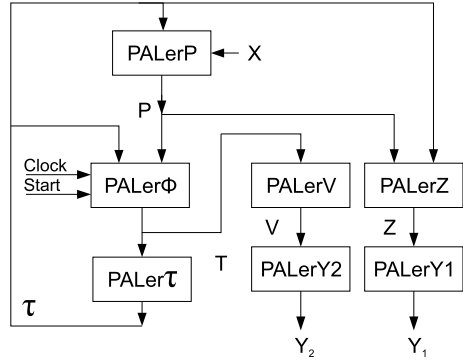
For example, replacing $x_l \in X$ by $p_g \in P$ allows transforming the equations (5.20):

$$\begin{aligned}
 y_1 &= [\bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 p_1 p_2 \vee \bar{\tau}_1 \bar{\tau}_2 \tau_3 p_1 p_2] \vee F_{12} \vee F_{16} = \\
 &= \bar{\tau}_1 \bar{\tau}_2 p_1 p_2 \vee \bar{\tau}_1 \tau_2 \tau_3 p_1 \bar{p}_2 \vee \bar{\tau}_1 \tau_2 \bar{p}_1 \bar{p}_2; \\
 D_1 &= \bar{\tau}_1 \tau_3 \vee \bar{\tau}_1 \tau_2 \bar{\tau}_3 \bar{p}_5 \bar{x}_7 \vee \tau_1 \bar{\tau}_2 \bar{\tau}_3 (p_2 \vee \bar{p}_1).
 \end{aligned} \tag{5.28}$$

Next, the replacing $x_l \in X$ by $p_g \in P$ in (5.25) gives the following equation:

$$z_1 = \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 p_1 \bar{p}_2 \vee \bar{\tau}_1 \bar{\tau}_2 \tau_3 \bar{p}_1 \vee \tau_2 \bar{\tau}_3 p_1 \vee \tau_2 \bar{\tau}_3 x_7 \vee \tau_1 \tau_2. \tag{5.29}$$

Fig. 5.18 Structural diagram of $MP_C Y1Y2$ CFSM



Let it be $Q(Y2)$ different CMOs $Y_q \subseteq Y2$. Let the following condition take place:

$$Q(Y2) \ll M. \quad (5.30)$$

It is enough R_{Q2} variables $v_r \in V$ to encode the CMOs $Y_q \subseteq Y2$:

$$R_{Q2} = \lceil \log_2 Q(Y2) \rceil. \quad (5.31)$$

Now, the system $Y2(T)$ is transformed into the system

$$Y2 = Y2(V). \quad (5.32)$$

If we apply this method to $MP_C Y1$ CSFM, then we will get the $MP_C Y1Y2$ CFSM. Its structural diagram is shown in Fig. 5.18.

In $MP_C Y1Y2$ CFSM, the PALerV implements the system

$$V = V(T). \quad (5.33)$$

The PALerY2 implements the system (5.32). The functions of the other blocks are the same as represented by (5.26).

Let us discuss how to get systems (5.32)–(5.33). Let us use the current example based on GSA Γ_9 . There are $Q(Y2) = 9$ CMOs $Y_q \subseteq Y2$ in the discussed case: $Y_1 = \emptyset$, $Y_2 = \{y_7, y_{11}\}$, $Y_3 = \{y_{10}\}$, $Y_4 = \{y_{11}\}$, $Y_5 = \{y_8, y_9, y_{11}\}$, $Y_6 = \{y_7\}$, $Y_7 = \{y_9, y_{11}\}$, $Y_8 = \{y_9\}$ and $Y_9 = \{y_8\}$. So, there is $R_{Q2} = 4$, $V = \{v_1, \dots, v_4\}$. Let us encode them as it is shown in Fig. 5.19.

Using GSA Γ_9 ans codes (Fig. 5.19) gives the following system:

$$\begin{aligned}
 y_7 &= Y_2 \vee Y_6 = \bar{v}_1 \bar{v}_2 v_4; & y_8 &= Y_3 \vee Y_5 \vee Y_9 = v_1 \bar{v}_2; \\
 y_9 &= Y_5 \vee Y_7 \vee Y_8 = v_1 \bar{v}_3; & y_{10} &= Y_3 = v_2 v_3; \\
 y_{11} &= Y_2 \vee Y_4 \vee Y_5 \vee Y_7 = \bar{v}_3 v_4.
 \end{aligned} \quad (5.34)$$

		$V_1 V_2$			
		00	01	11	10
$V_3 V_4$	00	Y_1	*	Y_8	*
	01	Y_2	Y_4	Y_7	Y_5
	11	Y_6	*	*	Y_9
	10	*	Y_3	*	*

Fig. 5.19 Codes of CMOs $Y_q \subseteq Y_2$

Table 5.12 Part of DST of $P_C Y_1(I_9)$

m	$K(a_m)$	q	$K(y_q)$	V_m	m	$K(a_m)$	q	$K(y_q)$	V_m
1	0000	1	0000	—	9	0010	8	1100	$v_1 v_2$
2	0101	2	0001	v_4	10	0001	6	0011	$v_3 v_4$
3	0100	3	0110	$v_2 v_3$	11	1100	3	0110	$v_2 v_3$
4	0111	4	0101	$v_2 v_4$	12	1111	9	1011	$v_1 v_3 v_4$
5	1011	5	1001	$v_1 v_4$	13	1101	6	0011	$v_3 v_4$
6	1010	6	0011	$v_3 v_4$	14	1110	3	0110	$v_2 v_3$
7	1000	1	0000	—	15	0110	1	0000	—
8	0011	7	1101	$v_1 v_2 v_4$	16	1010	8	1100	$v_1 v_2$

The block PALerV is represented by Table 5.12.

Using table of PALerV, it is possible to find the system (5.33). For example, the following equation can be found:

$$\begin{aligned}
 v_1 &= [A_5 \vee A_8] \vee [A_9 \vee A_{16}] \vee A_{12} = [\bar{T}_2 \bar{T}_3 T_4 \vee \bar{T}_2 T_3 \bar{T}_4] \vee A_{12} = \\
 &= \bar{T}_2 T_3 \vee T_1 T_2 T_3 T_4.
 \end{aligned} \tag{5.35}$$

All discussed models based on PES can be used in PLA-based CPLDs. To get the corresponding circuits, it is enough to replace the symbols “PALer” by “PLAer”. As follows from Fig. 5.18, there are maximum 7 different PLAers in these models. Some of them are represented by Table 5.13.

The symbols in the columns starting from P correspond to blocks PLAers generating these functions. There is an SBF generated by some PLAer for a given model written on the intersection of the row with the model and the column of the block.

If only a part of classes $B_i \in \Pi_A$ are represented by single generalized intervals of R-dimensional Boolean space, then it is possible to use the partial transformation of state codes. It leads to P_{C_1} CFSMs and its different variants. In this case, the optimization can be achieved due to the wider fan-in of PAL- and PLA-based MCs. We discuss this approach in Chap. 6.

Table 5.13 Models of PLA-based P_C CFSMs

Model	PLAer						
	P	Φ	τ	V	$Y2$	Z	$Y1$
P_C	–	$\Phi(\tau, X)$	$\tau(T)$	–	$Y2(T)$	–	$Y1(\tau, X)$
MP_C	$P(\tau, X)$	$\Phi(\tau, P)$	$\tau(T)$	–	$Y2(T)$	–	$Y1(\tau, P)$
$P_C Y1$	–	$\Phi(\tau, X)$	$\tau(T)$	–	$Y2(T)$	$Z(\tau, X)$	$Y1(Z)$
$P_C Y2$	–	$\Phi(\tau, X)$	$\tau(T)$	$V(T)$	$Y2(T)$	–	$Y1(\tau, X)$
$P_C Y1 Y2$	–	$\Phi(\tau, X)$	$\tau(T)$	$V(T)$	$Y2(T)$	$Z(\tau, X)$	$Y1(Z)$
$MP_C Y1$	$P(\tau, X)$	$\Phi(\tau, X)$	$\tau(T)$	–	$Y2(T)$	$Z(\tau, P)$	$Y1(Z)$
$MP_C Y2$	$P(\tau, X)$	$\Phi(\tau, X)$	$\tau(T)$	$V(T)$	$Y2(V)$	–	$Y1(\tau, P)$
$MP_C Y1 Y2$	$P(\tau, X)$	$\Phi(\tau, P)$	$\tau(T)$	$V(T)$	$Y2(V)$	$Z(\tau, P)$	$Y1(Z)$

5.5 Synthesis of CFSMs with Primary Encoding of Classes

There are two such methods. In both cases, state codes $K(a_m)$ are represented as concatenations of class codes $K(B_i)$ and: (1) the code $C(a_m)$ where a_m is an element of $B_i \in \Pi_A$ or (2) the code $K(Y_q)$ where $Y_q \subseteq Y_2$ is a CMO generated in the state $a_m \in B_i$. Let us repeat these formulae:

$$K(a_m) = K(B_i) * C(a_m); \quad (5.36)$$

$$K(a_m) = K(B_i) * K(Y_q). \quad (5.37)$$

In both cases, classes $B_i \in \Pi_A$ are encoded by variables $T_r \in T'$ having R_A bits (2.43). The codes $C(a_m)$ are created from variables $T_r \in T_A$, where $|T_A| = R_0$. The value of R_0 is determined by (3.50). The codes $K(Y_q)$ are based on variables $T_r \in T_Y$ where $|T_Y| = R_Q$. The value of R_Q is determined by (1.32).

The representation (5.36) leads to the expansion of code space [5]. The representation (5.37) is named a nonstandard representation of state codes [5]. The first method leads to P_E CFSMs, the second to $P_K Y2$ CFSMs. Let us discuss them. Let us use GSA Γ_9 to design these CFSMs.

In the case of P_E CFSM, there is the set $T = T' \cup T_A$, where $T' \cap T_A = \emptyset$. The representation (5.36) leads to the structural diagram shown in Fig. 5.20.

In P_E CFSM, the PALer Φ implements the system (3.28), the PALer $Y2$ the system (5.17) and the PALer $Y1$ the system

$$Y1 = Y1(T', X). \quad (5.38)$$

In this book, we propose the method of synthesis of P_E CFSM. It includes the following steps:

1. Marking the initial GSA Γ and creating the set A .
2. Creating the set of classes $B_i \in \Pi_A$.

Fig. 5.20 Structural diagram of P_E CFSM

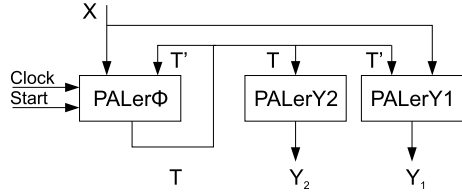


Fig. 5.21 Outcome of state assignment for $P_E(\Gamma_9)$

		$T_1T_2T_3$							
		000	001	011	010	110	111	101	100
T_4T_5	00	a_1	a_2	*	a_5	a_{14}	*	a_{11}	a_8
	01	*	a_3	*	a_6	a_{15}	*	a_{12}	a_9
	11	*	*	*	*	*	*	*	*
	10	*	a_4	*	a_7	a_{16}	*	a_{13}	a_{10}
		B_1		B_3		B_5		B_4	
		B_2				B_6			

3. Executing the representation (5.36) for states.
4. Creating the tables for blocks of PALs.
5. Implementing the CFSM logic circuit.

In the case of CFSM $P_E(\Gamma_9)$, the first two steps are already executed. Analysis of Γ_9 shows that there is $M_0 = 3$. Using (3.50) gives $R_0 = 2$. Using (2.43) gives $R_A = 3$. So, there is $T = \{T_1, T_2, T_3, T_4, T_5\}$, $T' = \{T_1, T_2, T_3\}$ and $T_A = \{T_4, T_5\}$. Let us encode the states as it is shown in Fig. 5.21

The following class codes follow from Fig. 5.21: $K(B_1) = 000$, $K(B_2) = 0*1$, $K(B_3) = 01*$, $K(B_4) = 100$, $K(B_5) = 1*1$ and $K(B_6) = 11*$. The classes are encoded in a way minimizing the number of ones in their codes. The state codes also could be optimizing: $C(a_1) = **$, $C(a_3) = *1$, $C(a_4) = 1*$. It gives the codes $K(a_m)$ with *. For example $K(a_1) = 000**$, $K(a_3) = 0*1*1$, and so on. The CFSM circuit can be optimized due to this fact.

The block PALer Φ is represented by Table 5.14. It is constructed using the system of GFT (5.19) and codes from the Karnaugh map (Fig. 5.21). The same table represents PALerY1.

Using Table 5.14, the functions (3.28) and (5.38) can be found. For example, there is $D_1 = [F_7 \vee F_8] \vee F_9 \vee [F_{11}, \dots \vee F_{16}] = \bar{T}_1 T_2 x_5 \vee \bar{T}_1 T_2 x_7 \vee T_1 \bar{T}_2$. In the same way, all other functions can be found.

To get the SBF (1.43), it is necessary to construct table of PALerY2. It has the following columns: a_m , $K(a_m)$, $Y(a_m)$, m . It is Table 5.15 in the discussed case.

For example, the following equation can be derived from Table 5.15: $y_7 = A_2 \vee A_6 \vee [A_{10} \vee A_{13}] = \bar{T}_1 T_3 \bar{T}_4 \bar{T}_5 \vee \bar{T}_1 T_2 T_5 \vee T_1 \bar{T}_2 T_4$. In the same manner, all other functions for $y_n \in Y_2$ can be derived.

Table 5.14 Table of PALer Φ of CFSM $P_E(\Gamma_9)$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	000	a_2	00100	x_1x_2	y_1y_4	D_3	1
		a_3	00101	$x_1\bar{x}_2$	y_2	D_3D_5	2
		a_4	00110	$\bar{x}_1$	–	D_3D_4	3
B_2	0*1	a_5	01000	x_3	$y_1y_2y_4$	D_2	4
		a_6	01001	$\bar{x}_3x_4$	$y_3y_5y_6$	D_2D_5	5
		a_7	01010	$\bar{x}_3\bar{x}_4$	y_5y_6	D_2D_4	6
B_3	01*	a_8	10000	x_5x_6	y_2y_6	D_1	7
		a_9	10001	$x_5\bar{x}_6$	y_5	D_1D_5	8
		a_{10}	10010	$\bar{x}_5x_7$	y_2y_4	D_1D_4	9
		a_7	01010	$\bar{x}_5\bar{x}_7$	–	D_2D_4	10
B_4	100	a_{11}	10100	x_1x_8	y_3y_6	D_1D_3	11
		a_{12}	10101	$x_1\bar{x}_8$	y_1y_5	$D_1D_3D_5$	12
		a_{13}	10110	$\bar{x}_1$	y_5	$D_1D_3D_4$	13
B_5	1*1	a_{14}	11000	x_2	y_2y_6	D_1D_2	14
		a_{15}	11001	$\bar{x}_2x_9$	y_5	$D_1D_2D_5$	15
		a_{16}	11010	$\bar{x}_2\bar{x}_9$	y_1y_4	$D_1D_2D_4$	16

Table 5.15 Table of PALerY2 of CFSM $P_E(\Gamma_9)$

a_m	$K(a_m)$	$Y(a_m)$	m	a_m	$K(a_m)$	$Y(a_m)$	m
a_1	000**	–	1	a_9	100*1	y_9	1
a_2	0*100	y_7y_{11}	2	a_{10}	1001*	y_7	2
a_3	0*1*1	y_{10}	3	a_{11}	1*100	y_{10}	3
a_4	0*11*	y_{11}	4	a_{12}	1*1*1	y_8	4
a_5	01*00	$y_8y_9y_{11}$	5	a_{13}	1*11*	y_7	5
a_6	01**1	y_7	6	a_{14}	11*00	y_{10}	6
a_7	01*1*	–	7	a_{15}	11**1	–	7
a_8	10000	y_9y_{11}	8	a_{16}	11*1*	y_9	8

Obviously, it is possible to use RSA to improve the characteristics of PALerY2. For example, if we place states a_2, a_6, a_{10} and a_{13} in the row $C(a_m) = 11$, we will get $y_7 = T_4T_5$. Of course, it results in changing the functions $\Phi(T', X)$ and $Y1(T', X)$. Remind, applying RSA leads to P_{ER} CFSMs. They have the same structural diagrams as P_E CFSMs.

To improve the hardware amount, it is possible to use the RLC. It leads to MP_E CFSM (Fig. 5.22). The block PALerP implements the system

$$P = P(T', X). \quad (5.39)$$

Fig. 5.22 Structural diagram of MP_E CFSM

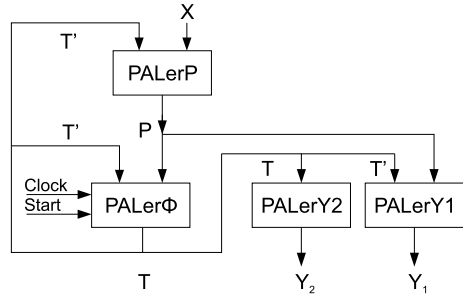
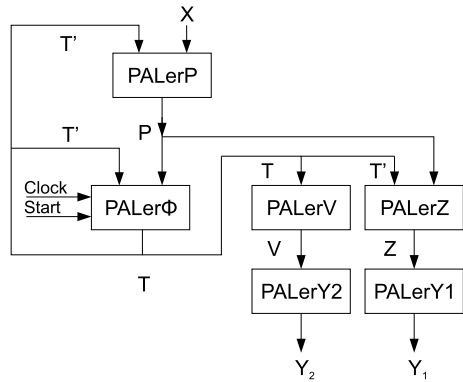


Fig. 5.23 Structural diagram of $MP_E Y1Y2$ CFSM



In the most extreme case, the following methods of structural decomposition can be used together: (1) RLC, (2) ECMO $Y_q \subseteq Y1$ and (3) ECMO $Y_q \subseteq Y2$. It leads to $MP_E Y1Y2$ CFSM (Fig. 5.23).

The blocks of $MP_E Y1Y2$ CFSM executes the following functions:

$$\begin{aligned} P &= P(T', X); & \Phi &= \Phi(T', P); \\ Z &= Z(T', P); & Y1 &= Y1(Z); \\ V &= V(T); & Y2 &= Y2(V). \end{aligned} \tag{5.40}$$

To get these systems, it is necessary to construct tables for corresponding blocks PALers. We discussed similar structures before (for example, $MP_C Y1Y2$ CFSM). If symbols of PALers are replaced by PLAers, we will have the PLA-based structural diagrams of CFSMs with the expansion of code space.

The model P_E should be applied if the majority of classes $B_i \in \Pi_A$ is represented by more than a single generalized interval of R-dimensional Boolean space. If the condition (5.30) takes place, then the model P_K can be used. It is based on representation (5.37). The structural diagram of P_K CFSM is shown in Fig. 5.24.

The functions of each block of PALs are clear from the previous text. In this book, we propose the method of synthesis of P_K CFSM. It includes the following steps:

1. Marking the initial GSA Γ and creating the set of states $A = \{a_1, \dots, a_M\}$.

Fig. 5.24 Structural diagram of P_K CFSM

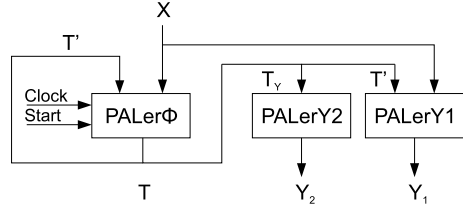


Fig. 5.25 Outcome of encoding of CMOs for $P_K(\Gamma_9)$

		$v_1 v_2$			
v_3		00	01	11	10
	0	Y_1	Y_3	*	Y_5
	1	Y_2	Y_4	*	Y_6

2. Constructing the partition Π_A of the set A by classes of PES.
3. Encoding of $B_i \in \Pi_A$.
4. Constructing the CMOs $Y_q \subseteq Y_2$.
5. Encoding of CMOs by binary codes $K(Y_q)$.
6. Constructing the tables for blocks of the circuit
7. Implementing the circuit of P_K CFSM.

Let us discuss an example of synthesis for CFSM $P_K(\Gamma_9)$. But let us change the CMOs $Y_q \subseteq Y_2$. Let it be $Y(a_1) = Y(a_7) = Y(a_{15}) = \emptyset$, $Y(a_2) = Y(a_6) = Y(a_{12}) = \{y_7, y_8\}$, $Y(a_3) = Y(a_5) = Y(a_{16}) = \{y_9\}$, $Y(a_4) = Y(a_8) = Y(a_{11}) = \{y_8, y_{10}\}$, $Y(a_9) = Y(a_{13}) = \{y_{11}\}$ and $Y(a_{10}) = Y(a_{14}) = \{y_7, y_{11}\}$. So, there is $Q(Y_2) = 6$, $R_{Q2} = 3$. It gives the set $V = \{v_1, v_2, v_3\}$. Let us leave class codes the same as for $P_E(\Gamma_9)$.

Let us encode the CMOs $Y_q \subseteq Y_2$ in a way optimizing the system $Y_2(V)$. One of the variants is shown in Fig. 5.25. In this Karnaugh map, we use the following CMOs: $Y_1 = \emptyset$, $Y_2 = \{y_7, y_8\}$, $Y_3 = \{y_9\}$, $Y_4 = \{y_8, y_{10}\}$, $Y_5 = \{y_{11}\}$ and $Y_6 = \{y_7, y_{11}\}$.

Using contents of CMOs and their codes (Fig. 5.25) gives the following system of MO $y_n \in Y_2$.

$$\begin{aligned}
 y_7 &= Y_2 \vee Y_6 = \bar{v}_2 v_3; & y_8 &= Y_2 \vee Y_4 = \bar{v}_1 v_3; \\
 y_9 &= Y_3 = v_2 \bar{v}_3; & y_{10} &= Y_4 = v_2 v_3; \\
 y_{11} &= Y_5 \vee Y_6 = v_1.
 \end{aligned} \tag{5.41}$$

So, there is $R = R_A + R_{Q2} = 6$. It gives the sets $T = \{T_1, \dots, T_6\}$, $T' = \{T_1, T_2, T_3\}$ and $T_Y = \{T_4, T_5, T_6\}$. Using codes $K(B_i)$ and $K(Y_q)$ gives the state codes (5.37). In the discussed case, they are represented by the Karnaugh map (Fig. 5.26).

In Fig. 5.26, we used the following relations:

$T_4 T_5 T_6$ \ $T_1 T_2 T_3$									
		000	001	011	010	110	111	101	100
000		a_1	*	*	a_7	a_{15}	*	*	*
001		*	a_3	*	a_6	*	*	a_{12}	*
011		*	a_4	*	*	*	*	a_{11}	a_8
010		*	a_3	*	a_5	a_{16}	*	*	*
110		*	*	*	*	*	*	*	*
111		*	*	*	*	*	*	*	*
101		*	*	*	*	a_{14}	*	*	a_{10}
100		*	*	*	*	*	*	a_{13}	a_9

Fig. 5.26 State codes of CFSM $P_K(\Gamma_9)$ **Table 5.16** Contents of $K(a_s)$, Φ_h for PALer Φ of $P_K(\Gamma_9)$

h	1	2	3	4	5	6	7	8
$K(a_s)$	001001	001010	001011	010010	010001	010000	100011	100100
Φ_h	$D_3 D_6$	$D_3 D_5$	$D_3 D_5 D_6$	$D_2 D_5$	$D_2 D_6$	D_2	$D_1 D_5 D_6$	$D_1 D_4$
h	9	10	11	12	13	14	15	16
$K(a_s)$	100101	010000	101011	101001	101100	110101	110000	110010
Φ_h	$D_1 D_4 D_5$	D_2	$D_1 D_3 D_5 D_6$	$D_1 D_3 D_6$	$D_1 D_3 D_4$	$D_1 D_2 D_4 D_6$	$D_1 D_2$	$D_1 D_2 D_5$

$$\begin{aligned}
K(a_1) &= K(B_1) * K(Y_1); & K(a_2) &= K(B_2) * K(Y_2); \\
K(a_3) &= K(B_2) * K(Y_3); & K(a_4) &= K(B_2) * K(Y_4); \\
K(a_5) &= K(B_3) * K(Y_3); & K(a_6) &= K(B_3) * K(Y_2); \\
K(a_7) &= K(B_3) * K(Y_1); & K(a_8) &= K(B_4) * K(Y_4); \\
K(a_9) &= K(B_4) * K(Y_5); & K(a_{10}) &= K(B_4) * K(Y_6); \\
K(a_{11}) &= K(B_5) * K(Y_4); & K(a_{12}) &= K(B_5) * K(Y_2); \\
K(a_{13}) &= K(B_5) * K(Y_5); & K(a_{14}) &= K(B_6) * K(Y_6); \\
K(a_{15}) &= K(B_6) * K(Y_1); & K(a_{16}) &= K(B_6) * K(Y_3).
\end{aligned} \tag{5.42}$$

To get the systems (3.28) and (5.38), it is necessary to construct the table of PALer Φ . In the case of $P_K(\Gamma_9)$, this table is practically the same as Table 5.14. But there is different information in the columns $K(a_s)$ and Φ_h . We show these columns organized as the rows of Table 5.16.

Using Tables 5.15 and 5.16, the following equations, for example, can be derived:

$$\begin{aligned}
D_1 &= [F_7 \vee F_8 \vee F_9] \vee [F_{11} \vee, \dots, \vee F_{16}] = \bar{T}_1 T_2 \vee T_1 \bar{T}_2 \bar{T}_3 \vee T_1 T_3; \\
D_4 &= F_8 \vee F_9 \vee F_{13} \vee F_{14} = \bar{T}_1 T_2 x_5 \bar{x}_6 \vee, \dots, T_1 T_3 x_2.
\end{aligned} \tag{5.43}$$

Obviously, the system (5.41) should be replaced by the following system:

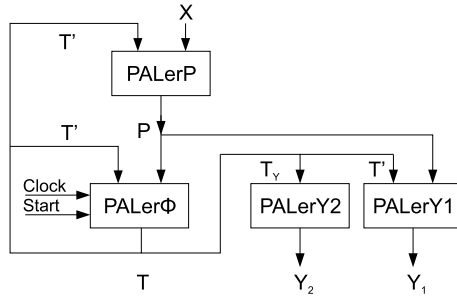


Fig. 5.27 Structural diagram of MP_K CFSM

$$\begin{aligned} y_7 &= \bar{T}_5 T_6; & y_8 &= \bar{T}_4 T_6; \\ y_9 &= T_5 \bar{T}_6; & y_{10} &= T_5 T_6; & y_{11} &= T_4. \end{aligned} \quad (5.44)$$

This transformation is connected with the fact that there is $v_r = T_{r+3}$ ($r \in \{1, 2, 3\}$) in the case of $P_K(I_9)$. In general case, the system of MOs $y_n \in Y_2$ is represented as

$$Y_2 = Y_2(T_Y). \quad (5.45)$$

The SOPs of $D_r \in \Phi$ are used to implement the circuit of PALer Φ . The circuit of PALerY1 is implemented using the system $Y_1(T', X)$. The circuit of PALerY2 is implemented using system (5.45).

To diminish the number of PALs in both PALer Φ and PALerY1, the RLC can be used. It leads to MP_K CFSM (Fig. 5.27).

In MP_K CFSM, the PALerP replaces logical conditions $x_l \in X$ by additional variables $p_g \in P$. Due to representation (5.37), only variables $T_r \in T' \subseteq T$ enter functions $p_g \in P$. So, the PALerP implements the system (5.39). The PALer Φ implements SBF (3.28). Outputs of MCs from PALer Φ are connected with D flip-flops. The PALerY1 implements the system (5.38) for Mealy outputs of CFSM. The PALerY2 implements the system (5.45). There are only variables $T_r \in T_Y$ in SOPs of $y_n \in Y_2$. There is no additional block for encoding of CMOs $Y_q \subseteq Y_2$. It is possible due to representation (5.37).

To diminish the number of PALs used to implement the system $Y_1(T', P)$, it is possible to encode the CMOs $Y_q \subseteq Y_1$. It leads to $MP_K Y_1$ CFSM (Fig. 5.28).

We hope that functions of all blocks are clear. The method of synthesis of $MP_K Y_1$ CFSM includes the following steps:

1. Making the initial GSA Γ and finding the set of states A .
2. Creating the partition Π_A of set A by classes of PES.
3. Executing the RLC $x_l \in X$ by variables $p_g \in P$.
4. Encoding of classes $B_i \in \Pi_A$.
5. Creating the table of PALerP and deriving the SBF (5.39).
6. Creating the CMOs $Y_q \subseteq Y_2$.

Fig. 5.28 Structural diagram of $MP_K Y1$ CFSM

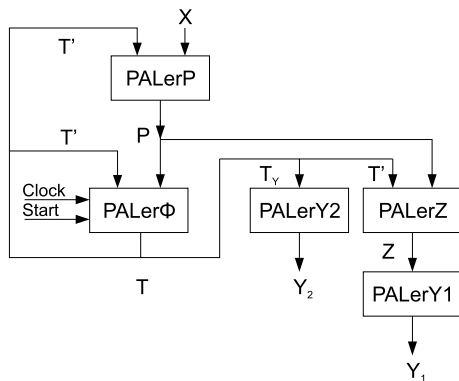


Table 5.17 Characteristic of CFSM $MP_K Y1(\Gamma_9)$

B_i	$K(B_i)$	a_m	Y_2			P			Y_1		
			q	Y_q	$K(Y_q)$	P_1	P_2	P_3	q	Y_q	$K(Y_q)$
B_1	000	a_1	1	—	000	x_1	x_2	—	1	—	0000
B_2	0*1	a_2	2	$y_7 y_8$	001	x_2	x_4	—	2	$y_1 y_2 y_4$	0100
		a_3	3	y_9	010	x_3	x_4	—	3	y_2	1000
		a_4	4	$y_8 y_{10}$	011	x_3	x_4	—	4	$y_1 y_2 y_4$	0101
B_3	01*	a_5	3	y_9	010	x_5	x_6	x_7	5	$y_3 y_5 y_6$	0011
		a_6	2	$y_7 y_8$	001	x_5	x_6	x_7	6	$y_5 y_6$	1011
		a_7	1	—	000	x_5	x_6	x_7	7	$y_2 y_6$	1001
B_4	100	a_8	4	$y_8 y_{10}$	011	x_1	x_8	—	8	$y_2 y_4$	1101
		a_9	5	y_{11}	100	x_1	x_8	—	9	$y_3 y_6$	0001
		a_{10}	6	$y_7 y_{11}$	101	x_1	x_8	—	10	$y_1 y_5$	0110
B_5	1*1	a_{11}	4	$y_8 y_{10}$	011	x_9	x_2		11	y_5	1110
		a_{12}	2	$y_7 y_8$	001	x_9	x_2		—	—	—
		a_{13}	5	y_{11}	100	x_9	x_2		—	—	—
B_6	11*	a_{14}	6	$y_7 y_{11}$	101	—	—	—	—	—	—
		a_{15}	1	—	000	—	—	—	—	—	—
		a_{16}	3	y_9	010	—	—	—	—	—	—

7. Encoding of CMOs $Y_q \subseteq Y_2$ and creating the state codes (5.37).
8. Creating the CMOs $Y_q \subseteq Y_1$ and encoding them.
9. Creating the tables of PALer Φ , PALerZ, PALerY1 and PALerY2.
10. Implementing circuits of PALers using particular PAL-based macrocells.

Let us discuss an example of synthesis for CFSM $MP_K Y1(\Gamma_9)$. Some of the steps are already executed. Let us create Table 5.17 with some characteristics of $MP_K Y1(\Gamma_9)$. They are taken from previous examples.

In Table 5.17, the codes $K(B_i)$ are taken from Table 5.14 the codes $K(Y_q)$ for $Y_q \subseteq Y_2$ from Fig. 5.25, the codes $K(Y_q)$ for $Y_q \subseteq Y_1$ from Fig. 5.14. Using (5.42)

Table 5.18 DST of CFSM $MP_K Y1(\Gamma_9)$

B_i	$K(B_i)$	a_s	$K(a_s)$	P_h	Z_h	Φ_h	h
B_1	000	a_2	001001	$p_1 p_2$	z_2	$D_3 D_6$	1
		a_3	001001	$p_1 \bar{p}_2$	z_1	$D_3 D_5$	2
		a_4	0010110	$\bar{p}_1$	—	$D_3 D_5 D_6$	3
B_2	0*1	a_5	010010	$p_1 p_2$	$z_2 z_4$	$D_2 D_5$	4
		a_6	010001	$p_1 \bar{p}_2$	$z_3 z_4$	$D_2 D_5$	5
		a_7	010000	$\bar{p}_1$	$z_1 z_3 z_4$	D_2	6
B_3	01*	a_8	100011	$p_1 p_2$	$z_1 z_4$	$D_1 D_5 D_6$	7
		a_9	100100	$p_1 \bar{p}_2$	$z_1 z_2 z_3$	$D_1 D_4$	8
		a_{10}	100101	$\bar{p}_1 x_7$	$z_1 z_2 z_4$	$D_1 D_4 D_6$	9
		a_7	010000	$\bar{p}_1 \bar{x}_7$	—	D_2	10
B_4	100	a_{11}	101011	$p_1 p_2$	z_4	$D_1 D_3 D_5 D_6$	11
		a_{12}	101001	$p_1 \bar{p}_2$	$z_2 z_3$	$D_1 D_3 D_6$	12
		a_{13}	101100	$\bar{p}_1$	$z_1 z_2 z_3$	$D_1 D_3 D_4$	13
B_5	1*1	a_{14}	110101	p_2	$z_1 z_4$	$D_1 D_2 D_6$	14
		a_{15}	110000	$\bar{p}_2 p_1$	$z_1 z_2 z_3$	$D_1 D_2$	15
		a_{16}	110010	$\bar{p}_2 \bar{p}_1$	z_2	$D_1 D_2 D_5$	16

and corresponding codes from Table 5.17 gives the state codes (5.37) which are the same as the ones shown in Fig. 5.26. We take the RLC from Table 5.10.

Using Table 5.17, we can create the DST of CFSM $MP_K Y1(\Gamma_9)$ on the base of Table 5.14. It is Table 5.18.

Table 5.18 represents the blocks PALer Φ and PALerZ. For example, the following minimized functions can be derived from Table 5.18:

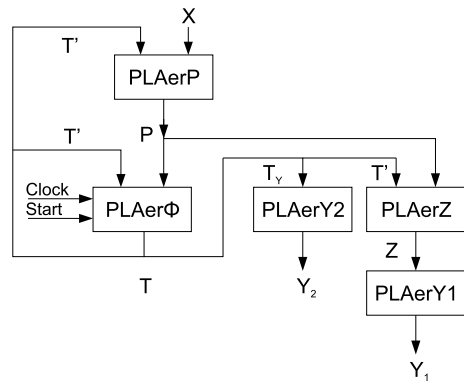
$$\begin{aligned}
 D_2 &= [F_4 \vee F_5 \vee F_6] \vee F_{10} \vee [F_{14} \vee F_{15} \vee F_{16}] = \\
 &= \bar{T}_1 T_3 \vee \bar{T}_1 T_2 \bar{p}_1 \bar{x}_7 \vee T_1 T_3 = T_3 \vee \bar{T}_1 T_2 \bar{p}_1 \bar{x}_7; \\
 z_1 &= F_2 \vee F_6 \vee [F_7 \vee F_8 \vee F_9] \vee F_{13} \vee [F_{14} \vee F_{15}] = \\
 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 p_1 \bar{p}_2 \vee \dots \vee T_1 T_3 p_2 \vee T_1 T_3 p_1.
 \end{aligned} \tag{5.46}$$

Using Table 5.17, it is possible to find functions representing PALerP. These equations are the same as (5.27). The system $Y1(Z)$ is the same as (5.23). The system $Y2(T_Y)$ is represented by (5.44). Of course, it is possible to construct tables for each block of PALs. But the same SBFs follow from tables.

If blocks of PALs are replaced by blocks of PLAs, we will get PLA-based circuits of CFSMs. For example, the structural diagram of $MP_K Y1$ CFSM is shown in Fig. 5.29.

Acting in the same manner, it is possible to obtain all PLA-based CFSMs. The methods of synthesis include the same steps as for PAL-based CFSMs. But now is necessary to place different subsystems of functions $P(T', X)$, $\Psi(T', P)$, $Z(T', P)$,

Fig. 5.29 PLA-based structural diagram of $MP_K Y1$ CFSM



$P(T', X)$ and $Y2(T_Y)$ among different PLAs. We do not discuss this problem in our book.

References

1. Achasova S (1987) Synthesis algorithms for automata with PLAs. M: Soviet radio (in Russian)
2. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
3. Baranov S, Skliarov V (1986) Digital devices with programmable LSIs with matrix structure. Radio and Sviaz (in Russian)
4. Barkalov A, Titarenko L (2009) Logic synthesis for FSM-based control units, vol 53. Lecture notes in electrical engineering. Springer, Berlin
5. Barkalov A, Titarenko L, Mielcarek K, Chmielewski S (2020) Logic synthesis for FPGA-based control units - Structural decomposition in logic design, vol 636. Lecture notes in electrical engineering. Springer, Berlin
6. Bibilo P (1992) Synthesis of combinational PLA-structures with VLSIs. Nauka i Technika, Minsk (in Russian)
7. Czerwinski R, Kania D (2013) Finite state machine logic synthesis for complex programmable logic devices, vol 231. Lecture notes in electrical engineering. Springer, Berlin
8. Klimovich AS, Solov'ev VV (2011) A method for minimizing Moore finite-state machines by merging two states. J Comput Syst Sci Int 50(6):907–920
9. McCluskey E (1986) Logic design principles. Prentice Hall, Englewood Cliffs
10. Skliarov V (1984) Synthesis of automata with matrix LSIs. Nauka i Technika, Minsk (in Russian)
11. Sklyarov V, Skliarova I, Barkalov A, Titarenko L (2014) Synthesis and optimization of FPGA-based systems, vol 294. Lecture notes in electrical engineering. Springer, Berlin
12. Solovjov V (1996) Design of functional blocks of digital systems with programmable logic devices. Bestprint (in Russian)
13. Solovjov V (2001) Design of digital systems basing on programmable logic integrated circuits. Hotline-Telecom, Moscow, Moscow (in Russian)

Chapter 6

Heterogeneous CPLD-based Synthesis of CFSMs



Abstract The Chapter deals with synthesis of CFSMs using the heterogeneous CPLDs, which include PAL-based macrocells and embedded memory blocks. In the beginning, the circuits based on embedded memory blocks are analysed. Next, different models of CFSMs with trivial state assignment are discussed. The EMBs are used to implement systems of regular functions, PAL-based macrocells implement irregular functions. Next, it is shown how to use EMBs for the replacement of logical conditions. Different optimization methods are discussed for CFSMs with the optimal state assignment. Next, the CFSM models based of the primary encoding of classes are proposed. The conditions are shown allowing execution of proposed methods. The last part of the chapter is devoted to synthesis methods based on the transformation of object codes. They are based on the representation of states by collections of microoperations and identifiers, as well as representation of microoperations by states and identifiers. All proposed methods target reducing the numbers of product terms in systems of Boolean functions representing CFSM circuits. The examples of synthesis are given for majority of proposed CFSM models.

6.1 Preliminary Information

There are two types of MCs in the heterogeneous CPLD chips [1, 20]: PAL-based and EMB-based macrocells. Each PAL-based MC has exactly a single output having possibility to be connected with a programmable FF. These MCs have wide fan-ins. They target implementing single functions: a single MC implements a circuit for one function.

The EMBs target implementing SBF [10, 15] having up to t_F BFs with $L(f_i) \leq S_A$. The pair $\langle S_A, t_F \rangle$ determines the configuration of EMB. The number of memory cells having t_F outputs and S_A inputs is determined as (2.30). The volume of an EMB is equal to V_0 determined as (2.29). We use the term “EMB” for memory of CPLD. As a rule, this term is used for memory blocks of FPGA chips by Xilinx [12, 13].

As follows from previous Chapters, there are three types of SBFs representing CFSM logic circuits. There are multiplexer, irregular and regular SBFs. Let us list them in this section.

There are the following multiplexer functions connected with RLC:

$$P = P(T, X); \quad (6.1)$$

$$P = P(\tau, X); \quad (6.2)$$

$$P = P(T', X). \quad (6.3)$$

There are five systems of regular functions. They do not depend on either $x_l \in X$ or $p_g \in P$. They are the following:

$$Y2 = Y2(T); \quad (6.4)$$

$$Y2 = Y2(T_Y); \quad (6.5)$$

$$Y2 = Y2(V); \quad (6.6)$$

$$Y1 = Y1(Z); \quad (6.7)$$

$$\tau = \tau(T). \quad (6.8)$$

All functions different from (6.1)–(6.8) belong to the class of irregular functions. The following functions are irregular: $D_r \in \Phi$, $y_n \in Y1$, $v_r \in V$ and $z_r \in Z$. They are represented by the following systems:

$$\Phi = \Phi(T, X); \quad (6.9)$$

$$\Phi = \Phi(T', X); \quad (6.10)$$

$$\Phi = \Phi(\tau, X); \quad (6.11)$$

$$\Phi = \Phi(T, P); \quad (6.12)$$

$$\Phi = \Phi(T', P); \quad (6.13)$$

$$\Phi = \Phi(\tau, P); \quad (6.14)$$

$$Y1 = Y1(T, X); \quad (6.15)$$

$$Y1 = Y1(T', X); \quad (6.16)$$

$$Y1 = Y1(\tau, X); \quad (6.17)$$

$$Y1 = Y1(T, P); \quad (6.18)$$

$$Y1 = Y1(T', P); \quad (6.19)$$

$$Y1 = Y1(\tau, P); \quad (6.20)$$

$$Z = Z(T, X); \quad (6.21)$$

$$Z = Z(T', X); \quad (6.22)$$

$$Z = Z(\tau, X); \quad (6.23)$$

$$Z = Z(T, P); \quad (6.24)$$

$$Z = Z(T', P); \quad (6.25)$$

$$Z = Z(\tau, P); \quad (6.26)$$

$$V = V(T, X); \quad (6.27)$$

$$V = V(T', X); \quad (6.28)$$

$$V = V(\tau, X); \quad (6.29)$$

$$V = V(T, P); \quad (6.30)$$

$$V = V(T', P); \quad (6.31)$$

$$V = V(\tau, P). \quad (6.32)$$

Designers of VLSI-based digital systems tend to get circuits that occupy as little the chip area as possible. To do this, it should be reduced the number of elements and their interconnections in a particular circuit. The best way to get this result is to use extensively the EMBs. Let the following condition take place:

$$2^{L+R}(R + N1 + N2) \ll V_0. \quad (6.33)$$

In this case, the CFSM can be implemented using a single EMB (Fig. 6.1).

In circuit (Fig. 6.1), the outputs $y_n \in Y2$ and $D_r \in \Phi$ are registered. If it is impossible, two EMBs can be used (Fig. 6.2).

In the circuit (Fig. 6.2), the outputs $y_n \in Y2$ and $D_r \in \Phi$ are registered. The outputs $y_n \in Y1$ are asynchronous.

The EMBs of modern CPLD are rather simple [10]. Analysis of library [11] show that only 6% of benchmark FSMs can be implemented in a way shown in Figs. 6.1 and 6.2. So, they are used to implement systems of regular functions. In the case of CFSM, it makes sense to implement the systems (6.4)–(6.8) with EMBs.

Each PAL-macrocell has S inputs and q terms. The number of inputs is large enough to implement any function from [11]. So, there is a wide fan-in of PAL-macrocells. Let the following condition take place:

$$L + R < S. \quad (6.34)$$

Fig. 6.1 EMB-based circuit of CFSM

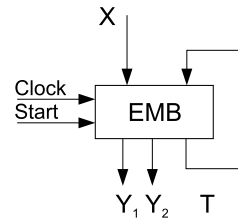
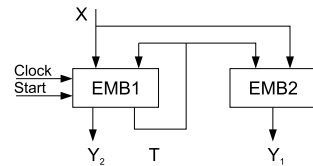


Fig. 6.2 CFSM with two EMBs



In this case, there are Δ_S free inputs:

$$\Delta_S = S - (L + R). \quad (6.35)$$

The free inputs can be used to receive some additional variables. For example, variables $T_r \in T$ and $\tau_r \in \tau$ can enter the same MC [6, 18]. We discuss this approach a bit later.

So, in the case of heterogeneous CPLD, the PAL-based MCs are used to implement both multiplexer and irregular functions. The EMBs are suitable for implementing regular functions (6.4)–(6.8). In addition, it is advisable to use EMBs for implementing some subsystems of irregular functions (6.9)–(6.32). The latter makes sense if the condition (6.33) takes place for such subsystems. An EMB implements a truth table [14, 15] of an SBF. Due to it, there is no need in optimization of functions from these SBFs.

6.2 Synthesis of CFSMs with Trivial State Assignment

Let us start from P CFSM. Its circuit is represented by SBFs (6.9), (6.15) and (6.4). So, there are two systems of irregular functions. They should be implemented as PALers. The system (6.4) can be implemented using EMBs. It leads to structural diagram shown in Fig. 6.3.

Obviously, the $PLAer\Phi$ implements (6.9), $PALerY1$ implements (6.15) and $EMBerY2$ the system (6.4). We assume that the latter block can include two and more EMBs. Because of it, we use the name $EMBer$.

The system (6.4) should be implemented as $EMBer$ if the following condition takes place:

$$2^R \leq V_0. \quad (6.36)$$

The condition (6.36) corresponds to the configuration $\langle S_A, t_F \rangle$, where

$$S_A = R. \quad (6.37)$$

Taking t_F from this pair gives the number of EMBs in $EMBerY2$:

Fig. 6.3 Structural diagram of P CFSM

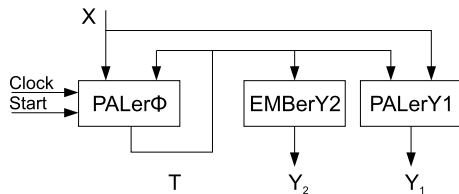
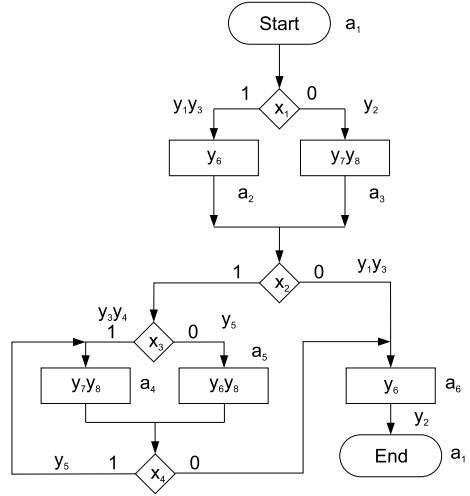


Fig. 6.4 Marked GSA Γ_{10} 

$$n_E = \left\lceil \frac{N2}{t_F} \right\rceil. \quad (6.38)$$

If condition (6.36) is violated for $\langle S_A, 1 \rangle$, then the system (6.4) should be implemented as PALerY2.

In this book, we propose a synthesis method for P CFSM implemented with heterogeneous CPLDs. It includes the following steps:

1. Marking the initial GSA Γ by states of Moore FSM.
2. Executing the trivial state assignment.
3. Creating the DST of P CFSM.
4. Deriving systems (6.9) and (6.15) from the DST.
5. Creating the table of EMBer for system (6.4).
6. Implementing the systems (6.9) and (6.15) with PALs.
7. Implementing the system (6.4) with EMBs.

Let us discuss an example of synthesis for CFSM $P(\Gamma_{10})$. The GSA Γ_{10} is shown in Fig. 6.4. It is marked by states of Moore FSM using the rules [2].

The following sets can be found for GSA Γ_{10} : $A = \{a_1, \dots, a_6\}$, $X = \{x_1, \dots, x_4\}$, $Y1 = \{y_1, \dots, y_5\}$, $Y2 = \{y_6, \dots, y_9\}$. It gives $M = 6$, $L = 4$, $N1 = 5$, $N2 = 4$, $N = 9$. Using (1.2) gives $R = 3$, $\Phi = \{D_1, D_2, D_3\}$, $T = \{T_1, T_2, T_3\}$. In the case of trivial state assignment, there is $K(a_m) = (m - 1)_2$. It gives $K(a_1) = 000$, $K(a_2) = 001, \dots, K(a_6) = 101$.

There are $H = 13$ transitions between the states $a_m \in A$ of CFSM $P(\Gamma_{10})$. So, there are $H = 13$ rows in DST (Table 6.1).

Using contents of columns a_m , $K(a_m)$, X_h , Φ_h allows deriving the SBF (6.9). For example, there is

Table 6.1 DST of CSFM $P(\Gamma_{10})$

a_m	$K(a_m)$	$Y(a_m)$	a_s	$K(a_s)$	$\bar{X}_h$	Y_h	Φ_h	h
a_1	000	—	a_2	001	x_1	$y_1 y_3$	D_3	1
			a_3	010	$\bar{x}_1$	y_2	D_2	2
a_2	001	y_6	a_4	011	$x_2 x_3$	$y_3 y_4$	$D_2 D_3$	3
			a_5	100	$x_2 \bar{x}_3$	y_5	D_1	4
			a_6	101	$\bar{x}_2$	$y_1 y_3$	$D_1 D_3$	5
a_3	010	$y_7 y_8$	a_4	011	$x_2 x_3$	$y_3 y_4$	$D_2 D_6$	6
			a_5	100	$x_2 \bar{x}_3$	y_5	D_1	7
			a_6	101	$\bar{x}_2$	$y_1 y_3$	$D_1 D_3$	8
a_4	011	$y_7 y_9$	a_4	011	x_4	y_5	$D_2 D_3$	9
			a_6	101	$\bar{x}_4$	—	$D_1 D_3$	10
a_5	100	$y_6 y_8$	a_4	011	x_4	y_5	$D_2 D_3$	11
			a_6	101	$\bar{x}_4$	—	$D_1 D_3$	12
a_6	101	y_6	a_1	000	1	y_2	—	13

$$D_1 = F_4 \vee F_5 \vee F_7 \vee F_8 \vee F_{10} \vee F_{12} = \bar{T}_1 \bar{T}_2 T_3 x_2 \bar{x}_3 \vee \bar{T}_1 \bar{T}_2 x_3 \bar{x}_2 \vee \dots \vee T_1 \bar{T}_3 \bar{x}_4 = D_1(T_1, T_2, T_3, x_2, x_3, x_4). \quad (6.39)$$

So, there is $NA(D_1) = 6$ and $Q(D_1) = 6$. It is enough a single PAL(S, q) with $S = q = 6$ to implement (6.39).

Using columns a_m , $K(a_m)$, X_h , Y_h gives functions $y_n \in Y1$. For example, the following equation can be derived:

$$y_1 = F_4 \vee F_5 \vee F_8 = \bar{T}_1 \bar{T}_2 \bar{T}_3 x_1 \vee \bar{T}_1 \bar{T}_2 T_3 \bar{x}_2 \vee T_2 \bar{T}_3 \bar{x}_2 = y_1(T_1, T_2, T_3, x_1, x_2). \quad (6.40)$$

The circuit for y_1 includes a single PAL cell having $S \geq 5$, $q \geq 3$.

Using the same approach, it is possible to get SBFs for PALer Φ and PALer $Y1$. Let it be

$$q_{max} = \max(Q(D_1), \dots, Q(y_n)); \quad (6.41)$$

$$S_{max} = \max(NA(D_1), \dots, NA(y_n)). \quad (6.42)$$

We estimate only the numbers of literals and terms in equations for $D_r \in \Phi$ and $y_n \in Y1$. If the conditions

$$q \geq q_{max}; \quad (6.43)$$

$$S \geq S_{max} \quad (6.44)$$

take places, then there is $R + N1$ MCs in PALer Φ and PALer $Y1$.

Table 6.2 Table of EMB for CSFM $P(\Gamma_{10})$

a_m	$K(a_m)$			$Y2$				m
	T_1	T_2	T_3	y_6	y_7	y_8	y_9	
a_1	0	0	0	0	0	0	0	1
a_2	0	0	1	1	0	0	0	2
a_3	0	1	0	0	1	1	0	3
a_4	0	1	1	0	1	0	1	4
a_5	1	0	0	1	0	1	0	5
a_6	1	0	1	1	0	0	0	6
–	1	1	0	*	*	*	*	*
–	1	1	1	*	*	*	*	*

If either (6.43) or (6.44) is violated, then it is necessary more than a single PAL to implement a particular function. We do not discuss this case. It is considerably discussed in many articles and books. For example, it can be found in [4, 7–9, 19].

Let the following conditions take place for some configuration $\langle S_A, t_F \rangle$:

$$S_A \geq R; \quad (6.45)$$

$$t_F \geq N2. \quad (6.46)$$

In this case, it is enough a single EMB to implement the system (6.4).

Let it be $S_A = 3$, $t_F = 4$. So, it is enough to use only a single EMB(3, 4) to implement system $y_n \in Y2$ of $P(\Gamma_{10})$.

There are 2^R rows in the table of EMB. It includes the columns a_m , $K(a_m)$, $Y2$, m . In the discussed case, it is Table 6.2.

The cells 110, 111 do not keep useful information. It is connected with the natural redundancy of memory blocks: they have exactly 2^{S_A} cells.

Let it be PALs with $S = 7$, $q = 8$ and EMB(3, 4). In this case, there are $R + N1 = 8$ MCs in the circuits of PALer Φ and PALer $Y1$. Also, there is a single EMB implementing $y_n \in Y2$ (Fig. 6.5).

Let us discuss a case of PY1 CFSM. In this case, the microoperations $y_n \in Y1$ are represented by the system (6.7). It leads to the structural diagram shown in Fig. 6.6. It includes two blocks of EMBs. The EMBer $Y1$ implements the system (6.7), the EMBer $Y2$ the system (6.4). The PALer Z implements the system (6.21), the PALer Φ the system (6.9).

Let the following condition take place:

$$2^{Rq+R}(N1 + N2) \leq V_0. \quad (6.47)$$

In this case, it is possible to use a single EMB to implement systems $y_n \in Y1 \cup Y2$. Let us discuss such a case for CFSM PY1(Γ_{10}).

Fig. 6.5 Logic circuit of CFSM $P(\Gamma_{10})$

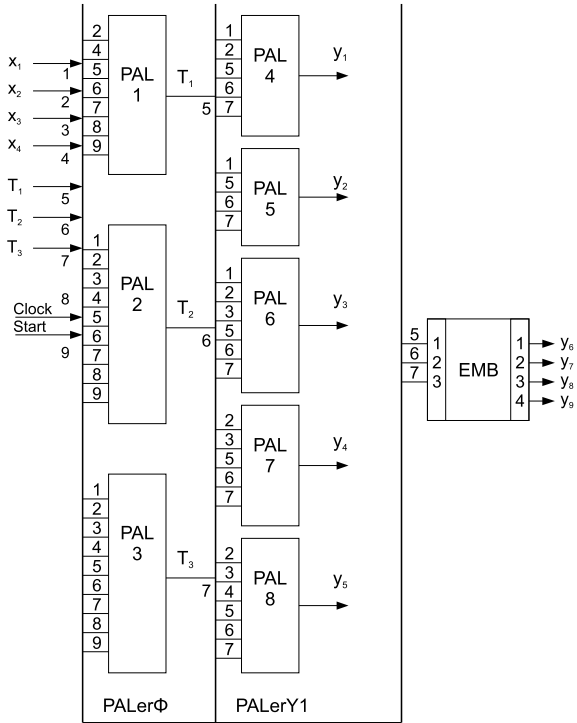
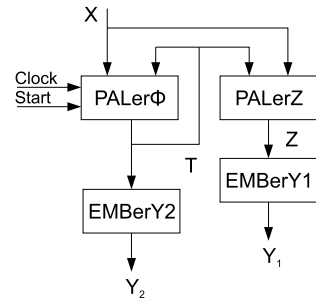


Fig. 6.6 Structural diagram of PY1 CFSM



There are the following CMOs $Y_q \subseteq Y1$ in the discussed case: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_2\}$, $Y_3 = \{y_2\}$, $Y_4 = \{y_3, y_4\}$, $Y_5 = \{y_5\}$. So, there is $Q = 5$. Using (1.32) gives $R_Q = 3$, $Z = \{z_1, z_2, z_3\}$.

Let us use the frequency approach [5] for encoding of CMOs $Y_q \subseteq Y1$. This approach is as follows: the more common a CMO is in DST, the more zeros its code contains. This reduce the number of terms in SOPs of functions $y_n \in Y1$.

Let $H(Y_q)$ be the number of rows in which the CMO Y_q occurs. Let us create the encoding queue γ in descending order of parameter $H(Y_q)$. There are $H(Y_1) = 2$, $H(Y_2) = 3$, $H(Y_3) = H(Y_4) = 2$, $H(Y_5) = 4$. It gives the queue

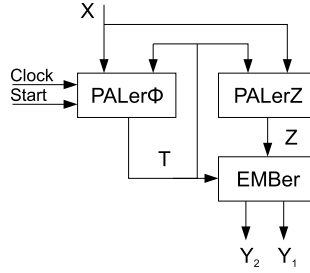


Fig. 6.7 Structural diagram of PY1 CFSM with a shared EMBer

Table 6.3 A part of table of EMBer for PY1(Γ_{10})

T			Z			Y1					Y2				h
T_1	T_2	T_3	z_1	z_2	z_3	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	
0	0	1	0	0	0	0	0	0	0	1	1	0	0	0	9
0	0	1	0	0	1	1	0	1	0	0	1	0	0	0	10
0	0	1	0	1	0	0	0	0	0	0	1	0	0	0	11
0	0	1	0	1	1	0	0	1	1	0	1	0	0	0	12
0	0	1	1	0	0	0	1	0	0	0	1	0	0	0	13
0	0	1	1	0	1	0	0	0	0	0	1	0	0	0	14
0	0	1	1	1	0	0	0	0	0	0	1	0	0	0	15
0	0	1	1	1	1	0	0	0	0	0	1	0	0	0	16

$\gamma = \langle Y_5, Y_2, Y_1, Y_3, Y_4 \rangle$. Now we have $K(Y_5) = 000$, $K(Y_2) = 001$, $K(Y_1) = 010$, $K(Y_3) = 100$ and $K(Y_4) = 011$.

Let it be the configuration $\langle 6, 9 \rangle$ for a given EMB. It gives $V_0 = 576$. So, we have the equality between resource requirements and availability. So, it is possible to implement both (6.4) and (6.7) using a single EMB(6, 9). It leads to a structural diagram (Fig. 6.7).

The block EMBer is shared between Y1 and Y2. It can be represented by table having $N_1 + N_2$ columns for MOs and 2^{R_0+R} rows. In the discussed case, there are 64 rows in such a table. We show some part of table of EMBer in Table 6.3.

There is $K(a_2)$ in the column T of Table 6.3. There are codes $K(Y_q)$ for CMOs $Y_q \subseteq Y_1$ in the rows 9–16 of Table 6.3. They are listed in the column Z. The MO $y_6 \in Y(a_2)$ is written in all these rows. The addresses XXX101, XXX110 and XXX111 correspond to insignificant assignments of variables $z_r \in Z$. Obviously, the addresses 110XXX and 111XXX correspond to insignificant assignments of state variables $T_r \in T$. All other rows are filled in the same way.

To diminish the number of inputs required by PALer Φ and PALerZ, it is possible to use the RLC. It leads to MPY1 CFSM (Fig. 6.8).

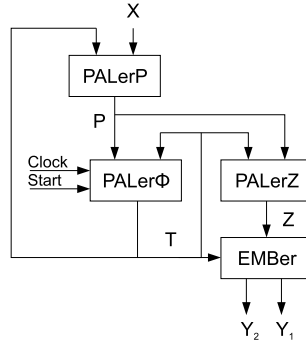


Fig. 6.8 Structural diagram of MPY1 CFSM with a shared EMBer

Fig. 6.9 Structural diagram of MPY1 CFSM with two EMBerS

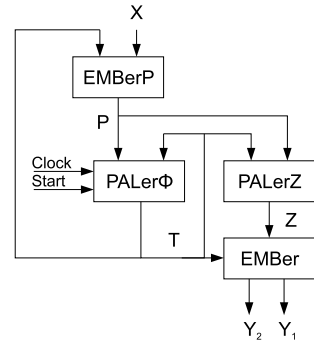


Table 6.4 Table of RLC for MPY1(Γ_9)

$P \backslash a_m$	a_1	a_2	a_3	a_4	a_5	a_6
p_1	x_1	x_2	x_2	x_4	x_4	-
p_2	-	x_3	x_3	-	-	-

We assume that the condition (6.47) takes place. In MPY1 CFSM, the PALer P implements the system (6.1), the PALer Φ the system (6.12). The PALer Z implements the system (6.24). The function of EMBer is the same as for PY1 CFSM.

Let condition (6.33) be violated but the following condition take place:

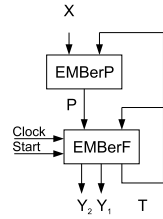
$$2^{R+L} \cdot G \leq V_0. \quad (6.48)$$

In this case, the system $P = P(T, X)$ can be implemented as EMBer P . It leads to MPY1 CFSM shown in Fig. 6.9.

Analysis of DST (Table 6.1) shows that there is $G = 2$. It gives the set $P = \{p_1, p_1\}$. Let us execute RLC as it is shown in Table 6.4.

Table 6.5 Table of EMBerP for MPY1(Γ_{10})

a_m	$K(a_m)$			X			P	h
	T_1	T_2	T_3	x_1	x_2	x_4	p_1	
a_3	0	1	0	0	0	0	0	17
	0	1	0	0	0	1	0	18
	0	1	0	0	1	0	1	19
	0	1	0	0	1	1	1	20
	0	1	0	1	0	0	0	21
	0	1	0	1	0	1	0	22

Fig. 6.10 Structural diagram of MP CFSM with two EMBers

There is $p_2 = x_3$. So, it is necessary to implement only the function $p_1 = A_1x_1 \vee (A_2 \vee A_3)x_2 \vee (A_4 \vee A_5)x_4$. So, it is enough to have an EMB with $S_A = 6$ and $t_F = 1$.

There are the following columns in table of EMBerP: a_m , $K(a_m)$, X, P, h. There are 64 rows in this table for MPY1(Γ_{10}). Some of them are represented by Table 6.5.

There are 8 combinations of $\langle x_1, x_2, x_4 \rangle$ for each state $a_m \in A$. So, the table for a_3 starts from the column 17. Because $p_1 = x_2$ for $a_3 \in A$, the column p_1 repeats the column x_2 of Table 6.5.

Let the following condition takes place:

$$2^{G+R}(R + N1 + N2) \leq V_0. \quad (6.49)$$

It means that functions (6.4), (6.12) and (6.18) can be implemented as a single EMB. It leads to MP CFSM (Fig. 6.10).

In MP CFSM, the EMBerP implements functions (6.1), the EMBerF functions (6.4), (6.12) and (6.18). If conditions (6.48)–(6.49) take places, each EMBer is implemented as a single EMB.

In the case of CFSM MP(Γ_{10}), the EMBerP is represented by Fig. 6.5. The table of EMBerF has the following columns: a_m , $K(a_m)$, P_1 , x_3 (they determine the address of memory cell), $Y1$, $Y2$, Φ (they include the contents of cells), w (the number of cell), $w \in \{1, \dots, W\}$. The value of W is determined as

$$W = 2^{R+G}. \quad (6.50)$$

Table 6.6 A part of table of EMBerF for CFSM MP(Γ_{10})

a_m	$K(a_m)$			P		$Y1$					$Y2$				Ψ			h	w
	T_1	T_2	T_3	P_1	x_3	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	D_1	D_2	D_3		
a_1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	2	1
	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	0	2	2
	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	1	1	3
	0	0	0	1	1	1	0	1	0	0	0	0	0	0	0	0	1	1	4
a_2	0	0	1	0	0	1	0	1	0	0	1	0	0	0	1	0	1	5	5
	0	0	1	0	1	1	0	1	0	0	1	0	0	0	1	0	1	5	6
	0	0	1	1	0	0	0	0	0	1	1	0	0	0	1	0	0	4	7
	0	0	1	1	1	0	0	1	1	0	1	0	0	0	0	1	1	3	8

There are $H(a_m)$ rows determining transitions from the state $a_m \in A$:

$$H(a_m) = 2^G. \quad (6.51)$$

In the discussed case, there is $W = 32$, $H(a_m) = 4$. A part of table of EMBerF is represented by Table 6.6.

To show the relationship between Tables 6.1 and 6.6, we entered the column h in Table 6.6. This facilitates the process of verification of the correctness of the contents of the rows of Table 6.6. We hope that the relationship among the tables is transparent and requires no explanation.

6.3 Synthesis of CFSMs with Optimal State Assignment

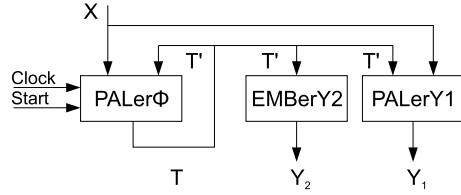
Let us use existence of PES in CFSMs. We start from the OSA. Let us point out there is no sense in using RSA because the system (6.4) is implemented by EMBs.

There are the same steps in synthesis methods of P and P_0 CFSMs. Let us discuss an example of synthesis of $P_0(\Gamma_{10})$. Its structural diagram is the same as it is shown in Fig. 6.3.

Using the Definition (3.1), we find the partition $\Pi_A = \{B_1, \dots, B_4\}$ where $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3\}$, $B_3 = \{a_4, a_5\}$ and $B_4 = \{a_6\}$. Let us encode the states in the optimal way (Fig. 6.11).

Fig. 6.11 Outcomes of OSA for $P_0(\Gamma_{10})$

		$T_1 T_2$			
		00	01	11	10
T_3	0	a_1	a_2	a_6	a_4
	1	*	a_3	*	a_5

Fig. 6.12 Structural diagram of P_0 CFSM**Table 6.7** DST of CFSM $P_0(\Gamma_{10})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	00*	a_2	010	x_1	$y_1 y_3$	D_2	1
		a_3	011	$\bar{x}_1$	y_2	$D_2 D_3$	2
B_2	01*	a_4	100	$x_2 x_3$	$y_3 y_4$	D_1	3
		a_5	101	$x_2 \bar{x}_3$	y_5	$D_1 D_3$	4
		a_6	110	$\bar{x}_2$	$y_1 y_3$	$D_1 D_2$	5
B_3	10*	a_4	100	x_4	y_5	D_1	6
		a_6	110	$\bar{x}_4$	—	$D_1 D_2$	7
B_4	11*	a_1	000	1	y_2	—	8

Using Fig. 6.11 gives $K(B_1) = 00*$, $K(B_2) = 01*$, $K(B_3) = 10*$ and $K(B_4) = 11*$. So, the transitions of CFSM $P_0(\Gamma_{10})$ are determined by the variables $T_r \in T' = \{T_1, T_2\}$ and LCs $x_l \in X$. It is the best case leading to the structural diagram shown in Fig. 6.12.

In P_0 CFSM, the PALer Φ implements the SBF (6.10), the PALerY1 the SBF (6.16). The EMBerY2 implements the system (6.4), which is represented by a truth table.

To get functions (6.10) and (6.16), it is necessary to construct the DST of P_0 CFSM. It is constructed on the base of the system of generalized formulae of transitions. This system is the following for CFSM $P_0(\Gamma_{10})$:

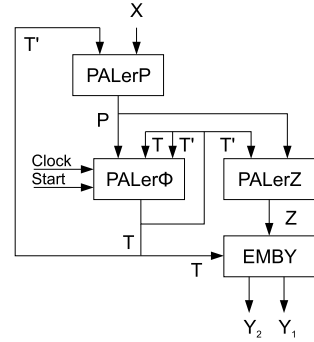
$$\begin{aligned}
 B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 a_3; \\
 B_2 &\rightarrow x_2 x_3 a_4 \vee x_2 \bar{x}_3 a_5 \vee \bar{x}_2 a_6; \\
 B_3 &\rightarrow x_4 a_4 \vee \bar{x}_4 a_6; \\
 B_4 &\rightarrow a_1.
 \end{aligned} \tag{6.52}$$

There is $H_{opt} = 8$ because the system (6.52) includes 8 terms. It determines the DST (Table 6.7).

The PALer Φ is represented by the system

$$\begin{aligned}
 D_1 &= \bar{T}_1 T_2 \vee T_1 \bar{T}_2; \\
 D_2 &= \bar{T}_1 \bar{T}_2 \vee \bar{T}_1 T_2 \bar{x}_2 \vee T_1 \bar{T}_2 \bar{x}_4; \\
 D_3 &= \bar{T}_1 \bar{T}_2 \bar{x}_1 \vee \bar{T}_1 T_2 x_2 \bar{x}_3.
 \end{aligned} \tag{6.53}$$

Fig. 6.13 Structural diagram of MP_0Y1 CFSM with a shared EMB



The PALerY1 is represented by the system

$$\begin{aligned}
 y_1 &= \bar{T}_1 \bar{T}_2 x_1 \vee \bar{T}_1 T_2 \bar{x}_2; \\
 y_2 &= \bar{T}_1 \bar{T}_2 \bar{x}_1 \vee T_1 T_2; \\
 y_3 &= \bar{T}_1 \bar{T}_2 x_1 \vee \bar{T}_1 T_2 x_3 \vee \bar{T}_1 T_2 \bar{x}_2; \\
 y_4 &= \bar{T}_1 T_2 x_2 x_3; \\
 y_5 &= \bar{T}_1 T_2 x_2 \bar{x}_3 \vee T_1 \bar{T}_2 x_4.
 \end{aligned} \tag{6.54}$$

For systems (6.53)–(6.54), there $q_{max} = 3$ and $S_{max} = 5$. So, it is enough to use 7 PAL(5, 3) macrocells to implement these systems.

To optimize the circuit of P_0 CFSM, it is possible to use: (1) the RLC, (2) ECMO and (3) both RLC and ECMO. Let us discuss the last case.

Let the condition (6.47) be violated, as well as (6.49). It means that EMBs can be used only for implementing systems (6.4) and (6.7). Let the condition (6.47) take place. So, both systems (6.4) and (6.7) are implemented by the same EMBY (Fig. 6.13).

In MP_0Y1 CFSM with a shared EMB, the PALerP implements the system (6.3). The functions $p_g \in P$ are used as arguments of systems (6.13) and (6.25). These systems are implemented by PALerΦ and PALerZ, respectively. At last, the EMBY implements both systems (6.4) and (6.7).

Let us discuss an example of synthesis for $MP_0Y1(\Gamma_{10})$. It includes the following general steps:

1. Marking the initial GSA Γ and creating the set of states A.
2. Creating the partition $\Pi_A = \{B_1, \dots, B_I\}$ of the set A by classes of PES.
3. Executing OSA.
4. Creating the table of RLC and finding the system $P = P(T', X)$.
5. Creating the CMOs $Y_q \subseteq Y1$ and execution of their encoding.
6. Creating the DST of MP_0Y1 CFSM.
7. Creating the systems $\Phi = (T', P)$ and $Z = Z(T', P)$.
8. Creating the table of EMBY.
9. Implementing CFSM logic circuit.

Table 6.8 Table of RLC for $MP_0Y1(\Gamma_{10})$

$P \setminus B_i$	B_1	B_2	B_3	B_4
p_1	x_1	x_2	x_4	–
p_2	–	x_3	–	–
$K(B_i)$	00*	01*	10*	11*

Table 6.9 DST of CSFM $P_0Y1(\Gamma_{10})$

B_i	$K(B_i)$	a_s	$K(a_s)$	P_h	Z_h	Φ_h	h
B_1	00*	a_2	010	p_1	$z_3 \ 2$	D_2	1
		a_3	011	$\bar{p}_1$	$z_1 \ 3$	$D_2 D_3$	2
B_2	01*	a_4	100	$p_1 x_3$	$z_2 z_3 \ 4$	D_1	3
		a_5	101	$p_1 \bar{x}_3$	– 5	$D_1 D_3$	4
		a_6	110	$\bar{p}_1$	$z_3 \ 2$	$D_1 D_2$	5
B_3	10*	a_4	100	p_1	– 5	D_1	6
		a_6	110	$\bar{p}_1$	$z_2 \ 1$	$D_1 D_2$	7
B_4	11*	a_1	000	1	$z_1 \ 3$	–	8

In the case of $MP_0Y1(\Gamma_{10})$, there are executed three first steps. The table of RLC is similar to Table 6.4. But the states $a_m \in A$ are replaced by classes $B_i \in \Pi_A$. It gives Table 6.8.

Using Table 6.8, we can get the system $P(T', X)$:

$$\begin{aligned} p_1 &= \bar{T}_1 \bar{T}_2 x_1 \vee \bar{T}_1 T_2 x_2 \vee T_1 \bar{T}_2 x_4; \\ p_2 &= x_3. \end{aligned} \quad (6.55)$$

So, it is enough a single PAL(5, 3) to execute the RLC in the discussed case.

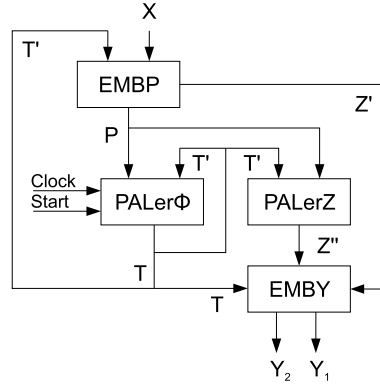
The step 5 is already executed during the synthesis of CFSM $PY1(\Gamma_{10})$. Let us have the same codes $K(Y_q)$ as for that example: $K(Y_1) = 010$, $K(Y_2) = 001$, $K(Y_3) = 100$, $K(Y_4) = 011$ and $K(Y_5) = 000$. There are the following CMOs: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_3\}$, $Y_3 = \{y_2\}$, $Y_4 = \{y_3, y_4\}$ and $Y_5 = \{y_5\}$.

To create the DST of MP_0Y1 CFSM, it is necessary to find the system of GFTs. To do it in the discussed case, it is enough to replace the variables $x_l \in X$ from (6.52) by variables $p_q \in P$. It gives the following system:

$$\begin{aligned} B_1 &\rightarrow p_1 a_2 \vee \bar{p}_1 a_3; \\ B_2 &\rightarrow p_1 x_3 a_4 \vee p_1 \bar{x}_3 a_5 \vee \bar{p}_1 a_6; \\ B_3 &\rightarrow p_1 a_4 \vee \bar{p}_1 a_6; \\ B_4 &\rightarrow a_1. \end{aligned} \quad (6.56)$$

There are the following columns in the DST of MP_0Y1 CFSM: B_i , $K(B_i)$, a_s , $K(a_s)$, P_h , Z_h , Φ_h , h . It is Table 6.9 in the discussed case.

Fig. 6.14 Structural diagram of MP_0Y1 CFSM with distributed functions Z



The $PALer\Phi$ is represented by the system of BFs similar to (6.53). But the variables $x_l \in X$ should be replaced by $p_q \in P$. For example, there is the following equation for function $D_2 \in \Phi$: $D_2 = \bar{T}_1 \bar{T}_2 \vee \bar{T}_1 T_2 \bar{p}_1 \vee T_1 \bar{T}_2 \bar{p}_1$.

The DST (Table 6.9) is the base to derive the system $Z = Z(T', P)$. It is the following:

$$\begin{aligned} z_1 &= \bar{T}_1 \bar{T}_2 \bar{p}_1 \vee T_1 T_2; \\ z_2 &= \bar{T}_1 T_2 p_1 x_3 \vee T_1 \bar{T}_2 \bar{p}_1; \\ z_3 &= \bar{T}_1 \bar{T}_2 p_1 \vee \bar{T}_1 T_2 x_3 \vee \bar{T}_1 T_2 \bar{p}_1. \end{aligned} \quad (6.57)$$

The table of $EMBY$ is similar to the Table 6.3. We do not discuss it for our example.

If EMBs are used to implement some parts of CFSM circuit, they can implement different SBFs (or their parts) by the same EMB. For example, let the following condition take place:

$$2^{R+L}(G + \Delta) = V_0. \quad (6.58)$$

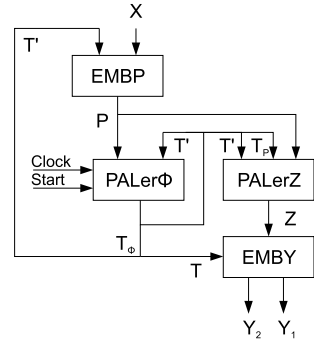
In this case, the EMB can be used to implement the system (6.3) and Δ equations from other systems. We should choose the most complex equations from $\Phi(T', P)$ and $Z(T', P)$. These equations are implemented by the EMB. For example, it leads to MP_0Y1 CFSM shown in Fig. 6.14.

In CFSM (Fig. 6.14), the $EMBP$ implements functions (6.3) and Δ the most complex functions from $Z(T', X)$. These functions form a set Z' , where $|Z'| = \Delta$. The functions $z_r \in Z''$ are implemented by the $PALerZ$. Here the following relations take place: $Z = Z' \cup Z''$, $Z' \cap Z'' = \emptyset$, $|Z'| = \Delta$ and $|Z''| = R_Q - \Delta$.

Obviously, it is possible to implement functions $T_r \in T_P$ using $EMBP$. In this case, the $PALer\Phi$ implements functions $T \in T_\Phi$ (Fig. 6.15).

Obviously, the following relations take place for CFSM (Fig. 6.15): $T = T_P \cup T_\Phi$, $T_P \cap T_\Phi = \emptyset$, $|T_P| = \Delta$ and $|T_\Phi| = R - \Delta$. Of course, it is possible to distribute both functions $D_r \in \Phi$ and $z_r \in Z$. We leave this task to our reader.

Fig. 6.15 Structural diagram of MP_0Y1 CFSM with distributed functions Φ



This approach can be used for any model of CFSM which circuit is implemented with PAL- and EMB-based MCs. For example, we can propose the following method for distribution of functions.

Let we have K blocks of EMBs. So, we should:

1. To estimate how many functions can be implemented using K EMBs. Let it be N_K functions.
2. To get the system of BFs representing the CFSM logic circuit.
3. To select N_K the most complex functions than require more than one PAL(S, q) for implementing the corresponding circuit.
4. To implement these N_K functions by EMBer.

Obviously, the remained functions are implemented as PALers. Of course, in this case interconnections between blocks could be more complex than in cases of CFSMs without the distribution of functions.

6.4 Synthesis of CFSMs with Primary Encoding of Classes

There are two such methods:

1. **The expansion of code space.** It is based on representation of $K(a_m)$ as (5.36). It leads to P_E CFSMs.
2. **The encoding of classes and CMOs** $Y_q \subseteq Y_2$. It is based on (5.37). This approach leads to $P_K Y_2$ CFSM.

Both methods can be used if the outcome of OSA is too far from optimal. It depends on the availability of insignificant assignments of state variables $T_r \in T$. The more of these assignments, the more chances for an outcome of OSA close to the optimal. Recall that the outcome of OSA is considered optimal if each class $B_i \in \Pi_A$ can be represented as a single generalized interval of R -dimensional Boolean space.

Let us consider the GSA Γ_{11} (Fig. 6.16). It is marked by states of Moore FSM.

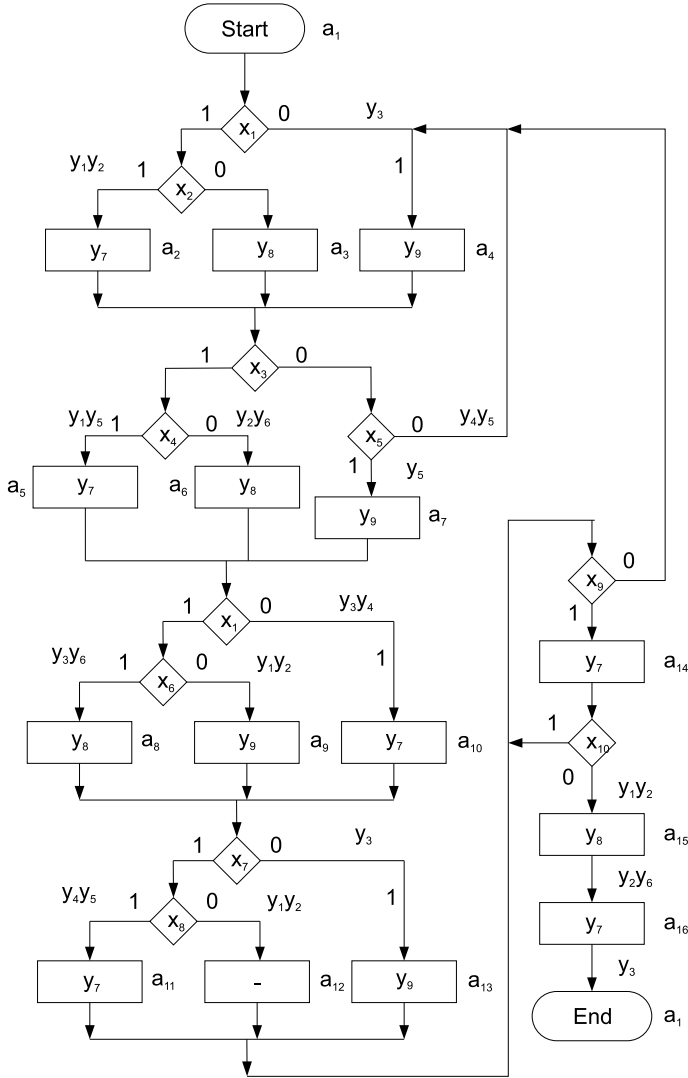


Fig. 6.16 Marked GSA Γ_{11}

There is $M = 16$. Using (1.2) gives $R = 4$ and $\Phi = \{D_1, \dots, D_4\}$, $T = \{T_1, \dots, T_4\}$. The following sets can be derived from GSA Γ_{11} : $X = \{x_1, \dots, x_{10}\}$, $Y1 = \{y_1, \dots, y_6\}$, $Y2 = \{y_7, y_8, y_9\}$. It gives $L = 10$, $N1 = 6$, $N2 = 3$, $N = 9$.

The following partition Π_A exists for Γ_{11} : $\Pi_A = \{B_1, \dots, B_8\}$ with $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6, a_7\}$, $B_4 = \{a_8, a_9, a_{10}\}$, $B_5 = \{a_{11}, a_{12}, a_{13}\}$, $B_6 = \{a_{14}\}$, $B_7 = \{a_{15}\}$, $B_8 = \{a_{16}\}$. So, there is $I = 8$. Let us execute OSA for CFSM $P_0(\Gamma_{11})$. The outcome of OSA is shown in Fig. 6.17.

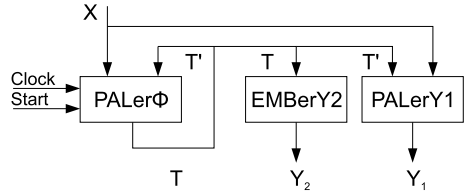
Fig. 6.17 Outcome of OSA
for $P_0(\Gamma_{11})$

$T_3T_4 \backslash T_1T_2$		00	01	11	10
		a_1	a_4	a_8	a_5
00	01	a_{11}	a_2	a_9	a_6
	11	a_{13}	a_3	a_{10}	a_7
10	01	a_{12}	a_{14}	a_{16}	a_{15}
	11				

Fig. 6.18 State codes of
CFSM $P_E(\Gamma_{11})$

$T_4T_5 \backslash T_1T_2T_3$		000	001	011	010	110	111	101	100
		a_1	a_2	a_8	a_5	a_{15}	a_{16}	a_{14}	a_{11}
T_4T_5	00	a_1	a_2	a_8	a_5	a_{15}	a_{16}	a_{14}	a_{11}
	01	*	a_3	a_9	a_6	*	*	*	a_{12}
	11	*	*	*	*	*	*	*	*
	10	*	a_4	a_{10}	a_7	*	*	*	a_{13}

Fig. 6.19 Structural
diagram of P_E CFSM



As follows from Fig. 6.17, classes $B_1, B_6, B_7, B_8 \in \Pi_A$ are represented by a single interval. Other classes require 2 generalized intervals to be represented.

The maximal number of DST rows is determined by (4.16). It is equal to 46 for $P(\Gamma_{11})$. If each class is represented by a single generalized interval, then there is $H_T = 20$. It follows from (4.18). For $P_0(\Gamma_{11})$, we can use (4.17) and get $H_{opt} = 33$. So, the numbers of terms can be reduced in 2,3 times due to primary encoding of classes $B_i \in \Pi_A$ for the discussed case.

Let us start from P_E CFSM. Analysis of GSA Γ_{11} gives $M_0 = 3$. So, using (3.50) gives $R_0 = 2$. Because of $I = 8$, there is $R_A = 3$. It gives, the sets $T = T' \cup T_A = \{T_1, T_2, T_3\} \cup \{T_4, T_5\}$. Let us encode states as it is shown in Fig. 6.18.

As follows from Fig. 6.18, the classes $B_i \in \Pi_A$ are encoded in the trivial way: $K(B_1) = 000$, $K(B_2) = 001, \dots, K(B_8) = 111$. The codes $C(a_m)$ can be found very simply. For example, there is $C(a_1) = C(a_2) = \dots = C(a_{16}) = 00$. The codes $K(a_m)$ are represented as (5.36). For example, $K(a_1) = K(B_1) * C(a_1) = 000**$, $K(a_2) = K(B_2) * C(a_2) = 00100$, and so on. Forming codes $K(a_m)$, we took into account the insignificant assignments of variables $T_r \in T_A$.

The structural diagram of P_E CFSM is shown in Fig. 6.19. Here, we assume that only the system (6.4) can be implemented using EMBs.

Table 6.10 A part of table of EMBerY2 for CFSM $P_E(\Gamma_{11})$

$T_1T_2T_3T_4T_5$	$y_7y_8y_9$	m
00000	000	1
00001	000	–
00010	000	–
00011	000	–
00100	100	2
00101	010	3
00110	001	4
00111	000	–
01000	100	5
01001	010	6

To implement the circuit of P_E CFSM, it is necessary to construct both DST and table of EMBerY2. The DST gives system (6.10) and (6.16). The table of EMBerY2 gives the truth table for system (6.4). To implement (6.4) as a single EMB, the conditions (6.45) and (6.46) should take places. If the condition (6.46) is violated, then it is necessary to use n_E blocks EMBs where n_E is determined as (6.38).

If the number of required EMBs exceeds their real number, then a part of MOs $y_n \in Y_2$ is implemented as PALerY2. Obviously, the most complex functions are implemented using EMBs. The complexity of a function is determined by the number of MCs in a corresponding circuit. It depends on parameters $Q(y_n)$ and S , where $y_n \in Y_2$.

Let it be the configuration $\langle 5, 4 \rangle$ in the used EMB. It means that it is enough a single EMB to implement the circuit of EMBerY2 for $P_E(\Gamma_{11})$. A part of this table is represented by Table 6.10.

We use state codes from Fig. 6.18 in Table 6.10. If there is the sign “*” in the Karnaugh map, the corresponded row of table is filled by zeros.

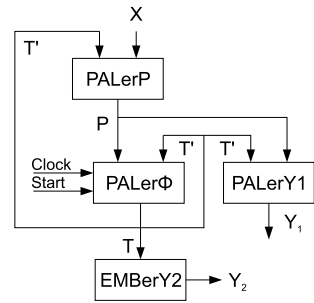
To get systems (6.10) and (6.16), let us construct the DST of CFSM $P_E(\Gamma_{11})$. It is constructed using the following system of GFT derived from GSA Γ_{11} :

$$\begin{aligned}
 B_1 &\rightarrow x_1x_2a_2 \vee x_1\bar{x}_2a_3 \vee \bar{x}_1a_4; \\
 B_2 &\rightarrow x_3x_4a_5 \vee x_3\bar{x}_4a_6 \vee \bar{x}_3x_5a_7 \vee \bar{x}_3\bar{x}_5a_4; \\
 B_3 &\rightarrow x_1x_6a_8 \vee x_1\bar{x}_6a_9 \vee \bar{x}_1a_{10}; \\
 B_4 &\rightarrow x_7x_8a_{11} \vee x_7\bar{x}_8a_{12} \vee \bar{x}_7a_{13}; \\
 B_5 &\rightarrow x_9a_{14} \vee \bar{x}_9a_4; \\
 B_6 &\rightarrow x_{10}x_9a_{14} \vee x_{10}\bar{x}_9a_4 \vee \bar{x}_{10}a_{15}; \\
 B_7 &\rightarrow a_{16}; \\
 B_8 &\rightarrow a_1.
 \end{aligned} \tag{6.59}$$

To construct the DST of $P_E(\Gamma_{11})$, we should use the system (6.59), state codes from Fig. 6.18, and CMOs from arks of GSA Γ_{11} . There are 20 terms in (6.59). So, there is $H = 20 = H_T$. A part of DST is represented by Table 6.11.

Table 6.11 A part of DST of CSFM $P_E(\Gamma_{11})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	000	a_2	00100	x_1x_2	y_1y_2	D_3	1
		a_3	00101	$x_1\bar{x}_2$	—	D_3D_5	2
		a_4	00110	$\bar{x}_1$	y_3	D_3D_4	3
B_2	001	a_5	01000	x_3x_4	y_1y_5	D_2	4
		a_6	01001	$x_3\bar{x}_4$	y_2y_6	D_2D_5	5
		a_7	01010	$\bar{x}_5$	y_5	D_2D_4	6
		a_4	00110	$\bar{x}_3\bar{x}_5$	y_4y_5	D_3D_4	7
B_3	010	a_8	01100	x_1x_6	y_3y_4	D_2D_3	8
		a_9	01101	$x_1\bar{x}_6$	y_1y_2	$D_2D_3D_5$	9
		a_{10}	01110	x_1	y_3y_4	$D_2D_3D_4$	10

Fig. 6.20 Structural diagram of MP_E CFSM

Using this table, we can get the first 10 terms of SBFs (6.10) and (6.16). There are the following terms: $F_1 = \bar{T}_1\bar{T}_2\bar{T}_3x_1x_2$, $F_2 = \bar{T}_1\bar{T}_2\bar{T}_3x_1\bar{x}_2$, ..., $F_{10} = \bar{T}_1\bar{T}_2\bar{T}_3\bar{x}_1$. Of course, the law of expansion [14] can be used to simplify the SOPs of functions.

For example, there is $D_3 = [F_1 \vee F_2 \vee F_3] \vee F_7 \cdots \vee F_{10} = \bar{T}_1\bar{T}_3 \vee \bar{T}_1\bar{T}_2\bar{T}_3\bar{x}_3\bar{x}_5$. The degree of SOP simplification depends largely on the configuration of a GSA Γ .

Let we can use only a single EMB with $S_A = 5$ and $t_F = 2$. Using (6.38) gives $n_E = 2$. So, two functions from Y_2 can be implemented by the EMB, and one by PAL MCs. To choose the candidates for implementing with PALs, let us analyze the values $Q(f_i)$ for MOs $y_n \in Y_2$. There is $Q(y_7) = 5$, $Q(y_8) = 4$ and $Q(y_9) = 3$. So, we should use EMB to implement functions y_7 and y_8 . The simplest function y_9 is implemented by PAL.

To diminish the numbers of literals in functions $D_r \in \Phi$ and $y_n \in Y_1$, let us use the RLC. It leads to MP_E CFSM (Fig. 6.20).

In MP_E CFSM, the PALerP implements the system (6.3). The PALerPhi implements the system (6.13), the PALerY1 the system (6.19). The functions of EMBerY2 are the same as for P_E CFSM.

To find (6.13) and (6.19), it is necessary to execute the RLC. Analysis of GSA Γ_{11} gives the following sets $X(B_i)$: $X(B_1) = \{x_1, x_2\}$, $X(B_2) = \{x_3, x_4, x_5\}$, $X(B_3) = \{x_1, x_6\}$, $X(B_4) = \{x_7, x_8\}$, $X(B_5) = \{x_9\}$, $X(B_6) = \{x_9, x_{10}\}$, $X(B_7) =$

Table 6.12 The RLC for CFSM $MP_E(\Gamma_{11})$

$P \setminus B_i$	B_1	B_2	B_3	B_4	B_5	B_6	B_7	B_8
p_1	x_1	x_3	x_1	x_7	x_9	x_9	—	—
p_2	x_2	x_4	x_6	x_8	—	x_{10}	—	—
p_3	—	x_5	—	—	—	—	—	—
$K(B_i)$	000	001	010	011	100	101	110	111

Table 6.13 A part of DST of CFSM $MP_E(\Gamma_{11})$

B_i	$K(B_i)$	a_s	$K(a_s)$	P_h	Y_h	Φ_h	h
B_1	000	a_2	00100	$p_1 p_2$	$y_1 y_2$	D_3	1
		a_3	00101	$p_1 \bar{p}_2$	—	$D_3 D_5$	2
		a_4	00110	$\bar{p}_1$	y_3	$D_3 D_4$	3
B_2	001	a_5	01000	$p_1 p_2$	$y_1 y_5$	D_2	4
		a_6	01001	$p_1 \bar{p}_2$	$y_2 y_6$	$D_2 D_5$	5
		a_7	01010	$\bar{p}_1 x_5$	y_5	$D_2 D_4$	6
		a_4	00110	$\bar{p}_1 \bar{x}_5$	$y_4 y_5$	$D_3 D_4$	7
B_3	010	a_8	01100	$p_1 p_z$	$y_3 y_4$	$D_2 D_3$	8
		a_9	01101	$p_1 \bar{p}_2$	$y_1 y_2$	$D_2 D_3 D_5$	9
		a_{10}	01110	$\bar{p}_1$	$y_3 y_4$	$D_2 D_3 D_4$	10

$X(B_8) = \emptyset$. The value of G is determined as

$$G = \max(|X(B_1)|, \dots, |X(B_I)|). \quad (6.60)$$

In the discussed case, we have $G=3$. So, there is the set $P = \{p_1, p_2, p_3\}$. Let us replace LCs $x_l \in X$ as it is represented by Table 6.12.

After minimizing, we can find the following equations:

$$\begin{aligned} p_1 &= \bar{T}_1 \bar{T}_3 x_1 \vee \bar{T}_1 \bar{T}_2 T_3 x_3 \vee T_2 T_3 x_7 \vee T_1 x_9; \\ p_2 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 x_2 \vee \bar{T}_1 \bar{T}_2 T_3 x_4 \vee T_2 \bar{T}_3 x_6 \vee T_2 T_3 x_8 \vee T_1 x_{10}; \\ p_3 &= x_5. \end{aligned} \quad (6.61)$$

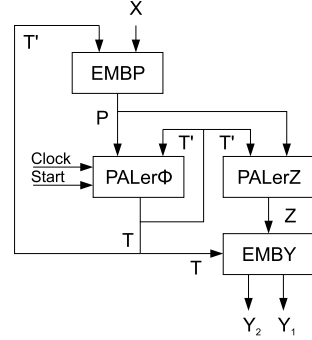
It is necessary to implement equations for p_1 and p_2 . There is $NA(p_1) = 7$, $Q(p_1) = 4$, $NA(p_2) = 8$, $Q(p_2) = 5$. So, it is enough to have PAL(8, 5) to implement the circuit of PALerP with 2 MCs (Fig. 6.20).

To get the functions $D_r \in \Phi$ and $y_n \in Y1$, it is necessary to replace $x_l \in X$ by $p_g \in P$ in the DST of P_E CFSM. In the discussed case, it gives Table 6.13.

Due to such state assignment, the system Y2(T) is simplified. It is the following:

Fig. 6.21 The outcome of RSA for CFSM $MP_E(\Gamma_{11})$

		$T_1 T_2 T_3$							
		000	001	011	010	110	111	101	100
$T_4 T_5$	00	a_1	*	*	*	*	*	*	a_{12}
	01	*	a_2	a_{10}	a_5	*	a_{16}	a_{14}	a_{11}
	11	*	a_3	a_8	a_6	a_{15}	*	*	*
	10	*	a_3	a_9	a_6	*	*	*	a_{12}

Fig. 6.22 Structural diagram of $MP_E Y1$ CFSM

$$\begin{aligned}
 y_7 &= A_2 \vee A_5 \vee A_{10} \vee A_{11} \vee A_{14} \vee A_{16} = \bar{T}_4 T_5; \\
 y_8 &= A_3 \vee A_6 \vee A_8 \vee A_{15} = T_4 T_5; \\
 y_9 &= A_4 \vee A_7 \vee A_9 \vee a_{13} = T_4 \bar{T}_5.
 \end{aligned} \tag{6.62}$$

Of course, changing state codes changes the functions $D_r \in \Phi$ and $y_n \in Y1$. But we just wanted to show how the proper state assignment can change SOPs of BFs.

If condition (6.48) takes place, then the $PALerP$ can be replaced by $EMBP$. If condition (6.47) takes place, then both systems $y_n \in Y1$ and $y_n \in Y2$ can be implemented using a single $EMBY$. If it is possible to use 2 $EMBs$ and conditions (6.47) and (6.48) are true, then a CFSM can be implemented as it is shown in Fig. 6.22.

In $MP_E Y1$ CFSM, the $EMBP$ executes RLC and generates functions (6.3). The $PALer\Phi$ implements functions (6.19). The $PALerZ$ generates variables $z_r \in Z$ encoding the CMOs $Y_q \subseteq Y1$ and represented as (6.24). The $EMBY$ implements systems (6.4) and (6.7). We hope that a reader can produce the method of synthesis of $MP_E Y1$ CFSM using previous methods.

Now let us discuss CFSMs based on the representation (5.37). In this case, the states $a_m \in A$ are determined by classes $B_i \in \Pi_A$ and CMOs $Y_q \subseteq Y2$. The set $T = T' \cup T_Y$, where $|T'| = R_A$ (2.43) and $|T_Y| = R_{Q2}$ (5.31). Of course, there is $T' \cap T_Y = \emptyset$. The MOs $y_n \in Y2$ are represented by the regular system (6.5).

Let the following condition take place:

$$2^{R_{Q2}} \cdot N2 \leq V_0. \tag{6.63}$$

Fig. 6.23 Structural diagram of P_K CFSM

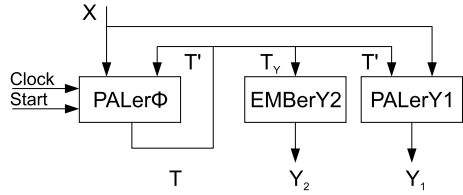
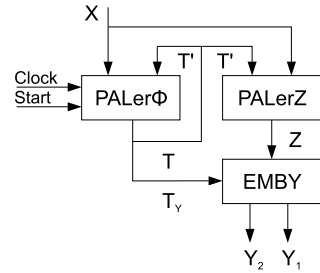


Fig. 6.24 Structural diagram of P_K Y1 CFSM with a shared EMB



In this case, the system (6.5) is implemented as EMBY2 (Fig. 6.23).

Let the following condition take place:

$$2^{R_Q + R_{Q2}}(N_1 + N_2) \leq V_0. \quad (6.64)$$

In this case, systems $y_n \in Y1$ and $y_n \in Y2$ can be implemented as a single EMBY. Of course, it is necessary to execute the ECMO $Y_q \subseteq Y1$. It leads to P_K Y1 CFSM (Fig. 6.24).

In this book, we propose a synthesis method for P_K Y1 CFSM. It includes the following steps:

1. Creating the set A.
2. Creating the partition $\Pi_A = \{B_1, \dots, B_I\}$ of the set A.
3. Creating the CMOs $Y_q \subseteq Y2$.
4. Encoding the classes of $B_i \in \Pi_A$ and CMOs $Y_q \subseteq Y2$.
5. Creating the codes of states $a_m \in A$.
6. Creating the CMOs $Y_q \subseteq Y1$.
7. Encoding the CMOs $Y_q \subseteq Y1$.
8. Creating the DST of P_K Y1 CFSM.
9. Deriving the systems $\Phi(T', X)$ and $Z(T', X)$ from DST.
10. Creating the table of EMBY.
11. Implementing CFSM circuit.

Let us discuss an example of synthesis of CFSM P_K Y1(Γ 11). The steps 1 and 2 are already executed. There is $M=16$ and $I=8$. So, there is $R_A = 3$, $T' = \{T_1, T_2, T_3\}$.

There are $Q(Y2)=4$ CMOs $Y_q \subseteq Y2$ in the discussed case. They are the following: $Y_1 = \emptyset$, $Y_2 = \{y_7\}$, $Y_3 = \{y_8\}$ and $Y_4 = \{y_9\}$. So, using (5.31) gives $R_{Q2} = 2$, $T_Y = \{T_4, T_5\}$. Let us encode CMOs in the trivial way: $K(Y_1) = 00, \dots, K(Y_4) = 11$.

		$T_1 T_2 T_3$								
		000	001	011	010	110	111	101	100	
$T_4 T_5$	00	a_1	*	*	*	*	*	*	a_{12}	y_1
	01	*	a_2	a_{10}	a_5	*	a_{16}	a_{14}	a_{11}	y_2
	11	*	a_4	a_9	a_7	*	*	*	a_{13}	y_4
	10	*	a_3	a_8	a_6	a_{15}	*	*	*	y_3
		B_1	B_2	B_4	B_3	B_7	B_8	B_6	B_5	

Fig. 6.25 State of codes of CFSM $P_K Y1(\Gamma_{11})$

Let us use the trivial codes of classes $B_i \in \Pi_A$: $K(B_1) = 000, \dots, K(B_8) = 111$. Using codes $K(B_i)$ and $K(Y_q)$ gives state codes $K(a_m)$ shown in Fig. 6.25.

There are the following CMOs $Y_q \subseteq Y1$ in GSA Γ_{11} : $Y_5 = \emptyset$, $Y_6 = \{y_1, y_2\}$, $Y_7 = \{y_3\}$, $Y_8 = \{y_1, y_5\}$, $Y_9 = \{y_2, y_6\}$, $Y_{10} = \{y_4, y_5\}$, $Y_{11} = \{y_5\}$, $Y_{12} = \{y_3, y_4\}$. So, there is $Q=8$, $R_Q = 3$, $Z = \{z_4, z_2, z_3\}$. Let us encode these CMOs in the trivial way $K(Y_5) = 000$, $K(Y_6) = 001, \dots, K(Y_{12}) = 111$.

Using the system (6.59), state codes (Fig. 6.25) and codes $K(Y_q)$ for $Y_q \subseteq Y1$, it is possible to construct the DST (Table 6.14).

The systems for PALer Φ and PALerZ are derived from Table 6.14. For example, there is the following equation: $D_1 = [F_{11} \vee F_{12} \vee F_{13}] \vee F_{14}, \vee F_{16} \vee F_{18} \vee F_{19} = \bar{T}_1 \bar{T}_2 \bar{T}_3 \vee T_1 \bar{T}_2 \bar{T}_3 x_9 \vee \dots \vee T_1 T_2 \bar{T}_3$.

The block EMBY is represented by the table having the columns Z, T_Y , Y1, Y2, h. This table has 32 rows in the discussed case. The first eight rows of the table of EMBY are represented by Table 6.15.

There is the code $K(Y_5)$ in the rows 1–4, the code $K(Y_6)$ in rows 5–8. The codes $K(Y_1)–K(Y_4)$ are repeated in every four rows of the table: 1–4, 5–8, 9–12 and so on. We hope that the contents of the rows are clear for our reader.

The last step of design is reduced to using some standard program tools [1, 20]. They executed all steps necessary to implement CFSM logic circuit.

To simplify circuits of PALer Φ and PALerZ, the RLC can be used. It leads to MP $_K Y1$ CFSM (Fig. 6.26). It includes the PALerP implementing the system $P(T', X)$. Now, the PALer Φ implements the system $\Phi(T', P)$ and the PALerZ the system $Z(T', P)$. We do not discuss this model in details.

6.5 Synthesis of CFSMs with Transformation of Object Codes

This approach [3, 16, 17] allows simplifying SBFs representing FSM circuits. It is reduced to presentation of some functions (the secondary objects) by other functions (the primary objects) and identifiers. For example, the states can be represented by MOs (or CMOs) and identifiers. In this case, the states are the secondary objects.

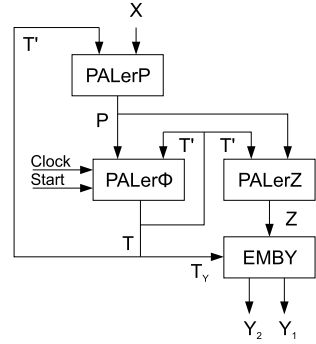
Table 6.14 DST of CSFM $P_K Y1(\Gamma_{11})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Z_h	Φ_h	h
B_1	000	a_2	00101	$x_1 x_2$	z_3	$D_3 D_5$	1
		a_3	00110	$x_1 \bar{x}_2$	—	$D_3 D_4$	2
		a_4	00111	$\bar{x}_1$	z_2	$D_3 D_4 D_5$	3
B_2	001	a_5	01001	$x_3 x_4$	$z_2 z_3$	$D_2 D_5$	4
		a_6	01010	$x_3 \bar{x}_4$	z_1	$D_2 D_5$	5
		a_7	01011	$\bar{x}_3 x_5$	$z_1 z_2$	$D_2 D_4 D_5$	6
		a_9	00111	$\bar{x}_1 \bar{x}_5$	$z_1 z_3$	$D_3 D_4 D_5$	7
B_3	010	a_8	01110	$x_1 x_6$	$z_1 z_2 z_3$	$D_2 D_3 D_4$	8
		a_9	01111	$x_1 \bar{x}_6$	z_3	$D_2 D_3 D_4 D_5$	9
		a_{10}	01101	$\bar{x}_1$	$z_1 z_2 z_3$	$D_2 D_3 D_5$	10
B_4	011	a_{11}	10001	$x_7 x_8$	$z_1 z_3$	$D_1 D_5$	11
		a_{12}	10000	$x_7 \bar{x}_8$	z_3	D_1	12
		a_{13}	10011	$\bar{x}_7$	z_2	$D_2 D_4 D_5$	13
B_5	100	a_{14}	10101	x_9	—	$D_1 D_3 D_5$	14
		a_4	00111	$\bar{x}_9$	$z_1 z_2 z_3$	$D_3 D_4 D_5$	15
B_6	101	a_{14}	10101	$x_{10} x_9$	—	$D_1 D_3 D_5$	16
		a_4	00111	$x_{10} \bar{x}_9$	$z_1 z_2 z_3$	$D_3 D_4 D_5$	17
		a_{15}	11010	$\bar{x}_{10}$	z_3	$D_1 D_2 D_4$	18
B_7	110	a_{16}	11101	1	z_1	$D_1 D_2 D_3 D_5$	19
B_8	111	a_1	00000	1	z_2	—	20

Table 6.15 A part of table of EMBY of $P_K Y1(\Gamma_{11})$

Z			T_Y		$Y1$						$Y2$			h
z_1	z_2	z_3	T_4	T_5	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	0	0	0	0	1	0	0	2
0	0	0	1	0	0	0	0	0	0	0	0	1	0	3
0	0	0	1	1	0	0	0	0	0	0	0	0	1	4
0	0	1	0	0	1	1	0	0	0	0	0	0	0	5
0	0	1	0	1	1	1	0	0	0	0	1	0	0	6
0	0	1	1	0	1	1	0	0	0	0	0	1	0	7
0	0	1	1	1	1	1	0	0	0	0	0	0	1	8

Fig. 6.26 Structural diagram of $MP_K Y1$ CFSM



Our analysis shows that the following methods of the transformation of object codes (TOC) can be used in the case of CFSMs:

1. Transformation of $y_n \in Y2$ into states $a_m \in A$.
2. Transformation of $y_n \in Y2$ into classes $B_i \in \Pi_A$.
3. Transformation of CMOs $Y_q \subseteq Y2$ into states $a_m \in A$.
4. Transformation of CMOs $Y_q \subseteq Y2$ into classes $B_i \in \Pi_A$.
5. Transformation of $y_n \in Y1$ into states $a_m \in A$.
6. Transformation of $y_n \in Y1$ into classes $B_i \in \Pi_A$.
7. Transformation of CMOs $Y_q \subseteq Y1$ into states $a_m \in A$.
8. Transformation of CMOs $Y_q \subseteq Y1$ into classes $B_i \in \Pi_A$.
9. Transformation of states $a_m \in A$ into MOs $y_n \in Y1$.
10. Transformation of states $a_m \in A$ into CMOs $Y_q \subseteq Y1$.
11. Transformation of classes $B_i \in \Pi_A$ into MOs $y_n \in Y1$.
12. Transformation of classes $B_i \in \Pi_A$ into CMOs $Y_q \subseteq Y1$.

Let us discuss the method 4, when classes $B_i \in \Pi_A$ are determined by CMOs $Y_q \subseteq Y2$ and identifiers $I_k \in SI$. Here $SI = \{I_1, \dots, I_K\}$ is a set of identifiers. So, each class $B_i \in \Pi_A$ is determined as some pair $\langle Y_q, I_k \rangle$. The CMOs $Y_q \subseteq Y2$ are written in operator vertices corresponding to states $a_m \in B_i$. Let us discuss how to find the set SI.

Let $B(Y_q) \subseteq \Pi_A$ by a set of classes $B_i \in \Pi_A$ such that CMO $Y_q \subseteq Y2$ is generated in some state $a_m \in B_i$. Let $K_q = |B(Y_q)|$. Let us find the value of K:

$$K = \max(K_1, \dots, K_{Q2}). \quad (6.65)$$

In (6.65), Q2 stands for the number of CMOs $Y_q \subseteq Y2$.

Let us analyze the GSA Γ_{11} . In this case, there is $Q2=4$: $Y_1 = \emptyset$, $Y_2 = \{y_7\}$, $Y_3 = \{y_8\}$, $Y_4 = \{y_9\}$. Let us find the sets $B(Y_q)$: $B(Y_1) = \{B_1, B_2\}$, $B(Y_2) = \{B_2, B_3, B_4, B_5, B_6, B_8\}$, $B(Y_3) = \{B_2, B_3, B_4, B_7\}$, $B(Y_4) = \{B_2, B_3, B_4, B_5\}$. So, there is $K_1 = 2$, $K_2 = 6$, $K_3 = K_4 = 4$. Using (6.65) gives $K=6$, so there is $SI = \{I_1, \dots, I_6\}$.

Let us create a table showing correspondence among CMOs, identifiers, classes of PES and states of CFSM. It is Table 6.16 in the discussed case.

Table 6.16 Correspondence among objects of CSFM $P(\Gamma_{11})$

B_1	a_1	Y_q	I_k	1	B_4	a_9	Y_q	I_k	9
B_2	a_2	Y_2	I_1	2	B_4	a_{10}	Y_2	I_3	10
B_2	a_3	Y_3	I_1	3	B_5	a_{11}	Y_2	I_4	11
B_2	a_4	Y_4	I_1	4	B_5	a_{12}	Y_1	I_2	12
B_3	a_5	Y_2	I_2	5	B_5	a_{13}	Y_4	I_4	13
B_3	a_6	Y_3	I_2	6	B_6	a_{14}	Y_2	I_5	14
B_3	a_7	Y_4	I_2	7	B_7	a_{15}	Y_3	I_4	15
B_4	a_8	Y_3	I_3	8	B_8	a_{16}	Y_2	I_6	16

There is $M=16$ rows in Table 6.16. Each row corresponds to a single state $a_m \in A$. A state is determined by some pair $\langle Y_q, I_k \rangle$. A class $B_i \in \Pi_A$ is determined by $|B_i|$ pairs $\langle Y_q, I_k \rangle$ corresponding to states $a_m \in B_i$.

Let us encode the classes in the trivial way: $K(B_1) = 000, \dots, K(B_8) = 111$. There is $R_{Q2} = 2$; it gives the set $V = \{v_1, v_2\}$. Let us encode CMOs in the trivial way: $K(Y_1) = 00, \dots, K(Y_4) = 11$. To encode the identifiers, it is necessary R_K variables $w_r \in W$:

$$R_K = \lceil \log_2 K \rceil. \quad (6.66)$$

There is $K = 6$ in the discussed case. Using (6.66) gives $R_K = 3$ and $W = \{w_1, w_2, w_3\}$. Let us encode the identifiers using the frequency approach. Let us organize the queue $\gamma = \langle I_1, I_2, \dots, I_6 \rangle$. It gives the codes $K(I_1) = 000, K(I_2) = 001, K(I_3) = 010, K(I_4) = 100, K(I_5) = 011, K(I_6) = 101$.

Using Table 6.16, it is possible to represent classes B_i as

$$B_i = B_i(V, W) \quad (i = \overline{1, I}). \quad (6.67)$$

Obviously, it gives the system

$$\tau = \tau(V, W). \quad (6.68)$$

To get variables $v_r \in V$ and $w_r \in W$, it is necessary to implement the SBFs

$$V = V(\tau, X); \quad (6.69)$$

$$W = W(\tau, X). \quad (6.70)$$

Obviously, the system Y1 should be implemented as (6.17).

Let us denote CFSM based on this transformation as P_{Y2} CFSM. Its structural diagram is shown in Fig. 6.27.

In P_{Y2} CFSM, the PALerVW implements systems (6.69)–(6.70). They are used by EMBerY2 to produce systems (6.69)–(6.68). The pulses *Clock* and *Start* enter

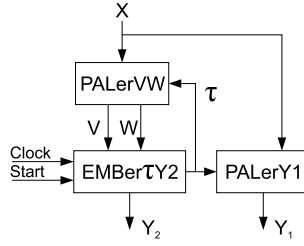


Fig. 6.27 Structural diagram of P_{Y2} CFSM

this block. Of course, they could enter the PALerVW. The PALerY1 implements the system (6.17).

To get the systems (6.69)–(6.70), it is necessary to construct the table of PALerVW. It includes the columns B_i , $K(B_i)$, a_s , $K(a_s)$, X_h , V_h , W_h , h . To get functions $v_r \in V$ and $w_r \in W$, the state codes should be represented as

$$K(a_m) = K(Y_q) * K(I_k). \quad (6.71)$$

The presentation (6.71) is obtained from a table similar to Table 6.16.

The table of PALerVW is constructed using the system of GFT. In the discussed case, the system (6.59) lead to Table 6.17. It has $H_T = 20$ rows. This table is a base to derive the functions $V(\tau, X)$ and $W(\tau, X)$.

For example, the following equation can be derived from Table 6.17:

$$w_1 = F_{14} \vee F_{19} = \tau_1 \bar{\tau}_2 \bar{\tau}_3 x_9 \vee \tau_1 \tau_2 \bar{\tau}_3 s. \quad (6.72)$$

The Eq. (6.72) determines the PAL macrocell in the circuit of PALerVW.

All other equations can be derived in the same manner. After minimizing, they are used for implementing corresponding subcircuits in the block PALerVW.

The EMBerTY2 implements a truth table of systems $\tau(V, W)$ and $Y2(V)$. It is implemented as a single EMB, if the following condition takes place:

$$2^{R_{Q2}+R_K} \cdot (R_A + N2) \leq V_0. \quad (6.73)$$

In the case of $P_{Y2}(\Gamma_{11})$, it is necessary to have an EMB with $S_A \geq 5$, $t_F \geq 6$. The EMBerTY2 is represented by the table having the columns V, W, τ , Y2, h . It is Table 6.18 with first rows of table corresponding to EMBerTY2.

We add the column m in Table 6.18 to show the correspondence with Table 6.16. Let us explain how to fill in Table 6.18. Consider row $h = 9$. There are $K(Y_2)$ and $K(I_1)$ in columns V and W for $h = 9$. As follows from Table 6.16, $K(Y_2) * K(I_1) = K(a_2)$. Because $a_2 \in B_2$, there is $K(B_2)$ in column τ for $h = 9$. Obviously, there is $y_7 = 1$ in this row. Consider row $h = 15$. There is no state corresponding to the pair

Table 6.17 Table of PALerVW for CSFM $P_K Y2(I_{11})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	V_h	W_h	h
B_1	000	a_2	00100	$x_1 x_2$	v_2	–	1
		a_3	10000	$x_1 \bar{x}_2$	v_1	–	2
		a_4	11000	$\bar{x}_1$	$v_1 v_2$	–	3
B_2	001	a_5	01001	$x_3 x_4$	v_2	w_3	4
		a_6	10001	$x_3 \bar{x}_4$	v_1	w_3	5
		a_7	11001	$\bar{x}_3 x_5$	$v_1 v_2$	w_3	6
		a_4	11000	$\bar{x}_1 \bar{x}_5$	$v_1 v_3$	–	7
B_3	010	a_8	10010	$x_1 x_6$	v_1	w_2	8
		a_9	11010	$x_1 \bar{x}_6$	$v_1 v_2$	w_2	9
		a_{10}	01010	$\bar{x}_1$	v_2	w_2	10
B_4	011	a_{11}	01011	$x_7 x_8$	v_2	$w_2 w_3$	11
		a_{12}	00001	$x_7 \bar{x}_8$	–	w_3	12
		a_{13}	11011	$\bar{x}_7$	$v_1 v_2$	$w_2 w_3$	13
B_5	100	a_{14}	01100	x_9	v_1	w_1	14
		a_4	11000	$\bar{x}_9$	$v_1 v_2$	–	15
B_6	101	a_{14}	01011	$x_{10} x_9$	v_2	$w - 2w_3$	16
		a_4	11000	$x_{10} \bar{x}_9$	$v_1 v_2$	–	17
		a_{15}	10011	x_{10}^-	v_1	$w_2 w_3$	18
B_7	110	a_{16}	01101	1	v_1	$w_1 w_3$	19
B_8	111	a_1	00000	1	–	–	20

Table 6.18 A part of table of EMBerrY2 of $P_{Y2}(I_{11})$

V		W			τ			Y2			h	m
v_1	v_2	w_1	w_2	w_3	τ_1	τ_2	τ_3	y_7	y_8	y_9		
0	1	0	0	0	0	0	1	1	0	0	9	2
0	1	0	0	1	0	1	0	1	0	0	10	5
0	1	0	1	0	0	1	1	1	0	0	11	10
0	1	0	1	1	1	0	0	1	0	0	12	11
0	1	1	0	0	1	0	1	1	0	0	13	14
0	1	1	0	1	1	1	0	1	0	0	14	16
0	1	1	1	0	0	0	0	0	0	0	15	–
0	1	1	1	1	0	0	0	0	0	0	16	–
1	0	0	0	0	0	0	1	0	1	0	17	3
1	0	0	0	1	0	1	0	0	1	0	18	6

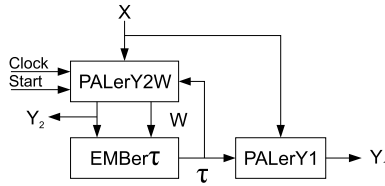


Fig. 6.28 Structural diagram of P_{OH2} CFSM

$\langle Y_2, 100 \rangle$. So, there are zeros in columns τ and Y_2 for $h = 15$. Other rows are filled in the same manner.

The table of PALerY1 is constructed as in the previous case. So, we skip this step.

Let the following condition take place:

$$2^{N^2+R_K} \cdot R_A \leq V_0. \quad (6.74)$$

In this case, it is possible to use one-hot codes of CMOs $Y_q \subseteq Y_2$ to produce functions $\tau_r \in \tau$. It leads to P_{OH2} CFSM (Fig. 6.28). The subscript OH2 means that we use one-hot encoding of CMOs $Y_q \subseteq Y_2$.

In P_{OH2} CFSM, the PALerY2W generate functions $y_n \in Y_2$ and $w_r \in W$. The outputs of MCs are connected with flip-flops. Due to, it the values of MOs $y_n \in Y_2$ are valid during the whole cycle. The EMBer τ implements functions

$$\tau = \tau(Y_2, W). \quad (6.75)$$

Let one-hot codes of CMOs correspond to the vectors $\langle y_7, y_8, y_9 \rangle$. It gives the state codes shown in Table 6.19. The PALerY2W implements systems

$$Y_2 = Y_2(\tau, X) \quad (6.76)$$

and (6.70). To construct the systems (6.70) and (6.76), it is necessary to find the table of PALerY2W. It is similar to Table 6.17. But the column V_h is replaced by the column Y_h .

There are the first 10 rows of table of PALerY2W represented by Table 6.20. We use the state codes (Table 6.19) in the column a_s of Table 6.20. There are no problem with deriving functions (6.70) and (6.20).

To implement the system (6.75), it is necessary to construct the table of EMBer τ . It is similar to Table 6.18, but the column V is replaced by the column Y2. Obviously, there are no MOs $y_n \in Y_2$ in the cells of this EMBer. We hope it is a very simple thing for our reader.

At last let us discuss the P_B CFSM where codes $K(B_i)$ and $K(I_k)$ are transformed into MOs $y_n \in Y_1$. This CFSM has the structural diagram shown in Fig. 6.29.

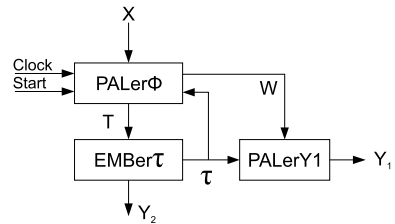
In P_B CFSM, the PALer Φ implements systems (6.11) and (6.70). The EMBer τ implements systems (6.4) and (6.8). The PALerY1 implements the system

Table 6.19 State codes of CFSM $P_{OH2}(\Gamma_{11})$

a_m	$K(Y_q)$	W	m	a_m	$K(Y_q)$	W	m
a_1	000	000	1	a_9	001	010	9
a_2	100	000	2	a_{10}	100	010	10
a_3	010	000	3	a_{11}	100	011	11
a_4	001	000	4	a_{12}	000	001	12
a_5	100	001	5	a_{13}	001	011	13
a_6	010	001	6	a_{14}	100	100	14
a_7	001	001	7	a_{15}	010	011	15
a_8	010	010	8	a_{16}	100	101	16

Table 6.20 A part of table of PALerY2W for CSFM $P_{OH2}(\Gamma_{11})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	W_h	h
B_1	000	a_2	100000	x_1x_2	y_7	—	1
		a_3	010000	$x_1\bar{x}_2$	y_8	—	2
		a_4	001000	$\bar{x}_1$	y_9	—	3
B_2	001	a_5	100001	x_3x_4	y_7	w_3	4
		a_6	010001	$x_3\bar{x}_4$	y_8	w_3	5
		a_7	001001	$\bar{x}_3x_5$	y_9	w_3	6
		a_4	001000	$\bar{x}_3\bar{x}_5$	y_9	—	7
B_3	010	a_8	010001	x_1x_6	y_8	w_3	8
		a_9	001010	$x_1\bar{x}_6$	y_9	w_2	9
		a_{10}	100010	$\bar{x}_1$	y_7	w_2	10

Fig. 6.29 Structural diagram of P_B CFSM

$$Y1 = Y1(\tau, W). \quad (6.77)$$

Comparison of (6.17) and (6.77) shows that variables $w_r \in W$ replace the LCs $x_l \in X$.

To design the PALerY1, each CMO $Y_q \subseteq Y1$ should be represented by pairs $\langle B_i, I_k \rangle$. To do it, let us create sets $Y(B_i)$ including CMOs $Y_q \subseteq Y1$ such that a transition to $a_m \in B_i$ is accompanied by producing the CMO $Y_q \subseteq Y1$.

Let us find the sets $Y(B_i)$ for Γ_{11} . There are $Q=8$ CMOs $Y_q \subseteq Y1$ discussed before: $Y_5 = \emptyset$, $Y_6 = \{y_1, y_2\}$, $Y_7 = \{y_3\}$, $Y_8 = \{y_1, y_5\}$, $Y_9 = \{y_2, y_6\}$, $Y_{10} = \{y_4, y_5\}$,

Table 6.21 Correspondence between objects of CFSM $P_B(\Gamma_{11})$

B_i	I_k	Y_q	B_i	I_k	Y_q	B_i	I_k	Y_q	B_i	I_k	Y_q
B_1	I_1	Y_7	B_3	I_1	Y_8	B_4	I_1	Y_6	B_5	I_3	Y_{10}
B_2	I_1	Y_5	B_3	I_2	Y_9	B_4	I_2	Y_{12}	B_6	I_1	Y_5
B_2	I_2	Y_6	B_3	I_3	Y_{10}	B_5	I_2	Y_6	B_8	I_1	Y_6
B_2	I_3	Y_7	B_3	I_4	Y_{11}	B_5	I_2	Y_7	B_8	I_1	Y_9

Table 6.22 A part of table of PALer Φ for CFSM $P_B(\Gamma_{11})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Φ_h	W_h	h
B_1	000	a_2	0001	x_1x_2	D_4	w_2	1
		a_3	0010	$x_1\bar{x}_2$	D_3	–	2
		a_4	0011	$\bar{x}_1$	D_3D_4	w_1	3
B_2	001	a_5	0100	x_3x_4	D_2	w_1	4
		a_6	0101	$x_3\bar{x}_4$	D_2D_4	–	5
		a_7	0110	$\bar{x}_3x_5$	D_2D_3	w_1	6
		a_4	0011	$\bar{x}_3\bar{x}_5$	D_3D_4	w_1	7

$Y_{11} = \{y_5\}$, $Y_{12} = \{y_3, y_4\}$. Analysis of GSA Γ_{11} allows constructing the following sets: $Y(B_1) = \{y_7\}$, $Y(B_2) = \{y_5, y_6, y_7\}$, $Y(B_3) = \{y_8, y_9, y_{10}, y_{11}\}$, $Y(B_4) = \{y_6, y_{12}\}$, $Y(B_5) = \{y_6, y_7, y_{10}\}$, $Y(B_6) = \{y_1\}$, $Y(B_7) = \{y_6\}$, $Y(B_8) = \{y_9\}$. So, it is necessary $K = 4$ identifiers to select a particular CMO $Y_q \in Y(B_i)$.

Let us construct a table showing the correspondence between B_i , I_k and Y_q . It is Table 6.21 in the discussed case.

Let us encode the states $a_m \in A$ in the trivial way: $K(a_1) = 0000, \dots, K(a_{16}) = 1111$. Let us encode the classes $B_i \in \Pi_A$ in the trivial way: $K(B_1) = 000, \dots, K(B_8) = 111$. There is $K=4$. Using (6.66) gives $R_K = 2$, $W = \{w_1, w_2\}$. Let us encode the identifiers in the trivial way: $K(I_1) = 00, \dots, K(I_4) = 11$.

Now, we can create the table of PALer Φ . It includes the columns B_i , $K(B_i)$, X_h , Φ_h , h . In the discussed case, this table is constructed using the system of GFT (6.59). A part of this table is represented by Table 6.22.

For example, consider the row $h = 3$. There is $a_s = a_4$ and $a_4 \in B_2$. There is $Y_q = Y_7$. Using Table 6.21 gives that $Y_7 \rightarrow \langle B_2, I_3 \rangle$. Because $K(I_3) = 10$, there is w_1 in the column W_h for $h = 3$. All other rows are filled in the same way.

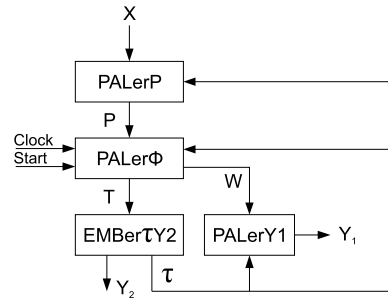
There are the following columns in the table of PALerY1: B_i , $K(B_i)$, I_k , $K(I_k)$, Y_q , h . In the discussed case, it is Table 6.23 constructed using Table 6.21 and codes of classes and identifiers.

The table of PALerY1 is a base to derive the system (6.77). For example, the following equation can be derived from Table 6.23:

$$y_6 = F_6 \vee F_{16} = \bar{\tau}_1 \tau_2 \bar{\tau}_3 \bar{w}_1 w_2 \vee \tau_1 \tau_2 \tau_3. \quad (6.78)$$

Table 6.23 Table of PALerY1 of CFSM $P_R(I_{11})$

B_i	$K(B_i)$	I_k	$K(i_k)$	Y_q	h
B_1	000	I_1	00	y_3	1
B_2	001	I_1	00	—	2
B_2	001	I_2	01	$y_1 y_2$	3
B_2	001	I_3	10	y_3	4
B_3	010	I_1	00	$y_1 y_5$	5
B_3	010	I_2	01	$y_2 y_6$	6
B_3	010	I_3	10	$y_4 y_5$	7
B_3	010	I_4	11	y_5	8
B_4	011	I_1	00	$y_1 y_2$	9
B_4	011	I_2	01	$y_3 y_4$	10
B_5	100	I_1	00	$y_1 y_5$	11
B_5	100	I_2	01	y_3	12
B_5	100	I_3	10	$y_4 y_5$	13
B_6	101	I_1	00	—	14
B_7	110	I_1	00	$y_2 y_6$	15
B_8	111	I_1	00	$y_2 y_6$	16

Fig. 6.30 Structural diagram of MP_B CFSM

The table of EMBer τ Y2 is constructed in the trivial way. It includes the columns: $K(a_m)$, τ , Y2, m. If $a_m \in B_i$, then there is $K(B_i)$ in the column τ and $Y(a_m)$ in the column Y2. We do not discuss this step, in view of its elementary.

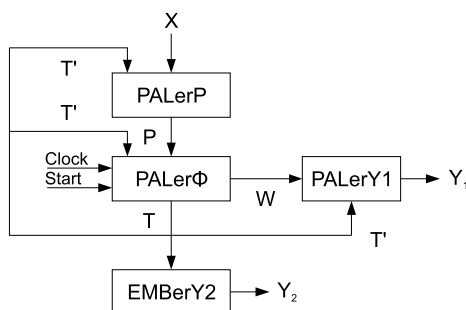
To diminish the number of MCs in PALer Φ , it is possible to execute the RLC. It leads to MP_B CFSM shown in Fig. 6.30.

Also, it is possible to execute the OSA and RLC. It simplifies the PALer Φ and EMBer. The EMBer should implement only the system Y2(T). It leads to MP_{OB} CFSM (Fig. 6.31).

To design the circuit of MP_{OB} CFSM, it is necessary to find the systems $P(T', X)$, $\Phi(T', P)$, $W(T', P)$, $Y2(T)$, $Y1(T', W)$. We do not discuss these models in details.

Let us point out that many possible structures of CFSMs remain outside the scope of this paragraph. we hope that our reader has seen all the possibilities related to

Fig. 6.31 Structural diagram of MP_{OB} CFSM



heterogenous CPLDs, as well as the transformation of objects for optimizing the CFSM logic circuits.

References

1. Altera (2020) <http://www.altera.com>. Accessed: January, 2020
2. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
3. Barkalov A, Barkalov A Jr (2005) Design of mealy finite-state machines with the transformation of object codes. *Int J Appl Math Comput Sci* 15(1):151–158
4. Barkalov A, Titarenko L (2009) Logic synthesis for FSM-based control units, vol 53. Lecture Notes in Electrical Engineering. Springer-Verlag, Berlin
5. Barkalov A, Titarenko L (2009) Synthesis of operational and control automata. UNITECH, Donetsk
6. Barkalov A, Titarenko L, Chmielewski S (2007) Reduction in the number of PAL macrocells into the circuit of a Moore FSM. *Int J Appl Math Comput Sci* 17(4):565–675
7. Czerwiński R, Kania D (2004) State assignment method for high speed FSM. In: *Proceedings of Programmable Devices and Systems*, pp 216–221
8. Czerwinski R, Kania D (2012) Area and speed oriented synthesis of FSMs for PAL-based CPLDs. *Microprocess Microsyst* 36(1):45–61
9. Czerwinski R, Kania D (2013) Finite state machine logic synthesis for complex programmable logic devices, vol 231. *Lecture Notes in Electrical Engineering*. Springer-Verlag, Berlin
10. Jenkins J (1995) Design with FPGAs and CPLDs. Prentice Hall, New York
11. Klimovich AS, Solov'ev VV (2011) A method for minimizing moore finite-state machines by merging two states. *J Comput Syst Sci Int* 50(6):907–920
12. Maxfield C (2004) The design warrior's guide to FPGAs. Academic Press Inc, Orlando, FL, USA
13. Maxfield C (2008) FPGAs: instant access. Newnes
14. McCluskey E (1986) Logic design principles. Prentice Hall, Englewood Cliffs
15. De Micheli G (1994) Synthesis and optimization of digital circuits. McGraw-Hill
16. Mielcarek K, Barkalov A, Titarenko L (2016) Designing LUT-based mealy FSM with transformation of collections of output functions. 5th International Conference on Modern Circuits and Systems Technologies—MOCAS 2016. IEEE, Thessaloniki, Grecia, pp 1–4
17. Mielcarek K, Barkalov O, Titarenko L (2017) Designing HFPGA-based mealy FSMs with transformation of output functions. In: 24th International Conference Mixed Design of Integrated Circuits and Systems—MIXDES
18. Skliarova I, Sklyarov V, Sudnitson A (2012) Design of FPGA-based circuits using hierarchical finite state machines. TUT Press, Tallinn

19. Solovjov V (2001) Design of digital systems basing on programmable logic integrated circuits. Hotline-Telecom, Moscow, Moscow (in Russian)
20. Xilinx (2020) <http://www.xilinx.com>. Accessed: January

Chapter 7

CPLD-Based Synthesis with Transformation of State Codes



Abstract The chapter is devoted to the hardware reduction based on transformation of state codes into the codes of classes of PES. The chapter starts from the complete state transformation. The method is applied for CFSMs implemented with either homogenous or heterogeneous CPLDs. These methods are applied if outcome of the optimal state assignment is far from optimal. It is shown how to combine this approach with other methods of structural decomposition. Next, the CFSMs with the partial state transformation are analysed. It is proposed to use two codes for classes of PES. It can be done due to a wide fan-in of PAL macrocells. Next, this approach is combined with other methods of structural decomposition. The same is done for CFSMs with PLA-based macrocells. Next, it is shown how to optimize hardware sharing EMBs and macrocells to implement the same systems of Boolean functions. At last, different methods of object transformation are discussed for heterogeneous CPLDs. All proposed methods target reducing the numbers of product terms in systems of Boolean functions representing CFSM circuits. The examples of synthesis are given for majority of proposed CFSM models.

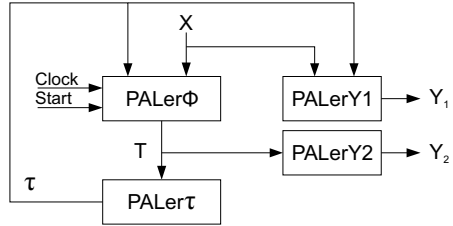
7.1 Synthesis with Complete State Transformation

The complete state transformation (CST) implies a transformation of state codes for all states $a_m \in A[2]$. If states $a_m \in B_i$, where B_i is a class of PES, then the codes $K(a_m)$ are transformed into a class code $K(B_i)$. The variables $\tau_r \in T$ are used for class encoding, where $|\tau| = R_A$. The value of R_A is determined as (2.43). The complete state transformation is reduced to replacing the state variables $T_r \in T$ by class variables $\tau_r \in T$. To execute such a replacement, it is necessary to use some resources of CPLD.

Let us start from the homogenous CFSMs based on CST. Let us denote it as P_C^1 CFSM. Its structural diagram is shown in Fig. 7.1.

In P_C^1 CFSM, the PALer Φ generates functions (6.11), the PALer $\mathcal{T}$ executes the CST and generates the functions (6.8). The PALerY1 implements the system (6.17), PALerY2 the system (6.4).

Fig. 7.1 Structural diagram of P_C^1 CFSM



In this chapter, we propose the method of synthesis of P_C^1 CFSM. It includes the following steps:

1. Marking the initial GSA Γ and finding the set of states A .
2. Creating the partition Π_A of the set A by classes of PES.
3. Executing the OSA and encoding of classes $B_i \in \Pi_A$.
4. Creating the DST of P_C^1 CFSM.
5. Creating the table of PALer $\mathcal{T}$.
6. Creating the table of PALer $Y2$.
7. Deriving systems (6.4), (6.11) and (6.17) from corresponding tables.
8. Implementing CFSM circuit with particular PAL macrocells.

Let us discuss an example of synthesis for CFSM $P_C^1(\Gamma_{12})$. The marked GSA Γ_{12} is shown in Fig. 7.2. The following sets can be found from analysis of Γ_{12} : $A = \{a_1, \dots, a_{16}\}$, $X = \{x_1, \dots, x_7\}$, $Y1 = \{y_1, \dots, y_6\}$, $Y2 = \{y_7, \dots, y_{10}\}$. It gives $M = 16$, $L = 7$, $N1 = 6$, $N2 = 4$, $N = 10$. Using (1.2) gives $R = 4$, $\Phi = \{D_1, \dots, D_4\}$ and $T = \{T_1, \dots, T_4\}$.

Using the Definition 3.1, we get the partition $\Pi_A = \{B_1, \dots, B_7\}$ with $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3\}$, $B_3 = \{a_4, a_5\}$, $B_4 = \{a_6, a_7, a_8\}$, $B_5 = \{a_9, a_{10}\}$, $B_6 = \{a_{11}, a_{12}, a_{13}\}$, $B_7 = \{a_{14}, a_{15}, a_{16}\}$. So, there is $I = 7$. Using (2.43) gives $R_A = 3$, $\mathcal{T} = \{\tau_1, \tau_2, \tau_3\}$.

Let us execute OSA for states of $P(\Gamma_{12})$. The outcome of OSA is shown in Fig. 7.3.

In the case of $P(\Gamma_{12})$, the following condition takes place:

$$M = 2^R. \quad (7.1)$$

It is clear that in this case the insignificant state assignments are absent. So, if a class $B_i \in \Pi_A$ has an odd number of states, at least two generalized intervals of R -dimensional Boolean space are required to represent it.

For $P(\Gamma_{12})$, the condition (7.1) takes place. There are odd numbers of states in classes B_1 , B_4 , B_6 and B_7 . Because $|B_1| = 1$, it is enough a single code 0000 to represent the class $B_i \in \Pi_A$. But it is necessary two intervals to represent classes $B_4, B_6, B_7 \in \Pi_A$. So, it makes sense to use the model $P_C^1(\Gamma_{12})$.

Let us encode the classes $B_i \in \Pi_A$. The following principle allows minimizing the system (6.8): the more states a class contains, the more zeros its code has. The outcome of class encoding is shown in Fig. 7.4.

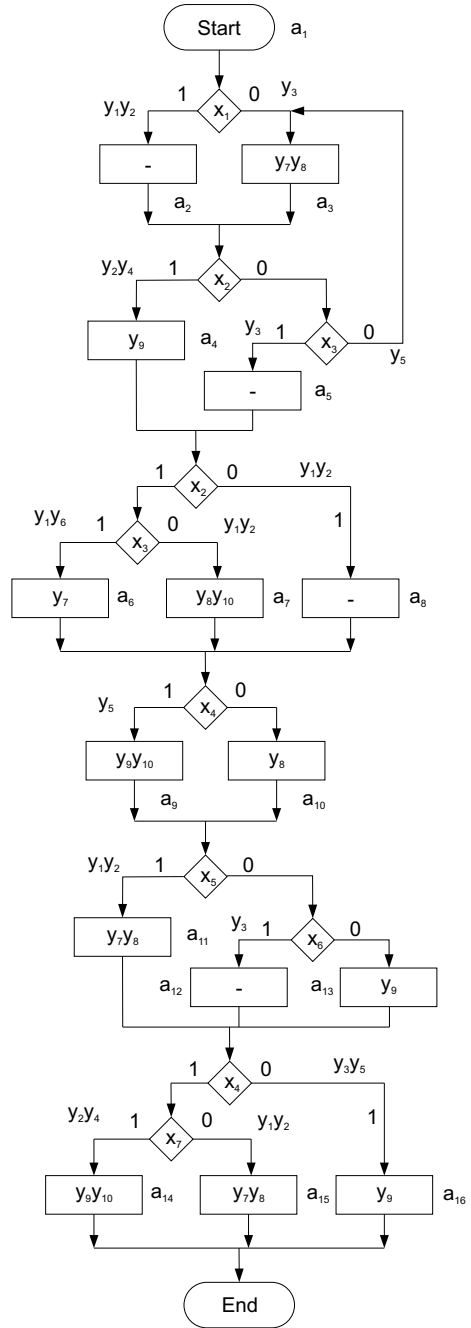
Fig. 7.2 Marked GSA Γ_{12} 

Fig. 7.3 Outcome of OSA
for $P(\Gamma_{12})$

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	a_1	a_6	a_{14}	a_9
	01	a_2	a_7	a_{15}	a_{10}
	11	a_3	a_8	a_{16}	a_{11}
	10	a_4	a_5	a_{13}	a_{12}

Fig. 7.4 Codes of classes
for $P_C^1(\Gamma_{12})$

		$\tau_1 \tau_2$			
		00	01	11	10
τ_3	0	B_4	B_6	B_3	B_7
	1	B_1	B_2	*	B_5

To create the DST of P_C^1 CFSM, it is necessary to find the system of GFT. It is the following in the discussed case:

$$\begin{aligned}
 B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 a_3; \\
 B_2 &\rightarrow x_2 a_4 \vee \bar{x}_2 x_3 a_5 \vee \bar{x}_2 \bar{x}_3 a_3; \\
 B_3 &\rightarrow x_2 x_3 a_6 \vee x_2 \bar{x}_3 a_7 \vee \bar{x}_2 a_8; \\
 B_4 &\rightarrow x_4 a_9 \vee \bar{x}_4 a_{10}; \\
 B_5 &\rightarrow x_5 a_{11} \vee \bar{x}_5 x_6 a_{12} \vee \bar{x}_5 \bar{x}_6 a_{13}; \\
 B_6 &\rightarrow x_4 x_7; \\
 B_7 &\rightarrow a_1.
 \end{aligned} \tag{7.2}$$

There is $H_T = 17$ for (7.2). So, there are 17 rows in DST (Table 7.1).

The table of PALerT includes the columns a_s , $K(a_m)$, B_i , $K(B_i)$, τ_m , m . It is Table 7.2 in the discussed case.

Let us represent the system $Y2(T)$ by the Karnaugh map (Fig. 7.5). Its cells include CMOs $Y_q \subseteq Y2$.

All functions from step 7 are created in the well-known way. For example, the following functions can be derived from Table 7.1:

$$\begin{aligned}
 D_1 &= F_9 \vee \dots \vee F_{16} = \bar{\tau}_1 \bar{\tau}_2 \vee \tau_1 \tau_3; \\
 y_1 &= F_1 \vee F_6 \vee F_8 \vee F_{11} = \bar{\tau}_1 \bar{\tau}_2 \tau_3 \vee \tau_1 \tau_2 \bar{\tau}_3 x_2 x_3 \\
 &\quad \vee \tau_1 \tau_2 \bar{\tau}_3 \bar{x}_2 \vee \tau_1 \tau_3 x_5.
 \end{aligned} \tag{7.3}$$

The system $\mathcal{T}(T)$ is derived from Table 7.2. For example, the following equation can be found:

$$\begin{aligned}
 \tau_1 &= [A_4 \vee A_5] \vee [a_5 \vee A_{10}] \vee [A_{14} \vee A_{15} \vee A_{16}] \\
 &= \bar{T}_1 T_3 \bar{T}_4 \vee \bar{T}_2 \bar{T}_3 T_4 \vee T_1 T_2 \bar{T}_3 \vee T_1 T_2 T_4.
 \end{aligned} \tag{7.4}$$

Table 7.1 DST of CSFM $P_C^1(\Gamma_{12})$

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
B_1	001	a_2	0001	x_1	$y_1 y_2$	D_4	1
		a_3	0011	$\bar{x}_1$	y_3	$D_3 D_4$	2
B_2	011	a_4	0010	x_2	$y_2 y_4$	D_3	3
		a_5	0110	$\bar{x}_2 x_3$	$y_3 y_5$	$D_2 D_3$	4
		a_3	0011	$\bar{x}_2 \bar{x}_3$	y_5	$D_3 D_4$	5
B_3	110	a_6	0100	$x_2 x_3$	$y_1 y_6$	D_2	6
		a_7	0101	$x_2 \bar{x}_3$	—	$D_2 D_4$	7
		a_8	0111	$\bar{x}_2$	$y_1 y_2$	$D_2 D_3 D_4$	8
B_4	000	a_9	1000	x_4	y_5	D_1	9
		a_{10}	1001	$\bar{x}_4$	—	$D_1 D_4$	10
B_5	101	a_{11}	1011	x_5	$y_1 y_2$	$D_1 D_3 D_4$	11
		a_{12}	1010	$\bar{x}_5 x_6$	y_3	$D_1 D_3$	12
		a_{13}	1110	$\bar{x}_5 \bar{x}_6$	—	$D_1 D_2 D_3$	13
B_6	010	a_{14}	1100	$x_4 x_7$	$y_2 y_4$	$D_1 D_2$	14
		a_{15}	1101	$x_4 \bar{x}_7$	—	$D_1 D_2 D_4$	15
		a_{16}	1111	$\bar{x}_4$	$y_3 y_5$	$D_1 D_2 D_3 D_4$	16
B_7	100	a_1	0000	1	—	—	17

Table 7.2 Table of PALer $\mathcal{T}$ of CFMSM $P_C^1(\Gamma_{12})$

a_m	$K(a_m)$	B_i	$K(B_i)$	τ_m	m
a_1	0000	B_1	001	τ_3	1
a_2	0001	B_2	011	$\tau_2 \tau_3$	2
a_3	0011	B_2	011	$\tau_2 \tau_3$	3
a_4	0010	B_3	110	$\tau_1 \tau_2$	4
a_5	0110	B_3	110	$\tau_1 \tau_2$	5
a_6	0100	B_4	000	—	6
a_7	0101	B_4	000	—	7
a_8	0111	B_4	000	—	8
a_9	1000	B_5	010	$\tau_1 \tau_3$	9
a_{10}	1001	B_5	010	$\tau_1 \tau_3$	10
a_{11}	1011	B_6	010	τ_2	11
a_{12}	1010	B_6	010	τ_2	12
a_{13}	1110	B_6	010	τ_2	13
a_{14}	1100	B_7	100	τ_1	14
a_{15}	1101	B_7	100	τ_1	15
a_{16}	1111	B_7	100	τ_1	16

Fig. 7.5 Karnaugh map for PALerY2

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	0	y_7	$y_9 y_{10}$	$y_9 y_{10}$
	01	0	$y_8 y_{10}$	$y_7 y_8$	y_8
	11	$y_7 y_8$	0	y_9	$y_7 y_8$
	10	y_9	0	y_9	0

The system $Y2(T)$ is obtained from Fig. 7.5:

$$\begin{aligned}
 y_7 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \vee T_1 T_2 \bar{T}_3 T_4 \vee \bar{T}_2 T_3 T_4; \\
 y_8 &= T_2 \bar{T}_3 T_4 \vee T_1 \bar{T}_2 T_4 \vee \bar{T}_2 T_3 T_4; \\
 y_9 &= \bar{T}_1 \bar{T}_2 T_3 \bar{T}_4 \vee T_1 T_2 T_3 \vee T_1 \bar{T}_3 \bar{T}_4; \\
 y_{10} &= \bar{T}_1 T_2 \bar{T}_3 T_4 \vee T_1 \bar{T}_2 \bar{T}_4.
 \end{aligned} \tag{7.5}$$

To get the system (7.5), we use the Karnaugh map (Fig. 7.5) as 4 different maps.

We do not discuss the last step. It is executed using standard program tools [1, 13].

In the case of heterogenous CPLDs, it is possible to replace either PALer $\mathcal{T}$ or PALerY2 (or both) by EMBer. The final structure depends on relations between t_F (with $S_A = R$) and $R_A + N2$.

Let the following condition take place:

$$t_F \geq R_A + N2. \tag{7.6}$$

In this case, both systems (6.4) and (6.8) are implemented by a single EMB. If $S_A = R$ and condition (7.6) is violated, then it is necessary n_E EMBs:

$$n_E = \left\lceil \frac{R_A + N2}{t_F} \right\rceil. \tag{7.7}$$

If only n_A EMBs can be used and $n_A < n_E$, then $n_A t_F$ functions are implemented by EMBer F and $(N2 + R_A) - n_A t_F$ functions by PALer F . It is the most general structure of P_C^2 CFSM (Fig. 7.6).

The superscript 2 means that P_C CFSM is implemented with PALs and EMBs. The following relations take places:

$$\tau_2 = \tau^1 \cup \tau^2 \quad (\tau_1 \cap \tau^2 = \emptyset); \tag{7.8}$$

$$Y2 = Y_2^1 \cup Y_2^2 \quad (Y_2^1 \cap Y_2^2 = \emptyset). \tag{7.9}$$

The set $\tau^1 \cup Y_2^1$ includes $n_A t_F$ the most complex functions depended on state variables.

Fig. 7.6 Structural diagram of P_C^2 CFSM

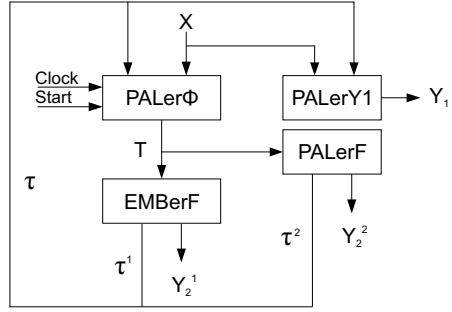
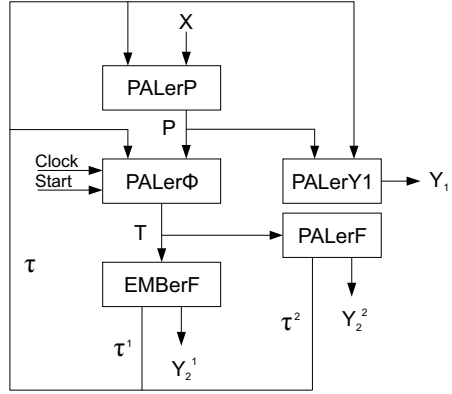


Fig. 7.7 Structural diagram of MP_C^2 CFSM



The method of synthesis for P_C^2 CFSM is practically the same as for P_C^1 . But there are different steps starting from step 5. They are:

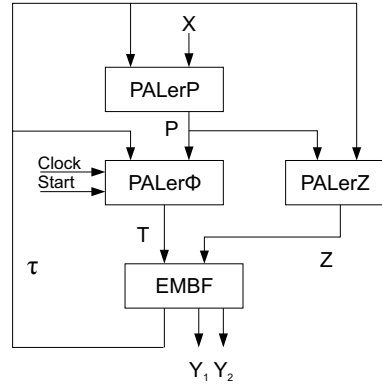
5. Finding the number of required EMBs, n_E .
6. Finding the sets τ^1 , τ^2 , Y_2^1 and Y_2^2 .
7. Creating the table of $EMBerF$ for functions from $\tau^1 \cup Y_2^1$.
8. Creating the table of $PALerF$ for functions $\tau^2 \cup Y_2^2$.
9. Implementing the CFSM logic circuit with particular PALs and EMBs.

To simplify functions Φ and $Y1$, it is possible to execute the RLC. Let us point out that RLC targets reducing the numbers of literals in SBFs [3, 12]. But it might reduce the numbers of terms, too.

For example, consider the function $y_1 = T_1 T_2 x_1 \vee T_1 \bar{T}_2 x_2$. If $x_1 = p_1$ for a_m with $K(a_m) = 11$ and $x_2 = p_1$ for a_m with $K(a_m) = 10$, then there is $y_1 = T_1 T_2 p_1 \vee T_1 \bar{T}_2 p_1 = T_1 p_1$. To get such an effect, it is necessary to execute the RLC targeting the reduction of terms. Such algorithms have not yet considered in the literature known to us.

Using the RLC, for example, turns P_C^2 CFSM into MP_C^2 CFSM. Its structural diagram is shown in Fig. 7.7.

Fig. 7.8 Structural diagram of MP_C^2Y1 CFSM with a shared EMB



The functions of all blocks of MP_C^2 CFSM are clear from previous material. In the same way, we can get the structural diagram of MP_C^1 CFSM. We leave this exercise to a reader.

The circuit of PALerY1 can be reduced using the ECMO $Y_q \subseteq Y1$. Let the following condition take place:

$$2^{R+R_\varrho}(R_A + N) \leq V_0. \quad (7.10)$$

In this case, the systems $\mathcal{T}(T)$, $Y2(T)$ and $Y1(Z)$ can be implemented using a single EMB. Applying this approach together with RLC leads to MP_C^2Y1 CFSM (Fig. 7.8).

In MP_C^2Y1 CFSM, the PALerP generates functions (6.2). They are used as arguments of functions (6.14) implemented by PALerΦ and (6.26) implemented by PALerZ. The EMBF transforms state variables $T_r \in T$ into functions (6.4) and (6.8). The variables $z_r \in Z$ are transformed into the functions (6.7).

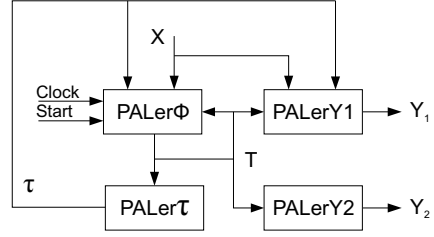
Also, it is possible to use: (1) the ECMO $Y_q \subseteq Y2$, (2) the transformation of codes $K(B_i)$ and identifiers into MOs $y_n \in Y1$ and other methods of the structural decomposition. We hope that it is not such a difficult problem for our reader. So, we do not discuss these models in our book.

7.2 Partial State Transformation: Homogenous CPLDs

Let us use ideas [5, 10] taking into account the features of CFSMs. Let us execute the OSA for some CFSM. Let us represent the partition Π_A as the unification of non-intersecting sets Π_B and Π_C . If a class $B_i \in \Pi_A$ is represented by a single generalized interval of R-dimensional Boolean space, then it is placed into the set Π_B . Otherwise, this class is placed into the set Π_C . For example, there are the following sets in the case of $P_0(\Gamma_{12})$: $\Pi_A = \{B_1, \dots, B_7\}$, $\Pi_B = \{B_1, B_2, B_3, B_5\}$ and $\Pi_C = \{B_4, B_6, B_7\}$.

Let us encode the classes $B_i \in \Pi_C$ by the binary codes $C(B_i)$ having R_C bits:

Fig. 7.9 Structural diagram of P_{C1}^1 CFSM



$$R_C = \lceil \log_2(|\Pi_C| + 1) \rceil. \quad (7.11)$$

In (7.11), the one is added to take into account the relation $B_i \notin \Pi_C$. The R_C determines the set $\mathcal{T}$.

Let the following condition take place:

$$S \geq L + R + R_C. \quad (7.12)$$

It means that some terms can be created from $x_l \in X$, $T_r \in T$ and $\tau_r \in \mathcal{T}$. It allows to transform only codes of states $a_m \in B_i$ where $B_i \in \Pi_C$.

So, the classes $B_i \in \Pi_B$ are represented by the variables $T_r \in T$, the classes $B_i \in \Pi_C$ by the variables $\tau_r \in \mathcal{T}$. It leads to P_{C1}^1 CFSM (Fig. 7.9).

In P_{C1}^1 CFSM, the PALer $\mathcal{T}$ implements the system (6.8), the PALerY2 the system (6.4). These systems are the same as for any CFSM without EMCO $Y_q \subseteq Y_2$.

But the functions $D_r \in \Phi$ and $y_n \in Y_1$ are significantly different. They are represented as

$$\Phi = \Phi(T, \mathcal{T}, X); \quad (7.13)$$

$$Y_1 = Y_1(T, \mathcal{T}, X). \quad (7.14)$$

In this chapter, we propose the method of synthesis of P_{C1}^1 CFSM. It includes the following steps:

1. Creating the set A for initial GSA Γ .
2. Constructing the partition Π_A with classes of PES.
3. Executing the optimal state assignment.
4. Constructing the sets Π_B and Π_C on the base of outcome of OSA.
5. Encoding of the classes $B_i \in \Pi_C$.
6. Creating the DST of P_{C1}^1 CFSM.
7. Deriving functions (7.13) and (7.14) from DST.
8. Creating the tables of PALer $\mathcal{T}$ and PALerY2.
9. Creating the functions (6.4) and (6.8).
10. Implementing the CFSM circuit using the particular macrocells.

Let us discuss an example of synthesis for CFSM $P_{C1}^1(\Gamma_{12})$. The steps 1–4 are already executed.

There is $|\Pi_C| = 3$. So, using (7.12) gives $R_C = 2$ and $\mathcal{T} = \{\tau_1, \tau_2\}$. Let us use the combination $\tau_1 = \tau_2 = 0$ for the relation $B_i \notin \Pi_C$. Let us encode the classes in the following way: $K(B_4) = 01$, $K(B_6) = 10$, $K(B_7) = 11$.

The codes of $B_i \in \Pi_B$ can be found from Fig. 7.3. They are the following: $K(B_1) = 0000$, $K(B_2) = 00*1$, $K(B_3) = 0*10$, $K(B_5) = 100*$.

The DST of $P_{C1}^1(\Gamma_{12})$ is constructed using the system (7.2). So, it includes $H_T = 17$ rows. It means that the PALer $\mathcal{T}$ is simpler than for $P_C^1(\Gamma_{12})$ but functions $y_n \in Y1$ and $D_r \in \Phi$ have the same number of terms as for $P_C^1(\Gamma_{12})$. The DST of CFSM P_{C1}^1 is represented by Table 7.3.

This table includes two columns ($\mathcal{T}$ and T) to represent the code of class $B_i \in \Pi_A$. If $\tau_1 = \tau_2 = 0$, then the code $K(B_i)$ is determined by the column T . If $\tau_1 \notin 0$ or $\tau_2 \notin 0$, then the code $K(B_i)$ is determined by column $\mathcal{T}$. In this case, the code in the column T is ignored. Because of it, we show all asterisks for such situations.

The systems (7.13)–(7.14) are derived in the same manner as for P_C^1 CFSM. But terms $F_h \in F$ are represented as

$$F_h = \bigwedge_{r=1}^R T_r^{l_{ir}} \cdot \bigwedge_{r=1}^{R_C} \tau_r^{l_{ir}} X_h. \quad (7.15)$$

The first member of (7.15) corresponds to the code $K(B_i)$ where $B_i \in \Pi_B$; l_{ir} is the value of r -th bit of $K(B_i)$ ($l_{ir} \in \{0, 1, *\}$, $T_r^0 = \bar{T}_r$, $T_r^1 = T_r$, $T_r^* = 1$). The second term corresponds to the code $K(B_i)$ where $B_i \in \Pi_C$. The same rule is true: $\tau_r^0 = \bar{\tau}_r$, $\tau_r^1 = \tau_r$, $\tau_r^* = 1$.

The DSTs of $P_C^1(\Gamma_{12})$ and $P_{C1}^1(\Gamma_{12})$ are constructed using the same system of GFT (7.2). The same codes (Fig. 7.3) are used in both Tables 7.1 and 7.3. Because of it, the contents of columns $a_m - \Phi_h$ are the same for these DSTs.

The systems $D_r \in \Phi$ and $y_n \in Y1$ depend on terms (7.15). The terms correspond to columns T , $\mathcal{T}$, X_h of a DST. For example, the following functions can be found from Table 7.3:

$$\begin{aligned} D_1 &= [F_9 \vee F_{10}] \vee [F_{11} \vee F_{12} \vee F_{13}] \vee [F_{14} \vee F_{15} \vee F_{16}] = \bar{\tau}_1 \vee \bar{\tau}_2; \\ y_1 &= F_1 \vee F_6 \vee F_8 \vee F_{11} = \bar{T}_1 \bar{T}_2 \bar{T}_3 T \tau_4 \bar{\tau}_1 \bar{\tau}_2 \\ &\quad \vee \bar{T}_1 T_3 \bar{T}_4 \bar{\tau}_1 \bar{\tau}_2 x_2 x_3 \vee \bar{T}_1 T_2 \bar{T}_3 \bar{\tau}_1 \bar{\tau}_2 \vee T_1 \bar{T}_2 \bar{T}_3 \bar{\tau}_1 \bar{\tau}_2 x_5; \\ y_5 &= [F_4 \vee F_5] \vee F_9 \vee F_{16} = \bar{T}_1 \bar{T}_2 T \tau_4 \bar{\tau}_1 \bar{\tau}_2 \bar{x}_2 \\ &\quad \vee \bar{\tau}_1 \tau_2 \vee \tau_1 \bar{\tau}_2 \bar{x}_4. \end{aligned} \quad (7.16)$$

The table of PALer $\mathcal{T}$ is constructed only for states $a_m \in B_i$, where $B_i \in \Pi_C$. It is Table 7.4 in the discussed case.

The following system can be found from Table 7.4:

Table 7.3 DST of CSFM $P_{C1}^1(\Gamma_{12})$

B_i	$K(B_i)$		a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
	T	τ						
B_1	0000	00	a_2	0001	x_1	$y_1 y_2$	D_4	1
			a_3	0011	$\bar{x}_1$	y_3	$D_3 D_4$	2
B_2	00*1	00	a_4	0010	x_2	$y_2 y_4$	D_3	3
			a_5	0110	$\bar{x}_2 x_3$	$y_3 y_5$	$D_2 D_3$	4
			a_3	0011	$\bar{x}_2 \bar{x}_3$	y_5	$D_3 D_4$	5
B_3	0*10	00	a_6	0100	$x_2 x_3$	$y_1 y_6$	D_2	6
			a_7	0101	$x_2 \bar{x}_3$	–	$D_2 D_4$	7
			a_8	0111	$\bar{x}_2$	$y_1 y_2$	$D_2 D_3 D_4$	8
B_4	****	01	a_9	1000	x_4	y_5	D_1	9
			a_{10}	1001	$\bar{x}_4$	–	$D_1 D_4$	10
B_5	100*	00	a_{11}	1011	x_5	$y_1 y_2$	$D_1 D_3 D_4$	11
			a_{12}	1010	$\bar{x}_5 x_6$	y_3	$D_1 D_3$	12
			a_{13}	1110	$\bar{x}_5 \bar{x}_6$	–	$D_1 D_2 D_3$	13
B_6	****	10	a_{14}	1100	$x_4 x_7$	$y_2 y_4$	$D_1 D_2$	14
			a_{15}	1101	$x_4 \bar{x}_7$	–	$D_1 D_2 D_4$	15
			a_{16}	1111	$\bar{x}_4$	$y_3 y_5$	$D_1 D_2 D_3 D_4$	16
B_7	****	11	a_1	0000	1	–	–	17

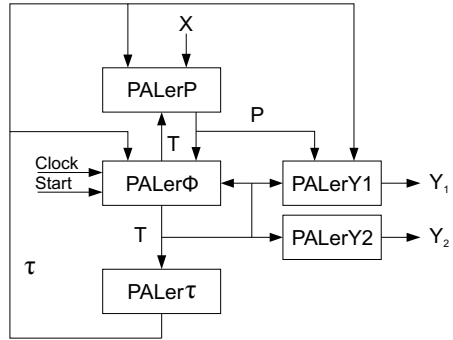
Table 7.4 Table of PALer $\mathcal{T}$ of CFMSM $P_{C1}^1(\Gamma_{12})$

a_m	$K(a_m)$	B_i	$K(B_i)$	τ_m	m
a_6	0100	B_4	01	τ_2	1
a_7	0101	B_4	01	τ_2	2
a_8	0111	B_4	01	τ_2	3
a_{11}	1011	B_6	10	τ_1	4
a_{12}	1010	B_6	10	τ_1	5
a_{13}	1110	B_6	10	τ_1	6
a_{14}	1100	B_7	11	$\tau_1 \tau_2$	7
a_{15}	1101	B_7	11	$\tau_1 \tau_2$	8
a_{16}	1111	B_7	11	$\tau_1 \tau_2$	9

$$\begin{aligned}
\tau_1 &= [A_{11} \vee A_{11}] \vee [A_{13} \vee \dots \vee A_{16}] = T_1 \bar{T}_2 T_3 \vee T_1 T_2; \\
\tau_2 &= [A_6 \vee A_{14}] \vee [(A_7 \vee \dots \vee A_8) \vee [A_{15} \vee \dots \vee A_{16}]] = T_2 T_3 \bar{T}_4 \vee T_2 T_4.
\end{aligned}
\tag{7.17}$$

The system $Y2(T)$ is represented by Fig. 7.5. So, there is no need in table of PALer $Y2$ in the discussed case. Obviously, the system $Y2(T)$ is represented by the system (7.5).

Fig. 7.10 Structural diagram of MP_{C1}^1 CFSM



We do not discuss the last step of the proposed method. Its execution requires using special CAD tools [1, 13].

Let us call this approach extended class encoding (ECE). In fact, the class codes $K(B_i)$ include parts $K_T(B_i)$ and $K_{\mathcal{T}}(B_i)$. The subscripts show which variables are used to encode the classes $B_i \in \Pi_A$.

The ECE gives some advantages in comparison with P_C^1 CFSMs:

1. There are fewer MCs in the circuit of PALer $\mathcal{T}$ because only a part of state codes is transformed.
2. There are H_T rows in the DST. So, the functions $D_r \in \Phi$ and $y_n \in Y1$ have the same amount of terms for P_C^1 and P_{C1}^1 equivalent CFSMs.

We propose to diminish the number of literals in functions generated by PALer Φ and PALerY1. It can be executed using the RLC. The RLC here is executed among the classes $B_i \in \Pi_A$. So, the functions $p_g \in P$ should be represented as

$$P = P(T, \mathcal{T}, X). \quad (7.18)$$

Now, the functions $D_r \in \Phi$ and $y_n \in Y1$ depend on $T_r \in T$, $\tau_r \in \mathcal{T}$ and $p_g \in P$. It leads to the systems

$$\Phi = \Phi(T, \mathcal{T}, P); \quad (7.19)$$

$$Y1 = Y1(T, \mathcal{T}, P). \quad (7.20)$$

The functions $\tau_r \in \mathcal{T}$ and $y_n \in Y2$ do not depend on logical conditions. So, they are still represented by the SBF (6.4) and (6.8).

These preliminaries lead to the structural diagram of MP_{C1}^1 CFSM (Fig. 7.10). In this model, the PALerP implements the system (7.18), PALer Φ the system (7.19) and PALerY1 the system (7.20).

Let us discuss an example of synthesis for CFSM $MP_{C1}^1(\Gamma_{12})$. Let us start from the distribution of LCs $x_l \in X$ between the variables $p_g \in P$.

Table 7.5 The RLC for CFMS $MP_{C1}^1(\Gamma_{12})$

P	B_i						
	B_1	B_2	B_3	B_4	B_5	B_6	B_7
p_1	x_1	x_2	x_2	—	x_5	x_7	—
p_2	—	x_3	x_3	x_4	x_6	x_4	—
$K(B_i)$	000000	00*100	0*1000	****01	100*00	****10	****11

Table 7.6 DST of CFMS $MP_{C1}^1(\Gamma_{12})$

B_i	$K(B_i)$		a_s	$K(a_s)$	P_h	Y_h	Φ_h	h
	T	τ						
B_1	0000	00	a_2	0001	p_1	$y_1 y_2$	D_4	1
			a_3	0011	$\bar{p}_1$	y_3	$D_3 D_4$	2
B_2	00*1	00	a_4	0010	p_1	$y_2 y_4$	D_3	3
			a_5	0110	$\bar{p}_1 p_2$	$y_3 y_5$	$D_2 D_3$	4
			a_3	0011	$\bar{p}_1 \bar{p}_2$	y_5	$D_3 D_4$	5
B_3	0*10	00	a_6	0100	$p_1 p_2$	$y_1 y_6$	D_2	6
			a_7	0101	$p_1 \bar{p}_2$	—	$D_2 D_4$	7
			a_8	0111	$\bar{p}_1$	$y_1 y_2$	$D_2 D_3 D_4$	8
B_4	****	01	a_9	1000	p_2	y_5	D_1	9
			a_{10}	1001	$\bar{p}_2$	—	$D_1 D_4$	10
B_5	100*	00	a_{11}	1011	p_1	$y_1 y_2$	$D_1 D_3 D_4$	11
			a_{12}	1010	$\bar{p}_1 p_2$	y_3	$D_1 D_3$	12
			a_{13}	1110	$\bar{p}_1 \bar{p}_2$	—	$D_1 D_2 D_3$	13
B_6	****	10	a_{14}	1100	$p_2 p_1$	$y_2 y_4$	$D_1 D_2$	14
			a_{15}	1101	$p_2 \bar{p}_1$	—	$D_1 D_2 D_4$	15
			a_{16}	1111	$\bar{p}_2$	$y_3 y_5$	$D_1 D_2 D_3 D_4$	16
B_7	****	11	a_1	0000	1	—	—	17

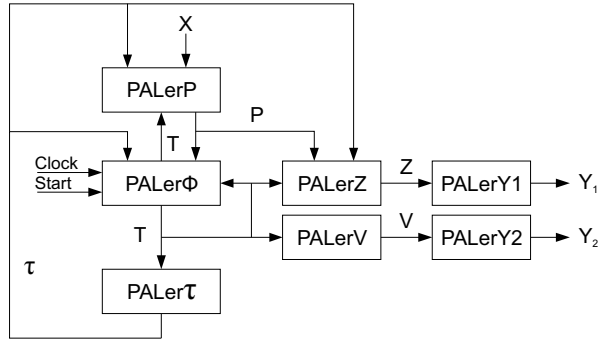
There are the following sets $X(B_i) : X(B_1) = \{x_1\}$, $X(B_2) = \{x_2, x_3\} = X(B_3)$, $X(B_4) = \{x_4\}$, $X(B_5) = \{x_5, x_6\}$, $X(B_6) = \{x_4, x_7\}$ and $X(B_7) = \emptyset$. It gives $G = 2$, $P = \{p_1, p_2\}$. The table of RLC is represented by Table 7.5.

Using Table 7.5 and executing the minimization, the following system is created:

$$\begin{aligned}
 p_1 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{\tau}_1 \bar{\tau}_2 x_1 \vee \bar{T}_1 \bar{T}_2 T_4 \bar{\tau}_1 \bar{\tau}_2 x_2 \\
 &\vee \bar{T}_1 T_3 \bar{T}_4 \bar{\tau}_1 \bar{\tau}_2 x_2 \vee T_1 \bar{T}_2 \bar{T}_3 \bar{\tau}_1 \bar{\tau}_2 x_5 \vee \tau_1 x_7; \\
 p_2 &= B_2 x_3 \vee B_3 x_3 \vee B_4 x_4 \vee B_5 x_6 \vee B_6 x_4.
 \end{aligned} \tag{7.21}$$

The functions (7.19)–(7.20) are derived from the DST of CFMS $MP_{C1}^1(\Gamma_{12})$. It is similar to Table 7.3, but the column X_h is replaced by P_h (Table 7.6).

Fig. 7.11 Structural diagram of $MP_{C1}^1 Y1Y2$ CFSM



Using Table 7.6 gives, for example:

$$\begin{aligned}
 D_1 &= \bar{\tau}_1 \vee \bar{\tau}_2; \\
 y_1 &= \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{\tau}_1 \bar{\tau}_2 \vee \bar{T}_1 T_3 \bar{T}_4 \bar{\tau}_1 \bar{\tau}_2 p_1 p_2 \\
 &\quad \vee \bar{T}_1 T_2 \bar{T}_3 \bar{\tau}_1 \bar{\tau}_2 \vee T_1 \bar{T}_2 \bar{T}_3 \bar{\tau}_1 \bar{\tau}_2 p_1; \\
 y_5 &= \bar{T}_1 \bar{T}_2 T_4 \bar{\tau}_1 \bar{\tau}_2 \bar{p}_1 \vee \bar{\tau}_1 \bar{\tau}_2 \vee \tau_1 \bar{\tau}_2 \bar{p}_2.
 \end{aligned} \tag{7.22}$$

There is no change for functions $\mathcal{T}(T)$ and $Y2(T)$. So, the functions $\mathcal{T}(T)$ are represented by (7.17).

Future optimization is possible due to:

1. Encoding of CMOs $Y_q \subseteq Y1$ leading to $MP_{C1}^1 Y1$ CFSM.
2. Encoding of CMOs $Y_q \subseteq Y2$ leading to $MP_{C1}^1 Y2$ CFSM.
3. Combined encoding of CMOs $Y_q \subseteq Y1 \cup Y2$. It leads to the most complex model of $MP_{C1}^1 Y1Y2$ CFSM (Fig. 7.11).

In the $MP_{C1}^1 Y1Y2$ CFSM, the PALerP generates the SBF (7.18). The PALerΦ implements functions (7.19), the PALerτ the functions (6.8). The PALerV implements functions

$$V = V(T). \tag{7.23}$$

The PALerY2 implements the functions (6.6). The PALerZ implements the system

$$Z = Z(T, \mathcal{T}, P), \tag{7.24}$$

the PALerY1 implements the system (6.7).

To design the circuit of $MP_{C1}^1 Y1Y2$ CFSM, it is necessary to apply a lot of methods discussed in the previous text. We hope that our reader will be able to do it.

Let us point out that all discussed structures can be used for PLA-based MCs. For example, the structural diagram of P_{C1}^1 CFSM is shown in Fig. 7.12.

Also, PLA-based MCs can be used for implementing CFSMs with the CST. For example, the structural diagram of P_C^1 CFSM (Fig. 7.13) can be obtained from Fig. 7.1.

Fig. 7.12 Structural diagram of P_{C1}^1 CFSM implemented with PLA-based MCs

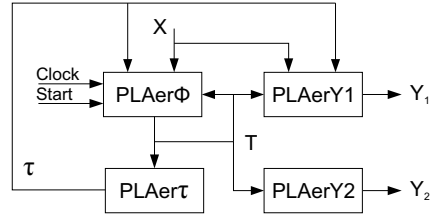
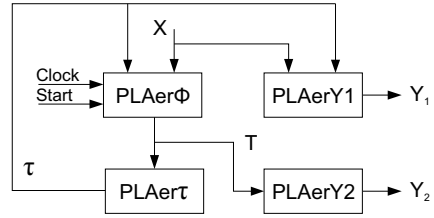


Fig. 7.13 Structural diagram of P_C^1 CFSM implemented with PLA-based MCs



All previously discussed methods can be used to get the SBFs representing PLA-based CFSMs. Next, the circuits should be implemented using the standard CAD tools [1, 13].

The structural diagrams shown in Figs. 7.12 and 7.13 are, in some sense, ideal. This implies, for example, that $\text{PLAer}\Phi$ implements only functions $D_r \in \Phi$. In reality, it is possible to implement functions $y_n \in Y1$ by $\text{PALer}\Phi$, too.

Let a CPLD chip include MCs $\text{PLA}(S, t, q)$ having S inputs, t outputs and q product terms. Let the following condition take place:

$$S \geq L + R_A. \quad (7.25)$$

It means that any function $f_i \in \Phi \cup Y1$ may be implemented using resources of a single MC. Of course, the following relation should take place:

$$Q(f_i) \leq q. \quad (7.26)$$

In the best case, it is enough n_B MCs to implement the systems Φ and $Y1$:

$$n_B = \left\lceil \frac{R + N1}{t} \right\rceil. \quad (7.27)$$

So, the system $\Phi \cup Y1$ should be implemented using n_B blocks ΦY_k such that

$$|\Phi Y_k| \leq t; \quad (7.28)$$

$$|F(\Phi Y_k)| \leq q. \quad (7.29)$$

Fig. 7.14 Structural diagram of P_C^1 CFSM with shared PLAers

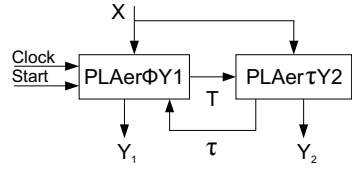
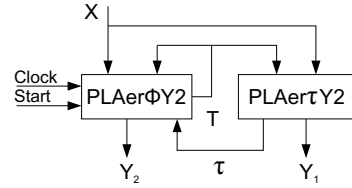


Fig. 7.15 Structural diagram of P_{C1}^1 CFSM with shared PLAers



In (7.29), the symbol $F(\Phi Y_i)$ stands for the number of terms in functions $f_i \in \Phi Y_k$.

To solve this problem, well-known methods [2, 9] can be used. The same can be done for functions $f_i \in T \cup Y2$ having the same arguments $T_r \in T$.

Now, the systems $\Phi \cup Y1$ are represented by the block PLAer $\tau Y1$. The systems $T \cup Y2$ are represented by the block PLAer $\tau Y2$. It leads to the structural diagram of P_C^1 CFSM shown in Fig. 7.14.

The same can be done for PLA-based CFSM P_{C1}^1 . The systems Φ and $Y1$ are implemented by PLAer $\Phi Y1$, the systems T and $Y2$ by PLAer $\tau Y2$. It turns the structural diagram (Fig. 7.12) into the structural diagram with two blocks of MCs (Fig. 7.15).

The structural diagrams with shared PLAers are more realistic. They are based on more sophisticated methods of synthesis than their counterparts without sharing of resources. Due to combining MCs for functions with the same arguments, it is possible to reduce the total number of MCs in CFSM circuits as compared to their counterparts without the shared PLAs.

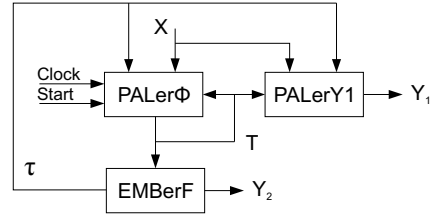
7.3 Partial State Transformation: Heterogenous CPLDs

There are two types of MCs in heterogenous CPLDs [6, 10]: PAL macrocells and configurable EMB macrocells. Generally, PALs implement irregular functions represented by minimized SOPs. The EMBs implement systems of regular functions. Let us point out that an EMB can implement any system having $L + R$ arguments, H terms and $R + N$ functions if the following relations are true:

$$R + L \leq S_A; \quad (7.30)$$

$$R + N \leq t_F. \quad (7.31)$$

Fig. 7.16 Structural diagram of P_{C1}^2 CFSM



In this case, the number of terms does not matter because the following condition takes place:

$$2^{S_A} \geq H = 2^{L+R}. \quad (7.32)$$

Let us discuss a case when it is not enough a single EMB to implement the circuit of CFSM. For example, it is not possible for 32% of benchmarks from the library [8].

Let the OSA be executed and the set $\Pi_A = \Pi_B \cup \Pi_C$ be constructed. Let us use PALers to implement functions (7.13) and (7.14). Let we have enough EMBs to implement functions $T(T)$ and $Y2(T)$. It leads to P_{C1}^2 CFSM (Fig. 7.16).

We use the PALer Φ to implement the functions (7.13) and PALerY1 the functions (7.14). The functions (6.4) and (6.8) are implement by the EMBerF.

If we compare Figs. 7.9 and 7.16, then it is obvious that the only difference is reduced to replacing PALer T and PALerY2 by EMBerF. So, methods of synthesis for these CFSMs are practically identical.

So, there are the same seven steps in design methods of P_{C1}^1 and P_{C1}^2 CFSMs. Next, we have the steps:

8. Creating the table of EMBerF.
9. Implementing the CFSM circuit with particular PAL macrocells and EMBs.

The structural diagram (Fig. 7.16) corresponds to the case when we have enough EMBs to implement the systems (6.4) and (6.8). It means that:

1. There is a configuration $\langle S_A, t_F \rangle$ such that the condition (6.36) takes place. In this case, an EMB can implement at least a single function from (6.4) or (6.8).
2. The number of available EMBs (n_A) exceeds their required number (n_R):

$$n_A \geq \left\lceil \frac{N2 + R_C}{t_F} \right\rceil = n_R. \quad (7.33)$$

If these conditions are violated, then it is not possible to use the model P_{C1}^2 . Let us discuss the possible solutions starting from GSA Γ_{13} (Fig. 7.17). It is marked by states of Moore FSM using the rules [2].

The following sets can be found from Γ_{13} : the set of states $A = \{a_1, \dots, a_{16}\}$, set of LCs $X = \{x_1, \dots, x_8\}$, set of MOs $Y = \{y_1, \dots, y_{11}\}$ with $Y1 = \{y_1, \dots, y_5\}$ and $Y2 = \{y_6, \dots, y_{11}\}$. So, there is $M = 16, L = 8, N1 = 5, N2 = 6$ and $N = 11$. Using (1.2) gives $R = 4, \Phi = \{D_1, \dots, D_4\}$ and $T = \{T_1, \dots, T_4\}$.

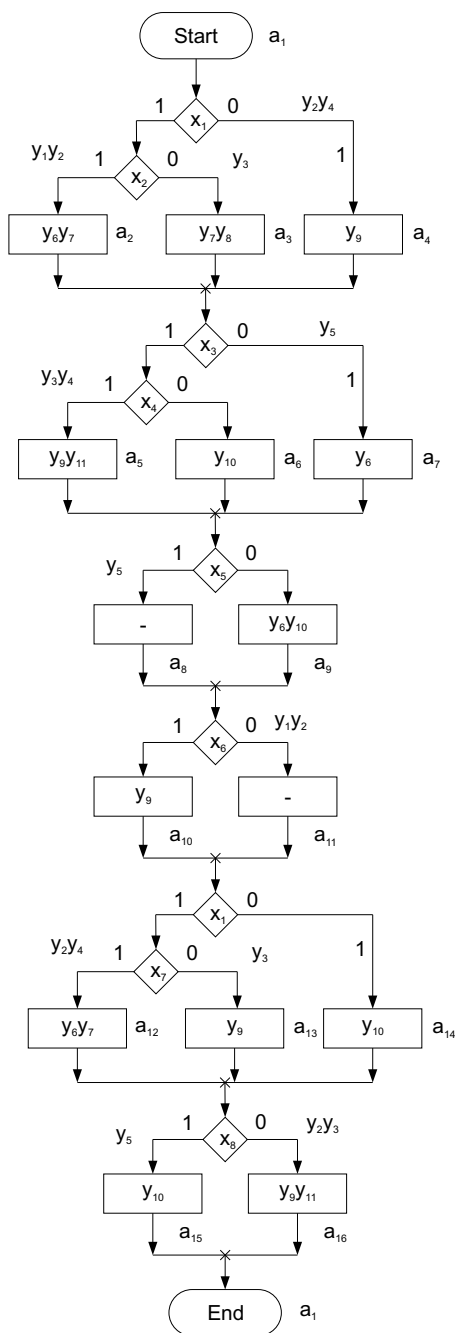
Fig. 7.17 Marked GSA Γ_{13} 

Fig. 7.18 Outcome of OSA
for $P(\Gamma_{13})$

		$T_1 T_2$			
$T_3 T_4$		00	01	11	10
	00	a_1	a_5	a_{12}	a_8
	01	a_2	a_6	a_{13}	a_9
	11	a_3	a_7	a_{14}	a_{10}
	10	a_4	a_{15}	a_{16}	a_{11}

There is the partition $\Pi_A = \{B_1, \dots, B_7\}$, where $B_i \in \Pi_A$ is a class of PES [7, 11]. There are the following classes $B_i \in \Pi_A$: $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6, a_7\}$, $B_4 = \{a_8, a_9\}$, $B_5 = \{a_{10}, a_{11}\}$, $B_6 = \{a_{12}, a_{13}, a_{14}\}$ and $B_7 = \{a_{15}, a_{16}\}$. Let us execute the OSA for CFSM $P(\Gamma_{13})$. Its outcome is shown in Fig. 7.18.

The classes $B_1, B_4, B_5, B_7 \in \Pi_A$ form the set Π_B . There are $K(B_1) = 0000$, $K(B_4) = 100*$, $K(B_5) = 101*$ and $K(B_7) = *110$. So, there is the set $\Pi_C = \{B_2, B_3, B_6\}$. So, there is $\Pi_B \neq \emptyset$ and $\Pi_C \neq \emptyset$. It means that the partial state transformation should be applied.

Let us have a CPLD with PAL macrocells and EMBs. So, we can try the model $P_{C1}^2(\Gamma_{13})$.

Using (7.12) gives $R_C = 2$, $\mathcal{T} = \{\tau_1, \tau_2\}$. Let us encode the classes $B_i \in \Pi_C$ in the following way: $K(B_2) = 01$, $K(B_3) = 10$, $K(B_6) = 11$. The input assignment 00 indicates that $B_i \notin \Pi_C$.

To derive functions $D_r \in \Phi$ and $y_n \in Y_1$ depended on terms (7.15), it is necessary to construct the DST of CFSM $P_{C1}^2(\Gamma_{13})$. It is constructed using the following GFTs:

$$\begin{aligned}
 B_1 &\rightarrow x_1 x_2 a_2 \vee x_1 \bar{x}_2 a_3 \vee \bar{x}_1 a_4; \\
 B_2 &\rightarrow x_3 x_4 a_5 \vee x_3 \bar{x}_4 a_6 \vee \bar{x}_3 a_7; \\
 B_3 &\rightarrow x_5 a_8 \vee \bar{x}_5 a_9; \\
 B_4 &\rightarrow x_6 a_{10} \vee \bar{x}_6 a_{11}; \\
 B_5 &\rightarrow x_1 x_7 a_{12} \vee x_1 \bar{x}_7 a_{13} \vee \bar{x}_1 a_{14}; \\
 B_6 &\rightarrow x_8 a_{15} \vee \bar{x}_8 a_{16}; \\
 B_7 &\rightarrow a_1.
 \end{aligned} \tag{7.34}$$

There are $H_T = 16$ product terms in (7.34). It gives the DST (Table 7.7) having 16 rows.

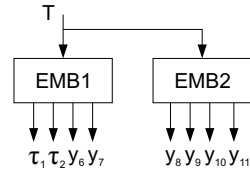
We use code **** to show that $B_i \notin \Pi_C$. The code 00 shows that $B_i \notin \Pi_B$. We use the same rules for constructing Tables 7.3 and 7.7. We hope that everything is clear for our reader.

The DST is a base to derive the systems (7.13) and (7.14). For example, the following functions can be derived from Table 7.7:

$$\begin{aligned}
 D_1 &= [F_7 \vee F_8] \vee [[F_9 \vee F_{10}] \vee [F_{11} \vee F_{12} \vee F_{13}]] \vee F_{15} \\
 &= \tau_1 \bar{\tau}_2 \vee T_1 \bar{T}_2 \bar{\tau}_1 \bar{\tau}_2 \vee \tau_1 \tau_2 \bar{x}_8; \\
 y_1 &= F_1 \vee F_{10} \vee \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{\tau}_1 \bar{\tau}_2 x_1 x_2 \vee T_1 \bar{T}_2 \bar{T}_3 \bar{\tau}_1 \bar{\tau}_2 \bar{x}_6.
 \end{aligned} \tag{7.35}$$

Table 7.7 DST of CSFM $P_{C1}^2(\Gamma_{13})$

B_i	$K(B_i)$		a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
	T	τ						
B_1	0000	00	a_2	0001	x_1x_2	y_1y_2	D_4	1
			a_3	0011	$x_1\bar{x}_2$	y_3	D_3D_4	2
			a_4	0010	$\bar{x}_1$	y_2y_4	D_3	3
B_2	****	01	a_5	0100	x_3x_4	y_3y_4	D_2	4
			a_6	0101	$x_3\bar{x}_4$	—	D_2D_4	5
			a_7	0111	$\bar{x}_3$	y_5	$D_2D_3D_4$	6
B_3	****	10	a_8	1000	x_5	y_5	D_1	7
			a_9	1001	$\bar{x}_5$	y_3	D_1D_4	8
B_4	100*	00	a_{10}	1011	x_6	—	$D_1D_3D_4$	9
			a_{11}	1010	$\bar{x}_6$	y_1y_2	D_1D_3	10
B_5	101*	00	a_{12}	1100	x_1x_7	y_2y_4	D_1D_2	11
			a_{13}	1101	$x_1\bar{x}_7$	y_3	$D_1D_2D_4$	12
			a_{14}	1111	$\bar{x}_1$	—	$D_1D_2D_3D_4$	13
B_6	****	11	a_{15}	0110	x_8	y_5	D_2D_3	14
			a_{16}	1110	$\bar{x}_8$	y_2y_3	$D_1D_2D_3$	15
B_7	*110	00	a_1	0000	1	—	—	16

Fig. 7.19 Implementing EMBerF with two EMBs

There are the following columns in the table of EMBerF: a_m , $K(a_m)$, τ_m , Y_2 , m . It includes $M = 16$ rows in the discussed case (Table 7.8).

We added the columns B_i , $K(B_i)$ to explain the content of the column τ_m . Some values of $\tau_r \in \mathcal{T}$ are generated only for states from the classes B_1 , B_3 , B_6 . For all other states, the code 00 should be generated because of $B_i \notin \Pi_C$.

Let we can use EMBs having the configuration $\langle 4, 4 \rangle$. There is $t_F = 4$, $R_C = 2$ and $N_2 = 6$. Using (7.33) gives $n_R = 2$. Let it be possible to use 2 EMBs. In this case, the EMBerF is implemented as it is shown in Fig. 7.19.

If the following condition

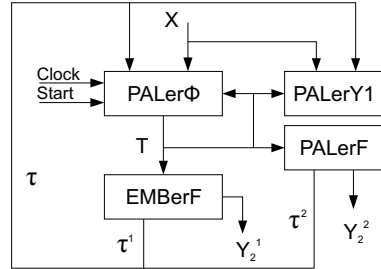
$$n_A < n_R \quad (7.36)$$

takes place, then the system $Y_2 \cup \mathcal{T}$ is implemented using EMBerF and PALerF. It leads to CFSM P_{C1}^{21} (Fig. 7.20).

As follows from Fig. 7.20, the systems (6.4) and (6.8) are derived by two subsystems. The corresponding subsets satisfy to the following relations:

Table 7.8 Table of EMBerF of CFSM $P_{C1}^2(\Gamma_{13})$

a_m	$K(a_m)$	τ_m	$Y2$	m	B_i	$K(B_i)$
a_1	0000	—	—	1	B_1	00
a_2	0001	τ_2	$y_6 y_7$	2	B_2	01
a_3	0011	τ_2	$y_7 y_8$	3	B_2	01
a_4	0010	τ_2	y_9	4	B_2	01
a_5	0100	τ_1	$y_9 y_{11}$	5	B_2	10
a_6	0101	τ_1	y_{10}	6	B_3	10
a_7	0111	τ_1	y_6	7	B_3	10
a_8	1000	—	—	8	B_4	00
a_9	1001	—	$y_6 y_{10}$	9	B_4	00
a_{10}	1011	—	y_9	10	B_5	00
a_{11}	1010	—	—	11	B_5	00
a_{12}	1100	$\tau_1 \tau_2$	$y_6 y_7$	12	B_6	11
a_{13}	1100	$\tau_1 \tau_2$	y_9	13	B_6	11
a_{14}	1111	$\tau_1 \tau_2$	y_{10}	14	B_6	11
a_{15}	0110	—	y_{10}	15	B_7	00
a_{16}	1110	—	$y_5 y_{11}$	16	B_7	00

Fig. 7.20 Structural diagram of P_{C1}^{21} CFSM

$$Y2 = Y21 \cup Y22, \quad Y21 \cap Y22 = \emptyset; \quad (7.37)$$

$$\mathcal{T} = \tau_1 \cup \tau_2, \quad \tau_1 \cap \tau_2 = \emptyset. \quad (7.38)$$

To derive the sets $Y2$ and $\mathcal{T}$, it is necessary:

1. To find the SOPs of functions $f_i \in \mathcal{T} \cup Y2$.
2. To arrange the queue γ in descending order of the parameter $Q(f_i)$.
3. To choose $n_A \cdot t_F$ first elements of the queue γ and implement corresponding functions with EMBs.
4. To implement the remaining $(N2 + R_C) - n_A \cdot t_F$ functions with PAL macrocells creating PALerF.

Fig. 7.21 Implementing systems $\mathcal{T}$ and Y_2 for CFSM $P_{C1}^{21}(\Gamma_{13})$

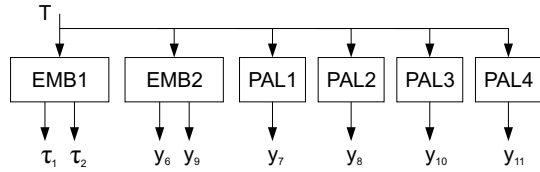
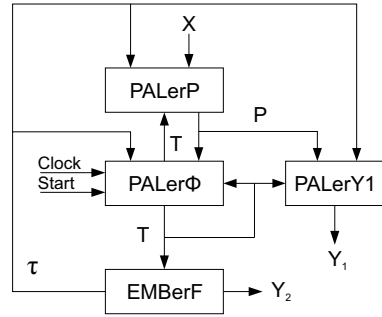


Fig. 7.22 Structural diagram of MP_{C1}^2 CFSM



Let us have EMBs with the configuration $\langle 4, 2 \rangle$. Let it be $n_A = 2$. Because $n_R = 4$, we should use the model P_{C1}^{21} for GSA Γ_{13} . To find the sets τ_1, τ_2, Y_{21} and Y_{22} , let us find the values of $Q(f_i)$ for $f_i \in \mathcal{T} \cup Y_2$. It can be done, for example, using Table 7.8.

After minimizing, we can get the following SBF:

$$\begin{array}{ll}
 \tau_1 = [A_5 \vee A_6] \vee A_7 \vee [A_{12} \vee A_{13}] \vee A_{14}; & Q(\tau_1) = 4; \\
 \tau_2 = [A_2 \vee A_3] \vee A_4 \vee [A_{12} \vee A_{13}] \vee A_{14}; & Q(\tau_2) = 4; \\
 y_6 = [A_2 \vee A_9] \vee A_7 \vee A_{12}; & Q(y_6) = 3; \\
 y_7 = [A_2 \vee A_3] \vee A_{12}; & y_8 = A_3; & Q(y_7) = 2; \\
 y_9 = [A_5 \vee A_{13}] \vee [A_{10} \vee A_{16}] \vee A_4; & Q(y_9) = 3; \\
 y_{10} = [A_6 \vee A_{15}] \vee A_9 \vee A_{14}; & Q(y_{10}) = 3; \\
 y_{11} = A_5 \vee A_{16}; & Q(y_{11}) = 2; & Q(y_8) = 1.
 \end{array} \tag{7.39}$$

We include the numbers of terms $Q(f_i)$ in (7.39). It allows creating the following queue: $\gamma = \langle \tau_1, \tau_2, y_6, y_9, y_{10}, y_7, y_{11}, y_8 \rangle$.

There is $n_A \cdot t_F = 4$, $N_2 + R_C = 8$, so four functions are implemented using 2 EMBs and four using 4 PALs. Using γ , we can get the following sets: $\tau_1 = \{\tau_1, \tau_2\}$, $\tau_2 = \emptyset$, $Y_{21} = \{y_6, y_9\}$, $Y_{22} = \{y_7, y_8, y_{10}, y_{11}\}$. It gives the circuit shown in Fig. 7.21.

It is possible to reduce the number of literals in functions $D_r \in \Phi$ and $y_n \in Y_1$ by replacing the LCs $x_l \in X$ by additional variables $p_g \in P$. For example, it turns P_{C1}^2 CFSM into MP_{C1}^2 CFSM (Fig. 7.22).

The PALerP is represented by system (7.18), the PALerPhi by (7.19), the PALerY1 by (7.20). The EMBerF is represented by systems (6.4) and (6.8).

Table 7.9 Distribution of LCs for $MP_{C1}^2(I_{13})$

P	B_i						
	B_1	B_2	B_3	B_4	B_5	B_6	B_7
p_1	x_1	x_3	x_5	—	x_1	x_8	—
p_2	x_2	x_4	—	x_6	x_7	—	—
$K(B_i)$	000000	****01	****10	100*00	101*00	****11	*11000

There are the following sets $X(B_i)$ in the discussed case: $X(B_1) = \{x_1, x_2\}$, $X(B_2) = \{x_3, x_4\}$, $X(B_3) = \{x_5\}$, $X(B_4) = \{x_6\}$, $X(B_5) = \{x_1, x_7\}$, $X(B_6) = \{x_8\}$ and $X(B_7) = \emptyset$. Analysis of $X(B_i)$ gives $G = 2$, $P = \{p_1, p_2\}$. Let us create the table of RLC (Table 7.9).

The following system can be derived from Table 7.9:

$$\begin{aligned} p_1 &= B_1x_1 \vee B_2x_3 \vee B_3x_5 \vee B_5x_1 \vee B_6x_8; \\ p_2 &= B_1x_2 \vee B_2x_4 \vee B_4x_6 \vee B_5x_7. \end{aligned} \quad (7.40)$$

Replacing symbols B_i in (7.40) by conjunctions corresponding to $K(B_i)$ leads to the system (7.18). For example, there is $B_2x_3 = \bar{\tau}_1\tau_2x_3$, $B_5x_1 = T_1\bar{T}_2T_3\bar{\tau}_1\bar{\tau}_2x_1$, and so on.

To get the systems (7.19)–(7.20), it is necessary to construct the DST of MP_{C1}^2 CFSM. To do it in the discussed case, it is necessary to replace LCs in (7.34) by variables $p_g \in P$. Using this transformed system of GFT turns Table 7.7 into Table 7.10. Of course, both tables have the same amount of rows ($H_T = 16$).

Using Table 7.10 gives systems (7.19)–(7.20). For example, the system (7.35) turns into the following system:

$$\begin{aligned} D_1 &= \tau_1\bar{\tau}_2 \vee T_1\bar{T}_2\bar{\tau}_1\bar{\tau}_2 \vee \tau_1\tau_2\bar{p}_1; \\ y_1 &= \bar{T}_1\bar{T}_2\bar{T}_3\bar{T}_4\bar{\tau}_1\bar{\tau}_2p_1p_2 \vee T_1\bar{T}_2\bar{T}_3\bar{\tau}_1\bar{\tau}_2\bar{p}_2. \end{aligned} \quad (7.41)$$

There is no connection of variables $p_g \in P$ with functions $\tau_r \in \mathcal{T}$ and $y_n \in Y_2$. So, there is no change for EMBerF.

To diminish the number of PALs in PALerY1, it is possible to execute the ECMO $Y_q \in Y_1$. It leads to MP_{C1}^2Y1 CFSM (Fig. 7.23).

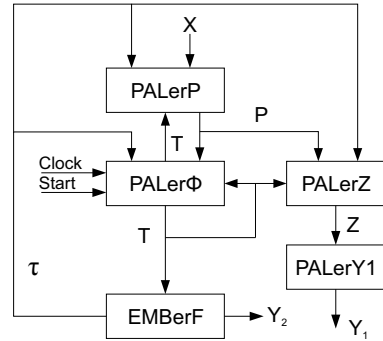
In MP_{C1}^2Y1 CFSM, the PALerZ implements the system (7.24), the PALerY1 the system (6.7). The functions of other blocks are the same as for MP_{C1}^2 CFSM. The system (6.7) includes regular functions. They can be implemented using EMBs.

Let the following condition take place:

$$2^{R+R_Q}(R_C + N) \leq V_0. \quad (7.42)$$

Table 7.10 DST of CSFM $MP_{C_2}^2(\Gamma_{13})$

B_i	$K(B_i)$		a_s	$K(a_s)$	P_h	Y_h	Φ_h	h
	T	τ						
B_1	0000	00	a_2	0001	$p_1 p_2$	$y_1 y_2$	D_4	1
			a_3	0011	$p_1 \bar{p}_2$	y_3	$D_3 D_4$	2
			a_4	0010	$\bar{p}_1$	$y_2 y_4$	D_3	3
B_2	****	01	a_5	0100	$p_1 p_2$	$y_3 y_4$	D_2	4
			a_6	0101	$p_1 \bar{p}_2$	—	$D_2 D_4$	5
			a_7	0111	$\bar{p}_1$	y_5	$D_2 D_3 D_4$	6
B_3	****	10	a_8	1000	p_1	y_5	D_1	7
			a_9	1001	$\bar{p}_1$	y_3	$D_1 D_4$	8
B_4	100*	00	a_{10}	1011	p_2	—	$D_1 D_3 D_4$	9
			a_{11}	1010	$\bar{p}_2$	$y_1 y_2$	$D_1 D_3$	10
B_5	101*	00	a_{12}	1100	$p_1 p_2$	$y_2 y_4$	$D_1 D_2$	11
			a_{13}	1101	$p_1 \bar{p}_2$	y_3	$D_1 D_2 D_4$	12
			a_{14}	1111	$\bar{p}_1$	—	$D_1 D_2 D_3 D_4$	13
B_6	****	11	a_{15}	0110	p_1	y_5	$D_2 D_3$	14
			a_{16}	1110	$\bar{p}_1$	$y_2 y_3$	$D_1 D_2 D_3$	15
B_7	*110	00	a_1	0000	1	—	—	16

Fig. 7.23 Structural diagram of $MP_{C_1}^2 Y1$ CFMS

In this case, the $EMBerF$ is implemented as a single EMB and functions $\tau_r \in \mathcal{T}$, $y_n \in Y_2$ are generated by this EMB. It leads to $MP_{C_1}^2 Y1$ CFMS with a shared EMB (Fig. 7.24).

Let a CPLD include only EMBs such that the following relations take places: 1) $S_A = R$ and 2) $t_F < R_C + N$. In this case, it is necessary n_R EMBs to implement the circuit of $EMBerF$. If there are no enough EMBs to implement functions $f_i \in \mathcal{T} \cup Y$, then some functions are implemented as $EMBerF$ and some as $PALerF$. It leads to $MP_{C_1}^{21} Y1$ CFMS (Fig. 7.25).

Fig. 7.24 Structural diagram of MP_{C1}^2Y1 CFSM with shared EMB

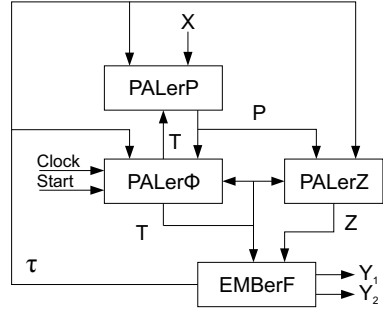
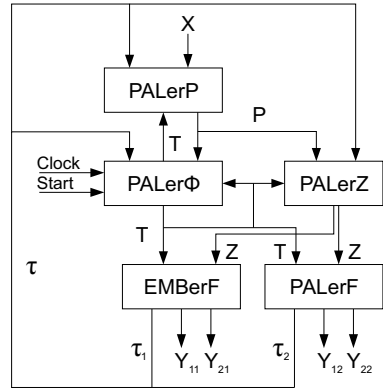


Fig. 7.25 Structural diagram of $MP_{C1}^{21}Y1$ CFSM



As follows from Fig. 7.25, there is a partition of each set $\mathcal{T}$, $Y1$ and $Y2$ by two subsets. Of course, there are no common elements in these subsets. The partition is executed using the method discussed before.

In this book, we propose the method of synthesis of $MP_{C1}^{21}Y1$ CFSM. It includes the following steps:

1. Marking the initial GSA Γ and finding the set A .
2. Finding the partition Π_A of the set A .
3. Executing the optimal state assignment.
4. Finding the sets Π_B and Π_C .
5. Executing the encoding of classes $B_i \in \Pi_C$.
6. Executing the RLC and finding the functions (7.18).
7. Constructing the CMOs $Y_q \subseteq Y1$ and their codes.
8. Constructing the DST of CFSM.
9. Finding the systems (7.19), (7.24) from the DST.
10. Constructing the systems $\mathcal{T}(T)$, $Y1(Z)$ and $Y2(T)$.
11. Partitioning the sets $\mathcal{T}$, $Y1$ and $Y2$.
12. Implementing CFSM circuit with PALs and EMBs.

Fig. 7.26 Codes of CMOs $Y_q \subseteq Y1$

		$z_1 z_2$			
z_3		00	01	11	10
	0	Y_1	Y_6	Y_5	Y_3
	1	Y_2	Y_4	*	Y_7

Table 7.11 The contents of the column Z_h

h	1	2	3	4	5	6	7	8
q	2	3	4	5	1	6	6	3
$K(Y_q)$	001	100	011	110	000	010	010	100
Z_h	z_3	z_1	$z_2 z_3$	$z_1 z_2$	—	z_2	z_2	z_1
h	9	10	11	12	13	14	15	16
q	1	2	4	3	1	6	7	1
$K(Y_q)$	000	001	011	100	000	010	101	000
Z_h	—	z_3	$z_2 z_3$	z_1	—	z_2	$z_1 z_3$	—

Let us discuss an example of synthesis for CFSM $MP_{C1}^{21}Y1(\Gamma_{13})$. Most of the steps have already been completed in previous examples. So, we should start this example with step 7.

There are $Q = 7$ CMOs $Y_q \subseteq Y1$ in the discussed case. They are the following: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_2\}$, $Y_3 = \{y_3\}$, $Y_4 = \{y_2, y_4\}$, $Y_5 = \{y_3, y_4\}$, $Y_6 = \{y_5\}$ and $Y_7 = \{y_2, y_3\}$. Using (1.32) gives $R_Q = 3$ and $Z = \{z_1, z_2, z_3\}$. Let us encode CMOs $Y_q \subseteq Y1$ as it is shown in Fig. 7.26.

The DST of $MP_{C1}^{21}Y1(\Gamma_{13})$ is practically the same as Table 7.10. But the column Y_h should be replaced by the column Z_h (Table 7.11).

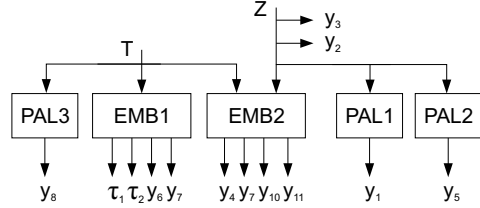
In Table 7.11 we show the way from Y_h to Z_h . For example, there is $Y_h = Y_4$ in the row $h = 3$. There is $K(Y_4) = 011$ (Fig. 7.26). It gives $z_1 = 0$, $z_2 = z_3 = 1$ in the row Z_h . In the same way, all other rows are filled for DST of CFSM $MP_{C1}^{21}Y1(\Gamma_{13})$.

The systems (7.19) are the same for $MP_{C1}^2Y1(\Gamma_{13})$ and $MP_{C1}^{21}Y1(\Gamma_{13})$. So, the equations for $D_1 \in \Phi$ are the same for these CFSMs. The system (7.24) is created using the same terms as the system (7.14). For example, the following equations can be derived from Tables 7.10 and 7.11:

$$\begin{aligned}
 z_1 &= F_2 \vee F_4 \vee F_8 \vee F_{12} \vee F_{15} = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{t}_1 \bar{t}_2 p_1 p_2 \\
 &\quad \vee \bar{t}_1 t_2 p_1 p_2 \vee \dots \vee t_1 t_2 \bar{p}_1; \\
 z_2 &= F_3 \vee F_4 \vee F_6 \vee F_7 \vee F_{11} \vee F_{14} = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{t}_1 \bar{t}_2 \bar{p}_1 \\
 &\quad \vee \bar{t}_1 t_2 p_1 p_2 \vee \dots \vee t_1 t_2 p_1; \\
 z_3 &= F_1 \vee F_3 \vee F_{10} \vee F_{11} \vee F_{15} = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{t}_1 \bar{t}_2 p_1 p_2 \\
 &\quad \vee \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{t}_1 t_2 \bar{p}_1 \vee \dots \vee t_1 t_2 \bar{p}_1.
 \end{aligned} \tag{7.43}$$

The systems $\mathcal{T}(T)$ and $Y2(T)$ are represented by (7.39). Let us find the system $Y1(Z)$. Using CMOs $Y_q \subseteq Y1$ and their codes (Fig. 7.26), we can find the minimized

Fig. 7.27 Implementing systems $\mathcal{T}$, $Y1$ and $Y2$ for CFSM $MP_{C1}^{21}Y1(\Gamma_{13})$



SOPs:

$$\begin{aligned}
 y_1 &= Z_2 = \bar{z}_1 \bar{z}_2 z_3; & y_2 &= Z_2 \vee Z_4 \vee Z_7 = z_3; \\
 y_3 &= Z_3 \vee Z_5 \vee Z_7 = z_1; & y_4 &= Z_4 \vee Z_5; \\
 y_5 &= Z_6 = \bar{z}_1 z_2 \bar{z}_3.
 \end{aligned} \tag{7.44}$$

The functions $y_2, y_3 \in Y1$ are represented by a single literal. So, they are generated by the PALerZ. It means that only MOs y_1, y_4, y_5, y_6 should be placed into the queue γ . There is $Q(y_1) = Q(y_5) = 1$ and $Q(y_4) = 2$.

Using $Q(f_i)$ from (7.39), we can create the following queue $\gamma = \langle \tau_1, \tau_2, y_6, y_9, y_{10}, y_4, y_7, y_{11}, y_1, y_5, y_8 \rangle$. Let we can use 2 EMBs with $S_A = R$ and $t_F = 4$. It is necessary n_R EMBs to implement functions, where:

$$n_R = \left\lceil \frac{R_C + N}{t_F} \right\rceil. \tag{7.45}$$

In the discussed case, we should use $N - 2 = 9$ in (7.42). It is connected with expressing two MOs by a single literal. So, we have $N_R = 3$, but there is $n_A = 2$. It means that $n_A \cdot t_F = 8$ functions from γ are implemented by EMBer. Obviously, $R_C + N - 2 - 8 = 3$ functions are implemented by PALs. It gives the circuit shown in Fig. 7.27.

Let us point out that other blocks of CFSMs may be implemented with EMBs. But we do not discuss these approaches in our book.

7.4 Combining Different Methods of Object Transformation

The transformation of state codes belongs to the group of methods of object transformation [4, 12]. It is possible to combine some methods from this group to achieve better result than for each method separately. Let us discuss the CFSMs with combining two methods: (1) the partial transformation of state codes into class codes and (2) the transformation of states into MOs $y_n \in Y1$. Let only PALs be used.

It leads to P_{C1A}^1 CFSM. Its structural diagram is shown in Fig. 7.28.

Fig. 7.28 Structural diagram of P_{C1A}^1 CFSM

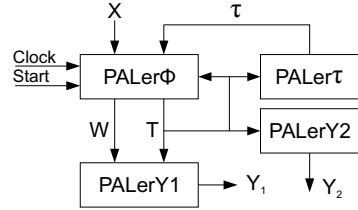
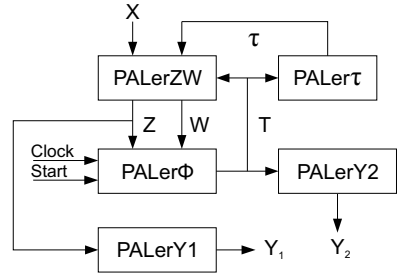


Fig. 7.29 Structural diagram of P_{C1Y1}^1 CFSM



The $\text{PALer}\Phi$ of P_{C1A}^1 CFSM implements the systems (7.13) and

$$W = W(T, \tau, X). \quad (7.46)$$

The variables $\omega_r \in W$ encode the identifiers $I_k \in SI$. Now, the PALerY1 generates functions

$$Y1 = Y1(T, W). \quad (7.47)$$

Other blocks generate the same functions as for P_{C1}^1 CFSM.

It is possible to transform the codes $K(Y_q)$ into state codes $K(a_m)$. In this case, it is necessary to represent the state code as (6.71). To do it, it is necessary:

1. To find the sets $A(Y_q)$ including states of transitions generated together with CMOs $Y_q \subseteq Y1$.
2. To find the set of identifiers SI using the expression (6.65).
3. To encode the identifiers using codes $K(I_k)$ having R_k bits, where R_k is determined by (6.66). The variables $\omega_r \in W$ encode the identifiers, where $|W| = R_k$.

This approach leads to P_{C1Y1}^1 CFSM. As follows from the superscript, only PAL-based MCs are used in its structural diagram shown in Fig. 7.29.

In P_{C1Y1}^1 CFSM, the PALerZW implements the systems (7.47) and

$$Z = Z(T, \tau, X). \quad (7.48)$$

The $\text{PALer}\Phi$ implements the system

$$\Phi = \Phi(Z, W). \quad (7.49)$$

Table 7.12 Correspondence between CMOs, identifiers and states for Γ_{13}

a_m	Y_q	I_k	a_m	Y_q	I_k	a_m	Y_q	I_k	a_m	Y_q	I_k
a_1	Y_1	I_1	a_5	Y_5	I_1	a_9	Y_3	I_2	a_{13}	Y_3	I_3
a_2	Y_2	I_1	a_6	Y_1	I_2	a_{10}	Y_1	I_3	a_{14}	Y_1	I_4
a_3	Y_3	I_1	a_7	Y_6	I_1	a_{11}	Y_2	I_2	a_{15}	Y_6	I_3
a_4	Y_4	I_1	a_8	Y_6	I_2	a_{12}	Y_4	I_2	a_{16}	Y_7	I_1

The PALerY1 implements the system (6.7) and PALer $\mathcal{T}$ the system (6.8).

Let us discuss an example of synthesis for CFSM $P_{C1Y1}^1 Y1(\Gamma_{13})$. Generally, the synthesis method includes the following steps:

1. Marking the initial GSA Γ and finding the set A .
2. Creating the partition $\Pi_A = \{B_1, \dots, B_I\}$.
3. Executing the optimal state assignment.
4. Creating the sets Π_B and Π_C based on OSA.
5. Creating the CMOs $Y_q \subseteq Y1$.
6. Encoding the CMOs $Y_q \subseteq Y1$ by codes $K(Y_q)$.
7. Representing the states $a_m \in A$ by pairs $\langle Y_q, I_k \rangle$.
8. Encoding the identifiers $I_k \in SI$.
9. Creating the DST of $P_{C1Y1}^1 Y1$ CFSM.
10. Creating the tables of PALer Φ , PALerY1, PALerY2 and PALer $\mathcal{T}$.
11. Implementing the CFSM logic circuit.

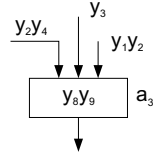
Some steps of synthesis have been already executed. There is $A = \{a_1, \dots, a_{16}\}$, $\Pi_A = \{B_1, \dots, B_7\}$, $\Pi_B = \{B_1, B_4, B_5, B_7\}$, $\Pi_C = \{B_2, B_3, B_6\}$. The outcome of OSA is represented by the Karnaugh map (Fig. 7.18). There are 7 CMOs $Y_q \subseteq Y1$: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_2\}$, $Y_3 = \{y_3\}$, $Y_4 = \{y_2, y_4\}$, $Y_5 = \{y_3, y_4\}$, $Y_6 = \{y_5\}$ and $Y_7 = \{y_2, y_3\}$. Let us encode them as it is shown in Fig. 7.26. There are the following codes: $K(Y_1) = 000$, $K(Y_2) = 001$, $K(Y_3) = 100$, $K(Y_4) = 011$, $K(Y_5) = 110$, $K(Y_6) = 010$ and $K(Y_7) = 101$. The codes of classes $B_i \in \Pi_B$ are the following: $K(B_1) = 0000$, $K(B_4) = 100*$, $K(B_5) = 101*$, $K(B_7) = *110$. The classes $B_i \in \Pi_C$ have the following codes: $K(B_2) = 01$, $K(B_3) = 10$ and $K(B_6) = 11$.

Let us construct the pairs $\langle Y_q, I_k \rangle$ for states $a_m \in A$. To do it, let us find the sets $A(Y_q)$. A set $A(Y_q)$ includes states $a_m \in A$ for transitions with the CMO $Y_q \subseteq Y1$. There are the following sets: $A(Y_1) = \{a_1, a_6, a_{10}, a_{14}\}$, $A(Y_2) = \{a_2, a_{11}\}$, $A(Y_3) = \{a_3, a_9, a_{13}\}$, $A(Y_4) = \{a_4, a_{12}\}$, $A(Y_5) = \{a_5\}$, $A(Y_6) = \{a_7, a_8, a_{15}\}$, $A(Y_7) = \{a_{16}\}$.

Using (6.65) gives $K = 4$ (it is the number of states into the set $A(Y1)$). Using (6.66) gives $R_k = Z$, $W = \{\omega_1, \omega_2\}$. So, there is the set $SI = \{I_1, \dots, I_4\}$. Let us create the table of correspondence between $Y_q \subseteq Y1$ and $a_m \in A$. It is Table 7.12.

As follows from Table 7.12, each state $a_m \in A$ is represented by a single pair $\langle Y_q, I_k \rangle$. But it is true only for a discussed example. Consider the fragment of GSA Γ_0 from Fig. 7.30.

There are three transitions into the state a_3 . All they are accompanied by different CMO $Y_q \subseteq Y1$. So, the state a_3 is represented by three pairs $\langle Y_q, I_k \rangle$.

Fig. 7.30 Fragment of GSA Γ_0 

Let us encode the identifiers $I_k \in SI$ in the trivial way: $K(I_1) = 00, \dots, K(I_4) = 11$. Now, it is possible to replace the codes $K(a_m)$ by codes $K(Y_q)$ and $K(I_k)$.

The DST of $P_{C1Y1}^1 Y1$ CFSM is represented by the system of GFT. It is the system (7.34) in the discussed case. Let us transform the system (7.34) replacing states $a_m \in A$ by pairs $\langle Y_q, I_k \rangle$ from Table 7.12. It gives the following system:

$$\begin{aligned}
 B_1 &\rightarrow x_1 x_2 \langle Y_2, I_1 \rangle \vee x_1 \bar{x}_2 \langle Y_3, I_1 \rangle \vee \bar{x}_1 \langle Y_4, I_1 \rangle; \\
 B_2 &\rightarrow x_3 x_4 \langle Y_5, I_1 \rangle \vee x_3 \bar{x}_4 \langle Y_1, I_2 \rangle \vee \bar{x}_3 \langle Y_6, I_1 \rangle; \\
 B_3 &\rightarrow x_5 \langle Y_6, I_2 \rangle \vee \bar{x}_5 \langle Y_3, I_2 \rangle; \\
 B_4 &\rightarrow x_6 \langle Y_1, I_3 \rangle \vee \bar{x}_6 \langle Y_2, I_2 \rangle; \\
 B_5 &\rightarrow x_1 x_7 \langle Y_4, I_2 \rangle \vee x_1 \bar{x}_7 \langle Y_3, I_3 \rangle \vee \bar{x}_1 \langle Y_1, I_4 \rangle; \\
 B_6 &\rightarrow x_8 \langle Y_6, I_3 \rangle \vee \bar{x}_8 \langle Y_7, I_1 \rangle; \\
 B_7 &\rightarrow \langle Y_1, I_1 \rangle.
 \end{aligned} \tag{7.50}$$

As follows from (7.50), there are the following columns in the DST of $P_{C1Y1}^1 Y1$ CFSM: $B_i, K(B_i), Y_q, K(Y_q), I_k, K(I_k), X_h, Z_h, W_h, h$. So, it is necessary to eliminate the columns $a_s, K(a_s), Y_h, \Phi_h$ from the DST of $P_{C1}^1 Y1$ CFSM. In the discussed case, the DST is represented by Table 7.13.

The DST is a base to derive the systems (7.46)–(7.47). For example, the following equations can be derived from Table 7.13:

$$\begin{aligned}
 \omega_1 &= F_9 \vee F_{12} \vee F_{13} \vee F_{14} = T_1 \bar{T}_2 \bar{T}_3 \bar{t}_1 \bar{t}_2 x_6 \vee \dots; \\
 z_1 &= F_2 \vee F_4 \vee F_8 \vee F_{12} \vee F_{15} = \bar{T}_1 \bar{T}_2 \bar{T}_3 \bar{T}_4 \bar{t}_1 \bar{t}_2 x_1 \bar{x}_2 \vee \dots
 \end{aligned} \tag{7.51}$$

The table of PALer Φ is constructed on the base of the table of correspondence. It has the following columns: $Y_q, K(Y_q), I_k, K(I_k), a_m, K(a_m), \Phi_m, m$. The column Φ_m contains the state variables $T_r \in T$ equal to 1 in the code $K(a_m)$. It is Table 7.14 in the discussed case. The following equations can be derived from Table 7.14:

$$\begin{aligned}
 D_1 &= F_8 \vee F_9 \vee \dots \vee F_{14} \vee F_{16} = \bar{z}_1 z_2 \bar{z}_3 \bar{\omega}_1 \bar{\omega}_2 \vee \dots \vee z_1 \bar{z}_2 z_3 \bar{\omega}_1 \bar{\omega}_2; \\
 D_2 &= F_5 \vee F_6 \vee F_7 \vee F_{12} \vee \dots \vee F_{16} = z_1 z_2 \bar{z}_3 \bar{\omega}_1 \bar{\omega}_2 \vee \dots \vee z_1 \bar{z}_2 z_3 \bar{\omega}_1 \bar{\omega}_2.
 \end{aligned} \tag{7.52}$$

The tables of PALer $Y1$, PALer $Y2$ and PALer T are constructed in the well-known way. We do not discuss them for the given example.

Let the following condition take place:

$$N1 + R_k \leq S. \tag{7.53}$$

Table 7.13 DST of CSFM $P_{C1Y1}^1 Y1(\Gamma_{13})$

B_i	$K(B_i)$		Y_q	$K(Y_q)$	I_k	$K(I_k)$	X_h	Z_h	W_h	h
	T	$\mathcal{T}$								
B_1	0000	00	Y_2	001	I_1	00	$x_1 x_2$	z_3	—	1
			Y_3	100	I_1	00	$x_1 \bar{x}_2$	z_1	—	2
			Y_4	011	I_1	00	$\bar{x}_1$	$z_2 z_3$	—	3
B_2	****	01	Y_5	110	I_1	00	$x_3 x_4$	$z_1 z_2$	—	4
			Y_1	000	I_2	01	$x_3 \bar{x}_4$	—	w_2	5
			Y_6	010	I_1	00	$\bar{x}_3$	z_2	—	6
B_3	****	10	Y_6	010	I_2	01	x_5	z_2	w_2	7
			Y_3	100	I_2	01	$\bar{x}_5$	z_1	w_2	8
B_4	100*	00	Y_1	000	I_3	10	x_6	—	w_1	9
			Y_2	001	I_2	01	$\bar{x}_6$	z_3	w_2	10
B_5	101*	00	Y_4	011	I_2	01	$x_1 x_7$	$z_2 z_3$	w_2	11
			Y_3	100	I_3	10	$x_1 \bar{x}_7$	z_1	w_1	12
			Y_{11}	000	I_4	11	$\bar{x}_1$	—	$w_1 w_2$	13
B_6	****	11	Y_6	010	I_3	10	x_8	z_2	w_1	14
			Y_7	101	I_1	00	$\bar{x}_8$	$z_1 z_3$	—	15
B_7	*110	00	Y_1	000	I_1	00	1	—	—	16

Table 7.14 Table of PALer Φ of CFSM $P_{C1Y1}^1 Y1(\Gamma_{13})$

Y_q	$K(Y_q)$	I_k	$K(I_k)$	a_m	$K(a_m)$	Φ_m	m
Y_1	000	I_1	00	a_1	0000	—	1
Y_2	001	I_1	00	a_2	0001	D_4	2
Y_3	100	I_1	00	a_3	0011	$D_3 D_4$	3
Y_4	011	I_1	00	a_4	0010	D_3	4
Y_5	110	I_1	00	a_5	0100	D_2	5
Y_1	000	I_2	01	a_6	0101	$D_2 D_4$	6
Y_6	010	I_1	00	a_7	0111	$D_2 D_3 D_4$	7
Y_6	010	I_2	01	a_8	1000	D_1	8
Y_3	100	I_2	01	a_9	1001	$D_1 D_4$	9
Y_1	000	I_3	10	a_{10}	1011	$D_1 D_3 D_4$	10
Y_2	001	I_2	01	a_{11}	1010	$D_2 D_3$	11
Y_4	011	I_2	01	a_{12}	1100	$D_1 D_2$	12
Y_3	100	I_3	10	a_{13}	1101	$D_1 D_2 D_4$	13
Y_1	000	I_4	11	a_{14}	1111	$D_1 D_2 D_3 D_4$	14
Y_6	010	I_3	10	a_{15}	0110	$D_2 D_3$	15
Y_7	101	I_1	00	a_{16}	1110	$D_1 D_2 D_3$	16

Fig. 7.31 Structural diagram of $P_{C1Y1}^1 Y1$ CFSM

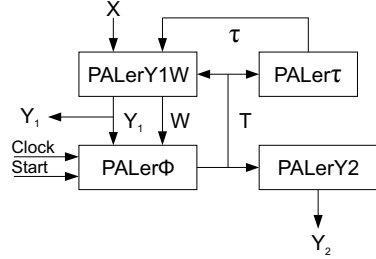


Table 7.15 Table of $PALer\Phi$ of CFSM $P_{C1Y1}^1(\Gamma_{13})$

Y_q	$K(Y_q)$	I_k	$K(I_k)$	a_m	$K(a_m)$	Φ_m	m
Y_1	00000	I_1	00	a_1	0000	—	1
Y_2	11000	I_1	00	a_2	0001	D_4	2
Y_3	00100	I_1	00	a_3	0011	$D_3 D_4$	3
Y_4	01010	I_1	00	a_4	0010	D_3	4
Y_5	00110	I_1	00	a_5	0100	D_2	5
Y_1	00000	I_2	01	a_6	0101	$D_2 D_4$	6

In this case, it is possible to use the one-hot codes of CMOs $Y_q \subseteq Y1$. In this case, the functions $D_r \in \Phi$ are represented by the system

$$\Phi = \Phi(Y1, W). \quad (7.54)$$

It leads to $P_{C1Y1}^1 Y1$ CFSM (Fig. 7.31). Using one-hot codes of CMOs $Y_q \subseteq Y1$ allows eliminating the $PALerY1W$ transforming $K(Y_q)$ into $y_n \subseteq Y_q$. The $PALer2W$ is replaced by the $PALerY1W$.

The methods are practically the same for synthesis of $P_{C1Y1}^1 Y1$ and P_{C1Y1}^1 CFSMs. But there is no step 6 in the second case. The one-hot codes $K(Y_q)$ are generated automatically. They contain $N1$ bits. In the case of (Γ_{13}) , the codes correspond to vectors $\langle y_1, y_2, \dots, y_5 \rangle$. For example, $Y_2 = \{y_1, y_2\}$ and $K(Y_2) = 11000$, $K(Y_3) = 00100$ because $Y_3 = \{y_3\}$ and so on.

To get the system (7.54), it is necessary to replace the binary codes of CMOs by one-hot codes. For example, Table 7.15 represents 6 first rows of the table of $PALer\Phi$ of CFSM $P_{C1Y1}^1(\Gamma_{13})$.

For example, the following part of SOP for D_2 can be derived from Table 7.15:

$$D_2 = F_5 \vee F_6 \vee \dots = \bar{y}_1 \bar{y}_2 y_3 y_4 \bar{y}_5 \bar{\omega}_1 \bar{\omega}_2 \vee \bar{y}_1 \bar{y}_2 \bar{y}_3 \bar{y}_4 \bar{y}_5 \bar{\omega}_1 \omega_2 \vee \dots \quad (7.55)$$

It is possible to execute the RLC to diminish the number of arguments in functions $Z, Y1, W$. It turns $P_{C1Y1}^1 Y1$ CFSM into $MP_{C1Y1}^1 Y1$ CFSM (Fig. 7.32) and P_{C1Y1}^1 CFSM into MP_{C1Y1}^1 CFSM (Fig. 7.33).

Fig. 7.32 Structural diagram of $MP_{C1Y1}^1 Y1$ CFSM

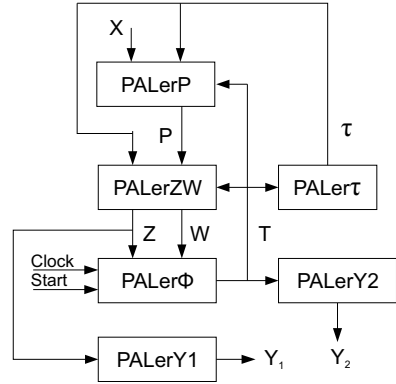
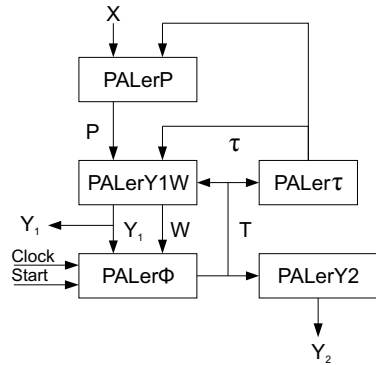


Fig. 7.33 Structural diagram of MP_{C1Y1}^1 CFSM



Obviously, all these structural diagrams can be turned into PLA-based CFSMs. We hope that it is a trivial task for our reader.

The transformation of object codes can be used in the case of heterogenous CPLDs. Let the following conditions take places:

$$2^{R_Q + R_k} \cdot R \leq V_0; \quad (7.56)$$

$$2^{R_Q + R} \cdot (N + R_A) \leq V_0. \quad (7.57)$$

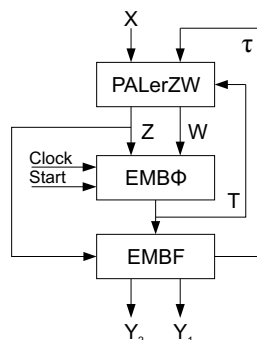
In this case, the $PALer\Phi$ in replaced by an EMB. Also, the $PALerY1$, $PALerY2$ and $PALerT$ can be replaced by a single EMB. It turns the $P_{C1Y1}^1 Y1$ CFSM (Fig. 7.29) into a $P_{C1Y1}^2 Y1$ CFSM (Fig. 7.34).

The method of synthesis of $P_{C1Y1}^2 Y1$ CFSM is practically the same as for $P_{C1Y1}^1 Y1$ CFSM. But the step 10 is the following:

10. Creating the tables of $EMB\Phi$ and $EMB F$.

So, there are a huge number of different structures of CPLD-based CFSMs. Naturally, this leads to need for formalization of the process of choosing a structure with

Fig. 7.34 Structural diagram of $P_{C1Y1}^2 Y1$ CFSM



optimal parameters. Unfortunately, there are no such methods, yet. The rightness of the choice depends on the experience of a designer. The same is true for FPGA-based CFSMs.

References

1. Altera. <http://www.altera.com>. Accessed: Jan 2020
2. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
3. Barkalov A (1998) Principles of logic optimization for Moore microprogram automaton. *Cybern Syst Anal* 34(1):54–60
4. Barkalov A, Barkalov A Jr (2005) Design of mealy finite-state machines with the transformation of object codes. *Int J Appl Math Comput Sci* 15(1):151–158
5. Barkalov A, Titarenko L, Chmielewski S (2007) Reduction in the number of PAL macrocells in the circuit of a Moore FSM. *Int J Appl Math Comput Sci* 17(4):565–675
6. Barkalov A, Titarenko L, Chmielewski S (2008) Optimization of Moore FSM on system-on-chip using PAL technology. In: *Proceedings of the international conference TCSET 2008*, pp 314–317, Lviv-Slavsko, Ukraine. Ministry of Education and Science of Ukraine, Lviv Polytechnic National University, Lviv, Publishing House of Lviv Polytechnic
7. Barkalov A, Węgrzyn M (2006) Design of control units with programmable logic. University of Zielona Góra Press
8. Klimovich AS, Solov'ev VV (2011) A method for minimizing Moore finite-state machines by merging two states. *J Comput Syst Sci Int* 50(6):907–920
9. Skliarov V (1984) Synthesis of automata with matrix LSIs. Nauka i Technika, Minsk (in Russian)
10. Skliarova I, Sklyarov V, Sudnitson A (2012) Design of FPGA-based circuits using hierarchical finite state machines. TUT Press, Tallinn
11. Sklyarov V, Skliarova I, Barkalov A, Titarenko L (2014) Synthesis and optimization of FPGA-based systems, vol 294. Lecture notes in electrical engineering. Springer, Berlin
12. Wolf W (2004) FPGA-based system design. Prentice Hall PTR, Upper Saddle River, NJ, USA
13. Xilinx. <http://www.xilinx.com>. Accessed: Jan 2020

Chapter 8

FPGA-Based Synthesis of CFSMs



8.1 Preliminary Information

Modern FPGA chips include look-up table (LUT) elements and embedded memory blocks (EMB) [2, 14]. To implement an CFSM logic circuit, it is necessary to find the SBFs (1.4), (1.42) and (1.43). Next, these functions are distributed between LUTs and EMBs.

There are S_L inputs in a LUT. So, a LUT can implement an arbitrary Boolean function f_i having $NA(f_i)$ arguments if the following condition takes place:

$$NA(f_i) \leq S_L. \quad (8.1)$$

If (8.1) is violated, a function should be decomposed and represented as a composition/indexcomposition of subfunctions satisfying to (8.1). To do it in the optimal way, the sophisticated methods of functional decomposition are used [10, 11]. We do not discuss these methods in our book.

An EMB is characterised by its configuration $\langle S_A, t_F \rangle$, where S_A is a number of address inputs and t_F is a number of outputs per a memory cell. Changing a configuration does not change the size V_0 of EMB. It is determined by (2.29). The typical configurations of modern EMBs are the following [2, 14]: $\langle 15, 1 \rangle$, $\langle 14, 2 \rangle$, $\langle 13, 4 \rangle$, $\langle 12, 8 \rangle$, $\langle 11, 16 \rangle$, $\langle 10, 32 \rangle$ and $\langle 9, 64 \rangle$. So, the parameters S_A and t_F can be adjusted to the parameters L , R , $N1$ and $N2$.

There are two trivial models of FPGA-based CFSMs. These models are homogeneous P CFSMs. In the first case, three LUTers are used (Fig. 8.1)

Obviously, the LUTer Φ implements the system (1.4). The outputs of LUTs corresponding to functions $D_r \in \Phi$ are connected with flip-flops. The LUTerY1 implements the system (1.42), the LUTerY2 the system (1.43).

The trivial EMB-based P CFSM is shown in Fig. 8.2. This circuit includes three EMBers. Of course, the EMBer Φ and EMBerY2 can be combined.

Fig. 8.1 LUT-based structural diagram of P CFSM

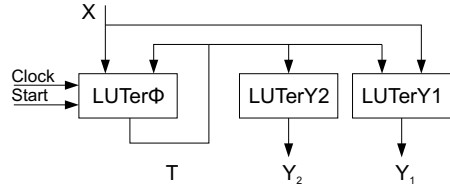


Fig. 8.2 EMB-based structural diagram of P CFSM

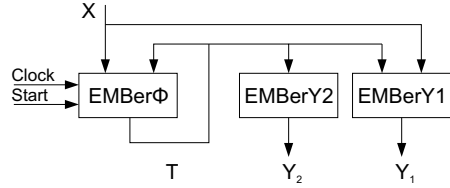
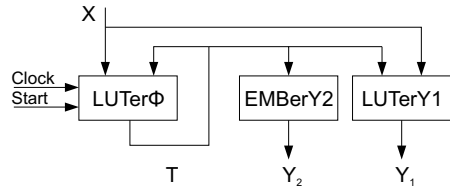


Fig. 8.3 Structural diagram of P^2 CFSM



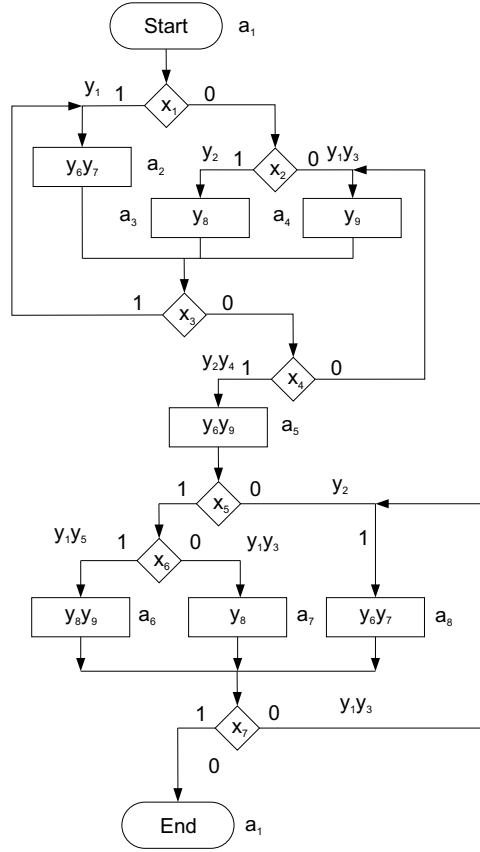
As a rule, a circuit of EMB-based CFSM has fewer blocks and interconnects than the circuit of equivalent LUT-based CFSM [4, 5]. As it is known, the fewer elements and interconnections the circuit has, the lower its power consumption [7]. Therefore, it is more preferable to use EMBs in CFSM design. But on the other hand, EMBs are intensively used for implementing different parts of a digital system controlled by a CFSM [12]. Due to it, the designers try to use as many EMBs as possible, but with their shortage, the only alternative is a heterogeneous circuits. This short analysis explains why we further consider only heterogeneous FPGA-based CFSMs.

Let us start from P CFSM for which the following conditions are true:

$$2^{L+R} > V_0; \quad (8.2)$$

$$2^R \leq V_0. \quad (8.3)$$

The condition (8.2) shows that SBFs (1.4) and (1.42) cannot be implemented as EMBs. They should be implemented using LUTerΦ and LUTerY1. The condition (8.3) shows that EMBs can be used to implement the system (1.43). It leads to P^2 CFSM (Fig. 8.3) where the superscript “2” means that the circuit is heterogeneous.

Fig. 8.4 Marked GSA Γ_{14} 

Obviously, there are the same steps in synthesis of CPLD- and FPGA-based P^2 CFSMs. We will not repeat them again. Let us just consider an example of synthesis for CFSM $P^2(\Gamma_{14})$. The marked GSA Γ_{14} is shown in Fig. 8.4.

The following sets and their cardinality numbers follow from GSA Γ_{14} : $A = \{a_1, \dots, a_8\}$, $M = 8$, $X = \{x_1, \dots, x_7\}$, $L = 7$, $Y1 = \{y_1, \dots, y_5\}$, $N1 = 5$, $Y2 = \{y_6, \dots, y_9\}$, $N2 = 4$ and $N = 9$. Using (1.2) gives $R = 3$, $T = \{T_1, T_2, T_3\}$ and $\Phi = \{D_1, D_2, D_3\}$.

Let us encode states $a_m \in A$ in the trivial way: $K(a_1) = 000, \dots, K(a_8) = 111$. Now, it is possible to construct the DST of CFSM $P^2(\Gamma_{14})$. It is constructed using the following system of formulae of transitions:

$$\begin{aligned}
 a_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 x_2 a_3 \vee \bar{x}_1 \bar{x}_2 a_4; \\
 a_2, a_3, a_4 &\rightarrow x_3 a_2 \vee \bar{x}_3 x_4 a_5 \vee \bar{x}_3 \bar{x}_4 a_4; \\
 a_5 &\rightarrow x_5 x_6 a_6 \vee x_5 \bar{x}_6 a_7 \vee \bar{x}_5 a_8; \\
 a_6, a_7, a_8 &\rightarrow x_7 a_1 \vee \bar{x}_7 a_8.
 \end{aligned} \tag{8.4}$$

Table 8.1 DST of CFSM $P^2(\Gamma_{14})$

a_m	$K(a_m)$	$Y(a_m)$	a_s	$K(a_s)$	X_h	Y_h	Φ_h	h
a_1	000	—	a_2	001	x_1	y_1	D_3	1
			a_3	010	$\bar{x}_1 x_2$	y_2	D_2	2
			a_4	011	$\bar{x}_1 \bar{x}_2$	$y_1 y_3$	$D_2 D_3$	3
a_2	001	$y_6 y_7$	a_2	001	x_3	—	D_3	4
			a_5	100	$\bar{x}_3 x_4$	$y_2 y_4$	D_1	5
			a_4	011	$\bar{x}_3 \bar{x}_4$	$y_1 y_3$	$D_2 D_3$	6
a_3	010	y_8	a_2	001	x_3	—	D_3	7
			a_5	100	$\bar{x}_3 x_4$	$y_2 y_4$	D_1	8
			a_4	011	$\bar{x}_3 \bar{x}_4$	$y_1 y_3$	$D_2 D_3$	9
a_4	011	y_9	a_2	001	x_3	—	D_3	10
			a_5	100	$\bar{x}_3 x_4$	$y_2 y_4$	D_1	11
			a_4	011	$\bar{x}_3 \bar{x}_4$	$y_1 y_3$	$D_2 D_3$	12
a_5	100	$y_6 y_9$	a_6	101	$x_5 x_6$	$y_1 y_5$	$D_1 D_3$	13
			a_7	110	$x_5 \bar{x}_6$	$y_1 y_3$	$D_1 D_2$	14
			a_8	111	$\bar{x}_5$	y_2	$D_1 D_2 D_3$	15
a_6	101	$y_8 y_9$	a_1	000	x_7	—	—	16
			a_8	111	$\bar{x}_7$	$y_1 y_5$	$D_1 D_2 D_3$	17
a_7	110	y_8	a_1	000	x_7	—	—	18
			a_8	111	$\bar{x}_7$	$y_1 y_5$	$D_1 D_2 D_3$	19
a_8	111	$y_6 y_7$	a_1	000	x_7	—	—	20
			a_8	111	$\bar{x}_7$	$y_1 y_5$	$D_1 D_2 D_3$	21

Obviously, states $a_2, a_3, a_4 \in A$ have the same formulae of transitions. Due to it, we combine them together. The same is true for states $a_6, a_7, a_8 \in A$. The system (8.4) determines the DST (Table 8.1).

Using Table 8.1, we can construct functions for all blocks of P^2 CFSM. For example, the following function can be derived:

$$y_5 = F_{13} \vee F_{17} \vee F_{19} \vee F_{21} = T_1 \bar{T}_2 \bar{T}_3 x_5 x_6 \vee T_1 \bar{T}_2 T_3 \bar{x}_7 \vee T_1 T_2 \bar{T}_3 \bar{x}_7 \vee T_1 T_2 T_3 \bar{x}_7. \quad (8.5)$$

There is $NA(y_5) = 6$.

Let it be LUTs with $S_L = 5$ in an FPGA chip. It means that the condition (8.1) is violated. To implement the circuit for $y_5 \in Y1$, it is necessary to apply the methods of functional decomposition [11]. It gives the following SOP:

$$y_5 = T_1 (\bar{T}_2 \bar{T}_3 x_5 x_6 \vee \bar{T}_2 T_3 \bar{x}_7 \vee T_2 \bar{T}_3 \bar{x}_7 \vee T_2 T_3 \bar{x}_7) = T_1 \cdot y_5^1. \quad (8.6)$$

Using (8.6) gives us the circuit implementing the function y_5 . It includes two LUTs (Fig. 8.5).

Fig. 8.5 LUT-based implementation of function y_5

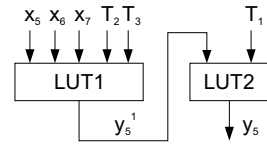


Table 8.2 A part of truth table for LUT1

T_2	T_3	x_5	x_6	x_7	y_5^1	h	T_2	T_3	x_5	x_6	x_7	y_5^1	h
0	0	0	0	0	0	1	0	1	0	0	0	1	9
0	0	0	0	1	0	2	0	1	0	0	1	0	10
0	0	0	1	0	0	3	0	1	0	1	0	1	11
0	0	0	1	1	0	4	0	1	0	1	1	0	12
0	0	1	0	0	0	5	0	1	1	0	0	1	13
0	0	1	0	1	0	6	0	1	1	0	1	0	14
0	0	1	1	0	1	7	0	1	1	1	0	1	15
0	0	1	1	1	1	8	0	1	1	1	1	0	16

Table 8.3 Table of EMBerY2 for CFSM $P^2(\Gamma_{14})$

T_1	T_2	T_3	y_6	y_7	y_8	y_9	m	T_1	T_2	T_3	y_6	y_7	y_8	y_9	m
0	0	0	0	0	0	0	1	1	0	0	1	0	0	1	5
0	0	1	1	1	0	0	2	1	0	1	0	0	1	1	6
0	1	0	0	0	1	0	3	1	1	0	0	0	1	0	7
0	1	1	0	0	0	1	4	1	1	1	1	1	0	0	8

To implement this circuit, it is necessary to create truth tables for LUT1 and LUT2. We can show 16 first rows of truth table for LUT1 (Table 8.2).

The rows 7 and 8 of Table 8.2 correspond to the conjunction $\bar{T}_2\bar{T}_3x_5x_6$ where $y_5^1 = 1$. Because of it, there are zeros in the column y_5^1 for rows 1–6. The rows 9–16 correspond to $\bar{T}_2T_3$. If $x_7 = 0$, then $y_5^1 = 1$ for this part of Table 8.2. We hope that everything is clear for our reader (Fig. 8.5).

The system (1.43) is constructed using three first columns of DST. It is represented by Table 8.3.

The system (1.43) is implemented with EMBs. So, there is no need in minimizing its equations. Let it be the configuration (3, 4) for EMBs of a particular chip. In this case, the system (1.43) is implemented by a single EMB.

In fact, both Table 8.2 and Table 8.3 contain “bit-streams” for programming LUT1 and EMB. To get the circuit of CFSM as a part of FPGA chip, it is necessary: (1) to create bit-streams for each logic element; (2) to create bit-streams for the matrix of programmable interconnections and (3) to write this data into the memory of configuration [8, 9].

Fig. 8.6 Structural diagram of MP^2 CFSM

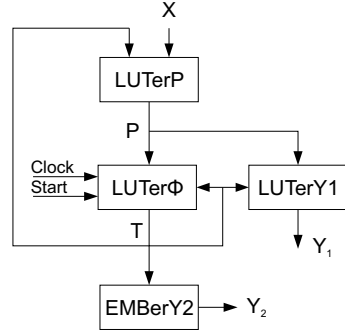


Table 8.4 Table of RLC for CFSM $MP^2(\Gamma_{14})$

P	A							
	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
p_1	x_1	x_3	x_3	x_3	x_5	x_7	x_7	x_7
p_2	x_2	x_4	x_4	x_4	x_6	—	—	—

As shown in [7, 13], there is no sense in increasing the number of inputs for LUTs. The value $S_L = 6$ provides a reasonable balance between design flexibility, number of LUTs per chip and power consumption. In this regard, the only way to optimize the characteristics of CFSM circuit is to reduce the number of literals in $SBF D_r \in \Phi$ and $y_n \in Y1$. The same is true for functions $y_n \in Y2$. But these functions have not more than R literals.

As it is known, the RLC is a very effective way for reducing the number of literals in $SBF D_r \in \Phi$ and $y_n \in Y1$. It can be applied for P^2 CFSM. It leads to MP^2 CFSM (Fig. 8.6).

In MP^2 CFSM, the LUTer P implements the system (6.1), the LUTer Φ the system (6.12) and the LUTer $Y1$ the system (6.18). The EMBer $Y2$ implements the system (6.4).

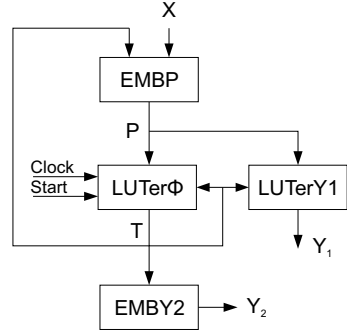
Let us discuss an example of synthesis for CFSM $MP^2(\Gamma_{14})$. Let us use LUTs with $S_L = 5$ and EMB with $S_A = 3$ and $t_F = 4$. The steps of synthesis are already known from previous chapters.

To create the set $P = \{p_1, \dots, p_G\}$, let us analyze the sets $X(a_m)$. We can use, for example, Table 8.1 to find these sets. As follows from Table 8.1, there are the following sets: $X(a_1) = \{x_1, x_2\}$, $X(a_2) = X(a_3) = X(a_4) = \{x_3, x_4\}$, $X(a_5) = \{x_5, x_6\}$, $X(a_6) = X(a_7) = X(a_8) = \{x_7\}$. So, there is $G = 2$, which gives the set $P = \{p_1, p_2\}$.

The RLC is executed in the well-known way. It leads to Table 8.4.

The table of RLC is a base to get the system (6.1). It is the following system in the discussed case:

Fig. 8.7 Structural diagram of MP^2 CFSM with two EMBs



$$\begin{aligned} p_1 &= A_1x_1 \vee [A_2 \vee A_3 \vee A_4]x_3 \vee A_5x_5 \vee [A_6 \vee A_7 \vee A_8]x_7; \\ p_2 &= A_1x_2 \vee [A_2 \vee A_3 \vee A_4]x_4 \vee A_5x_6. \end{aligned} \quad (8.7)$$

The column P_h replaces the column X_h in DST of MP^2 CFSM. To get the DST of CFSM $MP^2(\Gamma_{14})$, it is necessary to replace X_h of Table 8.1 by P_h . In this case, the SOP (8.5) is transformed into the following SOP:

$$y_5 = T_1\bar{T}_2\bar{T}_3p_1p_2 \vee T_1\bar{T}_2T_3\bar{p}_1 \vee T_1T_2T_3\bar{p}_1. \quad (8.8)$$

Now, there is $NA(y_5) = 5 = S_L$. Obviously, each function $D_r \in \Phi$ and $y_n \in Y_1$ has not more than 5 literals. So, there are $R = 3$ LUTs in LUTer Φ and $N1 = 5$ LUTs in LUTer Y_1 .

There is $NA(p_1) = 7$ and $NA(p_2) = 6$. So, these functions should be decomposed. We can show that: (1) there are 3 LUTs in circuit for p_1 and (2) there are two LUTs for implementing the circuit for p_2 . Both circuits have two levels of LUTs.

Let us discuss the following situation. There is an EMB such that: (1) the condition (8.2) takes place and (2) the system $P(T, X)$ can be implemented using a single EMB. It corresponds to condition

$$2^{R+L} \cdot G = V_0. \quad (8.9)$$

It leads to CFSM, where system (6.1) is implemented as EMBP (Fig. 8.7). We assume that only a single EMB is enough to implement the system $Y2(T)$.

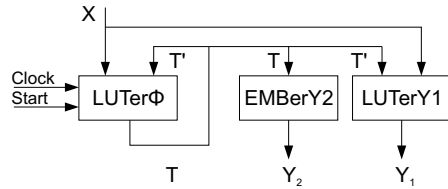
This circuit has 2 EMBs and $N1 + R$ LUTs. In the case of $MP^2(\Gamma_{14})$, there is $N1 + R = 8$. So, each function is implemented by a single-level circuit. It is the fastest solution for the existing combination of parameters $L, R, N1, N2$ and S_L, S_A, t_F . Let us discuss how to get the table of EMBP. It has inputs $x_l \in X_B$ and $T_r \in T$, variables $p_g \in P$ are the outputs.

Generally, there are H_P rows in the table of EMBP:

$$H_P = 2^{L+R}. \quad (8.10)$$

Table 8.5 A part of table of EMB for CFSM $MP^2(\Gamma_{14})$

Inputs										Outputs		h
T_1	T_2	T_3	x_7	x_6	x_5	x_4	x_3	x_2	x_1	p_1	p_2	
0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	1	1	0	2
0	0	0	0	0	0	0	0	1	0	0	1	3
0	0	0	0	0	0	0	0	1	1	1	1	4
0	0	0	0	0	0	0	1	0	0	0	0	5
0	0	0	0	0	0	0	1	0	1	1	0	6
0	0	0	0	0	0	0	1	1	0	0	1	7
0	0	0	0	0	0	0	1	1	1	1	1	8

Fig. 8.8 Structural diagram of P_0^2 CFSM

So, there are 1024 rows in the table of EMBP for the discussed example. Table 8.5 shows 8 first rows of this table.

There is $K(a_1)$ into these 8 rows. So, there is $p_1 = x_1$ and $p_2 = x_2$ for these rows. It explains the information from columns p_1, p_2 of Table 8.5. Acting in the same way, we can fill the remaining lines (1016 lines) of this table. But let us leave this task for the competent CAD tools.

8.2 Primary Encoding of Classes of PES for FPGA-Based GFSMs

There is a rather small amount of inputs per a LUT ($S_L \leq 6$) [7]. It means that optimization of logic circuit assumes reducing the number of arguments in Boolean functions to be implemented. This goal can be achieved by the using the classes of PES [12]. Let us start from the OSA. It leads to P_0^2 CFSM (Fig. 8.8).

In the best case, the functions $D_r \in \Phi$ and $y_n \in Y1$ depend on state variables $T_r \in T'$, where $T' \subset T$. So, there is $|T'| < R$. There in no change for functions $y_n \in Y2$. In general case, they depend on state variables.

In P_0^2 CFSM, the LUTerPhi implements the system (6.10), the LUTerY1 the system (6.16). The EMBerY2 implements the system (6.4).

Let us discuss an example of synthesis for CFSM $P_0^2(\Gamma_{14})$. There is the partition $\Pi_A = \{B_1, \dots, B_4\}$ with the classes $B_1 = \{a_1\}$, $B_2 = \{a_1, a_2, a_3\}$, $B_3 = \{a_5\}$ and

Fig. 8.9 Outcome of OSA
for $P_0^2 Y_2(\Gamma_{14})$

		$T_1 T_2$			
		00	01	11	10
T_3	0	a_1	a_2	a_8	a_6
	1	a_5	a_3	a_4	a_7

$B_4 = \{a_6, a_7, a_8\}$. Let us execute the OSA for $P_0^2(\Gamma_{14})$. Using the approach [1] gives the state codes shown in Fig. 8.9.

As follows from the Karnaugh map (Fig. 8.9), the classes B_2 and B_4 are represented by two generalized intervals. So, we can represent B_2 as B_2^1 and B_2^2 , the class B_4 by is represented as B_4^1 and B_4^2 . Let $B_2^1 = \{a_2, a_3\}$ and $B_2^2 = \{a_3, a_4\}$, $B_4^1 = \{a_6, a_7\}$ and $B_4^2 = \{a_6, a_8\}$.

The following codes $K(B_i)$ follow from Fig. 8.9: $K(B_1) = 000$, $K(B_2^1) = 01*$, $K(B_2^2) = *11$, $K(B_3) = 001$, $K(B_4^1) = 10*$ and $K(B_4^2) = 1*0$. To construct the DST of CFSM $P_0^2(\Gamma_{14})$, it is necessary to construct the system of GFT. To do it, we should replace states $a_m \in A$ by corresponding classes of $\Pi_A = \{B_1, B_2^1, B_2^2, B_3, B_4^1, B_4^2\}$. It gives the following system:

$$\begin{aligned}
 B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 x_2 a_3 \vee \bar{x}_1 \bar{x}_2 a_4; \\
 B_2^1, B_2^2 &\rightarrow x_3 a_2 \vee \bar{x}_3 x_4 a_5 \vee \bar{x}_3 \bar{x}_4 a_4; \\
 B_3 &\rightarrow x_5 x_6 a_6 \vee x_5 \bar{x}_6 a_7 \vee \bar{x}_5 a_8; \\
 B_4^1, B_4^2 &\rightarrow x_7 a_1 \vee \bar{x}_7 a_8.
 \end{aligned} \tag{8.11}$$

Using (8.11), the tables of LUTer Φ and LUTerY1 can be constructed. They can be extracted from DST of P_0^2 CFSM. It is Table 8.6 in the discussed case. To be able to construct the system (6.4), we replaced the column $Y(a_m)$ from Table 8.1 by the column $Y(a_s)$.

There is $H_{opt} = 16 < H = 21$. But there is $H_{opt} > H_T$. Using (4.18), we can find $H_T = 11$. There are no insignificant input assignments for state variables $T_r \in T$. Due to it, it is impossible to reduce the number of rows in DST of $P_0^2(\Gamma_{14})$ up to 11. To reach this value, we should apply other PES-based methods.

Using Table 8.6, we can derive functions (6.10) and (6.16). For example, the following equation can be derived:

$$y_5 = F_{10} \vee F_{14} \vee F_{16} = \bar{T}_1 \bar{T}_2 T_3 x_5 x_6 \vee T_1 T_2 \bar{x}_7 \vee T_1 \bar{T}_3 \bar{x}_7. \tag{8.12}$$

There is $NA(y_5) = 6$. So, there is no gain in the number of LUTs with $S_L = 5$ for implementing (8.12) as compared to (8.5).

The table of LUTer Φ is represented by columns $B_i - K(a_s)$, X_h , Φ_h , h of DST. The table of LUTerY1 is represented by the columns $B_i - K(a_s)$, X_h , Y_h , h of DST.

To optimize the circuit of P_0^2 CFSM, it is possible:

1. To execute the RLC. It leads to MP_0^2 CFSM shown in Fig. 8.10.
2. To execute the ECMO $Y_q \subseteq Y_1$. It leads to $P_0^2 Y_1$ CFSM (Fig. 8.11).

Table 8.6 DST of CFSM $P_0^2(\Gamma_{14})$

B_i	$K(B_i)$	a_s	$K(a_s)$	$Y(a_s)$	X_h	Y_h	Φ_h	h
B_1	000	a_2	010	y_6y_7	x_1	y_1	D_2	1
		a_3	011	y_8	$\bar{x}_1x_2$	y_2	D_2D_3	2
		a_4	111	y_9	$\bar{x}_1\bar{x}_2$	y_1y_3	$D_1D_2D_3$	3
B_2^1	01*	a_2	010	y_6y_7	x_3	—	D_2	4
		a_5	001	y_6y_9	$\bar{x}_3x_4$	y_2y_4	D_3	5
		a_4	111	y_9	$\bar{x}_3\bar{x}_4$	y_1y_3	$D_1D_2D_3$	6
B_2^2	*11	a_2	010	y_6y_7	x_3	—	D_2	7
		a_5	001	y_6y_9	$\bar{x}_3x_4$	y_2y_4	D_3	8
		a_4	111	y_9	$\bar{x}_3\bar{x}_4$	y_1y_3	$D_1D_2D_3$	9
B_3	001	a_6	100	y_8y_9	x_5x_6	y_1y_5	D_1	10
		a_7	101	y_8	$x_5\bar{x}_6$	y_1y_3	D_1D_3	11
		a_8	110	y_6y_7	$\bar{x}_5$	y_2	D_1D_2	12
B_4^1	10*	a_1	000	—	x_7	—	—	13
		a_8	110	y_6y_7	$\bar{x}_7$	y_1y_5	D_1D_2	14
B_4^2	1*0	a_1	000	—	x_7	—	—	15
		a_8	110	y_6y_7	$\bar{x}_7$	y_1y_5	D_1D_2	16

Fig. 8.10 Structural diagram of MP_0^2 CFSM

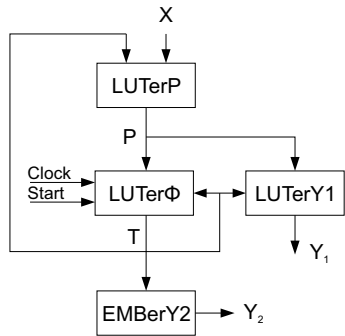


Fig. 8.11 Structural diagram of $P_0^2Y_1$ CFSM

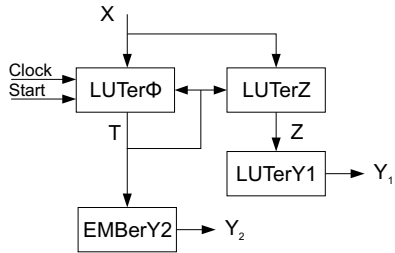


Fig. 8.12 Structural diagram of P_0^2 CFSM with a shared EMB

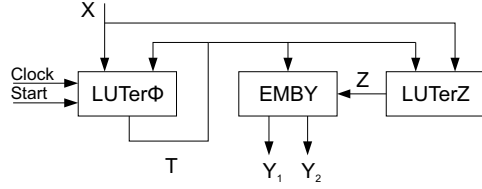


Table 8.7 A part of table of EMBY for CFSM $P^2(\Gamma_{14})$

$K(a_m)$			$K(Y_q)$			Y1					Y2					h
T_1	T_2	T_3	z_1	z_2	z_3	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	2
0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	3
0	0	0	0	1	1	1	0	1	0	0	0	0	0	0	0	4
0	0	0	1	0	0	0	1	0	1	0	0	0	0	0	0	5
0	0	0	1	0	1	1	0	0	0	1	0	0	0	0	0	6
0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	7
0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	8

If the condition (6.47) takes place, then a single EMBY implements both systems $Y1(Z)$ and $Y2(T)$. It leads to $P_0^2 Y1$ CFSM with a shared EMB.

To implement the circuit of EMBY, it is necessary to find a corresponding bit-stream. To find it, it is necessary: (1) to construct the CMOs $Y_q \subseteq Y1$, (2) to encode them by binary codes $K(Y_q)$ and (3) to construct the table of EMBY. To find the system $Z(T, X)$, it is necessary to construct the DST of $P_0^2 Y1$ CFSM (Fig. 8.11).

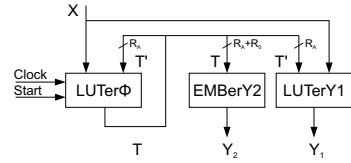
In the discussed case, there are the following CMOs $Y_q \subseteq Y1$: $Y_1 = \emptyset$, $Y_2 = \{y_1\}$, $Y_3 = \{y_2\}$, $Y_4 = \{y_1, y_3\}$, $Y_5 = \{y_2, y_4\}$, $Y_6 = \{y_1, y_5\}$. So, there is $Q = 6$. Using (1.32) gives $R_Q = 3$ and $Z = \{z_1, z_2, z_3\}$. Let it be the configuration of EMB with $S_A = 6$, $t_F = 10$. In this case, the model $P_0^2(\Gamma_{14})$ can be used.

Let us encode the CMOs $Y_q \subseteq Y1$ in the trivial way: $K(Y_1) = 000$, $K(Y_2) = 001$, ..., $K(Y_6) = 101$. There is no sense in optimizing SOPs for $y_n \in Y1$ because this system is implemented as EMB. There is $2^{R_Q+R} = 64$ rows in the table of EMBY. The table has the columns $K(a_m)$, $K(Y_q)$, $Y1$, $Y2$, h . We show the first 8 rows of table of EMBY for $P_0^2 Y1(\Gamma_{14})$ in Table 8.7.

Let us explain the filling of rows for Table 8.7. The rows 1–8 contain the $K(a_1)$ with $Y(a_1) = \emptyset$. So, there are only zeros in the column $Y2$ of Table 8.6. The column $K(Y_q)$ includes all possible input assignments for variables $z_r \in Z$. There is $K(Y1)$ in the row 1, $K(Y2)$ in the row 2, and so on. The corresponding row contains the one-hot codes of CMO $Y_q \subseteq Y1$. Input assignments 110 and 111 are insignificant. So, there are only zeros for them in the column $Y1$.

Now, let us analyse the influence of expansion of code space on the hardware amount in FPGA-based CFSMs. In this case, the concatenation of $K(B_i)$ and $C(a_m)$

Fig. 8.13 Structural diagram of P_E^2 CFSM



represents the code $K(a_m)$. It is represented by (4.39). The code $C(a_m)$ represents a state $a_m \in A$ as an element of the class $B_i \in \Pi_A$. The number of bits in $K(B_i)$ is determined by (2.43), the number of bits in $C(a_m)$ by (3.50). The set T is represented as $T' \cup T_A$, where R_A elements of T are used to encode the classes and R_0 elements the states inside the classes.

This approach leads to P_E^2 CFSM (Fig. 8.13).

In P_E^2 CFSM, the LUTer Φ generates $R = R_A + R_0$ functions $D_r \in \Phi$ represented as (6.10). The LUTerY1 implements $N1$ functions $y_n \in Y1$ represented as (6.16). The EMBerY2 implements the functions (6.4). Now, functions $y_n \in Y2$ depend on $R_A + R_0$ arguments. Let the following condition take place:

$$2^{R_A+R_0} \cdot N2 \leq V_0. \quad (8.13)$$

In this case, it is enough a single EMB to implement the system (6.4). If (8.13) is violated, then it is necessary n_R EMBs. The value of n_R is determined as

$$n_R = \left\lceil \frac{N2}{t_F} \right\rceil. \quad (8.14)$$

The EMBs can be used till the following condition takes place:

$$2^{R_A+R_0} \leq V_0. \quad (8.15)$$

Let us discuss an example of synthesis for CFSM $P_E^2(\Gamma_{14})$. There is $I = 4$; so, there is $R_A = 2$ and $T' = \{T_1, T_2\}$. The maximum number of states $a_m \in A$ belonged to a class $B_i \in \Pi_A$ is equal to 3 ($M_0 = 3$). Using (3.50) gives $R_0 = 2$ and $T_A = \{T_3, T_4\}$.

Let us encode the classes $B_i \in \Pi_A$ in the trivial way: $K(B_1) = 00, \dots, K(B_4) = 11$. Let us encode states as it is shown in Fig. 8.14.

From Fig. 8.14, we can get: $C(a_1) = **$, $C(a_2) = 00$, $C(a_3) = *1$, $C(a_4) = 1*$, and so on. Also, we can get that $K(a_1) = 00**$, $K(a_2) = 0100$, $K(a_3) = 01*1$, $K(a_4) = 011*$ and so on. These codes are used into the column $K(a_s)$ of DST of CFSM $P_E^2(\Gamma_{14})$. This table (Table 8.8) is constructed using the following system of GFT:

$$\begin{aligned} B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 x_2 a_3 \vee \bar{x}_1 \bar{x}_2 a_4; \\ B_2 &\rightarrow x_3 a_2 \vee \bar{x}_3 x_4 a_5 \vee \bar{x}_3 \bar{x}_4 a_4; \\ B_3 &\rightarrow x_5 x_6 a_6 \vee x_5 \bar{x}_6 a_7 \vee \bar{x}_5 a_8; \\ B_4 &\rightarrow x_7 a_1 \vee \bar{x}_7 a_8. \end{aligned} \quad (8.16)$$

Fig. 8.14 State assignment for CFSM $P_E^2(\Gamma_{14})$

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	a_1	a_2	a_6	a_5
	01	*	a_3	a_7	*
	11	*	*	*	*
	10	*	a_4	a_8	*

Table 8.8 DST of CFSM $P_E^2(\Gamma_{14})$

B_i	$K(B_i)$	a_s	$K(a_s)$	$Y(a_s)$	X_h	Y_h	Φ_h	h
B_1	00	a_2	0100	$y_6 y_7$	x_1	y_1	D_2	1
		a_3	0101	y_8	$\bar{x}_1 \bar{x}_2$	y_2	$D_2 D_4$	2
		a_4	0110	y_9	$\bar{x}_1 \bar{x}_2$	$y_1 y_3$	$D_2 D_3$	3
B_2	01	a_2	0100	$y_6 y_7$	x_3	—	D_2	4
		a_5	1000	$y_6 y_9$	$\bar{x}_3 \bar{x}_4$	$y_2 y_4$	D_1	5
		a_4	0110	y_9	$\bar{x}_3 \bar{x}_4$	$y_1 y_3$	$D_2 D_3$	6
B_3	10	a_6	1100	$y_8 y_9$	$x_5 x_6$	$y_1 y_5$	$D_1 D_2$	7
		a_7	1101	y_8	$x_5 \bar{x}_6$	$y_1 y_3$	$D_1 D_2 D_4$	8
		a_8	1110	$y_6 y_7$	$\bar{x}_5$	y_2	$D_1 D_2 D_3$	9
B_4	11	a_1	0000	—	x_7	—	—	10
		a_8	1110	$y_6 y_7$	$\bar{x}_7$	$y_1 y_5$	$D_1 D_2 D_3$	11

There are $H_T = 11$ rows in the DST of $P_E^2(\Gamma_{14})$. It is the best possible result. But more important is the reducing parameters $NA(f_i)$ where $f_i \in \Phi \cup Y1$. For example, we can derive the following functions from Table 8.8:

$$\begin{aligned} y_5 &= F_7 \vee F_{11} = T_1 \bar{T}_2 x_5 x_6 \vee T_1 T_2 x_7; \\ D_1 &= F_5 \vee [F_7 \vee F_8 \vee F_9] \vee F_{11} = \bar{T}_1 T_2 \bar{x}_3 \bar{x}_4 \vee T_1 \bar{T}_2 \vee T_1 T_2 \bar{x}_7. \end{aligned} \quad (8.17)$$

As follows from (8.16), there is $NA(y_5) = NA(D_1) = 5$. So, each of functions from (8.17) is implemented using only a single LUT with $S_L = 5$. In the case of $P_0^2(\Gamma_{14})$, it was necessary two LUTs for implementing each of these functions.

But, for example, there is $NA(y_1) = 8$. This function is represented by the following equation:

$$y_1 = \bar{T}_1 \bar{T}_2 x_1 \vee \bar{T}_1 \bar{T}_2 \bar{x}_2 \vee \bar{T}_1 T_2 \bar{x}_3 \bar{x}_4 \vee T_1 \bar{T}_2 x_5 \vee T_1 T_2 x_7. \quad (8.18)$$

So, there is $NA(y_1) > S_L$. It means that the function (8.18) should be decomposed. For example, the following expression can be found using [11]:

$$y_1 = \bar{T}_1 (\bar{T}_2 x_1 \vee \bar{T}_2 \bar{x}_2 \vee T_2 \bar{x}_3 \bar{x}_4) \vee (T_1 \bar{T}_2 x_5 \vee T_1 T_2 x_7) = \bar{T}_1 y_1^1 \vee y_1^2. \quad (8.19)$$

Fig. 8.15 Logic circuit for function y_1

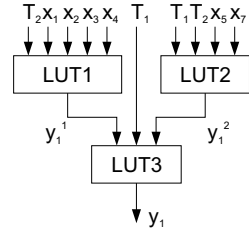


Fig. 8.16 Structural diagram of MP_E^2 CFSM

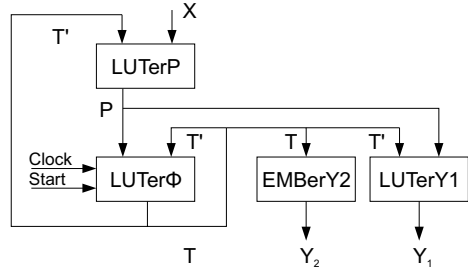


Table 8.9 Table of RLC for CFSM $MP_E^2(\Gamma_{14})$

P	B			
	B_1	B_2	B_3	B_4
p_1	x_1	x_3	x_5	x_7
p_2	x_2	x_4	x_6	—
$K(B_i)$	00	01	10	11

It is necessary three LUTs with $S_L = 5$ to implement the function (8.19). The circuit (Fig. 8.14) has 2 levels of logic (Fig. 8.15).

This example shows that it is necessary to diminish the number of arguments in functions $D_r \in \Phi$ and $y_n \in Y_1$. It can be done using the RLC. It results in MP_E^2 CFSM (Fig. 8.16).

In MP_E^2 CFSM, the LUTerP implements the system (6.3), the LUTerPhi the system (6.13), the LUTerY1 the system (6.19). The function of EMBerY2 is the same as for P_E^2 CFSM. Let us discuss some steps of synthesis of CFSM $P_E^2(\Gamma_{14})$.

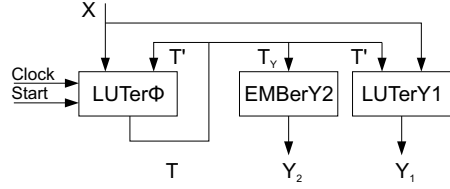
The RLC is executed for classes of PES. Its outcome is represented by Table 8.9.

Using Table 8.9 gives the system (6.3). It is the following system in the discussed example:

$$\begin{aligned} p_1 &= \bar{T}_1 \bar{T}_2 x_1 \vee \bar{T}_1 T_2 x_3 \vee T_1 \bar{T}_2 x_5 \vee T_1 T_2 x_7; \\ p_2 &= \bar{T}_1 \bar{T}_2 x_2 \vee \bar{T}_1 T_2 x_4 \vee T_1 \bar{T}_2 x_6. \end{aligned} \quad (8.20)$$

There is $NA(p_1) = 6$ and $NA(p_2) = 5$. If $S_L = 5$, then there are three LUTs (and two levels of logic) in the circuit of LUTerP.

Fig. 8.17 Structural diagram of P_K^2 CFSM



Replacing LCs $x_l \in X$ in (8.17) and (8.18) allows obtaining the following functions:

$$\begin{aligned} y_5 &= T_1 \bar{T}_2 p_1 p_2 \vee T_1 T_2 p_1; \\ y_1 &= \bar{T}_1 \bar{T}_2 p_1 \vee \bar{T}_1 \bar{T}_2 \bar{p}_2 \vee \bar{T}_1 T_2 \bar{p}_1 \bar{p}_2 \vee T_1 \bar{T}_2 p_1 \vee T_1 T_2 p_1; \\ D_1 &= \bar{T}_1 T_2 \bar{p}_1 p_2 \vee T_1 \bar{T}_2 \vee T_1 T_2 \bar{p}_1. \end{aligned} \quad (8.21)$$

For $S_L = 5$, the condition (8.1) takes place for each function from (8.21). Moreover, the following condition takes place in the discussed case:

$$G + R_A \leq S_L. \quad (8.22)$$

It means that the condition (8.1) is true for all functions derived from DST of $MP_E^2(\Gamma_{14})$. So, there are 13 LUTs and a single EMB in the logic circuit of CFSM $MP_E^2(\Gamma_{14})$.

Now, let us discuss the nonstandard representation of state codes (5.37). In this case, the $K(a_m)$ is represented by $K(B_i)$, where $a_m \in B_i$, and $K(Y_q)$, where $Y_q = Y(a_m)$. It leads to P_K^2 CFSM (Fig. 8.17).

The set T includes variables $T_r \in T'$ and $T_r \in T_Y$. There are R_{Q2} elements in the set T_Y where R_{Q2} is determined as (5.31). The system (6.5) represents functions $y_n \in Y2$. The system (6.5) is implemented by EMBerY2.

This approach makes sense if the condition (8.3) is violated, but the following condition is true:

$$2^{R_{Q2}} \cdot N2 \leq V_0. \quad (8.23)$$

In this case, it is enough only a single EMB to implement all functions $y_n \in Y2$.

Let us discuss some steps of synthesis for the CFSM $P_K^2(\Gamma_{14})$. There are $Q(Y2) = 6$ CMOs $Y_q \subseteq Y2$ in the discussed case. They are the following: $Y_1 = \emptyset$, $Y_2 = \{y_6, y_7\}$, $Y_3 = \{y_8\}$, $Y_4 = \{y_9\}$, $Y_5 = \{y_6, y_9\}$, $Y_6 = \{y_8, y_9\}$. So, it is enough $R_{Q2} = 3$ variables $T_r \in T_Y$ to encode the CMOs $Y_q \subseteq Y2$. There is $R = 3$. So, there is no sense in the encoding of CMOs in the discussed case.

The methods of synthesis are practically the same for CPLD- and FPGA-based P_K^2 CFSMs. So, we do not discuss this approach in details. We hope that our reader perfectly understands this approach.

If the condition

$$2^{R_A+L} \cdot G \leq V_0 \quad (8.24)$$

Fig. 8.18 Structural diagram of MP_K^2 CFSM with 2 EMBs

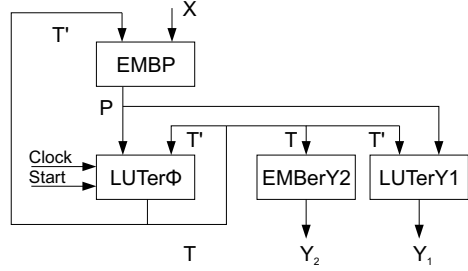
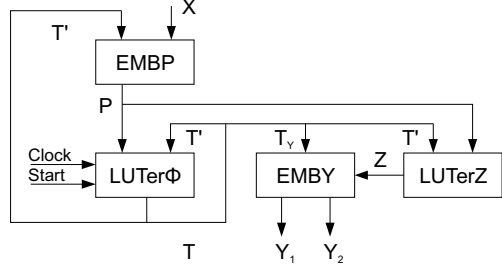


Fig. 8.19 Structural diagram of MP_K^2Y1 CFSM with a shared EMB



takes place, then the RLC can be executed by EMB. It leads to MP_K^2 CFSM with 2 EMBs (Fig. 8.18). Of course, this approach can be used if $n_A = 2$.

It is possible to diminish the number of LUTs in CFSM circuit, using the ECMO $Y_q \subseteq Y1$. In the best case, the following condition takes place:

$$2^{R_{Q2}+R_Q} \cdot (N1 + N2) \leq V_0. \quad (8.25)$$

In this case, systems $y_n \in Y1$ and $y_n \in Y2$ are implemented by a single EMB. It leads to MP_K^2Y1 CFSM with a shared EMB (Fig. 8.19).

The previous chapter contains information about the functions executed by each block of CFSMs shown in Figs. 8.18 and 8.19. A very interesting solution can be obtained if the following condition takes place for EMBP:

$$t_F > G. \quad (8.26)$$

For example, if $t_F - G = R_Q$, then functions $z_r \in Z$ can be represented by EMBPZ (Fig. 8.20).

In this case, the EMBPZ implements functions $P(T', X)$ and $Z(T', X)$, the EMBY functions $Y1(Z)$ and $Y2(T_Y)$. There is only a single block consisting from LUTs. It is the LUTerΦ implementing the IMFs $\Phi(T', P)$. Let the following conditions take places:

$$\begin{aligned} 2^{R_A+L} \cdot (G + R_Q) &= V_0; \\ 2^{R_A+G} \cdot G &= V_0; \\ 2^{R_{Q2}+R_Q} \cdot (N1 + N2) &= V_0. \end{aligned} \quad (8.27)$$

Fig. 8.20 Structural diagram of MP_K^2Y1 CFSM with shared EMBs

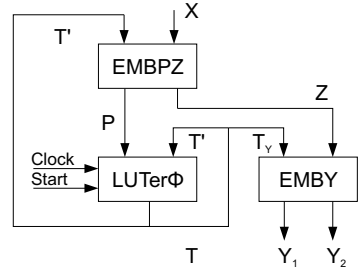
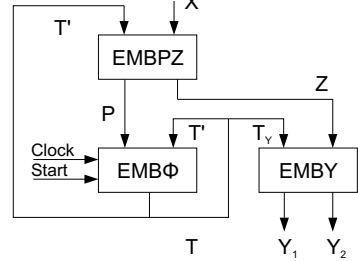


Fig. 8.21 Structural diagram of MP_K^2Y1 CFSM with three EMBs



In this case, it is necessary three EMBs to implement the circuit of MP_K^2Y1 CFSM (Fig. 8.21).

We do not discuss in details the methods for synthesis of CFSMs shown in Figs. 8.20 and 8.21. We hope that they are clear. Now, let us discuss synthesis based on the transformation of state codes into class codes.

8.3 FPGA-Based Synthesis with Transformation of Class Codes

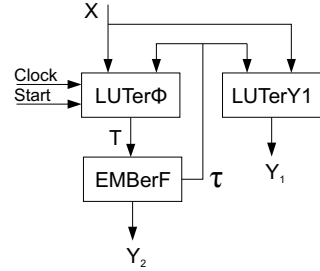
As a rule, $S_L \leq 6$, but $R + L \approx 30$ [3] even for FSMs with the average complexness. So, it is not possible to use the partial transformation. Due to it, let us discuss the CST for FPGA-based CFSMs.

Let us construct the partition $\Pi_A = \{B_1, \dots, B_I\}$ for the set A . Each class $B_i \in \Pi_A$ is a class of PES. Let us encode the classes $B_i \in \Pi_A$ using R_A bits. The value of R_A is determined by (2.43). Let us use class variables $\tau_r \in \mathcal{T}$ for creating codes $K(B_i)$. Now, we can represent a CFSM using the model of P_C^2 CFSM (Fig. 8.22).

In P_C^2 CFSM, the LUTerPhi implements the system (6.11), the LUTerY1 the system (6.17). The EMBerF executes the CST. It generates functions (6.4) and (6.8). It is possible if the following condition takes place:

$$2^R(R_A + N2) \leq V_0. \quad (8.28)$$

Fig. 8.22 Structural diagram of P_C^2 CFSM



In this chapter, we propose the method of synthesis for P_C^2 CFSM. It includes the following steps:

1. Marking the initial GSA Γ by states of Moore FSM.
2. Creating the partition $\Pi_A = \{B_1, \dots, B_I\}$ for set A .
3. Executing the trivial state assignment.
4. Encoding of classes $B_i \in \Pi_A$ using variables $\tau_r \in \mathcal{T}$.
5. Creating the DST of P_C^2 CFSM.
6. Creating the tables for LUTer Φ , LUTer Y_1 , and EMBer F .
7. Implementing the CFSM logic circuit with particular LUTs and EMBs.

Let us discuss an example of synthesis for CFSM $P_C^2(\Gamma_{15})$. The marked GSA Γ_{15} is shown in Fig. 8.23.

The following sets can be derived from GSA Γ_{15} : $A = \{a_1, \dots, a_{16}\}$, $X = \{x_1, \dots, x_{10}\}$, $Y_1 = \{y_1, \dots, y_9\}$, $Y_2 = \{y_{10}, \dots, y_{14}\}$. It gives $M = 16$, $L = 10$, $N_1 = 9$ and $N_2 = 5$. Using (1.2) gives $R = 4$, $T = \{T_1, \dots, T_4\}$ and $\Phi = \{D_1, \dots, D_4\}$.

Using Definition 3.1, we can get the partition of the states set A by classes of PES. In the discussed case, it is the partition $\Pi_A = \{B_1, \dots, B_6\}$ with the following classes of PES: $B_1 = \{a_1\}$, $B_2 = \{a_2, a_3, a_4\}$, $B_3 = \{a_5, a_6, a_7\}$, $B_4 = \{a_8, a_9, a_{10}\}$, $B_5 = \{a_{11}, a_{12}, a_{13}\}$ and $B_6 = \{a_{14}, a_{15}, a_{16}\}$. So, there is $I = 6$.

Because, functions (6.4) and (6.8) are implemented using an EMB, there is no influence of state codes on the hardware amount. Due to it, the states can be encoded in the trivial way. In the discussed case, it gives codes $K(a_1) = 0000$, $K(a_2) = 0001$, ..., $K(a_{16}) = 1111$.

Using (2.43) gives $R_A = 3$ and $\mathcal{T} = \{\tau_1, \tau_2, \tau_3\}$. Let us encode the classes $B_i \in \Pi_A$ in the trivial way: $K(B_1) = 000$, ..., $K(B_6) = 101$.

To construct the DST of P_C^2 CFSM, it is necessary to create the system of generalized formulae of transitions. This system is the following in the discussed case:

$$\begin{aligned}
 B_1 &\rightarrow x_1 a_2 \vee \bar{x}_1 x_2 a_3 \vee \bar{x}_1 \bar{x}_2 a_4; \\
 B_2 &\rightarrow x_3 x_4 a_2 \vee x_3 \bar{x}_4 a_5 \vee \bar{x}_3 x_5 a_6 \vee \bar{x}_3 \bar{x}_5 a_7; \\
 B_3 &\rightarrow x_5 x_6 a_2 \vee x_5 \bar{x}_6 a_8 \vee \bar{x}_5 x_3 a_9 \vee \bar{x}_5 \bar{x}_3 a_{10}; \\
 B_4 &\rightarrow x_7 x_8 a_{11} \vee x_7 \bar{x}_8 a_{12} \vee \bar{x}_7 a_{13}; \\
 B_5 &\rightarrow x_5 a_{14} \vee \bar{x}_5 x_9 x_{10} a_{15} \vee \bar{x}_5 x_9 \bar{x}_{10} a_{16} \vee \bar{x}_5 \bar{x}_9 a_{10}; \\
 B_6 &\rightarrow x_3 x_8 a_{11} \vee x_3 \bar{x}_8 a_{12} \vee \bar{x}_3 a_1.
 \end{aligned} \tag{8.29}$$

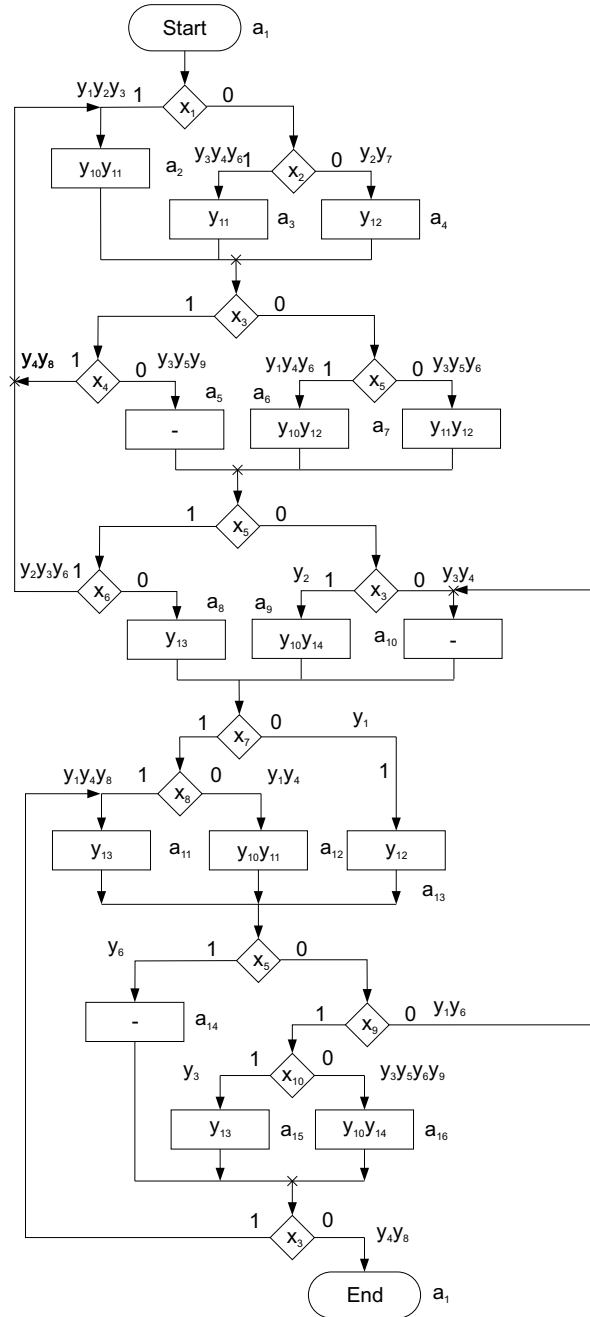
Fig. 8.23 Initial GSA Γ_{15} 

Table 8.10 DST of CFSM $P_C^2(\Gamma_{14})$

B_i	$K(B_i)$	a_s	$K(a_s)$	$Y(a_s)$	X_h	Y_h	Φ_h	h
B_1	000	a_2	0001	$y_{10}y_{11}$	x_1	$y_1y_2y_3$	D_4	1
		a_3	0010	y_{11}	$\bar{x}_1x_2$	$y_3y_4y_6$	D_3	2
		a_4	0011	y_{12}	$\bar{x}_1\bar{x}_2$	y_2y_7	D_3D_4	3
B_2	001	a_2	0001	$y_{10}y_{11}$	x_3x_4	y_4y_8	D_4	4
		a_5	0100	—	$x_3\bar{x}_4$	$y_3y_5y_9$	D_2	5
		a_6	0101	$y_{10}y_{12}$	$\bar{x}_3x_5$	$y_1y_4y_6$	D_2D_4	6
		a_7	0110	$y_{11}y_{12}$	$\bar{x}_3\bar{x}_5$	$y_3y_5y_6$	D_2D_3	7
B_3	010	a_2	0001	$y_{10}y_{11}$	x_5x_6	$y_2y_3y_6$	D_4	8
		a_8	0111	y_{13}	$x_5\bar{x}_6$	$y_1y_2y_7$	$D_2D_3D_4$	9
		a_9	1000	$y_{10}y_{14}$	$\bar{x}_5x_3$	$y_2y_7y_6$	D_1	10
		a_{10}	1001	—	$\bar{x}_5\bar{x}_3$	y_2y_3	D_1D_4	11
B_4	011	a_{11}	1010	y_{13}	x_7x_8	$y_1y_4y_8$	D_1D_3	12
		a_{12}	1011	$y_{10}y_{11}$	$x_7\bar{x}_8$	y_1y_4	$D_1D_3D_4$	13
		a_{13}	1100	y_{12}	$\bar{x}_7$	y_1	D_1D_2	14
B_5	100	a_{14}	1101	—	x_5	y_6	$D_1D_2D_4$	15
		a_{15}	1110	y_{13}	$\bar{x}_5x_9x_{10}$	—	$D_1D_2D_3$	16
		a_{16}	1111	$y_{10}y_{14}$	$\bar{x}_5x_9\bar{x}_{10}$	$y_3y_5y_6y_9$	$D_1D_2D_3D_4$	17
		a_{10}	1001	—	$\bar{x}_5\bar{x}_9$	y_1y_6	D_1D_4	18
B_6	101	a_{11}	1010	y_{13}	x_3x_8	$y_1y_4y_8$	D_1D_3	19
		a_{12}	1011	$y_{10}y_{11}$	$x_3\bar{x}_8$	y_1y_4	$D_1D_3D_4$	20
		a_1	0000	—	$\bar{x}_3$	y_4y_8	—	21

There is $H_T = 21$ in the DST of CFSM $P_C^2(\Gamma_{15})$. It is represented by Table 8.10. There are CMOs $Y_q \subseteq Y_2$ shown in the column $Y(a_s)$.

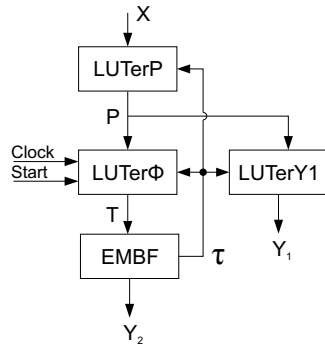
The DST is a base for deriving the functions $D_r \in \Phi$ and $y_n \in Y_1$, as well as $y_n \in Y_2$. For example, the following functions can be derived in the discussed case from Table 8.10:

$$\begin{aligned}
 D_1 &= [F_{10} \vee F_{11}] \vee [F_{12} \vee F_{13} \vee F_{14}] \vee [F_{15} \vee \dots \vee F_{18}] \vee [F_{19} \vee F_{20}] = \\
 &= \tau_2 \bar{\tau}_3 \bar{x}_5 \vee \tau_2 \tau_3 \vee \tau_1 \bar{\tau}_3 \vee \tau_1 \tau_3 x_3; \\
 y_1 &= F_1 \vee F_6 \vee F_9 \vee [F_{12} \vee F_{13} \vee F_{14}] F_{18} \vee F_{19} = \\
 &= \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 x_1 \vee \dots \vee \tau_1 \tau_3 x_3.
 \end{aligned} \tag{8.30}$$

There are the following columns in the table of EMBerF: a_m , $K(a_m)$, B_i , $K(B_i)$, $\mathcal{I}_m$, $Y(a_m)$, m . This table is constructed using the columns a_m , $K(a_m)$, and $Y(a_s)$ of a DST, as well as class codes $K(B_i)$. It is Table 8.11 in the discussed case. The code $K(a_m)$ determines an address of a memory cell. The codes of $B_i \in \Pi_A$ and $Y(a_m) \subseteq Y_2$ determine the contents of the cells. There are the following one-hot codes of CMOs $Y_q \subseteq Y_2$: $K(Y_1) = 00000$, $K(Y_2) = 11000$, $K(Y_3) = 01000$, and so on. We use vectors $\langle y_{10}y_{11}y_{12}y_{13}y_{14} \rangle$ to represent codes $K(Y_q)$.

Table 8.11 Table of EMBerF of CFSM $P_C^2(I_{15})$

a_m	$K(a_m)$	B_i	$K(B_i)$	τ_m	$Y(a_m)$	m
a_1	0000	B_1	000	—	—	1
a_2	0001	B_2	001	τ_3	$y_{10}y_{11}$	2
a_3	0010	B_2	001	τ_3	y_{11}	3
a_4	0011	B_2	001	τ_3	y_{12}	4
a_5	0100	B_3	010	τ_2	—	5
a_6	0100	B_3	010	τ_2	$y_{10}y_{12}$	6
a_7	0110	B_3	010	τ_2	$y_{11}y_{12}$	7
a_8	0111	B_4	011	$\tau_2\tau_3$	y_{13}	8
a_9	1000	B_4	011	$\tau_2\tau_3$	$y_{10}y_{14}$	9
a_{10}	1001	B_4	011	$\tau_2\tau_3$	—	10
a_{11}	1010	B_5	100	τ_1	y_{13}	11
a_{12}	1011	B_5	100	τ_1	$y_{10}y_{11}$	12
a_{13}	1100	B_5	100	τ_1	y_{12}	13
a_{14}	1101	B_6	101	$\tau_1\tau_3$	—	14
a_{15}	1110	B_6	101	$\tau_1\tau_3$	y_{13}	15
a_{16}	1111	B_6	101	$\tau_1\tau_3$	$y_{10}y_{14}$	16

Fig. 8.24 Structural diagram of MP_C^2 CFSM

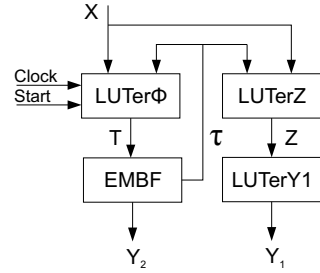
There is $R_A + L = 13$ in the discussed case. This number exceeds significantly the value of S_L for modern FPGA chips [2, 14]. It means that it makes sense to use the replacement of logical conditions by additional variables $p_g \in P$.

Let the condition (8.28) take place and only a single EMB be available. In this case, the RLC is executed by the LUTerP. It leads to MP_C^2 CFSM shown in Fig. 8.24.

In MP_C^2 CFSM, the LUTerP implements the SBF (6.1), the LUTerPhi the system (6.14), the LUTerY1 the system (6.20), the EMBF generates the same functions as the EMBerF in P_C^2 CFSM.

Table 8.12 Table of RLC for CFSM $MP_C^2(\Gamma_{15})$

P	B_i					
	B_1	B_2	B_3	B_4	B_5	B_6
p_1	x_1	x_3	x_3	x_7	x_9	x_3
p_2	x_2	x_4	x_6	x_8	x_{10}	x_8
p_3	—	x_5	x_5	—	x_5	—
$K(B_i)$	000	001	*10	*11	100	1*1

Fig. 8.25 Structural diagram of P_C^2Y1 CFSM

To design the MP_C^2 CFSM, it is necessary to execute the RLC and to find all SBFs to be implemented. Let us execute the RLC for CFSM $MP_C^2(\Gamma_{15})$.

There are the following sets $X(B_i)$ in the discussed case: $X(B_1) = \{x_1, x_2\}$, $X(B_2) = \{x_3, x_4, x_5\}$, $X(B_3) = \{x_3, x_5, x_6\}$, $X(B_4) = \{x_7, x_8\}$, $X(B_5) = \{x_5, x_9, x_{10}\}$ and $X(B_6) = \{x_3, x_8\}$. It gives $G_1 = G_4 = G_6 = 2$ and $G_2 = G_3 = G_5 = 3$, where $|X(B_i)| = G_i$. Using (2.17) gives $G = 3$, $P = \{p_1, p_2, p_3\}$. The table of RLC (Table 8.12) is constructed in the well-known way.

To optimize Eq. (6.2), we use the insignificant input assignments 110, 111 of class variables $\tau_r \in \mathcal{T}$. This is reflected in the row $K(B_i)$ of Table 8.12. The following system $P(\mathcal{T}, X)$ can be found from Table 8.12:

$$\begin{aligned}
 p_1 &= B_1x_1 \vee [B_2 \vee B_3 \vee B_4]x_3 \vee B_4x_7 \vee B_5x_9 = \\
 &= \bar{\tau}_1\bar{\tau}_2\bar{\tau}_3x_1 \vee \dots \vee \tau_1\bar{\tau}_2\bar{\tau}_3x_9; \\
 p_2 &= B_1x_2 \vee B_2x_4 \vee B_3x_6 \vee [B_4 \vee B_6]x_8 \vee B_5x_{10} = \\
 &= \bar{\tau}_1\bar{\tau}_2\bar{\tau}_3x_2 \vee \dots \vee \tau_1\bar{\tau}_2\bar{\tau}_3x_{10}; \\
 p_3 &= x_5.
 \end{aligned} \tag{8.31}$$

As follows from (8.31), only two functions are implemented by $LUTerP$. The variable x_5 is used instead of p_3 . There is $NA(p_1) = 6$, $NA(p_2) = 5$. If $S_L = 5$, then three LUTs create the circuit of $LUTerP$ in the discussed case. We do not discuss other steps of synthesis for CFSM $MP_C^2(\Gamma_{15})$.

To simplify the part of CFSM circuit implementing the MOs $y_n \in Y1$, it is possible to use the ECMO $Y_q \subseteq Y1$. It leads to P_C^2Y1 CFSM (Fig. 8.25).

Table 8.13 CMOs $Y_q \subseteq Y1$ for Γ_{15}

q	Y_q	q	Y_q	q	Y_q
1	—	7	$y_1 y_4 y_6$	13	$y_1 y_4 y_8$
2	$y_1 y_2 y_3$	8	$y_3 y_5 y_6$	14	$y_1 y_4$
3	$y_3 y_4 y_6$	9	$y_2 y_3 y_6$	15	y_1
4	$y_2 y_7$	19	$y_1 y_2 y_7$	16	y_6
5	$y_4 y_8$	11	$y_2 y_7 y_6$	17	$y_1 y_6$
6	$y_3 y_5 y_9$	12	$y_2 y_3$	18	$y_3 y_5 y_6 y_9$

In $P_C^2 Y1$ CFSM, the LUTerZ implements the system (6.23), the LUTerY1 the system (6.7). The functions of other blocks are the same as for P_C^2 CFSM.

There are $Q = 18$ CMOs $Y_q \subseteq Y1$ in the case of GSA Γ_{15} . They are listed in Table 8.13.

Let us place these CMOs into the set V . So, there is $V = \{Y_1, \dots, Y_{18}\}$. Using (1.22) gives $R_Q = 5$. Let it be $S_L = 3$. So, it is necessary 2 LUTs to transform $K(Y_q)$ into $y_n \in Y1$.

In [12], we propose the method of mixed encoding of CMOs. It allows reducing hardware in LUT-based circuits if

$$R_Q > S_L. \quad (8.32)$$

Let us explain this approach.

Let the set V include CMOs Y_i, Y_j such that $Y_i = Y_j \cup \{y_n\}$. So, the elimination of y_n from Y_i leads to the equality $Y_i = Y_j$. It leads to decrementing the number of CMOs $Y_q \subseteq V : |V| = Q - 1$. Let $I(y_n)$ be a number of pairs $\langle Y_j, Y_i \rangle$ such that the elimination of y_n from the collections $Y_i \subseteq Y1$ leads to the equality $Y_i = Y_j$. Therefore, the elimination of y_n from the collections Y_i leads to decrease in the number of CMOs by $I(y_n)$.

The elimination of y_n results in the transformation of the set V into a set V_1 having Q_1 elements, where $Q_1 = Q - I(y_n)$. It is enough R_1 bits to encode the CMO $Y_q \in V_1$

$$R_1 = \lceil \log_2 Q_1 \rceil. \quad (8.33)$$

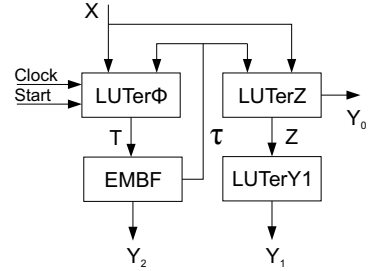
Let the following condition take place:

$$R_L \leq S_L. \quad (8.34)$$

In this case, the MOs y_n is generated by LUTerZ, the LUTerY1 implements the system $Y1 \setminus \{y_n\}$.

If the condition (8.34) is violated, then it is necessary to find a variable y_m such that its elimination from $Y_q \in V_1$ leads to the set V_2 having Q_2 elements. Now, it is necessary R_2 bits to encode the remaining CMOs:

Fig. 8.26 Structural diagram of $P_C^2Y1_m$ CFSM



$$R_2 = \lceil \log_2 Q_2 \rceil. \quad (8.35)$$

Let the following condition take place:

$$R_2 < R_1. \quad (8.36)$$

It means that elimination of y_m leads to further decreasing for the number of bits in codes of CMOs.

This process should be continued till the following condition will be true:

$$R_i = S_L. \quad (8.37)$$

The subscript “ i ” shows that $R_i = \lceil \log_2 Q_i \rceil$, where $Q_i = |V_i|$. There are $N - i$ output variables into CMOs $Y_q \in V_i$.

Let Y_0 be a set of eliminated MOs $y_n \in Y1$. If $y_n \in Y_0$, then it is generated by LUTerZ. Let Y_L be a set of MOs creating the CMOs $Y_q \in V_i$. These MOs are generated by LUTerY1. Due to (8.37), each MO $y_n \in Y_L$ is implemented as a single LUT. This approach leads to $P_C^2Y1_m$ CFSM (Fig. 8.26). The subscript “ m ” means that the principle of mixed encoding of MOs is used.

So, there is $Y1 = Y_0 \cup Y_L$, where

$$Y_0 = Y_0(T, X); \quad (8.38)$$

$$Y_L = Y_L(Z). \quad (8.39)$$

The set Y_L creates Q_L collections. To encode them, there is enough R_L variables:

$$R_L = \lceil \log_2 Q_L \rceil. \quad (8.40)$$

Obviously, the following condition should take place:

$$R_L = S_L. \quad (8.41)$$

Table 8.14 The process of dividing the set $Y1$

y_n	$Q(y_n)$	ΔQ_1	R_1	$Q(y_n)$	ΔQ_1	R_2
y_1	7			5	4	$3 \oplus$
y_2	7			4		
y_3	6			5		
y_4	5			4		
y_5	3			2		
y_6	8+	6	$4 \oplus$	–	–	–
y_7	3			1		
y_8	2			1		
y_9	2			1		
Y_0		y_6			y_1	

Table 8.15 The CMOs after step 1

q	Y_q	q	Y_q	q	Y_q
1	–	5	$y_4 y_8$	9	$y_2 y_3$
2	$y_1 y_2 y_3$	6	$y_3 y_5 y_9$	10	$y_1 y_2 y_7$
3	$y_3 y_4$	7	$y_1 y_4$	11	$y_1 y_4 y_8$
4	$y_2 y_7$	8	$y_3 y_5$	12	y_1

Let us illustrate this approach for Γ_{15} . There is $R_Q = 5$ and $Z = \{z_1, \dots, z_5\}$. Because $S_L = 3$, the condition (8.32) takes place. The process of creating sets Y_0 and Y_L is shown in Table 8.14

There are output variables $y_n \in Y$ shown in the column y_n . There are values of $Q(y_n)$ and $\Delta Q_i (i \in \{1, 2\})$ shown in the corresponding columns. If some y_n is taken for analysis, then there is the sign “+” in the corresponding row. The sign “ $\oplus$ ” shows that the corresponding output variable is included into Y_0 . The sign “–” shows that y_n is excluded from further analysis. There are output variables $y_n \in Y_0$ shown in the row Y_0 .

Using $Q(y_n)$ gives the queue $\gamma = \langle y_6, y_1, y_2, y_3, y_4, y_5, y_7, y_8, y_9 \rangle$. Including y_6 into Y_0 gives $\Delta Q_1 = 6$ and $Q - \Delta Q_1 = 12$. Using (8.33) gives $R_1 = 4$. Because the condition (8.32) is still true, the process should be continued. Now, there is the set $V_L = \{Y_1, \dots, Y_{12}\}$. There are new CMOs $Y_q \in V_L$ shown in Table 8.15.

Next, the following queue γ is formed: $\gamma = \langle y_1, y_3, y_2, y_4, y_5, y_7, y_8, y_9 \rangle$. If y_1 is included into Y_0 , then $\Delta Q_2 = 4$ are excluded from V_L . Now, there is $Q_2 = Q_L - \Delta Q_2 = 8$.

Using (8.35) gives $R_2 = 3$. There is $R_2 = S_L$. So, the further dividing of $Y1$ makes no sense. The algorithm is terminated. There are new CMOs Y_q shown in Table 8.16.

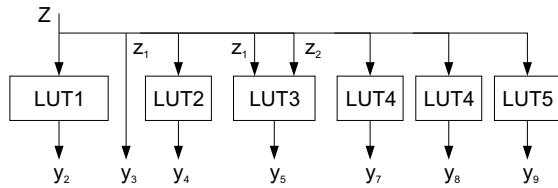
So, the set Y is divided by the sets $Y_0 = \{y_1, y_6\}$ and $Y_L = Y \setminus Y_0$. There is $Q = 8$, $R = 3$, $Z = \{z_1, z_2, z_3\}$. Now, each function $y_n \in Y_L$ is implemented by a single LUT.

Table 8.16 The CMOs after step 2

q	Y_q	q	Y_q	q	Y_q	q	Y_q
1	—	3	y_3y_4	5	y_4y_8	7	y_4
2	y_2y_3	4	y_2y_7	6	$y_3y_5y_9$	8	y_3y_5

Fig. 8.27 Codes of CMOs $Y_q \subseteq Y_L$

	z_1z_2			
z_3	00	01	11	10
0	Y_1	Y_4	Y_2	Y_6
1	Y_5	Y_7	Y_3	Y_8

Fig. 8.28 Logic circuit of LUTerY1 for CFSM $P_C^2Y1_m(\Gamma_{15})$ 

In this book, we propose the method of synthesis for $P_C^2Y1_m$ CFSM. It includes the following steps:

1. Constructing the set of states A for initial GSA Γ .
2. Executing the state assignment.
3. Constructing the partition Π_A for set A .
4. Executing the encoding of classes $B_i \in \Pi_A$.
5. Constructing the collection of MOs $Y_q \subseteq Y1$.
6. Executing the mixed encoding of CMOs $Y_q \subseteq Y1$.
7. Creating the DST of $P_C^2Y1_m$ CFSM.
8. Creating the systems $\Phi(\mathcal{T}, X)$, $\mathcal{T}(T)$, $Y2(T)$, $Z(\mathcal{T}, X)$, $Y0(\mathcal{T}, X)$ and $Y_L(Z)$.
9. Implementing CFSM circuit with particular LUTs and EMB.

Let us discuss an example of synthesis for CFSM $P_C^2Y1(\Gamma_{15})$. The first 5 steps have been already executed and we can start from the mixed encoding of CMOs.

There is the set $Y_L = \{y_2, y_3, y_4, y_5, y_7, y_8, y_9\}$. Let us encode the CMOs Y_q from Table 8.16 as it is shown in Fig. 8.27.

Using codes $K(Y_q)$ from (Fig. 8.27), we can get the following system of Boolean functions:

$$\begin{aligned}
 y_2 &= Y_2 \vee Y_4 = \bar{z}_1z_2\bar{z}_3 \vee z_1\bar{z}_2\bar{z}_3; & y_3 &= Y_2 \vee Y_3 \vee Y_6 \vee Y_8 = z_1; \\
 y_4 &= Y_3 \vee Y_5 \vee Y_7 = \bar{z}_2\bar{z}_3 \vee z_1z_3; & y_5 &= Y_6 \vee Y_8 = z_1z_2; \\
 y_7 &= Y_4 = \bar{z}_1\bar{z}_2\bar{z}_3; & y_8 &= Y_5 = \bar{z}_1\bar{z}_2\bar{z}_3; & y_9 &= Y_6 = z_1z_2\bar{z}_3.
 \end{aligned} \tag{8.42}$$

The system (8.42) determines the circuit of LUTerY1. It includes 6 LUTs having $S_L = 3$ (Fig. 8.28).

Table 8.17 DST of CFSM $P_C^2 Y1_m(\Gamma_{15})$

B_i	$K(B_i)$	a_s	$K(a_s)$	$Y(a_s)$	X_h	Y_{0h}	Z_h	Φ_h	h
B_1	000	a_2	0001	$y_{10}y_{11}$	x_1	y_1	z_1z_2	D_4	1
		a_3	0010	y_{11}	$\bar{x}_4x_2$	y_6	$z_1z_2z_3$	D_3	2
		a_4	0011	y_{12}	$\bar{x}_1\bar{x}_2$	—	z_2	D_3D_4	3
B_2	001	a_2	0001	$y_{10}y_{11}$	x_3x_4	—	z_3	D_4	4
		a_5	0100	—	$x_3\bar{x}_4$	—	z_1	D_2	5
		a_6	0101	$y_{10}y_{12}$	$\bar{x}_3x_5$	y_1y_6	z_2z_3	D_2D_4	6
		a_7	0110	$y_{11}y_{12}$	$\bar{x}_3\bar{x}_5$	y_6	z_1z_3	D_2D_3	7
B_3	010	a_2	0001	$y_{10}y_{11}$	x_5x_6	y_6	z_1z_2	D_4	8
		a_8	0111	y_{13}	$x_5\bar{x}_6$	y_1	z_2	$D_2D_3D_4$	9
		a_9	1000	$y_{10}y_{14}$	$\bar{x}_5x_3$	y_6	z_2	D_1	10
		a_{10}	1001	—	$\bar{x}_5\bar{x}_3$	—	z_1z_2	D_1D_4	11
B_4	011	a_{11}	1010	y_{13}	x_7x_8	y_1	z_3	D_1D_3	12
		a_{12}	1011	$y_{10}y_{11}$	$x_7\bar{x}_8$	y_1	z_2z_3	$D_1D_3D_4$	13
		a_{13}	1100	y_{12}	$\bar{x}_7$	y_1	—	D_1D_2	14
B_5	100	a_{14}	1101	—	x_5	y_6	—	$D_1D_2D_4$	15
		a_{15}	1110	y_{13}	$\bar{x}_5x_9x_{10}$	—	—	$D_1D_2D_3$	16
		a_{16}	1111	$y_{10}y_{14}$	$\bar{x}_5x_9\bar{x}_{10}$	y_6	z_1	$D_1D_2D_3D_4$	17
		a_{10}	1001	—	$\bar{x}_5\bar{x}_9$	y_1y_6	—	D_1D_4	18
B_6	101	a_{11}	1010	y_{13}	x_3x_8	y_1	z_3	D_1D_3	19
		a_{12}	1011	$y_{10}y_{11}$	$x_3\bar{x}_8$	y_1	z_2z_3	$D_1D_3D_4$	20
		a_1	0000	—	$\bar{x}_3$	—	z_3	—	21

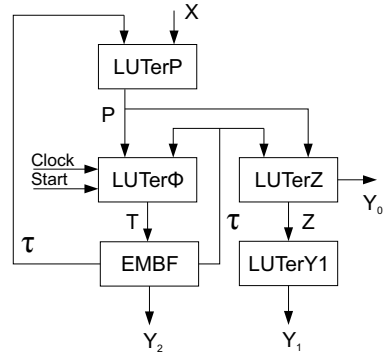
The CMOs $Y_q \subseteq Y1$ are represented by mixed codes. The MOs $y_1, y_6 \in Y1$ are represented by one-hot codes. The MOs $y_n \subseteq Y_L$ are represented by binary codes shown in Fig. 8.27. If we go back to initial CMOs $Y_q \subseteq Y1$, we can get the following codes $\langle y_1, y_6, z_1, z_2, z_3 \rangle$:

$$\begin{aligned}
y_1y_2y_3 &\rightarrow y_1Y_2 \rightarrow \langle 10110 \rangle; \\
y_3y_4y_6 &\rightarrow y_6Y_3 \rightarrow \langle 01111 \rangle; \\
y_2y_7 &\rightarrow Y_4 \rightarrow \langle 00010 \rangle; \\
&\vdots \\
y_3y_5y_6y_9 &\rightarrow y_6Y_6 \rightarrow \langle 01100 \rangle.
\end{aligned} \tag{8.43}$$

These codes are used in DST of $P_C^2 Y1_m(\Gamma_{15})$. It is constructed using the system of GFT (8.29). It is Table 8.17 in the discussed case.

There is a one-hot codes of CMOs $Y_q \subseteq Y_0$ in the column Y_{0h} . These MOs just separated from the column Y_h of DST (Table 8.10). The column Z_h includes variables $z_r = 1$ in the codes of CMOs $Y_q \subseteq Y_L$. For example, there is the CMO $\{y_1, y_2, y_3\}$ in the column Y_h of Table 8.10 (first row). So, it should be y_1 in the column Y_{0h} (Table

Fig. 8.29 Structural diagram of $MP_C^2Y1_m$ CFSM



8.17) in the row 1. Due to $\{y_2, y_3\} = Y_2$ and $K(Y_2) = 110$, there are variables z_1 and z_2 in the first row of Table 8.17 (in the column Z_h). Using the same approach, all other rows are filled.

The system (8.37) is derived from the DST using the data from Y_{0h} . For example, the following function can be derived from Table 8.17:

$$y_6 = F_2 \vee F_6 \vee F_7 \vee F_8 \vee F_{10} \vee F_{15} \vee F_{17} \vee F_{18} = \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 \bar{x}_1 x_2 \vee \dots \vee \tau_1 \bar{\tau}_2 \bar{\tau}_3 \bar{x}_5 x_9 \bar{x}_{10}. \quad (8.44)$$

The system (6.23) is constructed using the column Z_h of the DST. For example, the following function can be derived from Table 8.17.

$$z_2 = [F_1 \vee F_2 \vee F_3] \vee F_6 \vee [F_8 \vee \dots \vee F_{11}] \vee F_{13} \vee F_{20} = \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 \vee \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 \bar{x}_3 x_5 \vee \dots \vee \tau_1 \tau_3 x_3 \bar{x}_8. \quad (8.45)$$

The number of literals for functions $D_r \in \Phi$, $y_n \in Y_0$ and $z_r \in Z$ can be reduced due to the RLC. It leads to $MP_C^2Y1_m$ CFSM (Fig. 8.29).

In $MP_C^2Y1_m$ CFSM, the LUTerP executes the RLC and generates the functions (6.2). The LUTerPhi implements the system (6.14), the LUTerZ implements the systems (6.26) and

$$Y_0 = Y_0(\mathcal{T}, P). \quad (8.46)$$

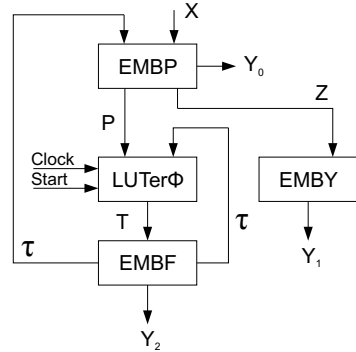
The functions of EMBF and LUTerY1 are clear from our analysis of $MP_C^2Y1_m$ CFSM.

Let it be $S_A < 2^{R_A+L}(R + N1)$, but the following condition take place:

$$2^{R_A+L} \cdot (G + R_Q) = V_0. \quad (8.47)$$

Let it be 2 EMBs available for implementing the CFSM circuit. In this case, we can propose the model of $MP_C^2Y1_m$ CFSM with a shared EMBP shown in Fig. 8.30.

Fig. 8.30 Structural diagram of $MP_C^2Y1_m$ CFSM with a shared EMBP



In the version of CFSM (Fig. 8.30), the EMBP implements functions $P(\mathcal{T}, X)$, $Y_0(\mathcal{T}, X)$ and $Z(\mathcal{T}, X)$. Because of it, the LUTerZ is eliminated. We hope that the steps of synthesis are clear for this CFSM.

Let us discuss how to create the table of EMBP. It contains the following columns: $K(B_i)$, X (it is the address of a memory cell), P , Y_0 , Z (it is the content of the cells), h .

There are H_P rows in the table of EMB:

$$H_P = 2^{R_A+L}. \quad (8.48)$$

It is necessary H_L rows to represent all possible transitions from the states $a_m \in B_i$:

$$H_L = 2^L. \quad (8.49)$$

In the case of CFSM $MP_C^2Y1_m(\Gamma_{15})$ CFSM, the formulae (8.48)–(8.49) give $H_P = 8192$ and $H_L = 1024$. Of course, it is a task of CAD tools to construct the bit-stream for such a huge table.

Let us discuss the following approach targeting FPGA-based synthesis of CFSMs.

8.4 Twofold State Assignment in CFSMs

Let us find the partition $\Pi_D = \{A^1, \dots, A^K\}$ of the set A by classes $A^k \in \Pi_D$ satisfying to the following condition:

$$R_k + L_k \leq S_L \quad (k = \overline{1, K}). \quad (8.50)$$

In (8.50), the symbol R_k stands for the number of state variables necessary to encode the states $a_m \in A^k$, L_k stands for the number of LCs determining transions from states $a_m \in A^k$ ($k = \overline{1, K}$). These LCs form a set $X^k \subseteq X$.

The value of R_k is determined as

$$R_k = \left\lceil \log_2(|A^k| + 1) \right\rceil \quad (k = \overline{1, K}). \quad (8.51)$$

The one in (8.51) is added to take into account the relation $a_m \notin A^k$. Let us use the variables $\tau_r \in \mathcal{T}^k$ to encode the states as the variables set A^k . Obviously, there is $|\mathcal{T}^k| = R_k$.

It necessary R_D variables $\tau_r \in \mathcal{T}$ to encode all states for all classes of Π_D . The value of R_D is determined as

$$R_D = \sum_{k=1}^K R_k. \quad (8.52)$$

Each class $A^k \in \Pi_D$ determines the sets $X^k, Y1^k, \Phi^k$. The set $Y1^k$ includes MOs $y_n \in Y1$ generated during transitions $\langle a_m, a_s \rangle$ where $a_s \in A^k$. The set Φ^k includes input memory functions $D_r \in \Phi$ equal to 1 during transitions into the states $a_m \in A^k$. Obviously, the following relations are true: $Y1^k \subseteq Y1$ and $\Phi^k \subseteq \Phi$.

Let us encode states $a_m \in A$ by binary codes $K(a_m)$ having R bits. The value of R is determined by (1.2). As usually, let us use state variables $T_r \in T$ for the state assignment. Now, each state $a_m \in A$ has two codes. The code $K(a_m)$ determines a state as the element of A , the code $C(a_m)$ as the element of $A^k \in \Pi_D$. Due to this fact, this approach is called twofold state assignment [6].

This approach leads to P_T^2 CFSM (Fig. 8.29). The subscript “ T ” means that the method of twofold state assignment (TSA) is used (Fig. 8.31).

In P_T^2 CFSM, the LUTerk implements the systems

$$\Phi^k = \Phi^k(\mathcal{T}^k, X^k) \quad (k = \overline{1, K}); \quad (8.53)$$

$$Y1^k = Y1^k(\mathcal{T}^k, X^k) \quad (k = \overline{1, K}). \quad (8.54)$$

The LUTer Φ implements the disjunctions of eponymous outputs of LUTerk:

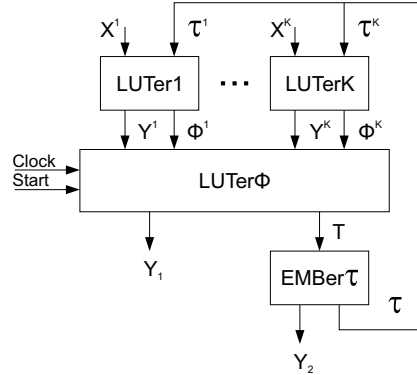
$$\Phi = \bigvee_{k=1}^K \Phi^k; \quad (8.55)$$

$$Y1 = \bigvee_{k=1}^K Y1^k. \quad (8.56)$$

The EMBer $\mathcal{T}$ implements the functions (6.4) and (6.8). We assume that the following condition takes place:

$$2^R(R_D + N2) \leq V_0. \quad (8.57)$$

Fig. 8.31 Structural diagram of P_T^2 CFSM



In this chapter, we propose the method of synthesis of P_T^2 CFSM. It includes the following steps:

1. Marking the states $a_m \in A$ for initial GSA Γ .
2. Executing the state assignment.
3. Finding the partition Π_D with minimum possible amount of blocks.
4. Executing the encoding of states as elements of the classes $A^k \in \Pi_D$.
5. Creating the tables of LUTer1–LUTerK.
6. Creating the tables of LUTer Φ .
7. Creating the tables of EMBer $\mathcal{T}$.
8. Implementing the CFSM logic circuit.

Let us discuss an example of synthesis for CFSM $P_C^2(\Gamma_{16})$. The marked GSA Γ_{16} is shown in Fig. 8.32. As follows from Fig. 8.32, there are the following sets: $A = \{a_1, \dots, a_9\}$, $X = \{x_1, x_2, x_3\}$, $Y1 = \{y_1, \dots, y_7\}$, $Y2 = \{y_8, \dots, y_{12}\}$, $Y = \{y_1, \dots, y_{12}\}$. There is $R = 4$. Let us use LUTs with $S_L = 4$.

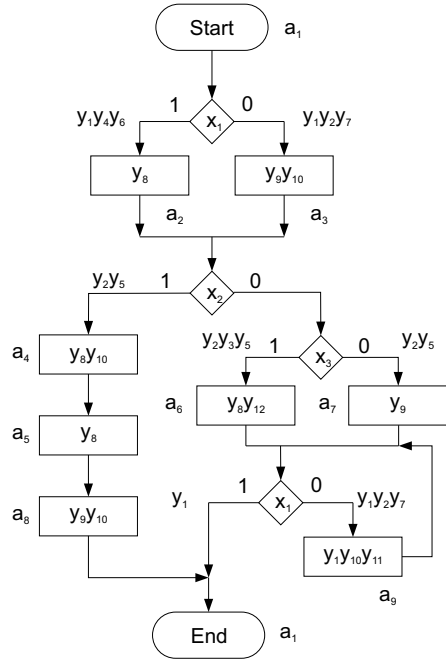
To reduce the numbers of literals in functions $Y2(T)$, it is possible to execute the refined state assignment. But due to (8.56), we can use any method of state assignment. Let us encode states $a_m \in A$ as it is shown in Fig. 8.33.

We do not discuss the methods of creation the partition Π_D . They can be found in [6]. Using them, we can find the partition $\Pi_D = \{A^1, A^2\}$ with $A^1 = \{a_1, a_4, \dots, a_9\}$ and $A^2 = \{a_2, a_3\}$. It gives the sets $X^1 = \{x_1\}$, $X^2 = \{x_2, x_3\}$. This partition satisfies to condition (8.50).

Using (8.51)–(8.52), we can get $R_1 = 3$, $R_2 = 2$, $R_D = 5$. It gives the sets $\mathcal{T}^1 = \{\tau_1, \tau_2, \tau_3\}$, $\mathcal{T}^2 = \{\tau_4, \tau_5\}$ and $\mathcal{T} = \{\tau_1, \dots, \tau_5\}$. Let us encode states $a_m \in A^k$ as it is shown in Table 8.18.

Each table of LUTerk can be viewed as a DST having the following columns: a_m , $C(a_m)$, a_s , $K(a_s)$, $Y(a_s)$, X_h^k , Y_h^k , Φ_h^k , k . In the discussed case, the LUTer1 is represented by Table 8.19, LUTer2 by Table 8.20.

Using Table 8.19 gives the systems $\Phi^1(\mathcal{T}^1, X^1)$ and $Y^1(\mathcal{T}^1, X^1)$. For example, the following functions can be derived:

Fig. 8.32 Marked GSA Γ_{16} **Fig. 8.33** State codes of CFSM $P_T^2(\Gamma_{16})$

		$T_1 T_2$			
		00	01	11	10
$T_3 T_4$	00	a_1	a_2	*	a_3
	01	*	a_4	*	a_8
	11	*	a_5	*	a_9
	10	a_7	a_6	*	*

Table 8.18 State codes for CFSM $P_T^2(\Gamma_{16})$

a_m	$\tau_1 \tau_2 \tau_3$	$\tau_4 \tau_5$	a_m	$\tau_1 \tau_2 \tau_3$	$\tau_4 \tau_5$	a_m	$\tau_1 \tau_2 \tau_3$	$\tau_4 \tau_5$
a_1	0 0 1	0 0	a_4	0 1 0	0 0	a_7	1 0 1	0 0
a_2	0 0 0	0 1	a_5	0 1 1	0 0	a_8	1 1 0	0 0
a_3	0 0 0	1 0	a_6	1 0 0	0 0	a_9	1 1 1	0 0

$$\begin{aligned}
 D_2^1 &= \bar{\tau}_1 \bar{\tau}_2 \tau_3 x_1 \vee \bar{\tau}_1 \tau_2 \bar{\tau}_3; \\
 y_3^1 &= \tau_1 \bar{\tau}_2 \bar{\tau}_3 x_1 \vee \tau_1 \bar{\tau}_2 \tau_3 x_1 \vee \tau_1 \tau_2 \tau_3 x_1.
 \end{aligned} \tag{8.58}$$

The LUTer2 is designed using systems $\Phi^2(\mathcal{T}^2, X^2)$ and $Y^1(\mathcal{T}^2, X^2)$. They are derived from Table 8.20. For example, the following functions can be derived:

$$\begin{aligned}
 D_2^2 &= \tau_5 x_2 \vee \tau_5 x_3 \vee \tau_4 x_2 \vee \tau_4 x_3; \\
 y_3^2 &= D_2^2.
 \end{aligned} \tag{8.59}$$

Table 8.19 DST of CFSM $P_C^2 Y 1_m(\Gamma_{15})$

a_m	$C(a_m)$	a_s	$K(a_s)$	$Y(a_s)$	X_h^1	Y_h^1	Φ_h^1	h
a_1	001	a_2	0100	y_8	x_1	$y_1 y_4 y_6$	D_2	1
		a_3	1000	$y_9 y_{10}$	$\bar{x}_1$	$y_1 y_2 y_7$	D_1	2
a_4	010	a_5	0111	y_8	1	y_1	$D_2 D_3 D_4$	3
a_5	011	a_8	1001	$y_9 y_{10}$	1	y_2	$D_1 D_4$	4
a_6	100	a_1	0000	—	x_1	y_3	—	5
		a_9	1011	$y_9 y_{10} y_{11}$	$\bar{x}_1$	$y_1 y_2 y_7$	$D_1 D_3 D_4$	6
a_7	101	a_1	0000	—	x_1	y_3	—	7
		a_9	1011	$y_9 y_{10} y_{11}$	$\bar{x}_1$	$y_1 y_2 y_7$	$D_1 D_3 D_4$	8
a_8	110	a_1	0000	—	—	—	—	9
a_9	111	a_1	0000	—	x_1	y_3	—	10
		a_9	1011	$y_9 y_{10} y_{11}$	$\bar{x}_1$	$y_1 y_2 y_7$	$D_1 D_3 D_4$	11

Table 8.20 Table of LUTer2 for CFSM $P_T^2(\Gamma_{16})$

a_m	$C(a_m)$	a_s	$K(a_s)$	$Y(a_s)$	X_h^2	Y_h^2	Φ_h^2	h
a_2	*1	a_4	0101	$y_8 y_{10}$	x_2	$y_3 y_6$	$D_2 D_4$	1
		a_6	0110	$y_8 y_{12}$	$\bar{x}_2 x_3$	$y_2 y_3 y_5$	$D_2 D_3$	2
		a_7	0010	$y_{11} y_{12}$	$\bar{x}_2 \bar{x}_3$	$y_2 y_5$	D_3	3
a_3	1*	a_4	0101	$y_8 y_{10}$	x_2	$y_3 y_6$	$D_2 D_4$	4
		a_6	0110	$y_8 y_{12}$	$\bar{x}_2 x_3$	$y_2 y_3 y_5$	$D_2 D_3$	5
		a_7	0010	$y_{11} y_{12}$	$\bar{x}_2 \bar{x}_3$	$y_2 y_5$	D_3	6

The table of LUTer Φ is constructed using systems (8.55)–(8.56). These functions are created in the trivial way. For example, using (8.58)–(8.59) gives:

$$\begin{aligned} D_2 &= D_2^1 \vee D_2^2; \\ y_3 &= y_3^1 \vee y_3^2. \end{aligned} \quad (8.60)$$

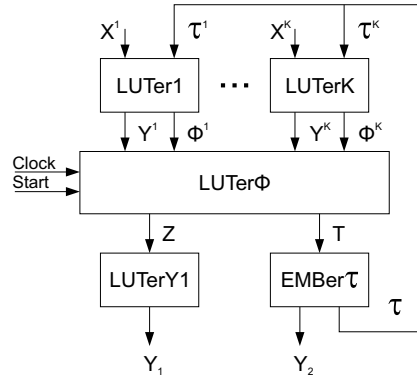
The table of EMBer $\mathcal{T}$ is constructed in the same way as it was before. So, we do not discuss this step, as well as the last step of synthesis.

The twofold state assignment produces logic circuits having a number of advantages [6]:

1. A circuit has exactly three levels of logic. There is a single level of LUTs in the LUTer k ($k = 1, \bar{K}$) due to condition (8.50). There is a single level of EMBs in the EMBer $\mathcal{T}$ due to condition (8.57). There is a single level of logic in the LUTer Φ , if the following condition takes place:

$$K \leq S_L. \quad (8.61)$$

Fig. 8.34 Structural diagram of P_T^2Y1 CFSM



- The circuit has the regular interconnections. For example, LCs $x_l \in X$ enter only the first level of logic circuit (LUTer1–LUTerK). The same is true for variables $\tau_r \in \mathcal{T}$. It makes the task of routing [8, 9] much easier. Also, it reduces the total length of interconnections and, therefore, the parasitic capacitance of a final circuit.

To reduce hardware, it is possible to encode the CMOs $Y_q \subseteq Y1$. In this case, the partition Π_D determines the sets $Z^1, \dots, Z^K$ implemented by LUTer1, ..., LUTerK, respectively. It leads to P_T^2Y1 CFSM (Fig. 8.34).

In P_T^2Y1 CFSM, the LUTerK implements the systems (8.53) and

$$Z^k = Z^k(\mathcal{T}^k, X^k) \quad (k = \overline{1, K}). \quad (8.62)$$

The LUTerPhi implements the systems (8.55) and

$$Z = \bigvee_{k=1}^K Z^k. \quad (8.63)$$

The LUTerY1 implements the system (6.7), the EMBerT the systems (6.4) and (6.8).

Let us discuss an example of synthesis for CFSM $P_T^2Y1(\Gamma_{16})$. Some steps of this example have been already executed.

There are $Q = 9$ CMOs $Y_q \subseteq Y1$ in the discussed example. They are the following: $Y_1 = \emptyset$, $Y_2 = \{y_1, y_4, y_6\}$, $Y_3 = \{y_1, y_2, y_7\}$, $Y_4 = \{y_3, y_6\}$, $Y_5 = \{y_1\}$, $Y_6 = \{y_2, y_3, y_5\}$, $Y_7 = \{y_2, y_5\}$, $Y_8 = \{y_2\}$, $Y_9 = \{y_3\}$. Using (1.32) gives $R_Q = 4$ and the set $Z = \{z_1, \dots, z_4\}$. Let us encode the CMOs $Y_q \subseteq Y1$ in a way minimizing the number of literals in the system (6.7). Using [1] gives the codes $K(Y_q)$ shown in Fig. 8.35.

Fig. 8.35 Codes of CMOs $Y_q \subseteq Y1$ for $P_T^2 Y1(\Gamma_{16})$

		$z_1 z_2$			
$z_3 z_4$		00	01	11	10
	00	Y_1	*	*	Y_8
	01	Y_9	Y_4	*	Y_6
	11	*	*	*	Y_7
	10	Y_5	Y_2	*	Y_3

Table 8.21 Table of LUTer1 of $P_T^2 Y1(\Gamma_{16})$

a_m	$C(a_m)$	a_s	$K(a_s)$	X_h^1	Z_h^1	Φ_h^1	h
a_1	001	a_2	0100	x_1	$z_2 z_3$	D_2	1
		a_3	1000	$\bar{x}_1$	$z_1 z_3$	D_1	2
a_4	010	a_5	0111	1	z_3	$D_2 D_3 D_4$	3
a_5	011	a_8	1001	1	z_1	$D_1 D_4$	4
a_6	100	a_1	0000	x_1	z_4	—	5
		a_9	1011	$\bar{x}_1$	$z_1 z_3$	$D_1 D_3 D_4$	6
a_7	101	a_1	0000	x_1	z_4	—	7
		a_9	1011	$\bar{x}_1$	$z_1 z_3$	$D_1 D_3 D_4$	8
a_8	110	a_1	0000	1	—	—	9
a_9	111	a_1	0000	x_1	z_4	—	10
		a_9	1011	$\bar{x}_1$	$z_1 z_3$	$D_1 D_3 D_4$	11

Using codes $K(Y_q)$ from Fig. 8.35 allows obtaining the following SBF:

$$\begin{aligned}
 y_1 &= Y_2 \vee Y_3 \vee Y_5 = z_1 \bar{z}_4; & y_4 &= Y_2 = z_2 z_3; \\
 y_2 &= Y_3 \vee Y_6 \vee Y_7 = z_1; & y_5 &= Y_6 \vee Y_7 = z_1 z_4; \\
 y_3 &= Y_4 \vee Y_6 \vee Y_9 = \bar{z}_3 z_4; & y_6 &= Y_2 \vee Y_4 = z_2; & y_7 &= Y_3 = z_1 z_3 \bar{z}_4.
 \end{aligned} \tag{8.64}$$

As follows from (8.64), there are only 5 LUTs in the circuit of LUTerY1. The functions y_2 and y_6 are generated by LUTer Φ .

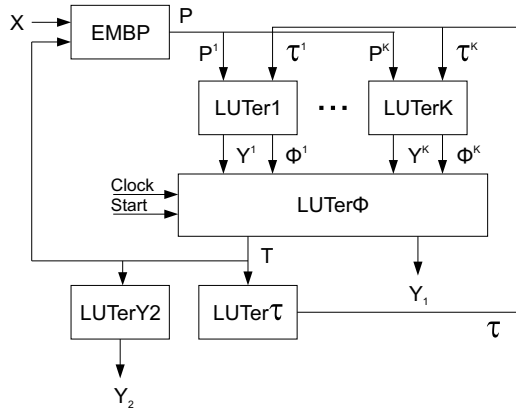
In the common case, there are $N1 \cdot R_Q = 21$ interconnections between the bus Z and LUTerY1. In the discussed case, there are only 11 literals in the system (8.63). Each literal corresponds to an interconnection. Each interconnection adds the parasitic capacitance. So, a proper encoding of CMOs can decrease the consumed power.

Using $K(Y_q)$ from Fig. 8.35 and tables of LUTer1 (Table 8.19) and LUTer2 (Table 8.20), it is possible to obtain tables for $P_T^2 Y1(\Gamma_{16})$. They are Table 8.21 and Table 8.22, respectively.

Using these tables, we can get, for example, the functions: $z_3^1 = \bar{\tau}_1 \bar{\tau}_2 \bar{\tau}_3 \vee \bar{\tau}_1 \tau_2 \bar{\tau}_3 \vee \tau_1 \bar{\tau}_2 \bar{x}_1 \vee \tau_1 \tau_3 \bar{x}_1$, $z_3^2 = \tau_5 \bar{x}_2 \bar{x}_3 \vee \tau_4 \bar{x}_2 \bar{x}_3$. Next, it is possible to get Eq. (8.63).

Table 8.22 Table of LUTer2 of $P_T^2Y1(\Gamma_{16})$

a_m	$C(a_m)$	a_s	$K(a_s)$	X_h^2	Z_h^2	Φ_h^2	h
a_2	01	a_4	0101	x_2	z_2z_4	D_2D_4	1
		a_6	0110	$\bar{x}_2x_3$	z_1z_4	D_2D_3	2
		a_7	0010	$\bar{x}_2\bar{x}_3$	$z_1z_3z_4$	D_3	3
	10	a_4	0101	x_2	z_2z_4	D_2D_4	4
		a_6	0110	$\bar{x}_2x_3$	z_1z_4	D_2D_3	5
		a_7	0010	$\bar{x}_2\bar{x}_3$	$z_1z_3z_4$	D_3	6

Fig. 8.36 Structural diagram of MP_T^2 CFSM

To diminish the number of LUTs in the first level of logic, it is possible to use the RLC. Let us have only a single EMB satisfying the condition

$$2^{R+L} \cdot G \leq V_0. \quad (8.65)$$

In this case, we propose to execute the RLC using the EMB. It leads to MP_T^2 CFSM (Fig. 8.36).

In MP_T^2 CFSM, the EMB P implements the system (6.1). The LUTer k implements the systems

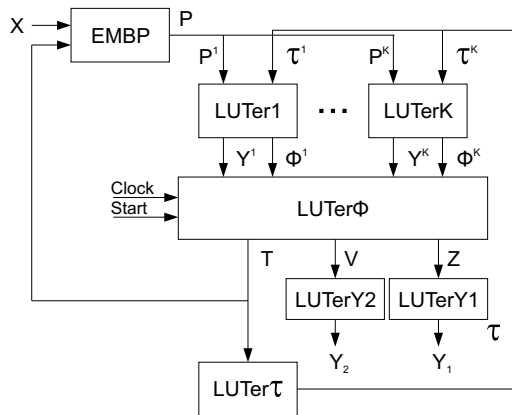
$$\Phi^k = \Phi^k(P^k, T^k) \quad (k = \overline{1, K}); \quad (8.66)$$

$$Y1^k = Y1^k(P^k, T^k) \quad (k = \overline{1, K}). \quad (8.67)$$

The functions of other blocks are clear from previous text.

We can combine together such approaches as: (1) the twofold state assignment; (2) the replacement of LCs; (3) the encoding of CMOs $Y_q \subseteq Y1$ and (4) the encoding of CMOs $Y_2 \subseteq Y$. It leads to MP_T^2Y1Y2 CFSM (Fig. 8.37).

Fig. 8.37 Structural diagram of MP_T^2Y1Y2 CFSM



In this model, the $LUTerk$ generates three systems of functions including the system

$$V^k = V^k(P^k, T^k) \quad (k = \overline{1, K}). \quad (8.68)$$

The variables $v_r \in V$ can be generated by $LUTerV$. But if there is $R > S_L$, then more than a single LUT is needed to generate any function $v_r \in V$. As follows from (8.50), $R_k \leq S_L$. So, the model (Fig. 8.36) gives a better result.

In this book, we have discussed all steps necessary to design the circuits for each block of MP_T^2Y1Y2 CFSM. We hope our reader can combine them together to produce the method of synthesis of MP_T^2Y1Y2 CFSM. On the one hand, it will be a good exercise. On the other hand, it will mean that the reader has understood all the methods previously considered. If so, then the reader is quite ready to develop his/her own synthesis methods of combined finite state machines.

References

1. Achasova S (1987) Synthesis algorithms for automata with PLAs. M: Soviet radio (in Russian)
2. Altera. <http://www.altera.com>. Accessed: Jan 2020
3. Baranov S (1994) Logic synthesis of control automata. Kluwer Academic Publishers
4. Barkalov A, Titarenko L, Visor J, Matvienko A (2015) Realization of combined microprogrammed automaton with FPGA. Probl Inf Control 3(47):6–12 (in Russian)
5. Barkalov A, Titarenko L, Visor J, Matvienko A (2015) Synthesis of combined microprogrammed automaton with FPGA. Comput Tools Netw Syst 14:32–39 (in Russian)
6. Barkalov O, Titarenko L, Mielcarek K (2018) Hardware reduction for LUT-based mealy FSMs. Int J Appl Math Comput Sci 28(3):595–607
7. Kuon I, Tessier R, Rose J (2008) FPGA architecture: survey and challenges - found trends. Electr Des Autom 2:135–253
8. Maxfield C (2004) The design warrior's guide to FPGAs. Academic Press Inc., Orlando, FL, USA
9. Maxfield C (2008) FPGAs: instant access. Newnes

10. Michalski T, Kokosiński Z (2016) Functional decomposition of combinational logic circuits with pkmin. Technical Transactions. Electrical Engineering = Czasopismo Techniczne. Elektrotechnika, Iss. 2-E:191–202
11. Scholl C (2001) Functional decomposition with application to FPGA synthesis. Kluwer Academic Publishers, Boston
12. Sklyarov V, Skliarova I, Barkalov A, Titarenko L (2014) Synthesis and optimization of FPGA-based systems, vol 294. Lecture notes in electrical engineering. Springer, Berlin
13. Trimberg S (2015) Three ages of FPGA: a retrospective on the first thirty years of FPGA technology. IEEE Proc 103(3):318–331
14. Xilinx. <http://www.xilinx.com>. Accessed: Jan 2020

Conclusion

Our time is characterised by the intensive development of design methods targeting VLSI-based digital systems. The complexity of digital systems to be designed increases drastically, as well as the complexity of different VLSI chips used for the implementing the digital systems and their blocks. For example, the up-to-day system-on-chips include up to seven billions of transistors.

Control units are very important blocks of digital systems. They are very complex, too. Development of control units with such complex logic elements is impossible without application of hardware description languages, computer-aided design tools and design libraries. But even the application of all these tools does not guarantee that some competitive product will be designed for appropriate time-to-market. To solve this problem, a designer should know not only CAD tools, but the design and optimization methods, too.

It is especially important in the case of such irregular devices as combined finite state machines. Because of irregularity, their logic circuits are implemented without using the standard library cells. In this case, the knowledge and experience of a designer become a crucial factor of the success. Many experiments conducted with use of standard industrial packages show that outcomes of their operation are, especially in case of complex control units design, too far from optimal. Thus, it is necessary to develop own program tools oriented on CFSM optimization and use them together with industrial packages. This problem cannot be solved without fundamental knowledge in the area of logic synthesis. Besides, to be able to develop new design and optimization methods, a designer should know the existed methods, as well as peculiarities of used logic elements.

We think that new CFSM models and design methods proposed in our book will help in solution of this very important problem. We hope that our book will be useful for the designers of digital systems and scholars developing synthesis and optimization methods targeting implementation VLSI-based logic circuits of combined finite state machines.

Index

B

Binary code, [5](#), [23](#), [222](#), [246](#)

Block

functional, [56](#)

logic, [52](#)

Boolean

functions, [146](#), [215](#)

C

Chip area, [40](#), [50](#), [84](#), [85](#), [88](#), [89](#), [114](#), [117](#), [127](#)

Class of

pseudoequivalent states, [77](#), [215](#), [233](#), [265](#)

Composition, [5](#), [61](#), [146](#), [147](#)

Configurations, [40](#), [54](#)

of EMB, [60](#), [179](#)

of pairs, [20](#)

Conjunction, [44](#), [48](#), [77](#), [85](#), [95](#), [237](#), [253](#)

Consumed power, [283](#)

Control

algorithm, [78](#), [97](#)

memory, [17](#), [21](#)

unit, [51](#)

D

Decomposition

functional, [61](#), [62](#), [249](#), [252](#)

structural, [25](#), [56](#), [77](#), [79](#), [172](#), [215](#), [222](#)

Design, [36](#), [37](#), [47](#), [54](#), [102](#), [103](#), [147](#), [163](#), [165](#), [169](#), [210–212](#), [228](#), [231](#)

Direct structure table, [56](#), [78](#), [100](#), [101](#), [149](#), [251](#), [253](#), [257](#), [261](#), [266](#), [276](#)

Disjunction, [48](#), [95](#), [111](#), [278](#)

E

Embedded memory block, [40](#), [51](#), [179](#), [249](#)

EMBer, [57](#), [63](#), [64](#), [182](#), [185](#), [187–189](#), [191](#), [197–199](#), [201](#), [206–209](#), [211](#), [212](#), [220](#), [221](#), [231](#), [234](#), [236–238](#), [241](#), [278](#), [281](#)

Encoding of

classes of PES, [102](#), [119](#), [130](#), [137](#), [145](#), [256](#)

collections of microoperations, [43](#), [47](#), [85](#), [228](#), [263](#), [271](#), [283](#)

fields of compatible microoperations, [90](#), [92](#)

states, [147](#), [170](#)

Expansion of, [35](#), [37](#), [104–106](#), [169](#), [172](#), [195](#)

code space, [259](#)

F

Field-Programmable Gate Array (FPGA), [39](#), [40](#), [47](#), [60](#), [179](#), [248](#), [249](#), [253](#), [263](#), [269](#)

Finite State-Machine (FSM), [5](#), [40](#), [42](#), [43](#), [47](#), [48](#), [54](#), [55](#), [57](#), [60](#), [62](#), [64](#)

combined (CFSM), [3](#), [25](#), [77](#), [111](#), [145](#), [179](#), [181](#), [182](#), [186–192](#), [194](#), [195](#), [197](#), [199](#), [201–205](#), [207](#), [209](#), [210](#), [212](#), [285](#)

Mealy, [7](#), [9](#), [41–48](#), [55–57](#), [59](#), [61–64](#), [93](#), [94](#), [98](#), [107](#)

Moore, 3, 8, 12, 48, 49, 60–62, 64, 77–79, 82, 86, 93–97, 101–103, 106, 107, 109, 183, 195

Firmware, 3

Flip-flop, 5, 39, 51, 52, 60, 63, 145, 150, 175, 208, 209, 249

Function

- Boolean, 39, 45, 54, 60, 61, 249, 256
- input memory, 42, 48, 63
- irregular, 54, 180, 182
- multiplexer, 180, 182
- regular, 54, 180–182
- tabular, 36

G

Generalized formula of transitions, 266

H

Hardware, 3, 14, 37, 43, 47, 48, 55, 61, 62, 66

Hardware reduction, 22, 25, 40, 85, 111, 215, 271, 282

I

Identifier, 65, 205, 210, 211, 222, 242–244

Input memory functions, 5, 77, 94, 157, 278

Interconnections, 34, 39, 40, 42, 60, 61, 79, 146, 156, 164, 250, 253, 281, 282

K

Karnaugh map, 15, 28, 46, 50, 83, 104, 105, 114, 123, 137, 138, 159, 164, 170, 173, 198, 218, 220, 243, 257

L

Logic

- circuit, 6, 12, 14, 17, 22, 29, 33, 34, 40, 42, 43, 48, 55, 56, 59, 60, 62, 147, 151, 152, 179, 186, 203, 204, 221, 243, 249, 263, 281
- element, 30, 33, 56, 64, 145, 253

Logical condition, 2, 20, 23, 25, 37, 43, 55, 65, 145, 157, 175, 226, 269

Look-Up Table (LUT) element, 39, 60, 249

LUTer, 60, 63, 64, 249, 254, 256, 262, 269–271, 278, 280, 281

M

Macrocell, 38, 39, 50–52, 54, 145, 150, 154, 156, 176, 181, 192, 207, 208, 215, 216, 223, 230, 231, 233, 235

Matrix, 40, 44, 45, 50, 111, 119, 121, 130, 132, 133, 135, 139, 141

- circuit, 41, 43–49, 78, 86, 92, 93, 106, 116, 117, 122, 124, 133, 136, 139, 140, 142
- implementation, 41, 42, 48, 77

Memory

- block, 51, 52, 179, 185, 249
- cell, 34, 179, 249, 268, 277

Microoperation, 42, 47, 50, 97, 98, 102, 185

Model, 63, 64, 88, 107, 108, 117, 125, 127, 133, 134, 159, 169, 172, 216, 222, 226, 228, 231, 233, 236, 259, 265, 276, 285

Multiplexer, 21, 38, 54

O

Object code, 246, 247

Object of CFSM, 249, 253, 255, 259, 264, 277, 282, 285

Object transformation, 215, 241

Optimal encoding of, 46–48, 87

P

Product term, 10, 36, 38, 40, 41, 51, 52, 54, 97, 98, 146, 215, 229, 233

Programmable Array Logic (PAL), 38, 145–147, 150, 151, 156–159, 179, 181, 182, 184, 185, 192–195, 199, 200, 207, 208, 215, 216, 221, 230, 231, 235–237, 241

Programmable Logic Array (PLA), 36, 145–147, 151–153, 215, 229, 230, 246, 247

Programmable Read-Only Memory (PROM), 34

Pseudoequivalent states, 14

R

Read-Only Memory (ROM), 21

Replacement of logical conditions, 43, 108, 179, 269

S

State, 78, 89, 90, 93, 94, 97

- code, 50, 82, 83, 90, 91, 94, 100–102, 106, 107, 257, 265, 266, 280
 - counter, 66
 - next, 63
 - pseudoequivalent, 48, 50, 64
 - register, 41, 60
 - variables, 256, 257, 277, 278
 - State assignment, 62, 182, 200
 - combined, 119
 - optimal, 64, 65, 111, 154, 190, 215, 223, 239, 242, 243
 - refined, 115, 138, 156
 - twofold, 277, 281
 - State variables, 55, 83, 86, 105, 128, 137, 138, 154, 157, 187, 195
 - Structural diagram, 55, 59, 60, 62–66, 77, 78, 81, 85, 88, 94, 96, 97, 102, 103, 107, 108, 111, 116, 118, 119, 122, 124, 129, 134, 140, 141, 182, 186–189, 191, 192, 194, 197, 199, 201, 202, 204, 205, 207, 209, 210, 212, 215, 221–223, 226, 228–231, 235, 236, 238, 239, 241, 242, 246–248, 250, 254, 256, 264, 285
 - Structure table of CFSM, 251, 257, 266
 - Sum Of Products (SOP), 36, 40, 45, 54, 61, 148, 151, 175, 230, 235, 240, 241, 246, 252, 255, 259
 - Synthesis, 113, 115, 119, 130, 134, 137, 141, 145, 147, 154, 159, 169, 179, 182, 190, 195, 203, 205
 - logic, 33, 263, 266, 277
 - System, 113, 114, 116, 117, 120, 130, 135, 138, 141, 180, 188, 191–194, 198, 200, 201, 206, 209
 - cyber-digital, 3
 - cyber-physical, 1
 - digital, 3, 33, 34
 - embedded, 1
 - System of
 - Boolean functions, 6, 7, 25, 26, 193–195, 274
 - generalized formulae of transitions, 100, 104, 114, 130, 191, 193, 207, 210, 211, 218, 224, 237, 244
- ## T
- Table of, 82, 90, 100, 101, 106, 115, 134
 - EMBer, 187, 189, 190, 198, 208, 221, 231, 234, 235
 - PALer, 163, 171, 206–208, 210–212, 216, 218, 221, 224, 225, 244–246
 - Transformation, 94, 159, 206
 - complete, 215
 - partial, 97, 168, 215, 222, 241, 265
 - Transformation of
 - initial GSA, 26, 89, 95, 106
 - object codes, 203, 205, 246, 247
 - state codes, 65, 145, 215, 241, 265
 - states, 205
- ## V
- Vertex, 64
 - conditional, 4, 19
 - operator, 4, 12, 19, 89, 102
 - Very Large Scale Integration (VLSI), 30, 33, 181